Du Token au Pixel : Transformer un Grand Modèle de Langage en Générateur d'Images

Tokenisation visuelle, modélisation autorégressive, budget computationnel et protocoles pratiques pour ingénieurs et chercheurs en apprentissage automatique

Oussama Ghorbel

Bibliothèque Oussama Ghorbel

© 2026 Oussama Ghorbel. Tous droits réservés.

Aucune partie de cet ouvrage ne peut être reproduite, stockée ou transmise, sous quelque forme ou par quelque moyen que ce soit, sans l’autorisation écrite préalable de l’auteur, sauf courtes citations dans le cadre d’une critique ou d’une recension, conformément aux lois sur la propriété intellectuelle.

Ouvrage conçu et dirigé par Oussama Ghorbel, rédigé avec l’assistance d’outils d’intelligence artificielle.

Bibliothèque Oussama Ghorbel — Première édition, 2026.

Avertissement

Le présent ouvrage est fourni à titre strictement éducatif et informatif. Il ne constitue en aucun cas un conseil financier personnalisé, une recommandation d'investissement, ni une sollicitation à acheter ou vendre un instrument financier.

Les informations contenues dans ce livre sont générales et ne tiennent pas compte de votre situation financière personnelle, de vos objectifs ni de votre tolérance au risque. Avant toute décision d'investissement, consultez un conseiller financier agréé et indépendant.

L'auteur et l'éditeur déclinent toute responsabilité quant aux pertes éventuelles résultant de l'utilisation des informations présentées dans cet ouvrage.

Table des matières

Introduction 8

Partie I — Fondations : le Token, le Pixel et la Représentation . . 29

Chapitre 1 – Anatomie d'un LLM : de l'embedding au next-token prediction 30

Chapitre 2 – Le pixel comme signal : résolution, canaux, entropie et redondance 43

Chapitre 3 – La relation token <-> pixel : formalisation et premiers calculs 57

Chapitre 4 – Panorama des approches génératives : autoregression, diffusion, GAN, flux 68

Partie II — Tokenisation Visuelle : du Continu au Discret 79

Chapitre 5 – Auto-encodeurs et VAE : compresser l'image avant de la tokeniser 80

Chapitre 6 – VQGAN et au-delà : tokenizers d'images haute fidélité . . 94

Chapitre 7 – Codebooks : taille, dimension, initialisation et entraînement 108

Chapitre 8 – Patchification et inducteurs visuels : du CNN au ViT . . . 120

Chapitre 9 – Tokenizers continus vs discrets : quel choix pour un LLM ? 130

Partie III — Modélisation Autorégressive : Ordre, Balayage et Génération 145

Chapitre 10 – Factorisation autoregressive de l'image : p(x) = p(x_i | x_{

Chapitre 11 – Ordres de balayage : raster, serpentin, Hilbert, spiral et apprentis 158

Chapitre 12 – Génération parallèle et semi-autoregressive : MAR, MaskGIT et au-delà 169

Chapitre 13 – Résolution progressive et génération multi-échelle . . . 181

Partie IV — Alignement Texte-Image, Entraînement et Architecture 191

Chapitre 14 – Unifier les vocabulaires : texte et image dans un même espace de tokens 192

Chapitre 15 – Cross-attention et conditionnement textuel : guider la génération 211

Chapitre 16 – Protocoles d'entraînement : données, curriculum, stabilité 222

Chapitre 17 – Adapter un LLM existant : GPT, LLaMA, Mistral — étude de cas 237

Partie V — Évaluation, Comparaison et Limites 250

Chapitre 18 – Métriques quantitatives : FID, IS, CLIP-score, LPIPS et au-delà 251

Chapitre 19 – Comparaison approfondie : autoregression par tokens vs diffusion 267

Chapitre 20 – Limites actuelles : artefacts, cohérence, coût et sécurité 282

Chapitre 21 – Ablations et diagnostic : comprendre ce qui compte vraiment 296

© 2026 Oussama Ghorbel

Partie VI — Protocoles Pratiques, Déploiement et Pistes de Recherche 307

Chapitre 22 – Protocole complet : adapter un LLM 7B en générateur 256 × 256 308

Chapitre 23 – Optimisation et déploiement : inférence efficace, quantification, serving 323

Chapitre 24 – Frontières de la recherche : ce qui vient après le token-pixel 334

Conclusion 345

Bibliographie commentée 352

© 2026 Oussama Ghorbel

Introduction

Introduction

Une convergence longtemps attendue

Il y a une décennie à peine, générer une image photoréaliste à partir d'une description textuelle relevait de la science-fiction. Les systèmes de vision par ordinateur reconnaissaient des objets ; les modèles de langage produisaient des phrases. Deux mondes parallèles, séparés par un fossé représentationnel que peu imaginaient franchissable. Puis les architectures transformer ont bouleversé l'ordre établi. En traitant le texte comme une séquence de tokens discrets, les grands modèles de langage — GPT, LLaMA, Mistral et leurs successeurs — ont démontré qu'une seule fonction objectif, la prédiction du token suivant, pouvait engendrer des capacités d'une richesse stupéfiante : raisonnement, traduction, synthèse, code. La question qui s'est alors imposée, presque mécaniquement, fut la suivante : si l'on peut réduire la langue à une suite de tokens, peut-on réduire une image à la même abstraction ? Et si oui, le même moteur autorégressif peut-il générer un pixel aussi naturellement qu'il génère un mot ?

Ce livre est né de cette question. Il n'est ni un survol journalistique des dernières avancées en génération d'images, ni un tutoriel pas-à-pas pour fine-tuner un modèle sur un GPU de bureau. Il est une exploration systématique, rigoureuse et pratique du chemin qui mène d'un LLM entraîné sur du texte à un système capable de produire des images cohérentes, contrôlables et esthétiquement acceptables. Ce chemin passe par la tokenisation visuelle, la modélisation autorégressive de grilles de patches, l'alignement intermodal, l'ingénierie de l'entraînement et l'optimisation de l'inférence. Il est semé de choix architecturaux, de compromis computationnels et de pièges expérimentaux que j'ai voulu documenter avec la plus grande honnêteté possible.

Le sujet : du token au pixel, et retour

Le titre de cet ouvrage n'est pas une métaphore. Il désigne un trajet représentationnel précis. D'un côté, le *token* : unité discrète, indexée dans un vocabulaire fini, manipulée par des opérations d'embedding,

© 2026 Oussama Ghorbel

d'attention et de projection linéaire. De l'autre, le *pixel* : valeur continue (ou quantifiée sur 8 bits), inscrite dans une grille bidimensionnelle, porteuse d'information chromatique, spatiale et fréquentielle. Entre les deux se trouve un ensemble de transformations — encodeurs, quantifieurs, décodeurs — dont la conception détermine presque entièrement la qualité, la fidélité et le coût de la génération.

Comprendre ce trajet, c'est comprendre pourquoi un VQGAN avec un codebook de 1 024 entrées et une dimension de 256 produit des reconstructions sensiblement différentes d'un VQ-VAE à 8 192 codes ; c'est saisir pourquoi l'ordre de balayage d'une grille 32 × 32 influence la convergence de l'entraînement ; c'est mesurer pourquoi un modèle à 7 milliards de paramètres, pensé pour le texte, doit être adapté avec précaution pour ne pas catastrophiquement oublier ses capacités linguistiques tout en acquérant une compétence visuelle. Ce livre traite chacun de ces points, non comme des curiosités isolées, mais comme les maillons d'une chaîne dont la solidité conditionne le résultat final.

J'ai souhaité que le lecteur puisse, à la dernière page, répondre à des questions concrètes : combien de tokens faut-il pour représenter une image 256 × 256 avec une perte perceptuelle acceptable ? Quel budget GPU est raisonnable pour entraîner un adaptateur visuel sur un LLM open-source ? Comment diagnostiquer un effondrement de codebook ou une dérive de la norme des gradients lors d'un entraînement multimodal ? Ce sont ces questions, prosaïques mais décisives, qui structurent la matière de l'ouvrage.

La démarche : formaliser, expérimenter, transmettre

Trois principes guident la rédaction.

Le premier est la **formalisation**. Chaque concept — entropie d'une distribution de patches, facteur de compression d'un auto-encodeur, divergence de KL dans un VAE, score FID entre deux distributions d'images — est défini mathématiquement avant d'être illustré. Je ne cherche pas l'élégance formelle pour elle-même ; je cherche la précision qui évite les malentendus. Quand j'écris que la tokenisation visuelle « discrétise » l'espace latent, je précise quelle quantification est utilisée, selon quelle métrique de distorsion, et avec quelles garanties (ou absences de garanties) théoriques.

© 2026 Oussama Ghorbel

Le deuxième principe est l'**ancrage expérimental**. Les protocoles présentés — notamment celui du chapitre 22, qui détaille l'adaptation d'un LLM 7B en générateur 256 × 256 — s'appuient sur des configurations réalistes : types de GPU, tailles de batch, durées d'entraînement, courbes de perte typiques. Je ne promets aucun résultat miracle. La génération d'images autorégressive reste un domaine où la variance entre runs est élevée, où un hyperparamètre mal réglé peut diviser la qualité par deux, où les comparaisons entre méthodes dépendent fortement du budget computationnel alloué. Je m'efforce de présenter les ordres de grandeur honnêtement, en signalant les incertitudes.

Le troisième principe est la **pédagogie progressive**. Le livre est conçu pour être lu linéairement, chaque chapitre s'appuyant sur les notions introduites précédemment. Cependant, des encadrés de rappel permettent à un lecteur déjà familier avec les transformers ou les auto-encodeurs de sauter les sections introductives sans perdre le fil. Des figures schématiques, des tableaux comparatifs et des extraits de pseudocode jalonnent le texte pour ancrer l'abstraction dans le concret.

Le public visé

Ce livre s'adresse en priorité à trois profils.

L'ingénieur en apprentissage automatique qui maîtrise les fondamentaux du deep learning — rétropropagation, optimisation par descente de gradient, architectures convolutionnelles et attentionnelles — et qui souhaite comprendre comment étendre un pipeline LLM existant vers la génération d'images. Pour ce lecteur, les chapitres pratiques (16, 17, 22, 23) constituent le cœur de l'ouvrage, mais ils ne prennent leur pleine saveur qu'après la lecture des parties théoriques qui les précèdent.

Le chercheur ou le doctorant en vision par ordinateur, en traitement du langage naturel ou en apprentissage génératif, qui cherche une synthèse structurée des approches par tokens discrets, de leurs fondements informationnels et de leurs limites. Les chapitres 3, 7, 10, 11, 18 et 21 offrent un état de l'art critique, avec des références aux travaux fondateurs et aux résultats récents, sans prétendre à l'exhaustivité encyclopédique.

© 2026 Oussama Ghorbel

Le curieux techniquement instruit — développeur senior, architecte logiciel, passionné d'IA — qui veut comprendre *pourquoi* et *comment* un modèle de langage peut devenir un générateur d'images, au-delà des annonces marketing et des démonstrations spectaculaires. Pour ce lecteur, j'ai veillé à ce que chaque équation soit accompagnée d'une interprétation intuitive, et chaque choix architectural d'une justification en termes de compromis.

Un prérequis raisonnable : une familiarité avec l'algèbre linéaire de base, les probabilités élémentaires et le fonctionnement général d'un réseau de neurones. Aucun cadre logiciel spécifique n'est imposé ; les extraits de code, lorsqu'ils apparaissent, sont en Python avec PyTorch, mais les concepts sont indépendants de toute bibliothèque.

Comment lire ce livre

L'ouvrage est organisé en six parties, totalisant vingt-quatre chapitres. Chaque partie peut se lire comme un bloc cohérent, mais je recommande vivement une première lecture séquentielle, car les définitions introduites en Partie I sont réutilisées sans redondance dans les parties suivantes. Les chapitres 1 à 4 posent le vocabulaire commun ; les chapitres 5 à 9 construisent la brique de tokenisation ; les chapitres 10 à 13 explorent la génération proprement dite ; les chapitres 14 à 17 traitent de l'entraînement et de l'adaptation ; les chapitres 18 à 21 offrent les outils de diagnostic ; les chapitres 22 à 24 fournissent le protocole intégral et ouvrent sur l'avenir.

Le lecteur pressé pourra se rendre directement au chapitre 22 pour le protocole complet, puis revenir en arrière pour approfondir les points qui l'intéressent. Le lecteur chercheur privilégiera les chapitres 11, 12, 19 et 24, où les questions ouvertes et les comparaisons méthodologiques sont développées avec le plus de détail.

Annonce du plan

Partie I — Fondations : le Token, le Pixel et la Représentation. Quatre chapitres pour établir le terrain. Le chapitre 1 dissèque l'anatomie d'un LLM : embeddings positionnels, couches d'attention, tête de prédiction, fonction de perte cross-entropie. Le chapitre 2 aborde l'image comme signal : résolution, canaux RVB, entropie de Shannon appliquée

© 2026 Oussama Ghorbel

aux distributions de pixels, redondance spatiale et spectrale. Le chapitre 3 formalise la relation token-pixel : combien de bits d'information porte un pixel, combien un token, et quel est le coût représentationnel du passage de l'un à l'autre. Le chapitre 4 dresse un panorama des grandes familles génératives — autoregression, diffusion, GAN, modèles de flux — en situant l'approche par tokens dans ce paysage.

Partie II — Tokenisation Visuelle : du Continu au Discret. Cinq chapitres dédiés à la transformation de l'image en séquence de tokens. Le chapitre 5 introduit les auto-encodeurs et les VAE comme étape de compression. Le chapitre 6 approfondit VQGAN et ses successeurs, en analysant les mécanismes de quantification vectorielle et de régularisation adversariale. Le chapitre 7 se concentre sur le codebook : taille, dimension, stratégies d'initialisation, phénomènes d'effondrement et remèdes. Le chapitre 8 traite de la patchification et des inducteurs visuels, du CNN au Vision Transformer. Le chapitre 9 compare tokenisation discrète et tokenisation continue, en discutant les implications pour l'intégration dans un LLM.

Partie III — Modélisation Autorégressive : Ordre, Balayage et Génération. Quatre chapitres sur la mécanique de génération. Le chapitre 10 pose la factorisation autoregressive de la distribution conjointe des pixels (ou des tokens visuels). Le chapitre 11 compare les ordres de balayage — raster, serpentin, courbe de Hilbert, spirale — et leurs effets sur la qualité et la vitesse. Le chapitre 12 présente les approches parallèles et semi-autorégressives : MaskGIT, MAR, et les méthodes par masquage itératif. Le chapitre 13 explore la génération multi-échelle et la résolution progressive.

Partie IV — Alignement Texte-Image, Entraînement et Architecture. Quatre chapitres sur l'intégration multimodale. Le chapitre 14 montre comment unifier les vocabulaires textuel et visuel dans un espace de tokens commun. Le chapitre 15 détaille les mécanismes de cross-attention et de conditionnement textuel. Le chapitre 16 aborde les protocoles d'entraînement : constitution des données, curriculum learning, stabilité numérique. Le chapitre 17 propose une étude de cas : adapter concrètement un LLM existant (GPT, LLaMA, Mistral) à la génération d'images, avec les modifications architecturales nécessaires.

© 2026 Oussama Ghorbel

Partie V — Évaluation, Comparaison et Limites. Quatre chapitres pour mesurer et comprendre. Le chapitre 18 passe en revue les métriques quantitatives : FID, Inception Score, CLIP-score, LPIPS, et leurs limites. Le chapitre 19 propose une comparaison approfondie entre autoregression par tokens et diffusion, en termes de qualité, de coût, de contrôlabilité et de scalabilité. Le chapitre 20 discute honnêtement les limites actuelles : artefacts de quantification, incohérences spatiales, coût computationnel, enjeux de sécurité et de biais. Le chapitre 21 introduit les méthodologies d'ablation pour isoler les facteurs qui comptent réellement.

Partie VI — Protocoles Pratiques, Déploiement et Pistes de Recherche. Trois chapitres pour passer à l'action et regarder devant. Le chapitre 22 fournit un protocole complet, étape par étape, pour adapter un LLM 7 milliards de paramètres en générateur d'images 256 × 256 : choix du tokenizer, modification de l'architecture, entraînement, évaluation itérative. Le chapitre 23 traite de l'optimisation et du déploiement : inférence efficace, quantification, techniques de serving, réduction de latence. Le chapitre 24 ouvre sur les frontières de la recherche : ce qui vient après le paradigme token-pixel, les modèles unifiés, la génération vidéo, les interfaces neuronales et les questions ouvertes.

Conventions typographiques et notes

Les termes techniques anglais couramment utilisés dans la littérature (token, patch, codebook, embedding, cross-attention, fine-tuning, etc.) sont conservés en anglais et signalés en *italique* à leur première occurrence, avec une définition ou un équivalent français entre parenthèses. Les équations sont numérotées par chapitre. Les références bibliographiques sont données entre crochets [Auteur, année] et regroupées en fin d'ouvrage. Les extraits de pseudocode sont présentés dans des blocs monospaces.

J'ai fait le choix de ne pas multiplier les notes de bas de page, préférant intégrer les remarques contextuelles directement dans le corps du texte. Lorsqu'une affirmation repose sur un résultat empirique spécifique, je cite la source. Lorsqu'il s'agit d'une observation issue de la pratique expérimentale, je le signale explicitement.

© 2026 Oussama Ghorbel

Un mot sur l'état du domaine

La génération d'images est un champ en évolution rapide. Les méthodes décrites ici reflètent l'état des connaissances au moment de la rédaction. Certaines architectures mentionnées seront peut-être supplantées avant même la publication de cet ouvrage ; d'autres, plus fondamentales — la quantification vectorielle, la factorisation autoregressive, l'attention multi-têtes — constituent des briques dont la pertinence dépasse les modes. J'ai tâché de distinguer clairement ce qui relève du principe durable et ce qui relève de l'implémentation contingente, afin que le lecteur puisse extrapoler au-delà de ce texte.

Je ne promets pas qu'à la fin de ce livre vous saurez générer des images indiscernables de photographies avec un simple ordinateur portable. Ce serait malhonnête. Je promets, en revanche, que vous comprendrez précisément ce qui rend cette tâche difficile, quels leviers existent pour l'approcher, et comment concevoir un protocole expérimental rigoureux pour progresser, étape par étape, vers des résultats solides.

Remerciements et avertissement

Ce livre a bénéficié des échanges avec de nombreux collègues, chercheurs et praticiens dont les retours critiques ont affiné ma pensée. Je les remercie collectivement, sans les nommer individuellement, car leurs contributions sont trop entremêlées pour être attribuées ligne à ligne. Je remercie également les communautés open-source dont les implémentations publiques ont rendu possible l'expérimentation à moindre coût.

Un avertissement, enfin : les considérations de coût computationnel abordées dans cet ouvrage sont indicatives. Les prix du matériel, les tarifs cloud et les performances des puces évoluent rapidement. Les ordres de grandeur donnés visent à fournir une intuition, non à constituer un devis. De même, les questions éthiques liées à la génération d'images — consentement des données d'entraînement, biais représentationnels, usages malveillants — sont évoquées mais ne sauraient être traitées exhaustivement dans un ouvrage technique. J'encourage le lecteur à consulter les travaux dédiés à ces enjeux sociétaux.

© 2026 Oussama Ghorbel

En route

Le token est une abstraction. Le pixel est une réalité physique. Entre les deux, il y a un choix d'ingénierie, un compromis informationnel, une architecture. Ce livre est l'histoire de ce passage. Commençons par le commencement : comprendre ce qu'est un token, ce qu'est un pixel, et pourquoi leur rencontre est à la fois si naturelle et si délicate.

Bonne lecture.

Chapitre 1 — Anatomie d'un LLM : de l'embedding au next-token prediction

Un grand modèle de langage n'a jamais vu un mot. Au niveau de ses opérations internes, il n'a jamais vu non plus une phrase, une langue ou un sens. Il voit des entiers.

L'entrée est une suite d'identifiants. La sortie est une distribution de probabilité sur un ensemble fini d'identifiants. Entre les deux, des multiplications de matrices, une normalisation, une fonction non linéaire, et encore des multiplications de matrices. Tout le reste — la grammaire, la logique, la capacité à résumer un contrat ou à corriger une fonction Python — émerge de l'optimisation de cette fonction numérique sur un volume considérable de séquences.

C'est précisément cette austérité opérationnelle qui rend le projet de ce livre possible. Si le moteur ne sait traiter que des entiers ordonnés, rien dans son architecture n'interdit que ces entiers désignent autre chose que des fragments de texte. Un indice dans un *codebook* visuel est un entier comme un autre. On peut l'envoyer dans la même table d'embedding, le faire circuler dans les mêmes couches d'attention, et le prédire avec la même tête de sortie.

Mais cette possibilité architecturale ne doit pas être confondue avec une facilité statistique. Le moteur est neutre ; les données ne le sont pas. Une image n'a pas la même redondance, la même locality, ni la même tolérance à l'erreur qu'une phrase. Le propos de ce chapitre est de disséquer le moteur avec suffisamment de précision pour que, lorsque nous y introduirons des tokens visuels, nous sachions exactement quelles hypothèses nous conservons et lesquelles nous brisons.

© 2026 Oussama Ghorbel

1.1 L'interface discrète : ce que le modèle consomme vraiment

Un LLM ne manipule pas du texte. Il manipule une suite $x_{1:T} = (x_1, x_2, \ldots, x_T)$ où chaque x_t appartient à un vocabulaire fini $\mathcal{V} = \{0, 1, \ldots, |\mathcal{V}| - 1\}$. Le passage du caractère à cet entier est assuré par un tokenizer, le plus souvent de type *byte-pair encoding* — BPE, une algorithme de fusion statistique décrit par Sennrich et ses collaborateurs [Sennrich et al., 2016].

Le BPE part d'un alphabet de bytes et fusionne récursivement les paires les plus fréquentes dans le corpus d'entraînement. Il en résulte un vocabulaire hybride : certains indices correspondent à des mots entiers, d'autres à des sous-mots, d'autres à des fragments orthographiques ou à des séquences d'octets. Un terme technique rare peut occuper cinq positions dans la séquence ; un article fréquent en occupe une. De cette simple asymétrie découlent des conséquences pratiques que nous retrouverons au chapitre 16 : la longueur d'une séquence n'est pas une mesure du nombre de mots, et la densité informationnelle varie considérablement d'un token à l'autre.

Deux remarques méritent d'être fixées dès maintenant.

Premièrement, **un token n'est pas une unité sémantique**. C'est une unité de compression séquentielle. Pensser le token comme « un mot » est une heuristic commode pour le lecteur, mais une erreur pour l'ingénieur : le modèle ne dispose d'aucun privilège architectural sur les boundaries lexicales.

Deuxièmement, **l'entier lui-même ne porte pas de sens**. L'indice 4 821 n'est ni « plus proche » de 4 822 que de 912. La numérotation est un convention d'adressage. Toute la géométrie sémantique est portée par la table d'embedding :

$$h_t^{(0)} = E[x_t], \qquad E \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

Cette table est l'unique point de contact entre le monde discret et le monde continu. Elle assigne à chaque indice un vecteur de d réels, appris par rétropropagation. Deux tokens qui apparaissent dans des contextes semblables voient leurs vecteurs converger vers des régions voisines de $\mathbb{R}^d$ — non parce que quelqu'un l'a prescrit, mais parce que c'est une

© 2026 Oussama Ghorbel

solution économique pour le modèle.

Certaines architectures multiplient ensuite ce vecteur par $\sqrt{d}$ pour équilibrer les échelles avec les couches suivantes ; d'autres non. C'est un détail d'implémentation, mais un détail qui peut faire échouer un transfert de poids si on l'omet.

Pour notre projet, un troisième élément est décisif : les **tokens spéciaux**. Les LLM modernes disposent d'indices réservés marquant le début et la fin d'un document, le rôle d'un interlocuteur, ou la fin d'une génération. Ces jetons sont la couture sur laquelle nous coudrons la modalité visuelle. Un marqueur `<img_start>`, suivi de deux cents indices de codebook, puis d'un marqueur `<img_end>`, n'est pour le transformeur qu'une séquence parfaitement ordinaire. Le chapitre 14 exploitera cette coutrure en détail.

1.2 La position : donner un ordre à un ensemble

L'attention telle qu'introduite par Vaswani et ses collaborateurs [Vaswani et al., 2017] est, sans dispositif supplémentaire, insensible à l'ordre des éléments. Permuter la séquence permute le résultat de la même manière : aucun mécanisme ne dit à la couche « ce token vient avant celui-là ».

Or l'ordre est précisément ce qui fait d'une suite de tokens un énoncé. « Le chat poursuit le chien » et « le chien poursuit le chat » contiennent les mêmes éléments dans des ordres différents. Le modèle doit donc recevoir l'information positionnelle explicitement.

Trois familles de solutions coexistent. Les *embeddings positionnels appris* attribuent à chaque rang un vecteur entraîné, ajouté à l'embedding du token. Les *encodages sinusoïdaux* utilisent des fonctions périodiques fixes de différentes fréquences. La troisième, aujourd'hui dominante dans les modèles ouverts, est la *RoPE — Rotary Position Embedding* [Su et al., 2021] — qui n'ajoute rien au vecteur mais rotationne les requêtes et les clés dans des sous-espaces de deux dimensions.

Pour un indice de fréquence r, la RoPE définit une angle $\theta_r = b^{-2r/d_h}$, où d_h est la dimension d'une tête et b une base généralement grande. Le vecteur de requête à la position m est rotationné de $m\theta_r$, celui de clé à la position n de $n\theta_r$. La propriété essentielle est que leur produit scalaire ne

© 2026 Oussama Ghorbel

dépend alors que du décalage $m - n$:

$$\tilde{q}_m^\top \tilde{k}_n \;=\; \mathrm{Re}\sum_r q_{m,r}\,\overline{k_{n,r}}\; e^{\mathrm{i}(m-n)\theta_r}.$$

Le modèle apprend ainsi des relations relatives « deux positions plus loin », « bien plus tôt dans le contexte » — plutôt que des coordonnées absolues. C'est un inducteur puissant, et c'est aussi un piège pour la génération d'images.

Une image n'est pas une ligne. C'est une grille. Lorsque nous aplatirons une carte latente 16×16 en une séquence de 256 éléments, la RoPE unidimensionnelle verra deux patches adjacents verticalement comme séparés de seize positions, alors que deux patches adjacents horizontalement ne seront séparés que d'une. Cette distorsion n'est pas fatale : le modèle peut l'apprendre. Mais elle gaspille de la capacité et ralentit la convergence.

Nous verrons au chapitre 11 comment factoriser l'encodage positionnel en deux axes, et au chapitre 8 comment la patchification elle-même constitue un choix d'induction spatiale. Pour l'instant, retenez seulement ceci : **l'ordre de balayage n'est pas un détail d'implémentation, c'est une hypothèse structurelle sur le monde**.

1.3 Le bloc transformeur : lire et écrire dans un résidu

Un LLM décodeur standard empile des blocs identiques dans leur forme et distincts dans leurs paramètres. Chaque bloc consomme un tenseur $h \in \mathbb{R}^{T \times d}$ et en produit un autre de même forme. La structure moderne est dite *pré-norm* : on normalise avant chaque sous-module, et on ajoute le résultat à l'entrée.

$$h' = h + \mathrm{Attn}(\mathrm{Norm}(h)), \qquad h'' = h' + \mathrm{FFN}(\mathrm{Norm}(h')).$$

La normalisation est le plus souvent une RMSNorm, qui divise le vecteur par la racine de sa moyenne quadratique puis le rescale par un vecteur appris g :

$$\mathrm{RMSNorm}(z) = \frac{z}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} z_j^2 + \varepsilon}} \odot g.$$

Contrairement à une LayerNorm classique, elle ne recentre pas les activations. C'est un choix d'efficacité, pas une hypothèse sur la nature

© 2026 Oussama Ghorbel

du langage.

L'attention. On projette les représentations normalisées en trois familles de vecteurs — requêtes Q, clés K, valeurs V — puis on calcule :

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V.$$

Le masque causal M vaut zéro lorsque la position consultée est antérieure ou égale à la position courante, et $-\infty$ sinon. Il garantit qu'aucune information du futur ne fuit dans la prédiction du présent. Cette contrainte est ce qui permet d'entraîner sur toute la séquence en un seul passage, tout en définissant une véritable distribution autorégressive.

La division par $\sqrt{d_h}$ stabilise les logits avant le softmax : sans elle, la variance des produits scalaires croît avec la dimension et pousse l'attention vers des distributions dégénérées. Multiple heads signifie simplement que l'on répète l'opération dans plusieurs sous-espaces, puis que l'on concatène les résultats avant une projection de sortie.

Une variante importante pour la suite : le *grouped-query attention*, où plusieurs têtes de requête partagent une même clé et une même valeur. Le coût mémoire du cache d'inférence en est réduit proportionnellement. Nous y reviendrons au chapitre 23 ; sachez dès maintenant qu'un modèle à 7 milliards de paramètres peut voir son cache divisé par quatre par ce seul choix.

Le réseau feed-forward. Après l'attention, chaque position est traitée individuellement par un petit réseau à deux couches. Les architectures récentes utilisent une gating de type SwiGLU :

$$\mathrm{FFN}(z) = W_d(\mathrm{SiLU}(W_g z) \odot (W_u z)).$$

Trois matrices au lieu de deux, mais une nonlinearité plus expressive. Shazeer a discuté cette famille de gates dans un article court et influent [Shazeer, 2020].

Le résidu. L'équation (1.3) mérite une lecture particulière. Le tenseur h n'est pas écrasé à chaque couche : il est enrichi. Chaque bloc lit une partie de h via ses projections, calcule une correction, et l'ajoute. Cette métaphore du *résidu* comme bande passante commune — une sorte de tableau noir sur lequel les couches écrivent et se relisent — n'est

© 2026 Oussama Ghorbel

pas une description mécaniste définitive, mais c'est un modèle mental remarquablement utile.

Pour notre projet, il suggère où intervenir : ajouter une modalité ne nécessite pas de remplacer le tableau noir, seulement d'apprendre à y écrire un nouveau type d'information sans effacer l'ancien. C'est exactement la tension que nous étudierons au chapitre 17.

Où sont les paramètres ? À titre d'exemple, la configuration ouverte usuellement désignée comme LLaMA-2 7B [Touvron et al., 2023] présente les ordres de grandeur suivants :

Composant	Calcul	Ordre de grandeur
Table d'embedding	$32\,000 \times 4\,096$	131 M
Attention par couche	$4d^2$	67 M
Feed-forward par couche	$3 \times d \times d_{ff}$, $d_{ff} = 11\,008$	135 M
32 blocs complets	32×202 M	6,47 B
Tête de sortie non partagée	$4\,096 \times 32\,000$	131 M

Le total approche 6,7 milliards. Deux constatations en découlent. D'abord, la majeure partie des paramètres n'est pas dans l'interface discrète mais dans le traitement contextuel : c'est là que réside la capacité de modélisation. Ensuite, la table d'embedding et la tête de sortie représentent un volume non négligeable — environ 4 % chacune. Lorsque nous agrandirons le vocabulaire avec des tokens visuels, ce coût n'est pas annexe.

1.4 La tête de sortie : d'un vecteur à une distribution

Au sommet du réseau, une dernière normalisation précède une projection linéaire vers le vocabulaire :

$$z_t = W_U \,\mathrm{Norm}(h_t), \qquad W_U \in \mathbb{R}^{|\mathcal{V}| \times d},$$
$$p_\theta(x_{t+1} \mid x_{1:t}) = \mathrm{softmax}(z_t).$$

Les z_t sont les *logits* : des scores réels, non normalisés, dont le softmax fait une distribution. Notez l'alignement : la sortie à la position t prédit le token à la position $t+1$. Ce décalage d'une unité, apparemment anodin, est une source classique de bugs dans les implémentations

© 2026 Oussama Ghorbel

multimodales. Nous le reverrons dans le protocole du chapitre 22.

Dans de nombreuses architectures, W_U est lié à la table d'embedding E : les mêmes vecteurs servent à encoder l'entrée et à scorer la sortie. Cette *weight tying* réduit le nombre de paramètres et impose une géométrie cohérente entre ce que le modèle reconnaît et ce qu'il produit. Pour un vocabulaire visuel, la question du partage devient un choix de design à part entière, discuté au chapitre 14.

De la distribution au token. La prédiction la plus probable n'est pas nécessairement le choix souhaitable. Le *greedy decoding* sélectionne $\arg\max$; l'échantillonnage tire selon la distribution, éventuellement temperée :

$$p_i \propto \exp(z_i/\tau).$$

Une température $\tau < 1$ accentue le pic ; $\tau > 1$ aplati la distribution. Les filtres *top-k* et *top-p* restreignent l'échantillonnage aux candidats les plus plausibles.

Pour le texte, ces leviers contrôlent la créativité. Pour l'image, ils contrôlent autre chose : la texture, la diversité des modes, la stabilité de la composition. Une température trop basse tend à produire des surfaces lisses et répétitives ; une température trop élevée désorganise la scène. Il n'existe pas de valeur universelle, ce qui explique pourquoi les protocoles sérieux prévoient une grille de températures plutôt qu'un réglage unique hérité du langage.

Le coût de la génération. Une fois la séquence de contexte traitée, on peut conserver pour chaque couche les clés et les valeurs déjà calculées : c'est le *KV cache*. Générer un token supplémentaire ne requiert alors qu'une passe sur un seul vecteur, et non sur toute la séquence.

La mémoire du cache suit une loi simple :

$$\text{Mémoire} = 2 \cdot L \cdot T \cdot n_{kv} \cdot d_h \cdot b,$$

où L est le nombre de couches, T la longueur, n_{kv} le nombre de têtes de clé/valeur, et b le nombre d'octets par élément. Avec 32 couches, 32 têtes, une dimension de tête de 128 et du demi-précision, on obtient environ 0,5 Mio par position. Une séquence de 4 096 positions consomme donc près de 2 Go de cache — pour un seul utilisateur.

© 2026 Oussama Ghorbel

Cette équation sera l'une des plus utiles de l'ouvrage. Elle explique pourquoi la résolution d'une image, via le nombre de tokens qu'elle impose, se traduit directement en coût mémoire et en latence.

1.5 L'objectif : la cross-entropie comme contrat

L'entraînement d'un LLM repose sur une fonction objectif d'une sobriété trompeuse :

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

On minimise la négatif log-vraisemblance moyen des tokens observés. Le *teacher forcing* consiste à fournir au modèle la séquence réelle comme contexte, plutôt que sa propre génération précédente. Grâce au masque causal, toutes les positions peuvent être supervisées simultanément : un unique passage avant et un passage arrière entraînent le modèle à prédire chaque token à partir de son préfixe.

C'est un point crucial pour la suite. **L'entraînement autorégressif est parallèle dans le temps ; l'inférence autorégressive ne l'est pas.** Cette asymétrie explique pourquoi un modèle peut être entraîné efficacement sur des séquences longues alors que sa génération reste séquentielle et coûteuse. Le chapitre 12 explorera les architectures qui cherchent à briser cette contrainte.

La cross-entropie possède une interprétation informationnelle exacte. Si $p^\star$ désigne la véritable distribution des séquences, alors :

$$\mathcal{L}(\theta) = H(p^\star) + \mathbb{E}_{x\sim p^\star}\left[D_{KL}(p^\star(\cdot \mid x_{<t}) \,\|\, p_\theta(\cdot \mid x_{<t}))\right].$$

Le premier terme est une constante incompressible : l'entropie intrinsèque du données. Le second mesure l'excès de redondance du modèle. Minimiser la perte, c'est réduire cet écart — jamais faire disparaître le premier terme.

La *perplexité*, $\exp(\mathcal{L})$, est une reformulation commode : elle se lit comme un nombre effectif de choix auxquels le modèle est confronté à chaque position. En divisant la perte par $\ln 2$, on obtient directement un nombre de bits par token. Ce conversion sera notre outil principal au chapitre 3.

© 2026 Oussama Ghorbel

Que se passe-t-il quand le vocabulaire devient multimodal ? L'équation (1.10) ne change pas formellement. Mais sa pondération implicite, si. Dans un corpus mélangeant texte et images, chaque image contribue plusieurs centaines de positions visuelles contre quelques dizaines de positions textuelles. Une loss moyenne non pondérée devient donc un objectif majoritairement visuel, même si le corpus est majoritairement textuel en volume sémantique.

Trois stratégies coexistent : masquer certaines positions de la perte en les marquant comme ignorées ; pondérer les deux familles de tokens ; ou séparer les phases d'entraînement. Aucune n'est neutre. Le chapitre 16 les compare en termes de stabilité et de rétention des capacités linguistiques.

1.6 Quand les tokens ne sont plus des mots

Nous pouvons maintenant identifier précisément les hypothèses que le passage à l'image vient briser. Elles sont au nombre de quatre, et chacune irriguera plusieurs chapitres.

Première rupture : la géométrie du vocabulaire. Dans un vocabulaire BPE, les indices n'ont aucune structure intrinsèque ; toute la géométrie est apprise dans E. Dans un codebook visuel issu d'une quantification vectorielle, les indices correspondent à des vecteurs centraux qui possèdent déjà une métrique : deux codes voisins dans l'espace latent sont, par construction, des représentations semblables d'un même contenu visuel.

Cette structure est à la fois une opportunité et une contrainte. Une opportunité, car le modèle peut exploiter une similarité préexistante plutôt que l'apprendre ex nihilo. Une contrainte, car la répartition des fréquences est souvent très inégale : quelques codes dominent, d'autres presque jamais utilisés. Nous verrons au chapitre 7 comment ce déséquilibre conduit à l'effondrement du codebook.

Une subtilité mérite d'être signalée : réindexer bijectivement un codebook ne change pas la classe de distributions représentables. Cela change néanmoins la géométrie d'initialisation de la table d'embedding et de la tête de sortie, donc la dynamique d'optimisation. Deux implémentations mathématiquement équivalentes peuvent converger

© 2026 Oussama Ghorbel

vers des qualités très différentes.

Deuxième rupture : la nature de la dépendance. Le texte présente une localité forte et une structure hiérarchique : la fin d'une phrase dépend de son début, mais souvent par l'intermédiaire de quelques entités saillantes. Une image présente une localité bidimensionnelle et des contraintes globales : l'ombre portée en bas à droite doit correspondre à une source de lumière définie en haut à gauche.

L'aplatissement raster transforme cette géométrie en une chaîne où les relations longues deviennent littéralement longues. L'attention peut les apprendre, mais elle n'y est pas induite. D'où l'importance des chapitres 11 et 13.

Troisième rupture : le coût de l'erreur. Une erreur de token dans du texte est souvent locale et réparable : un mot mal choisi n'empêche pas la phrase suivante d'être correcte. Dans une image générée token par token, une erreur précoce est une décision structurelle qui conditionne tout la suite. Le modèle qui place un bord d'objet au mauvais endroit ne pourra pas le déplacer ensuite ; il devra accommoder le reste de la scène autour de cette erreur.

C'est le *exposure bias* — le décalage entre un entraînement sur préfixes réels et une inférence sur préfixes auto-produits — qui prend ici une dimension nouvelle. En génération d'images, il ne se manifeste pas seulement par une dégradation progressive de la qualité, mais par des incohérences globales : deux perspectives, trois sources de lumière, un objet dédoublé.

Quatrième rupture : ce que mesure la perte. La cross-entropie pénalise uniformément chaque erreur d'indice. L'œil, lui, ne pénalise pas uniformément chaque erreur de patch. Confondre deux codes représentant des textures de feuillage proches est perceptuellement bénin ; confondre un code de ciel avec un code de contour sombre est catastrophique.

Deux modèles peuvent donc afficher une perte identique et produire des images de qualité très différente. C'est sans doute la leçon la plus importante de ce chapitre : **la perte d'entraînement n'est pas une mesure de qualité visuelle**. Elle est un signal d'optimisation, pas un verdict. Le chapitre 18 sera consacré aux métriques qui tentent de

© 2026 Oussama Ghorbel

combler cet écart.

Le tableau suivant résume ces ruptures.

Dimension	Texte	Image tokenisée
Unité de base	Fragment orthographique	Patch latent quantifié
Structure du vocabulaire	Arbitraire, géométrie apprise	Métrique préexistante dans l'espace latent
Topologie des dépendances	Linéaire, hiérarchique	Bidimensionnelle, avec contraintes globales
Tolérance à l'erreur	Locale, souvent rattrapable	Structurelle, accumulative
Objectif de qualité	Fidélité sémantique locale	Cohérence perceptuelle globale
Métrique naturelle	Perplexité, exactitude	FID, CLIP-score, évaluation humaine

1.7 Premier calcul : une image 256 × 256 coûte combien ?

Avant d'écrire une ligne d'entraînement, faisons un compte. Ce type de calcul, fait tôt, évite des semaines d'égarement.

Prenons une image de 256×256 pixels en RVB 8 bits. Sa représentation brute occupe :

$$256 \times 256 \times 3 \times 8 = 1\,572\,864 \text{ bits.}$$

Appliquons maintenant un auto-encodeur quantifié avec un facteur de compression spatial de 16. La carte latente mesure 16×16, soit 256 positions. Avec un codebook de 1 024 entrées, chaque position tient sur $\log_2 1024 = 10$ bits. La séquence visuelle complète occupe donc 2 560 bits.

Le facteur de compression est d'environ 614.

Il faut immédiatement nuancer ce chiffre : il ne signifie pas que l'image contient 1,57 million de bits d'information. Il signifie que sa représentation brute utilise 1,57 million de bits, dont une immense partie est redondante. Toute la question de la tokenisation visuelle est de savoir quelle partie de cette redondance peut être supprimée sans perdre ce qui compte perceptuellement. Le chapitre 2 posera les outils pour y répondre.

© 2026 Oussama Ghorbel

Traduisons maintenant en coût de calcul. Ajoutons un prompt textuel de 64 tokens et deux marqueurs spéciaux : la séquence complète compte 322 positions. Avec la configuration évoquée plus haut, le KV cache représente environ 161 Mio. Les poids du modèle en demi-précision occupent près de 13 Go. La génération de l'image exige 256 passes avant séquentielles.

Si le facteur de compression passe de 16 à 8, la carte latente devient 32×32 : 1 024 tokens, soit quatre fois plus de passes, quatre fois plus de cache, et un coût d'attention qui croît plus vite encore à cause du terme quadratique. **La résolution ne coûte pas linéairement : elle coûte au moins quadratiquement dans le pire cas.**

Voici à quoi ressemble l'ossature d'un pas d'entraînement, une fois le vocabulaire visuel intégré :

Et l'ossature de la génération :

Deux lignes méritent une attention particulière. La première est `ignore_index` : c'est par ce mécanisme que l'on choisit quelles positions contribuent à la perte. La seconde est `visual_slice` : rien n'oblige la tête de sortie à rester libre sur l'ensemble du vocabulaire pendant la génération d'une image. En masquant les logits textuels entre les marqueurs visuels, on garantit que le modèle ne peut pas produire un token qui n'a pas de décodage image. Cette contrainte simple élimine une classe entière de bugs.

1.8 Diagnostic : quand la perte ment

Un LLM adapté à la génération d'images peut échouer de nombreuses manières, et certaines sont contre-intuitives. Voici les cinq signatures qu'il convient de distinguer en premier, parce qu'elles orientent le diagnostic vers des causes très différentes.

Signature observée	Cause probable à vérifier en premier
Perte élevée et plate dès l'initialisation	Décalage d'une position entre logits et cibles ; indices visuels mal alignés avec la table d'embedding
Perte très basse, images bruitées ou incohérentes	Bug de décodage : ordre de balayage inversé, normalisation du décodeur incorrecte, mauvaise reshape de la carte latente

© 2026 Oussama Ghorbel

Images structurées mais textures répétitives	Goulot d'étranglement du codebook : trop peu de codes, ou effondrement d'une partie des entrées
Motifs périodiques ou seams visibles	Inadéquation entre l'encodage positionnel 1D et la topologie 2D de la grille
Détérioration du texte après adaptation visuelle	Oubli catastrophique : taux d'apprentissage trop élevé sur les poids partagés, absence de régularisation ou de replay textuel

La deuxième ligne est la plus frustrante, car elle ressemble à un succès du point de vue de l'optimisation. Un modèle peut parfaitement apprendre à prédire une séquence de codes que le décodeur restitue ensuite dans le désordre. La perte ne le saura jamais. Seul un contrôle visuel régulier, sur un ensemble fixe de prompts, révèle ce type de rupture.

C'est pourquoi je recommande une discipline simple dès le premier jour d'expérimentation : fixer une douzaine de prompts, les conserver inchangés pendant toute la durée du projet, et visualiser leurs générations à intervalles réguliers. Pas les prompts d'évaluation finale. Une petite collection personnelle, représentative des cas que vous souhaitez maîtriser. La courbe de perte vous dit si l'optimisation progresse ; cette planche visuelle vous dit si le modèle comprend ce que vous lui demandez.

Le chapitre 21 généralisera cette démarche en méthodologie d'ablation : modifier un seul facteur à la fois, conserver les mêmes graines, et documenter ce qui ne change pas autant que ce qui change.

1.9 Ce qu'il faut emporter

Ce chapitre a volontairement traité le LLM comme ce qu'il est opérationnellement : une machine à prédire des entiers conditionnés par un préfixe. Quelques idées doivent maintenant être solidement ancrées.

L'interface discrète est une table d'adressage, pas une ontologie. Toute la sémantique — linguistique ou visuelle — est portée par des vecteurs appris. L'attention est un mécanisme de mélange conditionnel, rendu causal par un masque, et positionnel par un encodage externe ou par rotation. Le résidu est un espace de computation partagé dans lequel on peut apprendre à écrire une nouvelle modalité. La tête de sortie

© 2026 Oussama Ghorbel

transforme un vecteur en distribution, et le choix d'échantillonnage fait partie intégrante de la définition du modèle.

L'objectif de cross-entropie est un contrat mathématiquement propre mais perceptuellement aveugle. Il faut le compléter par des évaluations qui parlent le langage de l'image.

Enfin, le coût de génération est une fonction directe du nombre de tokens. Ce chiffre — 256, 1 024, 4 096 — est probablement la variable la plus déterminante de tout le projet. Il conditionne la mémoire, la latence, la longueur de contexte nécessaire, et la faisabilité même de l'entraînement.

Nous savons maintenant ce qu'est un token du point de vue du moteur. Il nous reste à comprendre ce qu'est un pixel du point de vue du signal : sa redondance, son entropie, sa structure fréquentielle, et la quantité d'information qu'il faut réellement conserver pour qu'une image paraisse juste. C'est l'objet du chapitre suivant.

© 2026 Oussama Ghorbel

Partie I — Fondations : le Token, le Pixel et la Représentation

© 2026 Oussama Ghorbel

Chapitre 1 – Anatomie d'un LLM : de l'embedding au next-token prediction

Avant de transformer un modèle de langage en générateur d'images, il faut comprendre ce qu'il est réellement : une machine à prédire le prochain élément d'une séquence discrète. Dans ce chapitre, je démonte pièce par pièce l'architecture d'un grand modèle de langage (LLM) de type décodeur, non pas pour en faire une revue exhaustive — la littérature en regorge déjà — mais pour identifier, à chaque étage, les points de contact où un signal visuel pourra s'insérer. C'est cette lecture « en creux » qui guidera l'ensemble de l'ouvrage.

1.1 — Le transformer décodeur : attention causale, couches feed-forward, normalisation

Le transformer décodeur, popularisé par la famille GPT, repose sur un empilement de blocs identiques. Chaque bloc comporte trois sous-mécanismes que l'on retrouve, à quelques variantes près, dans LLaMA, Mistral, Phi ou Qwen.

L'attention causale multi-têtes. Le mécanisme d'attention calcule, pour chaque position i de la séquence, une combinaison pondérée des représentations des positions antérieures. Le mot « causale » signifie que le masque appliqué aux scores d'attention est triangulaire supérieur : la position i ne peut jamais « voir » les positions $j > i$. Concrètement, on projette chaque vecteur caché $h_i \in \mathbb{R}^d$ en trois vecteurs — requête q_i, clé k_i, valeur v_i — via des matrices apprises W_Q, W_K, W_V. Le score d'attention entre i et j vaut :

$$a_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad \text{si } j \le i, \qquad -\infty \text{ sinon.}$$

Après un softmax sur la dimension j, on obtient la sortie de la tête. L'attention multi-têtes consiste à effectuer cette opération H fois en parallèle avec des sous-espaces de dimension $d_k = d/H$, puis à concaténer et projeter. Cette redondance permet au modèle de capturer simultanément des dépendances syntaxiques, sémantiques et positionnelles.

Les couches feed-forward (FFN). Chaque bloc contient ensuite un réseau de neurones à deux couches pleinement connectées, souvent avec une non-linéarité GeLU ou SwiGLU. La dimension interne est typiquement $4d$ (ou $\frac{8}{3}d$ pour SwiGLU). C'est dans ces couches que se concentre une large part des paramètres du modèle et, empiriquement, une grande partie de la « mémoire factuelle ».

Normalisation et connexions résiduelles. Deux stratégies dominent : la normalisation post-couche (Post-LN), historique, et la normalisation pré-couche (Pre-LN), adoptée par la quasi-totalité des LLM modernes depuis GPT-2. En Pre-LN, on applique une normalisation de couche (LayerNorm) ou une normalisation racine (RMSNorm, utilisée par LLaMA) *avant* chaque sous-couche, puis on ajoute le résultat au flux résiduel :

$$h_{\ell+1} = h_\ell + \mathrm{SubLayer}(\mathrm{Norm}(h_\ell)).$$

Cette formulation stabilise les gradients sur des dizaines de couches et rend l'entraînement de modèles à 7, 70 ou 405 milliards de paramètres envisageable.

Ce qu'il faut retenir pour la suite. Le décodeur ne « comprend » pas le texte ; il transforme une séquence de vecteurs en une autre séquence de vecteurs, position par position. La nature linguistique du signal n'intervient qu'en amont (tokenisation) et en aval (interprétation du logit). C'est précisément cette abstraction qui rendra possible, au chapitre 14, l'injection de tokens visuels dans le même pipeline.

1.2 — Tokenisation textuelle : BPE, SentencePiece, WordPiece et taille de vocabulaire

Un LLM ne lit pas des caractères ni des mots : il lit des *tokens*, unités discrètes issues d'un algorithme de segmentation. Trois familles dominent.

Byte-Pair Encoding (BPE). Introduit en NLP par Sennrich, Haddow et Birch (2016), BPE part d'un vocabulaire de caractères (ou de bytes) et fusionne itérativement la paire de symboles la plus fréquente dans le corpus d'entraînement. Après V itérations, on obtient un vocabulaire de taille $|V_{\text{base}}| + V$. GPT-2 utilise 50 257 tokens ; LLaMA 2, 32 000 ; Mistral, 32 000 également.

© 2026 Oussama Ghorbel

WordPiece. Similaire dans l'esprit, mais le critère de fusion est la vraisemblance du corpus plutôt que la fréquence brute. Historiquement associé à BERT et à la famille Encoder.

SentencePiece. Plus qu'un algorithme, c'est une bibliothèque qui traite le texte comme un flux de caractères bruts (sans pré-tokenisation par espaces), ce qui la rend agnostique à la langue. Elle implémente BPE ou Unigram et est utilisée par LLaMA, T5 ou Gemma.

Taille de vocabulaire : un compromis. Un vocabulaire trop petit allonge les séquences (donc le coût quadratique de l'attention) ; un vocabulaire trop grand gonfle la matrice d'embedding et la couche de projection finale (deux matrices de taille $|V| \times d$). Pour un modèle de dimension $d = 4096$ et un vocabulaire de 32 000 tokens, ces deux matrices représentent à elles seules environ 262 millions de paramètres — soit près de 4 % d'un modèle 7B.

L'observation clé pour notre projet : la tokenisation est une *interface*. Rien n'empêche, en principe, d'étendre le vocabulaire avec des tokens visuels, à condition de leur associer des embeddings cohérents. Nous y reviendrons longuement en Partie II.

1.3 — Embeddings positionnels : absolus, relatifs, RoPE, ALiBi

Le mécanisme d'attention est, par construction, invariant par permutation : sans information positionnelle, l'ordre des tokens serait perdu. Plusieurs stratégies ont été proposées.

Embeddings absolus appris. GPT-2 additionne au token embedding un vecteur positionnel appris $e_p \in \mathbb{R}^d$ pour chaque position $p \in \{0, \dots, L_{\max} - 1\}$. Simple, mais limite la longueur de contexte à $L_{\max}$ et généralise mal au-delà.

Encodages relatifs et ALiBi. ALiBi (Attention with Linear Biases, Press et al., 2022) supprime tout embedding positionnel et ajoute directement aux scores d'attention une pénalité proportionnelle à la distance $|i - j|$, avec une pente propre à chaque tête. L'avantage : une extrapolation de longueur raisonnable sans réentraînement.

© 2026 Oussama Ghorbel

Rotary Position Embedding (RoPE). Introduit par Su et al. (2021) et adopté par LLaMA, Mistral, Qwen et la plupart des modèles récents, RoPE encode la position en appliquant une rotation d'angle θ_p aux paires de coordonnées des vecteurs q et k. Le produit scalaire $q_i \cdot k_j$ ne dépend alors que de la position *relative* $i - j$, ce qui confère une invariance translationnelle élégante. La formule en dimension 2 (généralisable par blocs) :

$$\mathrm{RoPE}(x, p) = \begin{matrix} \cos(p\theta) & -\sin(p\theta) \\ \sin(p\theta) & \cos(p\theta) \end{matrix} x.$$

Pourquoi cela compte pour l'image. Une image n'a pas une seule dimension séquentielle : elle en a deux (hauteur, largeur). Les schémas positionnels 1D devront être adaptés — soit en aplatissant la grille selon un ordre de balayage (Chapitre 11), soit en concevant des embeddings 2D ou 3D. RoPE, par exemple, se généralise naturellement en appliquant des rotations distinctes sur des sous-espaces dédiés à chaque axe spatial.

1.4 — La fonction objectif next-token : entropie croisée et température d'échantillonnage

L'entraînement d'un LLM décodeur est d'une simplicité conceptuelle remarquable : à chaque position t, le modèle produit un vecteur de logits $z_t \in \mathbb{R}^{|V|}$; on le compare au token réellement observé x_{t+1} via l'entropie croisée :

$$\mathcal{L} = -\sum_{t=1}^{T-1} \log \frac{\exp(z_t[x_{t+1}])}{\sum_{v=1}^{|V|} \exp(z_t[v])}.$$

Cette perte est minimisée par rétropropagation sur des corpus de plusieurs billions de tokens. Il n'y a pas de supervision explicite par tâche : la « tâche » *est* la prédiction du token suivant.

Température et échantillonnage. À l'inférence, on convertit les logits en distribution via un softmax tempéré :

$$p_v = \frac{\exp(z_v / \tau)}{\sum_{v'} \exp(z_{v'} / \tau)}.$$

Une température $\tau < 1$ sharpen la distribution (sorties plus déterministes) ; $\tau > 1$ l'aplatit (sorties plus diverses). On combine souvent ce mécanisme avec le *top-k* ou le *top-p* (nucleus sampling) pour tronquer

© 2026 Oussama Ghorbel

la queue de distribution.

Lien avec la génération d'images. Lorsque le vocabulaire contiendra des tokens visuels, la même entropie croisée s'appliquera. La température contrôlera alors le compromis entre fidélité (reproduction exacte du codebook appris) et créativité (exploration de combinaisons inédites). Ce point sera crucial au Chapitre 22 lors du protocole pratique.

1.5 — Pourquoi un LLM est « modalement agnostique » : la séquence comme abstraction universelle

J'aime formuler les choses ainsi : un LLM ne sait pas qu'il traite du langage. Il reçoit une séquence d'indices entiers $x_1, x_2, \dots, x_T$, les projette dans $\mathbb{R}^d$, applique L blocs transformer, puis produit une distribution sur le prochain indice. Nulle part dans les équations n'apparaît la notion de « mot », de « phonème » ou de « sens ».

Cette indifférence modale est une propriété *structurelle* de l'architecture :

1. **L'embedding est une table de lookup.** Tant qu'on fournit un indice valide, le vecteur correspondant existe. Qu'il encode « chat » ou le patch n° 437 d'un codebook VQGAN est transparent pour les couches suivantes.

2. **L'attention opère sur des vecteurs continus.** Elle ne distingue pas un embedding textuel d'un embedding visuel ; elle calcule des similarités et des combinaisons linéaires.

3. **La perte next-token est universelle.** Prédire le prochain indice d'une séquence de codes visuels est formellement identique à prédire le prochain mot.

Cette observation, que j'ai voulue au cœur de cet ouvrage, est ce qui légitime le titre : passer *du token au pixel*, c'est exploiter l'agnosticisme modal du transformer pour y faire transiter un signal d'une tout autre nature. Les travaux de Yu et al. (DALL-E, 2021), de Sun et al. (Transfusion, 2024) ou de la série Chameleon (Meta, 2024) confirment empiriquement cette thèse.

© 2026 Oussama Ghorbel

1.6 — Cartographie des points d'injection : où brancher un signal visuel

Terminons ce chapitre par une vue synoptique des endroits où un ingénieur peut intervenir pour greffer la modalité image sur un LLM existant. Je distingue quatre points d'injection :

Extension du vocabulaire et de la matrice d'embedding. On ajoute K tokens visuels (par exemple 8 192 codes issus d'un VQGAN) au vocabulaire. La matrice d'embedding passe de $|V| \times d$ à $(|V| + K) \times d$. Les nouveaux vecteurs peuvent être initialisés aléatoirement puis affinés, ou initialisés à partir des centroïdes du codebook.

Encodeur visuel en amont (approche « adapter »). On conserve un encodeur d'image séparé (ViT, CNN, VAE) qui produit des vecteurs continus, projetés dans l'espace du LLM par un petit réseau (MLP, Q-Former, Perceiver). C'est la stratégie de LLaVA, Flamingo ou BLIP-2. L'avantage : on ne modifie pas le vocabulaire. L'inconvénient : la génération d'image reste externe au LLM.

Tokenisation discrète native. L'image est quantifiée en tokens discrets *avant* d'entrer dans le LLM, exactement comme le texte. C'est la voie de DALL-E, Parti, Transfusion et celle que nous privilégierons dans cet ouvrage, car elle préserve l'homogénéité du pipeline next-token.

Tête de décodage visuelle en aval. En sortie du LLM, une tête spécialisée (décodeur VAE, décodeur VQGAN) reconstruit l'image à partir des tokens visuels prédits. Cette tête n'est pas différentiable pendant l'entraînement du LLM si l'on utilise des codes discrets ; elle intervient uniquement à l'inférence.

Chacun de ces points sera développé dans les parties suivantes. La cartographie ci-dessus constitue le fil conducteur de l'ouvrage : chaque chapitre peut être lu comme un zoom sur l'un de ces quatre nœuds.

Exercices

Exercice 1 — Tracer le flux d'un token dans un GPT minimal.

© 2026 Oussama Ghorbel

Considérez un décodeur à 2 couches, dimension $d = 64$, $H = 4$ têtes d'attention (donc $d_k = 16$), FFN de dimension interne 256, vocabulaire de 100 tokens. Prenez la séquence d'entrée $[7, 23, 51]$ (trois indices de tokens).

• Écrivez la forme de chaque tenseur à chaque étape : embedding, ajout positionnel, Q/K/V par tête, scores d'attention (avec masque causal), sortie d'attention concaténée, projection de sortie, résidu + LayerNorm, FFN, résidu + LayerNorm, logits.

• Indiquez explicitement où le masque causal intervient et combien d'entrées sont masquées pour la position 1 (indice 0).

• Calculez le nombre total de paramètres de ce modèle miniature.

Exercice 2 — Taille mémoire d'un embedding.

Calculez la taille en mémoire (en Mio) d'une matrice d'embedding de vocabulaire 50 000 × dimension 1 024, stockée en FP16 (2 octets par paramètre). Même question si l'on ajoute 8 192 tokens visuels au vocabulaire. Quel pourcentage de la mémoire totale du modèle 7B (env. 14 Go en FP16) ces matrices représentent-elles ?

Exercice 3 — Étendre un tokenizer BPE avec des tokens visuels.

À l'aide de la bibliothèque `tokenizers` (Hugging Face) :

1. Chargez un tokenizer BPE pré-entraîné (par exemple `gpt2` via `transformers`).

2. Ajoutez 8 192 tokens spéciaux `<vis_0>` à `<vis_8191>` au vocabulaire.

3. Sauvegardez le tokenizer modifié et mesurez la taille du fichier JSON.

4. Instanciez un modèle `GPT2LMHeadModel`, redimensionnez les embeddings (`model.resize_token_embeddings(len(tokenizer))`) et comparez le nombre de paramètres avant/après.

5. Calculez le surcoût mémoire en FP16 induit par cette extension.

Cet exercice préfigure directement la manipulation que nous réaliserons au Chapitre 22 sur un modèle 7B.

© 2026 Oussama Ghorbel

Dans le chapitre suivant, nous quitterons le monde discret du token pour plonger dans la nature continue, redondante et hautement structurée du signal pixel. C'est là que commencera véritablement le pont entre langage et vision.

Indications pour les exercices

Exercice 1 — Éléments de résolution. Le nombre total de paramètres se décompose ainsi. Pour chaque couche : la projection Q/K/V demande $3 \times d \times d = 3 \times 64 \times 64 = 12\,288$ paramètres (sans biais pour simplifier) ; la projection de sortie de l'attention ajoute $d \times d = 4\,096$; le FFN contribue $d \times 256 + 256 \times d = 32\,768$; les deux LayerNorm apportent $2 \times 2d = 256$ paramètres (gain et biais). Soit environ $49\,408$ paramètres par couche, $98\,816$ pour deux couches. La matrice d'embedding (100 × 64) et la tête de projection (64 × 100, souvent liée par *weight tying*) ajoutent $6\,400$ paramètres. Le total avoisine $105\,000$ paramètres — un modèle jouet, mais suffisant pour visualiser chaque tenseur.

Pour le masque causal : à la position 0, aucune entrée n'est masquée (le token ne voit que lui-même). À la position 1, une entrée est masquée (la position 2). À la position 2, deux entrées sont masquées. Le masque est donc strictement triangulaire supérieur, avec $\frac{T(T-1)}{2} = 3$ entrées masquées au total pour $T = 3$.

Exercice 2 — Calcul. La matrice d'embedding occupe $50\,000 \times 1\,024 \times 2$ octets $= 102\,400\,000$ octets, soit environ 97,7 Mio. Avec 8 192 tokens visuels supplémentaires, on passe à $58\,192 \times 1\,024 \times 2 \approx 119{,}2$ Mio. Le surcoût est de $8\,192 \times 1\,024 \times 2 \approx 16{,}8$ Mio. Rapporté à un modèle 7B stocké en FP16 (≈ 14 Gio), l'embedding initial représente $\frac{97{,}7}{14\,336} \approx 0{,}68\,\%$ et l'extension visuelle ajoute $\approx 0{,}12\,\%$. Ces chiffres, modestes en apparence, deviennent significatifs lorsqu'on considère que la tête de projection finale (souvent liée à l'embedding par *weight tying*) double cet impact si elle n'est pas partagée.

Exercice 3 — Pistes d'implémentation. Après avoir ajouté les 8 192 tokens spéciaux via `tokenizer.add_tokens([f"<vis_{i}>" for i in range(8192)])`, la méthode `resize_token_embeddings` alloue de nouvelles lignes dans la matrice d'embedding, initialisées par défaut selon la même loi normale que les lignes existantes. Le fichier JSON du

© 2026 Oussama Ghorbel

tokenizer croît linéairement avec le nombre de tokens ajoutés (chaque entrée occupe quelques dizaines d'octets). Pour GPT-2, le passage de 50 257 à 58 449 tokens augmente le nombre de paramètres de $8\,192 \times 768 = 6\,291\,456$ (embedding) auxquels s'ajoutent autant pour la tête LM si elle n'est pas liée — soit environ 12,6 millions de paramètres supplémentaires, c'est-à-dire ≈ 10 Mio en FP16.

Points clés à retenir

- Un transformer décodeur est une fonction $f_\theta : \{1, \ldots, |V|\}^T \to \Delta^{|V|-1}$ qui, à chaque position, produit une distribution sur le prochain indice. Aucune hypothèse linguistique n'est codée dans les poids.
- La tokenisation (BPE, SentencePiece, WordPiece) est une *interface discrétisation* réversible en principe, extensible en pratique.
- L'information positionnelle (absolue, ALiBi, RoPE) est le seul mécanisme qui distingue l'ordre des tokens ; elle devra être repensée en deux dimensions pour l'image.
- La perte next-token (entropie croisée) est universelle : elle s'applique à toute séquence discrète, qu'elle encode des mots ou des codes visuels.
- Quatre points d'injection permettent de greffer la vision sur un LLM : extension du vocabulaire, encodeur adapter en amont, tokenisation discrète native, tête de décodage en aval.

Notes bibliographiques

Pour approfondir l'architecture transformer décodeur, le lecteur pourra se reporter à :

- Vaswani, A. *et al.* « Attention Is All You Need. » *NeurIPS*, 2017. — L'article fondateur.
- Radford, A. *et al.* « Language Models are Unsupervised Multitask Learners. » (GPT-2), 2019. — Introduction du Pre-LN à grande échelle.

© 2026 Oussama Ghorbel

• Touvron, H. *et al.* « LLaMA: Open and Efficient Foundation Language Models. » *arXiv:2302.13971*, 2023. — RMSNorm, RoPE, SwiGLU combinés.

• Su, J. *et al.* « RoFormer: Enhanced Transformer with Rotary Position Embedding. » *arXiv:2104.09864*, 2021.

• Press, O., Smith, N., Lewis, M. « Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. » *ICLR*, 2022. — ALiBi.

• Sennrich, R., Haddow, B., Birch, A. « Neural Machine Translation of Rare Words with Subword Units. » *ACL*, 2016. — BPE en NLP.

• Yu, A. *et al.* « Zero-Shot Text-to-Image Generation. » (DALL-E), *ICML*, 2021. — Première démonstration à grande échelle de l'autorégression sur tokens visuels.

Transition

Nous avons disséqué la machine à prédire le prochain token. Mais le pixel, lui, n'est pas un token. Il est un nombre réel, corrélé à ses voisins, redondant, porteur d'une entropie bien inférieure à celle du langage naturel. Avant de pouvoir le « brancher » sur le décodeur, il faut comprendre sa nature propre : c'est l'objet du chapitre suivant, où nous traiterons l'image comme un signal — résolution, canaux, compression et structure statistique. Le pont entre le token et le pixel commence par la compréhension de chacune de ses deux rives.

Lexique du chapitre

Avant de tourner la page, je propose un glossaire minimal des termes techniques mobilisés dans ce chapitre. Il servira de référence rapide pour la suite de l'ouvrage.

Terme	Définition opérationnelle
Token	Unité discrète élémentaire produite par un algorithme de segmentation (BPE, WordPiece, SentencePiece). C'est l'atome d'information que manipule le transformer.

© 2026 Oussama Ghorbel

Embedding	Vecteur dense $\in \mathbb{R}^d$ associé à chaque token par une table de lookup apprenable. Première transformation du signal discret en représentation continue.
Attention causale	Mécanisme d'attention où la position i ne peut consulter que les positions $j \leq i$, garantissant l'autorégressivité. Implémenté par un masque triangulaire supérieur sur les scores.
Tête d'attention	Sous-espace de projection indépendant dans lequel s'effectue un calcul d'attention. Un modèle à H têtes réalise H calculs parallèles de dimension $d_k = d/H$.
FFN (Feed-Forward Network)	Réseau à deux couches linéaires séparées par une non-linéarité (GeLU, SwiGLU), appliqué position par position. Dimension interne typique : $4d$ ou $\frac{8}{3}d$.
Pre-LN / Post-LN	Positionnement de la normalisation de couche par rapport à la sous-couche : avant (Pre-LN, standard actuel) ou après (Post-LN, historique).
RMSNorm	Variante de LayerNorm qui omet la soustraction de la moyenne et ne conserve que la mise à l'échelle par la racine carrée de la moyenne des carrés. Adoptée par LLaMA, Mistral.
RoPE	Rotary Position Embedding. Encodage positionnel par rotation des vecteurs q et k dans des sous-espaces 2D, rendant le produit scalaire dépendant uniquement de la position relative.
ALiBi	Attention with Linear Biases. Ajout d'une pénalité linéaire en distance aux scores d'attention, sans embedding positionnel explicite.
Entropie croisée	Fonction de perte $\mathcal{L} = -\log p(x_{t+1} \mid x_{\leq t})$ mesurant l'écart entre la distribution prédite et le token observé. Objectif unique de l'entraînement next-token.
Température τ	Paramètre scalaire divisant les logits avant softmax. $\tau \to 0$: argmax déterministe ; $\tau \to \infty$: distribution uniforme.

© 2026 Oussama Ghorbel

Top-k / Top-p	Troncature de la distribution d'échantillonnage aux k tokens les plus probables (top-k) ou au plus petit ensemble dont la masse cumulée dépasse p (nucleus, top-p).
Weight tying	Partage de la matrice d'embedding et de la matrice de projection finale (tête LM), réduisant le nombre de paramètres de $\lvert V \rvert \times d$.
Codebook	Ensemble fini de vecteurs prototypes (centroïdes) utilisé par un quantificateur vectoriel (VQ, VQGAN) pour représenter une image comme une séquence de tokens discrets.
Logit	Score brut produit par la couche de projection finale avant application du softmax. Un logit par token du vocabulaire, à chaque position.

Encadré — Ordres de grandeur utiles pour la suite

Je regroupe ici quelques chiffres que le lecteur sera amené à manipuler régulièrement dans les chapitres suivants. Ils ancrent les ordres de grandeur et évitent les surprises lors du dimensionnement d'un protocole pratique.

Coût d'une passe avant (forward) sur une séquence de longueur T. Pour un décodeur à L couches, dimension d, H têtes, FFN de dimension d_{ff}, le nombre d'opérations en virgule flottante (FLOPs) est approximativement :

$$\text{FLOPs} \approx 2\,T\,L(4d^2 + 2d \cdot d_{\text{ff}}) + 2\,T^2\,L\,d.$$

Le premier terme correspond aux projections et au FFN (linéaire en T) ; le second à l'attention elle-même (quadratique en T). Pour un modèle 7B ($L = 32$, $d = 4\,096$, $d_{\text{ff}} = 11\,008$) et une séquence de $T = 4\,096$ tokens, on obtient environ 2×10^{13} FLOPs par passe avant — soit une vingtaine de millisecondes sur un GPU A100 en FP16.

Mémoire des activations. Chaque couche stocke $T \times d$ activations pour la connexion résiduelle, plus les tenseurs intermédiaires de l'attention ($T \times H \times d_k$ pour Q, K, V) et du FFN ($T \times d_{\text{ff}}$). Pour $T = 4\,096$ et $d = 4\,096$, une seule couche occupe environ 200 Mio en FP16. Sur 32 couches, on approche 6,4 Gio d'activations — ce qui explique pourquoi

© 2026 Oussama Ghorbel

la génération d'images, qui multiplie T par le nombre de tokens visuels (256 à 1 024 pour une image 256 × 256 tokenisée), impose des stratégies d'optimisation mémoire spécifiques (Chapitre 23).

Taille d'un codebook typique. Un VQGAN entraîné sur des images 256 × 256 avec un facteur de compression spatial 16 produit une grille de $16 \times 16 = 256$ tokens. Un codebook de 8 192 entrées en dimension 256 occupe $8\,192 \times 256 \times 4 \approx 8$ Mio en FP32. Ces 256 tokens visuels s'ajoutent aux tokens textuels dans la séquence, allongeant d'autant le contexte. Pour une image 512 × 512 avec le même facteur de compression, on passe à $32 \times 32 = 1\,024$ tokens visuels. Le coût quadratique de l'attention devient alors un facteur limitant que nous devrons contourner (Chapitres 12 et 23).

Une dernière remarque avant de passer au pixel

J'ai insisté tout au long de ce chapitre sur l'agnosticisme modal du transformer décodeur. Il serait tentant d'en conclure que la greffe d'un signal visuel est triviale — qu'il suffit d'étendre un vocabulaire et de laisser l'entraînement faire le reste. Ce serait une erreur. L'indifférence *structurelle* de l'architecture ne garantit pas l'indifférence *statistique* : la distribution des tokens visuels, leur corrélation spatiale, leur redondance et leur entropie diffèrent radicalement de celles du langage naturel. Le modèle peut *accepter* ces tokens ; il ne les *modélisera* correctement que si l'entraînement, la tokenisation et l'ordre de génération sont adaptés.

C'est précisément l'objet de la Partie II : construire le pont discret — le tokenizer visuel — qui traduira le continu du pixel en une séquence de codes que le décodeur pourra traiter avec la même fonction objectif. Mais avant cela, il faut regarder le pixel en face, dans sa nudité numérique : un tableau de nombres réels, corrélés, redondants, porteurs d'une structure que le langage naturel ne possède pas.

Le chapitre 2 commence par cette immersion dans le signal.

Fin du chapitre 1.

© 2026 Oussama Ghorbel

Chapitre 2 – Le pixel comme signal : résolution, canaux, entropie et redondance

Un grand modèle de langage ne sait pas prédire des images. Il sait prédire un symbole parmi un vocabulaire fini, conditionnellement à une suite de symboles déjà és. Cette phrase, volontairement réductrice, contient tout le problème du livre : pour transformer un prédicteur de tokens en générateur d'images, il faut décider *ce qu'est un token visuel*.

La réponse naïve consiste à prendre le pixel. Un pixel est un nombre, ou trois ; un nombre se quantifie sur huit bits ; huit bits donnent 256 symboles ; 256 symboles appartiennent parfaitement à un vocabulaire de langage. L'argument semble irrésistible. Il est aussi catastrophique, pour des raisons que nous allons chiffrer.

Mais il serait trop facile de se contenter de dire que « c'est trop long ». Il faut comprendre *pourquoi* une image est compressible, quel genre de redondance exploiter, et ce que l'on accepte de perdre en la supprimant. C'est l'objet de ce chapitre : traiter l'image non comme un tableau de couleurs, mais comme un signal statistique, avec une entropie, des corrélations et un coût de description.

Je propose de suivre un chemin précis. Nous définirons d'abord l'objet mathématique « image ». Nous mesurerons son incertitude réelle. Nous compterons ce que coûterait sa modélisation brute. Puis nous examinerons deux familles de réponses — les transformées classiques et la patchification — avant de comprendre pourquoi un millier de symboles peut, dans certaines conditions, porter une image entière.

2.1 — L'image comme tenseur : H × W × C, espaces colorimétriques

Une image numérique est un tenseur. Notons-le $x \in \mathbb{R}^{H \times W \times C}$, où H est la hauteur, W la largeur et C le nombre de canaux. Pour une photographie couleur standard, $C = 3$. Pour une image médicale en niveaux de gris, $C = 1$. Pour un cliché multispectral, C peut atteindre plusieurs dizaines ; pour un buffer RGBA, $C = 4$.

© 2026 Oussama Ghorbel

Deux conventions cohabitent dans les implémentations : `(H, W, C)` dans la plupart des bibliothèques d'affichage, `(C, H, W)` dans les tenseurs PyTorch. Cette confusion apparemment anodine produit des bugs tenaces lorsqu'on aplatit une image en séquence. Je recommande de l'écrire explicitement dans chaque fonction de tokenisation, et de tester sur une image non carrée : un bogue de transposition est invisible sur un carré.

Dans un pipeline de génération, l'image est en pratique **quantifiée**. On travaille non sur $\mathbb{R}^{H \times W \times C}$ mais sur $\{0, 1, \dots, 2^b - 1\}^{H \times W \times C}$, avec $b = 8$ le plus souvent. Cette quantification n'est pas un détail d'implémentation : elle définit le support sur lequel toute notre analyse entropique aura un sens.

RGB : un repère, pas une vérité. Le modèle RGB décrit une couleur par l'intensité de trois primaries. C'est un système additif, adapté aux écrans. Mais ses axes ne sont pas orthogonaux au sens perceptif : une variation de 5 unités sur le rouge et une variation de 5 unités sur le bleu ne produisent pas la même sensation visuelle. De plus, les trois canaux d'une photographie sont fortement corrélés. Un pixel gris satisfait $R = G = B$; une peau, un ciel, un feuillage occupent des régions étroites du cube RGB. Modéliser les trois canaux comme indépendants revient à gaspiller de la capacité à décrire des corrélations que l'on connaissait d'avance.

YCbCr : séparer ce qui varie vite de ce qui varie lentement. La transformation usuelle (ITU-R BT.709) construit une luminance et deux differencechrominances :

$$Y = 0{,}2126\,R + 0{,}7152\,G + 0{,}0722\,B$$

$$C_b = B - Y, \qquad C_r = R - Y$$

avec des mises à l'échelle adaptées au stockage numérique. L'intérêt est double. D'une part, la vision humaine résout mieux les détails de luminance que ceux de chrominance. D'autre part, les canaux de chrominance sont statistiquement plus lisses. Le sous-échantillonnage 4:2:0 stocke une luminance par pixel mais une chrominance seulement par bloc de 2×2 : le nombre d'échantillons passe de trois à 1,5 par pixel, soit une division par deux du volume, avec une dégradation souvent peu visible.

© 2026 Oussama Ghorbel

C'est la première leçon quantitative du chapitre : **la redondance n'est pas uniformément répartie entre les canaux**. Un tokenizer intelligent ne devrait pas traiter C comme une dimension homogène.

CIELAB : un repère approximativement perceptif. Lab sépare une lightness L^* (de 0 à 100) de deux axes chromatiques a^* (vert–rouge) et b^* (bleu–jaune). Sa construction vise une uniformité perceptuelle : une distance euclidienne y corrèle mieux avec une différence ressentie qu'une distance RGB. La conversion dépend d'un point blanc de référence, typiquement D65.

Lab est précieux pour les pertes perceptuelles et certaines métriques. Il n'est pas pour autant un meilleur espace de tokenisation par défaut. Un espace appris, comme nous le verrons aux chapitres 5 et 6, peut dépasser les trois repères classiques parce qu'il n'est pas contraint par une histoire de l'imprimerie ou de la télévision.

Enfin, un point souvent oublié : le RGB stocké en 8 bits est déjà un codage perceptuel. La transfert function sRGB applique une courbe proche d'une gamma avant quantification, afin de répartir les pas de quantification selon la sensibilité visuelle. Les « 24 bits par pixel » ne sont donc pas 24 bits d'information perceptuelle. Cette nuance compte lorsque nous comparerons des débits.

2.2 — Entropie d'une image naturelle : estimation, corrélations spatiales, redondance

Soit une variable aléatoire discrète X prenant K valeurs avec probabilités p_k. Son entropie de Shannon est

$$H(X) = -\sum_{k=1}^{K} p_k \log_2 p_k.$$

Pour un pixel de niveau de gris quantifié sur 8 bits, $K = 256$ et $H(X) \leq 8$ bits. L'égalité n'est atteinte que pour une distribution uniforme : c'est le cas d'un bruit blanc quantifié.

Beaucoup de photographies en niveaux de gris présentent une entropie marginale de l'ordre de 6 à 7 bits par pixel. Un ciel uniforme tombe près de zéro. Un dégradé régulier oscille autour de 1 à 2 bits. Dès ici, une première conclusion émerge : **le codage brut sur 8 bits gaspille**

© 2026 Oussama Ghorbel

typiquement un à deux bits par pixel, rien qu'au niveau de la distribution marginale.

Mais cette entropie marginale est la moins intéressante des trois quantités qu'il faut distinguer.

Première quantité : l'entropie marginale d'une image. C'est un descriptif statistique local : « dans cette image, comment se répartissent les niveaux ? ». Elle ne dit rien sur la difficulté de prédire un pixel à partir de ses voisins. Une image de bruit et une image de texte peuvent avoir des histogrammes similaires et des compréssibilités radicalement différentes.

Deuxième quantité : l'entropie de la source. C'est l'entropie de la distribution d'où l'on tire les images, estimée sur un ensemble. C'est elle qui fixe, asymptotiquement, la limite de compression sans perte. L'estimer sur une seule image est un abus de langage fréquent.

Troisième quantité : l'entropie conditionnelle. C'est celle qui nous intéresse au premier chef :

$$H(x_{i,j} \mid x_{i',j'} \text{ déjà observés}).$$

C'est exactement ce qu'un modèle autorégressif cherche à modéliser. Et c'est là que la redondance spatiale devient spectaculaire.

Corrélations et information mutuelle. Pour deux variables, $H(X, Y) \leq H(X) + H(Y)$, avec égalité si et seulement si elles sont indépendantes. L'information mutuelle mesure l'écart :

$$I(X; Y) = H(X) + H(Y) - H(X, Y).$$

Sur des luminances de photographies naturelles, les pixels voisins partagent typiquement plusieurs bits d'information mutuelle. Un pixel à l'intérieur d'un mur clair permet de prédire son voisin avec une incertitude bien inférieure à 8 bits. Formellement, en notant $x_{<t}$ l'ensemble des pixels déjà balayés :

$$H(x_t) \;\geq\; H(x_t \mid x_{t-1}) \;\geq\; H(x_t \mid x_{<t}).$$

Chaque contexte supplémentaire réduit l'incertitude, ou la laisse inchangée. Dans les régions homogènes, l'entropie conditionnelleGiven un voisinage suffisant peut descendre très bas. Dans les contours et les textures fines, elle remonte.

© 2026 Oussama Ghorbel

On définit ainsi la **redondance** relative à un codage maximal :

$$R = 1 - \frac{H_{\text{conditionnelle}}}{H_{\max}}.$$

Une image dont les pixels pourraient être prédits avec 2 bits d'incertitude moyenne alors qu'ils sont stockés sur 8 présente une redondance de 75 %. C'est exactement l'espace de manœuvre que cherche à exploiter un tokenizer.

Comment estimer tout cela en pratique ? On construit un histogramme. Avec N pixels et n_k occurrences du niveau k, l'estimateur plug-in est $\hat{p}_k = n_k/N$ et $\hat{H} = -\sum_k \hat{p}_k \log_2 \hat{p}_k$.

Deux précautions s'imposent. La première est le biais d'estimation : sur un échantillon fini, l'entropie plug-in sous-estime systématiquement l'entropie vraie. À premier ordre, le biais est de $-(K-1)/(2N\ln 2)$ bits lorsque les K cellules sont occupées. La correction de Miller–Madow ajoute $(\hat{K}-1)/(2N)$ nats, où $\hat{K}$ est le nombre de cellules non vides. Sur une image 64×64, ce biais reste de l'ordre du centième de bit : il n'explique pas à lui seul les grandes écarts.

La seconde précaution est plus profonde : **un histogramme ne mesure pas la structure spatiale**. Deux images peuvent avoir exactement le même histogramme et des entropies conditionnelles très différentes. Pour mesurer la redondance, il faut des statistiques d'ordre supérieur : histogrammes joint de paires de pixels, de triplets, ou entropie des résidus après prédiction linéaire.

C'est ici qu'apparaît une propriété essentielle des images naturelles : leurs résidus de prédiction sont concentrés autour de zéro, avec des queues lourdes, souvent bien approximées par une loi laplacienne. Le signal n'est pas dans la valeur absolue du pixel ; il est dans l'écart au predictable.

Un dernier avertissement théorique. Pour une image réellement continue, il faudrait parler d'entropie différentielle, qui peut être négative et ne se compare pas directement à une entropie discrète. Tous les chiffres en bits de ce chapitre supposent une quantification fixée à l'avance. Sans cette convention, les comparaisons de débit n'ont pas de sens.

© 2026 Oussama Ghorbel

2.3 — Coût brut de la modélisation pixel-par-pixel : 256^3 niveaux, séquence de H·W·C tokens

Comptons maintenant ce que coûte la naïveté.

Prenons une image 256×256 en RGB 8 bits. Le nombre de scalaires est

$$H \times W \times C = 256 \times 256 \times 3 = 196\,608.$$

Le volume brut est de 196 608 octets, soit 1 572 864 bits. Deux encodages autorégressifs distincts se présentent alors.

Encodage A : un token par scalaire. Le vocabulaire contient 256 symboles. La séquence compte $N = 196\,608$ tokens. L'ordre peut être canal-major — tous les rouges, puis tous les verts, puis tous les bleus — ou interleaved, rgb, rgb, rgb… Le premier est séduisant par sa simplicité mais terrible pour l'apprentissage : les 65 536 premiers tokens sont prédits sans aucun accès à la chrominance correspondante. Le second préserve les corrélations locales au prix d'un ordre plus artificiel. Nous reviendrons sur ce choix au chapitre 11.

Le coût d'attention est quadratique. Même en ne comptant que les paires causales, on arrive à

$$\frac{N^2}{2} \approx \frac{(1{,}966 \times 10^5)^2}{2} \approx 1{,}9 \times 10^{10}$$

paires de positions à mettre en relation **pour une seule image**. Un transformer standard n'est pas conçu pour cela.

Encodage B : un token par pixel couleur. On regroupe (R, G, B) en un seul symbole. Le vocabulaire contient alors

$$256^3 = 16\,777\,216$$

valeurs possibles, et la séquence ne compte plus que $N = 65\,536$ tokens. La longueur diminue par trois ; le problème change de nature.

La couche de sortie doit produire une distribution sur 16,7 millions de classes. Avec une dimension cachée $d = 4096$, la matrice de sortie pèse $4096 \times 16\,777\,216 \approx 6{,}9 \times 10^{10}$ paramètres. En demi-précision, cela représente environ 137 Go — rien que pour la tête de génération, avant d'avoir écrit une seule couche du transformer. Et il faut normaliser une telle distribution à chaque pas.

© 2026 Oussama Ghorbel

On peut tenter un compromis : trois têtes de 256 classes, appliquées séquentiellement à chaque pixel. Cela revient en réalité à l'encodage A, avec un ordre particulier. Ou bien on suppose R, G, B conditionnellement indépendants étant donné le contexte ; c'est une approximation fausse, et elle coûte précisément les bits d'information mutuelle inter-canaux que nous avons identifiés en 2.1.

Le coût temps est tout aussi brutal. Un décodage autorégressif strict exige un passage par le modèle par token. Même en supposant, de façon très optimiste, une milliseconde par pas, 196 608 pas représentent près de trois minutes et demie par image — et cette hypothese ignore totalement la croissance du contexte.

Le cache KV fournit une seconde illustration. Prenons une configuration de 32 couches, 8 têtes de clé/valeur et une dimension de tête de 128. Chaque token ajoute $2 \times 32 \times 8 \times 128 \times 2$ octets, soit 128 Kio. Pour 196 608 tokens, le cache atteint environ 26 Go. Pour 1 024 tokens, il atteint 128 Mio. Le rapport est de 192 ; le rapport de coût d'attention, de 36 864.

La conclusion n'est pas que l'autorégression est incompatible avec l'image. Elle est que **l'autorégression ne peut pas s'exercer sur la représentation native du pixel**. Il faut changer d'espace avant de changer de modèle.

2.4 — Transformées classiques (DCT, ondelettes) comme pré-compression

Avant les tokenizers appris, l'ingénierie de la compression avait déjà répondu à la question de la redondance. Deux familles dominent : les transformées en cosinus et les ondelettes.

La DCT. La transformée en cosinus discrète de type II s'écrit, en une dimension :

$$X(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos\left[\frac{\pi(2x+1)u}{2N}\right],$$

avec $\alpha(0) = \sqrt{1/N}$ et $\alpha(u) = \sqrt{2/N}$ pour $u > 0$. En deux dimensions, on l'applique séparément selon les lignes puis selon les colonnes. Contrairement à la transformée de Fourier, la DCT ne suppose pas de

© 2026 Oussama Ghorbel

périodicité du bloc ; elle implique une symétrie aux bords, ce qui limite les artefacts de discontinuité sur des tuiles d'image.

Son intérêt s'appelle **compaction d'énergie**. Dans un bloc naturel relativement lisse, la quasi-totalité de l'énergie se concentre dans les quelques premiers coefficients de basse fréquence. Les coefficients de haute fréquence sont petits, souvent proches de zéro. On peut donc les quantifier grossièrement, voire les mettre à zéro, avec une perte perceptuelle modérée.

C'est exactement le principe de JPEG : découpage en blocs 8×8, DCT, quantification par une matrice adaptée, parcours en zigzag, codage entropique des coefficients. Notez l'ordre des opérations : la DCT ne compresse rien. Elle est orthogonale, donc inversible, et préserve l'entropie différentielle. **La compression vient de la quantification et du codage entropique**, pas de la transformée elle-même. La DCT se contente de rendre la redondance accessible.

Les ondelettes. Une décomposition ondelette sépare une image en une approximation grossière et des détails orientés — horizontaux, verticaux, diagonaux — à plusieurs échelles. Avec une Haar, par exemple, on obtient quatre sous-bandes : *LL*, *LH*, *HL*, *HH* ; la première est récursivement décomposée.

L'avantage sur la DCT par blocs est double : il n'y a pas de frontière de bloc visible, et la représentation est simultanément locale en espace et en fréquence. Les schémas de codage comme EZW ou EBCOT exploitent la propagation zerotree : si un coefficient de détail est négligeable, ses descendants le sont souvent aussi.

Pourquoi ne pas simplement tokeniser des coefficients DCT ? C'est une idée légitime, historiquement explorée. Elle se heurte à trois difficultés.

Premièrement, ces transformées sont **fixes**. Elles ne savent pas qu'un œil est plus important qu'un mur. Elles allouent de la capacité selon une géométrie fréquentielle, pas selon une sémantique.

Deuxièmement, l'ordre naturel des coefficients — zigzag, ou parcours par sous-bandes — ne coïncide pas avec l'ordre spatial. Un modèle autorégressif doit alors apprendre une causalité artificielle,

© 2026 Oussama Ghorbel

moins alignée avec la structure du signal.

Troisièmement, même après quantification aggressive, le nombre de coefficients significatifs reste élevé. On réduit le coût de codage, pas suffisamment le coût de **prédiction**. Or notre objectif n'est pas de stocker des images : c'est de laisser un transformer en générer.

Cette dernière distinction justifie à elle seule les chapitres suivants. Ce qu'il faut au LLM, ce n'est pas une représentation compacte au sens de JPEG. C'est une représentation compacte **et prédictible**.

2.5 — Du pixel brut au patch : première réduction de dimension

Il existe une réduction plus simple, que tout utilisateur de Vision Transformer connaît : la patchification.

On choisit une taille de patch P. On découpe l'image en tuiles non chevauchantes de $P \times P$ pixels, on aplatit chaque tuile, et on obtient une séquence de vecteurs :

$$p_j \in \mathbb{R}^{P^2 C}, \qquad n = \frac{H}{P} \cdot \frac{W}{P}.$$

Pour 256×256 en RGB, quelques valeurs méritent d'être mémorisées :

Taille de patch P	Nombre de patches n	Dimension de chaque patch
4	4 096	48
8	1 024	192
16	256	768
32	64	3 072

La patchification est une **bijection**. Aucun bit n'est perdu, aucun bit n'est créé : on a simplement regroupé les coordonnées. C'est un reshaping, pas une compression.

Cette distinction est cruciale et souvent mal comprise. Passer de 196 608 scalaires à 256 vecteurs de dimension 768 réduit la longueur de séquence par un facteur 768. Mais le modèle doit toujours, à chaque pas, produire 768 nombres. Le problème de sortie n'a pas disparu ; il a changé de forme.

© 2026 Oussama Ghorbel

Trois stratégies se présentent. On peut factoriser la prédiction des 768 coordonnées en sous-étapes, ce qui recrée une séquence longue. On peut prédire une distribution continue — régression gaussienne, mélange de gaussiennes, ou tête plus expressive — ce qui abandonne la softmax catégorique au profit d'une vraisemblance continue. On peut enfin **quantizer le patch** dans un codebook appris, ce qui transforme le vecteur de 768 dimensions en un indice parmi quelques milliers de possibilités.

C'est cette dernière stratégie qui nous intéresse. Et elle révèle la différence fondamentale entre les deux opérations :

- la **patchification** réduit la longueur de séquence sans réduire le contenu ;
- la **quantification vectorielle** réduit le contenu en acceptant une approximation.

Une analogie avec le texte est éclairante. Le BPE regroupe des caractères fréquents en un symbole parce qu'il a *appris* que ce groupe se répète. La patchification regroupe des pixels parce qu'ils sont *proches géométriquement*. La première est une compression statistique ; la seconde est un changement de coordonnées. Un bon tokenizer visuel doit faire les deux : grouper spatialement, puis compresser statistiquement.

Reste une question d'ordre. En aplatissant les patches en raster, on détruit la structure bidimensionnelle sauf à la réintroduire par un encodage de position. Et le masque causal du transformer impose que chaque patch ne voie que ceux qui le précèdent dans la séquence — pas nécessairement ceux qui le entourent dans l'image. Ce décalage entre géométrie spatiale et géométrie séquentielle sera le sujet du chapitre 11.

2.6 — Pourquoi 1 024 tokens suffisent (parfois) pour une image 256 × 256

Nous pouvons maintenant assembler les pièces.

Considérons un auto-encodeur variationnel quantifié dont l'encodeur réduit la résolution par un facteur $f = 8$. Une image 256×256 devient une grille latente de 32×32, soit **1 024 positions**. Chaque position est

© 2026 Oussama Ghorbel

associée au vecteur le plus proche d'un codebook de $K = 8\,192$ entrées. Un token est donc un entier entre 0 et 8 191, soit 13 bits.

Faisons le compte. La séquence générative contient 1 024 tokens. Son débit nominal est de $1024 \times 13 = 13\,312$ bits, contre 1 572 864 bits pour l'image brute : un facteur d'environ **118**. Rapporté au pixel, cela donne environ 0,20 bit par position spatiale pour les trois canaux, contre 24 bits dans le codage natif.

Comment est-ce possible ? Un patch de 8×8 pixels en RGB contient 192 nombres. Le représenter avec 13 bits semble défier le bon sens.

La réponse tient en une phrase : **les patchs naturels ne remplissent pas leur espace de définition**. Un vecteur de 192 dimensions décrivant un fragment de mur, une portion de peau, une bordure d'objet ou une touffe d'herbe appartient à un ensemble extrêmement contraint. Les corrélations spatiales locales, que nous avons mesurées en 2.2, font qu'un tel patch est bien approximé par l'un des quelques milliers de prototypes d'un codebook appris.

Le codebook est un quantificateur vectoriel joint. Il ne quantifie pas 192 nombres indépendamment ; il choisit une *configuration* parmi 8 192 configurations représentatives. C'est cette jointure qui produit le gain.

Mais il y a un second acteur, sans lequel le calcul ne tiendrait pas : **le décodeur**.

Les 1 024 codes ne déterminent pas une image de manière unique. Ils déterminent une grille latente approximative. Le décodeur, entraîné avec des objectifs de reconstruction, de perception et souvent adverses, apprend à transformer cette grille en pixels plausibles. Il restaure des textures, affine des contours, invente du détail haute fréquence cohérent avec le contexte.

Autrement dit, une partie de l'information n'est pas stockée : elle est **reconstituée par un prior appris**. C'est une compression avec perte, au sens strict de la théorie du débit–distorsion. Parler de « 118× de compression » sans préciser la distorsion obtenue serait trompeur.

C'est ici que se dessine la division du travail qui structure tout ce livre :

© 2026 Oussama Ghorbel

• le **tokenizer** l'espace : il transforme une redondance spatiale locale en une séquence courte de symboles ;

• le **LLM** modélise la prédiction : il apprend quelles séquences de symboles sont plausibles étant donné une description textuelle.

Le LLM n'a plus à prédire du bruit de capteur. Il prédit des unités sémantiquement structurées. Son entropie conditionnelle cible n'est plus celle du pixel, mais celle du code — une quantité plus courte à décrire et, si le tokenizer est bien conçu, plus régulière.

Pourquoi « parfois » ? Parce que le facteur 8 est un compromis, pas une loi. Une seule position latente couvre 64 pixels. Un trait d'un pixel de large, une glyph de petite taille, une rangée de vis sur une machine, un texte dense dans une enseigne : autant de structures dont l'information tient précisément dans les hautes fréquences que le quantificateur a jetées. Sur ces contenus, la reconstruction présente des artefacts caractéristiques — lettres fusionnées, motifs répétés, textures plastifiées — et aucun entraînement du LLM ne les fera disparaître. Ils sont perdus avant que le générateur n'intervienne.

De même, un codebook trop petit produit des collisions : deux textures distinctes partagent le même indice. Un codebook trop grand produit des codes rares, mal entraînés, et une softmax plus coûteuse. La taille K, le facteur f et la dimension des vecteurs sont trois hyperparamètres couplés, que nous dissecterons au chapitre 7.

Et à plus haute résolution ? À 512×512 avec le même $f = 8$, la grille devient 64×64, soit 4 096 tokens. La séquence quadruple ; le coût d'attention, lui, est multiplié par seize. La génération haute résolution n'est donc pas gratuite, même avec un excellent tokenizer. C'est l'une des raisons pour lesquelles les stratégies multi-échelles et progressives, étudiées au chapitre 13, ne sont pas un luxe.

Le tableau suivant résume les ordres de grandeur pour une image 256×256 :

Représentation	Longueur N	Vocabulaire	Paires d'attention causales
Scalaires 8 bits	196 608	256	$\approx 1{,}9 \times 10^{10}$
Pixels RGB entiers	65 536	16 777 216	$\approx 2{,}1 \times 10^{9}$

© 2026 Oussama Ghorbel

Patches 16×16 continus	256	tête continue	$\approx 3{,}3 \times 10^4$
Codes VQ, $f = 8$	1 024	8 192	$\approx 5{,}2 \times 10^5$

Deux leçons se dégagent. La patchification seule peut rendre la séquence très courte, mais laisse un problème de sortie massif. La quantification apprisseeule réduit ce problème, mais doit être combinée à une réduction spatiale pour rendre la séquence traitable. **Les deux opérations sont complémentaires, non alternatives.**

C'est, à mon sens, la idée fondamentale à emporter de ce chapitre : on ne compresse pas une image pour la stocker. On la reformate pour la rendre *prédictible par un modèle qui ne connaît que des symboles*.

Exercices

Exercice 2.1 — Entropie empirique d'une image 64 × 64.

Chargez une photographie en niveaux de gris, redimensionnez-la en 64×64, puis computez son histogramme sur 256 niveaux. Calculez $\hat{H}$ en bits par pixel. Renouvelez l'expérience sur (a) un bruit blanc uniforme, (b) un dégradé horizontal, (c) une image de contours synthétiques. Classez les quatre images par entropie, puis par difficulté subjective de compression. Expliquez l'éventuel désaccord. Refaites le calcul après quantification sur 4 bits et après calcul de l'histogramme des différences horizontales $x_{i,j} - x_{i,j-1}$. *Piste : normalisez l'histogramme, éliminez les cellules nulles avant le logarithme, et comparez votre résultat à la correction de Miller–Madow.*

Exercice 2.2 — Décompte de tokens à 512 × 512.

Pour une image $512 \times 512 \times 3$, calculez le nombre de tokens dans quatre configurations : (a) un token par scalaire 8 bits ; (b) un token par pixel RGB ; (c) des patches non chevauchants de 16×16 ; (d) un tokenizer quantifié avec facteur $f = 8$. Pour chaque cas, donnez la longueur de séquence, la taille de vocabulaire nécessaire et le nombre approximatif de paires d'attention causales. Calculez ensuite le débit en bits de la configuration (d) avec un codebook de 16 384 entrées, et comparez-le aux 6 291 456 bits du codage brut. *Piste : distinguez soigneusement réduction de longueur et réduction d'information ; les deux ne se comptent pas de la même manière.*

© 2026 Oussama Ghorbel

Exercice 2.3 — Carte d'autocorrélation spatiale.

Écrivez un script Python qui charge une image naturelle en luminance, puis affiche une carte de chaleur de la corrélation spatiale normalisée

$$\rho(\Delta x, \Delta y) = \frac{\mathrm{Cov}(x_{i,j},\, x_{i+\Delta x, j+\Delta y})}{\mathrm{Var}(x)}$$

pour les décalages $|\Delta x|, |\Delta y| \leq 16$. Centrez les données par la moyenne globale et n'agréez que les positions valides. Produisez la même carte pour une image de texture synthétique et pour un bruit blanc. Mesurez la distance à laquelle ρ tombe en dessous de 0,5. *Piste : comparez la carte du canal luminance à celles des trois canaux RGB. L'anisotropie éventuelle de la carte vous dira quelque chose sur le contenu de la scène — et sur l'ordre de balayage à privilégier.*

© 2026 Oussama Ghorbel

Chapitre 3 – La relation token <-> pixel : formalisation et premiers calculs

Dans le chapitre précédent, nous avons ausculté le pixel dans sa nature la plus brute : un signal continu, multidimensionnel, regorgeant de redondances spatiales et spectrales. Nous avons compris que l'image, dans sa représentation matricielle classique, est un candidat médiocre pour la modélisation autoregressive directe. Le Grand Modèle de Langage (LLM), lui, ne jure que par le discret. Il exige des symboles, des indices, des éléments finis qu'il peut manipuler, prédire et enchaîner.

Le défi fondamental de l'ingénieur en apprentissage automatique qui souhaite transformer un LLM en générateur d'images réside donc dans la construction d'un pont rigoureux entre ces deux mondes. Comment traduire la continuité lumineuse d'une photographie en une séquence finie de symboles discrets, sans trahir l'essence de la scène visuelle ? C'est l'objet de ce chapitre. Nous allons y définir formellement la correspondance entre le token et le pixel, établir les équations qui régissent ce mariage, et calculer les budgets computationnels et mémoriels qui en découlent. Car en ingénierie des systèmes d'IA, avant même de parler d'architecture ou d'attention, il faut savoir compter.

3.1 — Définition formelle : du token à la région pixelique

Pour instaurer un dialogue entre le pixel et le token, nous devons d'abord définir mathématiquement les espaces dans lesquels ils évoluent et les opérateurs qui permettent de passer de l'un à l'autre.

Soit $X \in \mathbb{R}^{H \times W \times C}$ une image d'entrée, où H et W sont les dimensions spatiales (hauteur et largeur) et C le nombre de canaux (généralement $C = 3$ pour RVB). L'opération de tokenisation ne s'applique presque jamais au pixel individuel. Comme nous le verrons en détail dans la Partie II, le coût computationnel et la perte de contexte spatial rendent la tokenisation pixel-par-pixel prohibitive. Nous opérons donc par blocs, ou *patches*.

Définissons une fonction de partitionnement Φ qui divise l'image X en N régions disjointes (ou se chevauchant, selon l'inducteur visuel

© 2026 Oussama Ghorbel

choisi, bien que le pavage strict soit la norme pour les LLMs) :

$$\Phi(X) = \{R_1, R_2, \ \dots, R_N\}$$

où chaque région $R_i \in \mathbb{R}^{p \times p \times C}$ est un patch de taille $p \times p$.

Le processus de tokenisation est une composition de trois fonctions distinctes :

1. **L'encodage continu (E)** : Une fonction paramétrique (souvent un réseau de neurones convolutif ou un transformateur) qui projette chaque patch dans un espace latent continu de dimension d :

$$z_i = E(R_i) \in \mathbb{R}^d$$

2. **La quantification (Q)** : L'opération cruciale qui fait basculer le signal du continu vers le discret. Elle associe le vecteur latent z_i à l'indice du vecteur le plus proche dans un dictionnaire fini (le *codebook*) $\mathcal{C} = \{e_1, e_2, \ \dots, e_K\}$:

$$t_i = Q(z_i) = \underset{k \in \{1, \ \dots, K\}}{\operatorname{argmin}} \ \|z_i - e_k\|_2$$

Le résultat t_i est un entier, notre **token visuel**.

3. **Le décodage (D)** : La fonction qui reconstruit la région pixelique à partir du token. Puisque le token t_i n'est qu'un indice, le décodeur commence par récupérer le vecteur du codebook $\hat{z}_i = e_{t_i}$, puis applique un réseau de décodage pour retrouver l'espace pixelique :

$$\hat{R}_i = D(\hat{z}_i) \in \mathbb{R}^{p \times p \times C}$$

La relation fondamentale qui unit le token t_i à la région originale R_i est donc une chaîne de transformations avec goulot d'étranglement informationnel :

$$R_i \to^{E} z_i \to^{Q} t_i \to^{\text{lookup}} e_{t_i} \to^{D} \hat{R}_i$$

Il est impératif de noter que la fonction de quantification Q est une fonction en escalier, non différentiable (sa dérivée est nulle presque partout, et infinie aux frontières). Cela pose un problème majeur pour la rétropropagation du gradient lors de l'entraînement de l'auto-encodeur. Nous aborderons les solutions à ce problème (comme l'estimateur *Straight-Through*) dans les chapitres dédiés à l'entraînement, mais il faut dès à présent retenir que le token t_i est une représentation

© 2026 Oussama Ghorbel

irréversiblement compressée de R_i. L'erreur de reconstruction $\varepsilon_i = R_i - \hat{R}_i$ est la rançon de cette discrétisation.

3.2 — Résolution effective et géométrie du pavage

Lorsque l'on traite du texte, la longueur de la séquence est dictée par la sémantique et la syntaxe : une phrase fait dix mots, un paragraphe en fait cent. Pour l'image, la longueur de la séquence N est une variable purement géométrique et algorithmique, dictée par la résolution de l'image et la taille du patch.

Si nous supposons un pavage régulier et sans chevauchement (la configuration standard pour des modèles comme VQGAN ou les Vision Transformers originaux), le nombre total de tokens N est donné par l'équation triviale mais fondatrice :

$$N = \left(\frac{H}{p}\right) \times \left(\frac{W}{p}\right) = \frac{H \cdot W}{p^2}$$

Cette équation soulève immédiatement une contrainte d'ingénierie : H et W doivent être des multiples entiers de p. Dans la pratique, les jeux de données d'images brutes ont des résolutions hétérogènes. Avant la tokenisation, un pipeline de prétraitement doit imposer une géométrie stricte, soit par redimensionnement (ce qui altère le ratio d'aspect), soit par recadrage central (ce qui perd de l'information), soit par rembourrage (*padding*). Le *padding* par réflexion est souvent préféré au *padding* par zéros pour éviter d'introduire des artefacts de bordure à haute fréquence que le tokenizer tenterait vainement d'apprendre.

Ce que j'appelle la **résolution effective** d'un modèle autoregressif visuel n'est pas $H \times W$, mais $\frac{H}{p} \times \frac{W}{p}$. C'est la grille sémantique sur laquelle le LLM va opérer ses prédictions de *next-token*.

Prenons une image de 1024×1024 pixels. C'est la résolution standard pour une génération de haute qualité aujourd'hui. Si nous choisissons une taille de patch $p = 32$, le LLM ne "voit" pas un million de pixels. Il manipule une grille de 32×32, soit une séquence de $N = 1024$ tokens. C'est une longueur de séquence tout à fait confortable pour un LLM moderne. En revanche, si nous réduisons la taille du patch à $p = 8$ pour capturer des détails plus fins, la séquence explose à $128 \times 128 = 16384$ tokens.

© 2026 Oussama Ghorbel

Ce changement de résolution effective a des conséquences dramatiques sur le budget computationnel du Transformateur. La complexité de l'attention standard étant quadratique $\mathcal{O}(N^2)$, passer de 1024 à 16384 tokens multiplie le coût de l'attention par un facteur 256. C'est pourquoi le choix de p n'est pas seulement une question de qualité visuelle, c'est le levier principal de l'architecture système globale.

3.3 — Le codebook comme dictionnaire visuel

Le token t_i n'a de sens que par rapport au dictionnaire qui lui donne sa valeur sémantique et pixelique : le *codebook* $\mathcal{C}$. Ce dernier peut être vu comme une matrice d'embeddings de taille $K \times d$, où K est la taille du vocabulaire visuel et d la dimension de l'espace latent.

Dans les modèles de langage textuel, K est souvent compris entre 32 000 et 100 000 (le vocabulaire couvre des mots, des sous-mots, des caractères rares, du code). Pour les images, les valeurs typiques de K sont beaucoup plus modestes : 1024, 4096, 8192, ou parfois 16384. Pourquoi cette différence ?

D'abord, parce que l'espace des concepts visuels atomiques (une texture de brique, un dégradé de ciel bleu, un contour d'œil) semble pouvoir être factorisé en un nombre de "mots visuels" plus restreint que la combinatoire du langage humain. Ensuite, et c'est une raison purement pratique, un codebook trop large souffre du phénomène d'effondrement (*codebook collapse*). Lors de l'entraînement, le réseau a tendance à n'utiliser qu'une petite fraction des K vecteurs disponibles, laissant les autres "morts" (jamais mis à jour par le gradient). Maintenir un codebook de 65 536 entrées pleinement utilisé et équilibré est un défi d'optimisation majeur que nous explorerons au chapitre 7.

La dimension d (souvent 256 ou 512) détermine la capacité de chaque vecteur du codebook à encoder de l'information complexe. Un d trop faible limite la fidélité de la reconstruction ; un d trop grand alourdit la mémoire et les calculs du réseau de décodage.

Évaluons maintenant la **capacité expressive** théorique de ce système. Si notre image est composée de N tokens, et que chaque token peut prendre K valeurs distinctes, l'espace des images possibles représentables par ce tokenizer est de :

© 2026 Oussama Ghorbel

$$|\mathcal{X}_{\text{représentable}}| = K^N$$

Pour une image de 256×256 avec $p = 16$ ($N = 256$) et $K = 1024$, l'espace est de $1024^{256} = (2^{10})^{256} = 2^{2560}$. C'est un nombre astronomique, bien supérieur au nombre d'atomes dans l'univers observable. Pourtant, la quasi-totalité de ces combinaisons ne produira pas une image naturelle, mais un bruit statique coloré et incohérent.

Le rôle du LLM autoregressif n'est pas d'explorer cet espace uniformément, mais d'apprendre la sous-variété de dimension infiniment plus petite qui correspond à la distribution des images naturelles $P(X_{\text{naturel}})$. Le codebook fournit la palette de couleurs et les coups de pinceau élémentaires ; le LLM apprend la composition, l'ordre et la probabilité conditionnelle d'agencer ces coups de pinceau pour former une scène cohérente.

3.4 — Bits par pixel (bpp) contre bits par token

En compression d'image et en théorie de l'information, la métrique reine pour évaluer l'efficacité d'un codec est le taux de compression, souvent exprimé en **bits par pixel (bpp)**. C'est une mesure qui permet de comparer objectivement un JPEG, un WebP et notre tokenizer basé sur un LLM, indépendamment de la résolution de l'image.

Calculons le bpp de notre représentation tokenisée.

Chaque token t_i est un indice entier compris entre 0 et $K - 1$. Pour stocker ou transmettre cet indice en binaire, il faut théoriquement $\log_2(K)$ bits. Par exemple, si $K = 1024$, chaque token nécessite exactement 10 bits. Si $K = 8192$, il en faut 13.

Le nombre total de bits B nécessaires pour représenter l'image complète est donc :

$$B = N \times \log_2(K) = \frac{H \cdot W}{p^2} \log_2(K)$$

Le bpp se définit comme le nombre total de bits divisé par le nombre total de pixels ($H \times W$) :

$$\text{bpp} = \frac{B}{H \cdot W} = \frac{\frac{H \cdot W}{p^2} \log_2(K)}{H \cdot W}$$

© 2026 Oussama Ghorbel

En simplifiant, nous obtenons l'une des équations les plus élégantes et importantes de ce livre :

$$\text{bpp} = \frac{\log_2(K)}{p^2}$$

Cette formule est remarquable car elle démontre que **le taux de compression (bpp) est totalement indépendant de la résolution de l'image** $H \times W$. Il ne dépend que de deux hyperparamètres architecturaux : la taille du vocabulaire K et la taille du patch p.

Prenons un exemple concret pour mesurer l'ampleur de cette compression. Une image RVB standard en 8 bits par canal possède un bpp brut de $3 \times 8 = 24$ bpp.

Si nous utilisons un tokenizer avec $K = 1024$ (donc 10 bits par token) et une taille de patch $p = 16$:

$$\text{bpp} = \frac{10}{16^2} = \frac{10}{256} \approx 0,039 \text{ bpp}$$

Le ratio de compression est de $\frac{24}{0,039} \approx 615$. Notre représentation tokenisée est environ 600 fois plus compacte que l'image brute. C'est une compression extrême, bien supérieure à ce que permettent les codecs classiques sans pertes, et même très agressive comparée au JPEG (qui opère généralement entre 0,5 et 2 bpp pour une bonne qualité).

Cependant, cette formule révèle aussi le compromis inévitable. Si l'on souhaite augmenter la fidélité visuelle (baisser la distorsion), on doit soit augmenter K (ce qui augmente le bpp logarithmiquement), soit diminuer p (ce qui augmente le bpp de façon quadratique). Réduire la taille du patch de 16 à 8 multiplie le bpp par 4, le faisant passer à 0,156 bpp, et multiplie la longueur de la séquence pour le LLM par 4. C'est l'arbitrage constant de l'ingénieur : la qualité pixelique contre le budget de séquence du modèle autoregressif.

3.5 — Surcoût de la quantification : l'arbitrage taux-distorsion

La compression que nous venons de calculer n'est pas sans perte. Le passage du vecteur continu z_i à l'indice discret t_i introduit une erreur de quantification incompressible. C'est ici qu'intervient la théorie classique de l'information de Shannon, et plus spécifiquement la théorie du

© 2026 Oussama Ghorbel

taux-distorsion (*Rate-Distortion theory*).

Le "Taux" (R) est notre bpp, la quantité d'information que nous conservons. La "Distorsion" (D) est l'erreur entre l'image originale X et l'image reconstruite $\hat{X}$. Le théorème du codage avec distorsion stipule que pour un taux R donné, il existe une distorsion minimale inévitable $D(R)$, et inversement.

Dans notre pipeline, l'erreur de quantification pour un patch i est :

$$\varepsilon_i = z_i - e_{t_i}$$

où e_{t_i} est le vecteur du codebook le plus proche de z_i. Cette erreur ε_i est ensuite propagée à travers le décodeur D, ce qui génère des artefacts visuels dans l'espace pixelique.

Quelles sont les manifestations visuelles de cette distorsion ?

1. **Le flou et la perte de hautes fréquences** : Si p est grand et K est petit, le codebook n'a pas assez de "mots" pour décrire les textures complexes (les feuilles d'un arbre, les cheveux, le grain de la peau). Le décodeur va produire des moyennes locales, résultant en un aspect "peint à l'huile" ou flou.

2. **Les artefacts de grille (Grid Artifacts)** : Puisque la quantification est opérée patch par patch de manière indépendante (dans les approches VQ standards), les frontières entre les patches peuvent présenter des discontinuités de couleur ou de texture, créant une grille visible à l'œil nu, surtout dans les zones de dégradé uniforme comme le ciel.

3. Le Color Banding : La limitation de restreint le nombre de nuances de couleurs disponibles pour un patch donné, transformant les dégradés subtils en bandes de couleurs plates.

L'ingénieur doit donc naviguer sur la courbe de Pareto du taux-distorsion. Mais contrairement à la compression classique (comme le JPEG) où la distorsion est mesurée par l'Erreur Quadratique Moyenne (MSE), en génération d'images pour l'IA, la MSE est une métrique trompeuse. Une image légèrement décalée d'un pixel aura une MSE terrible, bien qu'elle soit perceptuellement identique pour un humain.

© 2026 Oussama Ghorbel

C'est pourquoi l'évaluation de la distorsion dans les tokenizers visuels modernes s'appuie sur des métriques perceptuelles (comme LPIPS) et des pertes adversariales (GAN). On accepte volontiers une erreur pixelique (MSE) plus élevée si cela permet de générer des textures nettes et réalistes, même si ces textures ne correspondent pas *exactement* à la position originale des détails de l'image source. Le token ne doit pas être vu comme une archive parfaite de l'image, mais comme une capsule sémantique et structurelle suffisante pour "halluciner" une reconstruction visuellement plausible.

3.6 — Tableau de correspondance et ordres de grandeur

Pour ancrer ces concepts théoriques dans la réalité de l'ingénierie système, un tableau de correspondance est présenté. Ce tableau est un outil de dimensionnement que vous devriez garder à portée de main lors de la conception de vos architectures.

Il croise les résolutions d'image standards avec les tailles de patches usuelles, en supposant un codebook de taille $K = 1024$ (soit 10 bits par token, et un bpp constant pour une taille de patch donnée). La dernière colonne indique le pourcentage de la fenêtre de contexte (context window) consommé par l'image, en supposant un LLM avec une limite stricte de 8192 tokens (une taille courante pour les modèles de base avant extension).

Résolution ($H \times W$)	Taille Patch (p)	Grille Effective	Nb Tokens (N)	bpp ($K = 1024$)	Séquence vs Context Window (8k)
256 × 256	16	16 × 16	256	0,039	3,1 % (Très confortable)
256 × 256	32	8 × 8	64	0,009	0,8 % (Extrêmement compact)
512 × 512	16	32 × 32	1024	0,039	12,5 % (Standard pour le fine-tuning)
512 × 512	32	16 × 16	256	0,009	3,1 % (Bonne marge pour le texte)

© 2026 Oussama Ghorbel

1024 × 1024	16	64 × 64	4096	0,039	50,0 % (Limite système, KV-cache lourd)
1024 × 1024	32	32 × 32	1024	0,009	12,5 % (Optimal pour la génération HD)
1024 × 1024	64	16 × 16	256	0,002	3,1 % (Perte de détails fins)

Analyse du tableau :

Ce tableau met en lumière le "mur de la résolution". Générer du 1024×1024 avec des patches de 16×16 produit 4096 tokens. Si votre LLM doit également conditionner cette génération sur un prompt textuel de 500 tokens, et que vous utilisez des techniques d'inférence comme le *classifier-free guidance* qui doublent les passes, vous saturez rapidement les limites matérielles de la mémoire KV-cache des GPU.

C'est précisément pour contourner ce mur que la recherche s'oriente vers des patches plus grands ($p = 32$) couplés à des codebooks plus riches ($K = 8192$ ou 16384), ou vers des architectures de génération parallèle et multi-échelle que nous détaillerons dans la Partie III. Le choix de $p = 32$ pour du 1024px semble être le point d'équilibre actuel de l'industrie (le "sweet spot"), offrant 1024 tokens visuels, ce qui laisse suffisamment de room pour l'attention croisée avec le texte et maintient un bpp de 0,009, à condition que le codebook soit suffisamment expressif pour compenser la perte de résolution spatiale.

Exercices de fin de chapitre

La maîtrise de ces budgets n'est pas optionnelle ; elle conditionne la faisabilité même de vos expériences sur GPU. Les exercices suivants sont conçus pour ancrer ces formules dans votre pratique quotidienne.

Exercice 1 : Construction du tableau de budget mémoire

© 2026 Oussama Ghorbel

Construisez un tableau complet évaluant l'impact des hyperparamètres sur la mémoire. Pour chaque combinaison de résolution ($H \in \{256, 512, 1024\}$) et de taille de patch ($p \in \{8, 16, 32\}$), calculez :

1. Le nombre de tokens N.
2. Le bpp (en supposant $K = 8192$).
3. La taille mémoire exacte du codebook stocké en précision `float32`, en supposant une dimension latente $d = 512$.

Indice pour la mémoire du codebook : Le codebook est une matrice de K vecteurs de dimension d. Chaque float32 prend 4 octets. N'oubliez pas de convertir le résultat final en Mégaoctets (Mo) pour une lecture humaine. Observez comment la mémoire du codebook est indépendante de la résolution de l'image, mais pèse lourdement sur le chargement initial du modèle.

Exercice 2 : Démonstration de la loi d'échelle quadratique

Démontrez formellement, sur papier, que doubler la résolution spatiale d'une image (passer de $H \times W$ à $2H \times 2W$) tout en maintenant la taille du patch p constante résulte en un quadruplement exact du nombre de tokens N, et donc de la complexité mémoire de la séquence.

Ensuite, expliquez en trois lignes pourquoi cette propriété géométrique est le principal cauchemar des ingénieurs en déploiement de LLMs visuels, en faisant le lien avec la complexité en $\mathcal{O}(N^2)$ du mécanisme de *Self-Attention*.

Exercice 3 : Implémentation du calculateur de budget

Implémentez en Python une fonction utilitaire `budget_tokens` qui servira d'outil de diagnostic dans vos scripts d'entraînement. Cette fonction doit prendre en entrée les dimensions de l'image, la taille du patch, la taille du codebook et la dimension latente, et retourner un dictionnaire structuré contenant toutes les métriques de budget.

Voici la signature et l'implémentation de référence attendue :

Testez cette fonction avec les paramètres du VQGAN original (256×256, $p = 16$, $K = 1024$, $d = 256$) et comparez le résultat avec un tokenizer de nouvelle génération pour la haute définition (1024×1024, $p = 32$, $K = 16384$, $d = 512$). Vous constaterez que l'ingénierie des

© 2026 Oussama Ghorbel

tokenizers modernes consiste à maintenir le nombre de tokens N sous contrôle tout en augmentant massivement la richesse sémantique du codebook K et sa dimension d.

© 2026 Oussama Ghorbel

Chapitre 4 – Panorama des approches génératives : autoregression, diffusion, GAN, flux

Avant de plonger dans les mécanismes fins de la tokenisation visuelle et de la modélisation autorégressive qui occupent le cœur de cet ouvrage, il me semble indispensable de dresser une cartographie honnête du paysage génératif contemporain. Trop souvent, on présente l'autoregression sur tokens comme une curiosité historique ou une alternative exotique face à la diffusion. La réalité est plus nuancée : chaque famille de modèles incarne un compromis différent entre expressivité, coût computationnel, facilité de conditionnement et compatibilité avec les infrastructures existantes. Mon objectif dans ce chapitre est de vous donner les repères nécessaires pour situer la génération par tokens dans cet écosystème, comprendre ce qu'elle gagne et ce qu'elle concède par rapport à ses rivales, et surtout justifier pourquoi elle constitue le chemin le plus naturel lorsqu'on dispose déjà d'un grand modèle de langage entraîné.

4.1 — Modèles autoregressifs : factorisation chaîne de p(x), PixR, ImageGPT, DALL·E 1

Le principe est d'une élégance désarmante : on factorise la distribution conjointe des pixels — ou des tokens visuels — en un produit de distributions conditionnelles :

$$p(x) = \prod_{i=1}^{N} p(x_i \mid x_1, x_2, \ \dots, x_{i-1})$$

Chaque élément est prédit séquentiellement, conditionnellement à tout ce qui précède. C'est exactement la même factorisation qu'un modèle de langage applique aux mots. La différence tient uniquement à la nature de l'alphabet : au lieu de 32 000 ou 128 000 tokens textuels, on travaille avec des indices de codebook visuel, typiquement entre 1 024 et 16 384 entrées.

PixR (Ramesh et al., 2021) a démontré qu'un transformer autoregressif de 12 milliards de paramètres, entraîné sur des paires

© 2026 Oussama Ghorbel

texte-image via un vocabulaire VQ-VAE de 8 192 codes, pouvait produire des images cohérentes à partir de prompts textuels. Le modèle traite texte et image comme une séquence unique de tokens, et la génération se fait par échantillonnage nucléus sur la portion image. La qualité restait modeste comparée aux standards actuels, mais le principe d'unification était posé.

ImageGPT (Chen et al., 2020) a appliqué la même logique à la génération inconditionnelle d'images CIFAR-10 et ImageNet 64 × 64, en traitant chaque pixel comme un token d'un vocabulaire de 512 valeurs. L'insight clé : un GPT-2 standard, sans modification architecturale, apprend déjà des représentations visuelles exploitables.

DALL·E 1 a repris la philosophie de PixR en la rendant plus robuste : meilleur tokenizer (dVAE avec codebook de 8 192), entraînement sur 250 millions de paires, et un mécanisme de reranking CLIP en inférence. La génération reste séquentielle, token par token, sur une grille de 32 × 32 codes pour une image 256 × 256 — soit 1 024 étapes autoregressives.

L'avantage fondamental de cette famille est la réutilisation directe de l'infrastructure LLM : même architecture transformer, même procédure d'entraînement par cross-entropie, même stratégie de decoding (top-k, top-p, beam search). La limite principale est le coût séquentiel : chaque token dépend du précédent, ce qui interdit la parallélisation en génération et impose une latence proportionnelle à N.

4.2 — Modèles de diffusion : DDPM, score matching, Stable Diffusion, DiT

Les modèles de diffusion reposent sur une idée radicalement différente : on apprend à inverser un processus de corruption graduelle. Le processus forward ajoute du bruit gaussien à l'image en T étapes (typiquement T = 1 000) jusqu'à obtenir du bruit pur. Le processus backward, paramétré par un réseau _ , apprend à prédire le bruit ajouté à chaque étape.

DDPM (Ho et al., 2020) a formalisé ce cadre avec une perte simple — la MSE entre le bruit prédit et le bruit réel — et a montré que la qualité générative rivalisait avec les GAN sur des benchmarks standards. La génération nécessite T étapes de débruitage séquentiel, chacune

© 2026 Oussama Ghorbel

impliquant une passe complète du réseau.

Le **score matching** (Song et Ermon, 2019 ; Song et al., 2021) offre une perspective complémentaire : au lieu de prédire le bruit, le réseau apprend le gradient du log de la densité, _x log p(x), à différents niveaux de bruit. Cela unifie DDPM et les modèles basés sur les SDE (équations différentielles stochastiques) dans un cadre commun.

Stable Diffusion (Rombach et al., 2022) a introduit l'idée décisive de travailler dans l'espace latent d'un auto-encodeur pré-entraîné. Plutôt que de débruiter les pixels directement (coût prohibitif en haute résolution), on opère sur un latent de dimension réduite — par exemple 32 × 32 × 4 pour une image 256 × 256. Le conditionnement textuel passe par cross-attention sur les embeddings d'un encodeur de texte gelé (CLIP ou T5).

DiT (Peebles et Xie, 2023) remplace l'U-Net traditionnel par un transformer pur, où chaque patch du latent devient un token traité par attention. Cette architecture scale remarquablement bien et préfigure la convergence entre diffusion et transformers.

La force de la diffusion réside dans la qualité d'image exceptionnelle et la flexibilité du conditionnement (ControlNet, IP-Adapter, inpainting natif). Sa faiblesse structurelle : la génération reste itérative (25 à 50 étapes minimum en pratique), et chaque étape nécessite une évaluation complète du réseau. L'inférence est donc coûteuse, même si des distillations (LCM, SDXL-Turbo) réduisent ce nombre à 1-4 étapes au prix d'une perte de qualité.

4.3 — GAN et modèles à base d'énergie : StyleGAN, StyleGAN3

Les réseaux antagonistes génératifs, introduits par Goodfellow et al. (2014), mettent en compétition un générateur G et un discriminateur D dans un jeu minimax. Le générateur produit des images à partir d'un bruit latent z ; le discriminateur tente de distinguer les vraies images des fausses. À l'équilibre, G produit des échantillons indiscernables.

StyleGAN (Karras et al., 2019) et surtout **StyleGAN3** (Karras et al., 2021) ont poussé la qualité photoréaliste à un niveau spectaculaire sur des domaines spécifiques (visages, paysages). StyleGAN3 introduit une

© 2026 Oussama Ghorbel

invariance aux translations et rotations par conception, éliminant les artefacts de « texture sticking » des versions précédentes.

Les avantages des GAN sont une génération en une seule passe (pas d'itération) et une netteté souvent supérieure à la diffusion à résolution égale. Les inconvénients sont nombreux : instabilité de l'entraînement, mode collapse, difficulté à couvrir la diversité de la distribution, et surtout une interface de conditionnement textuel beaucoup moins naturelle que pour les transformers autoregressifs ou la diffusion.

Du point de vue de cet ouvrage, les GAN sont intéressants comme repère historique et comme source d'inspiration pour les loss perceptuelles, mais ils ne constituent pas une base viable pour adapter un LLM. L'architecture n'est tout simplement pas compatible.

4.4 — Normalizing flows et modèles continus : Glow, Flow Matching

Les normalizing flows apprennent une bijection explicite entre une distribution simple (gaussienne) et la distribution des données. **Glow** (Kingma et Dhariwal, 2018) utilise des couches de couplage affine inversibles pour construire cette bijection. L'avantage théorique est majeur : la vraisemblance est calculable exactement, ce qui permet un entraînement par maximum de vraisemblance sans approximation.

En pratique, les flows souffrent de contraintes architecturales lourdes (inversibilité imposée sur chaque couche) et d'une expressivité limitée par rapport aux modèles implicites. Ils ont été largement supplantés pour la génération d'images haute résolution.

Le **Flow Matching** (Lipman et al., 2023) renouvelle l'intérêt pour les modèles continus en apprenant un champ de vitesse qui transporte une distribution source vers la distribution cible le long d'un chemin d'interpolation. Contrairement à la diffusion qui opère dans un espace de bruit croissant, le flow matching définit une trajectoire directe entre bruit et données. L'entraînement est plus stable, le nombre d'étapes d'inférence peut être réduit, et le cadre se prête naturellement à une formulation en espace latent.

Pour notre propos, les flows et le flow matching représentent une alternative continue élégante, mais ils ne s'insèrent pas dans le

© 2026 Oussama Ghorbel

paradigme « next-token prediction » d'un LLM. Ils nécessitent une architecture dédiée et un objectif d'entraînement spécifique, ce qui les rend incompatibles avec une adaptation directe d'un modèle de langage pré-existant.

4.5 — Hybrides : MAR, Transfusion, Chameleon — mélanger autoregression et diffusion

La frontière entre familles s'estompe. Plusieurs travaux récents hybrident autoregression et diffusion pour capturer le meilleur des deux mondes.

MAR (Li et al., 2024) — Masked Autoregressive Models — combine la génération parallèle par masquage (à la MaskGIT) avec un décodeur de diffusion léger qui produit chaque token continu à partir de son contexte. L'autoregression se fait par blocs de tokens démasqués itérativement, tandis qu'à l'intérieur de chaque bloc, un petit modèle de diffusion génère les valeurs continues. Résultat : une image 256 × 256 en 8 à 16 itérations seulement, avec un FID compétitif.

Transfusion (Meta, 2024) entraîne un seul transformer avec deux objectifs simultanés : la perte de next-token prediction pour le texte et la perte de diffusion pour les patchs d'image. Le modèle alterne entre les deux modalités au sein d'une même séquence. C'est peut-être l'architecture hybride la plus aboutie à ce jour, car elle préserve la capacité de raisonnement séquentiel du texte tout en bénéficiant de la qualité de la diffusion pour les pixels.

Chameleon (Meta, 2024) pousse l'unification plus loin : texte et image sont tous deux tokenisés en tokens discrets, et un seul modèle autoregressif traite l'ensemble. Les images sont encodées via un VQ-VAE en séquences de 1 024 tokens. L'avantage est une simplicité conceptuelle totale ; le défi est que la qualité d'image reste en retrait par rapport à la diffusion pure.

Ces hybrides confirment une tendance que je défends dans ce livre : l'autoregression n'est pas morte, elle se réinvente. Elle reste le squelette séquentiel autour duquel s'articulent des mécanismes de génération locale plus expressifs.

© 2026 Oussama Ghorbel

4.6 — Critères de choix : qualité, coût, contrôle, scalabilité

Face à ce panorama, comment choisir ? Je propose quatre axes de comparaison.

Qualité d'image. À budget computationnel d'entraînement équivalent, la diffusion latente (Stable Diffusion 3, DALL·E 3, Imagen 3) produit aujourd'hui les images les plus fidèles et les plus diversifiées. L'autoregression discrète souffre encore d'artefacts de quantification, surtout en haute résolution. Les GAN excellent sur des domaines étroits mais manquent de diversité.

Coût d'inférence. L'autoregression pure est la plus coûteuse : N étapes séquentielles pour N tokens. La diffusion nécessite 20 à 50 passes réseau mais chacune est parallélisable spatialement. Les GAN et les flows n'exigent qu'une seule passe. Les hybrides (MAR, Transfusion) réduisent le nombre d'étapes à 8-16.

Contrôle et conditionnement. La diffusion offre le contrôle le plus fin : guidance par classifier, ControlNet, inpainting, outpainting, édition par attention. L'autoregression permet un contrôle séquentiel naturel (on peut intervenir token par token) mais manque d'outils d'édition globale. Les GAN permettent l'interpolation dans l'espace latent mais pas un conditionnement textuel riche.

Scalabilité et compatibilité LLM. C'est ici que l'autoregression sur tokens prend un avantage décisif. Un LLM de 7, 13 ou 70 milliards de paramètres est déjà un moteur autoregressif. Ajouter un vocabulaire visuel, étendre l'embedding, poursuivre l'entraînement : la procédure est incrémentale. Adapter ce même LLM à la diffusion exige de greffer un encodeur de bruit, un décodeur U-Net ou DiT, un scheduler — bref, une architecture parallèle qui double la complexité.

4.7 — Pourquoi l'autoregression sur tokens est naturelle pour un LLM existant

J'arrive au point qui justifie l'existence même de ce livre. Supposons que vous disposiez d'un LLM entraîné — LLaMA, Mistral, GPT — avec ses centaines de milliards de tokens textuels absorbés, ses capacités de raisonnement, de suivi d'instructions, de cohérence contextuelle. Vous souhaitez lui adjoindre la capacité de générer des images.

© 2026 Oussama Ghorbel

La voie diffusion exige de construire un pipeline séparé : un VAE pour encoder les images en latent, un U-Net ou DiT pour le débruitage, un encodeur textuel pour le conditionnement, un scheduler pour le processus d'échantillonnage. Ce pipeline vit à côté du LLM, communique avec lui par des ponts d'attention ou des adaptateurs, mais ne partage ni ses poids, ni sa dynamique d'entraînement, ni sa mémoire contextuelle unifiée.

La voie autoregressive, au contraire, ne demande qu'une extension de vocabulaire. On entraîne un tokenizer visuel (VQGAN, par exemple, cf. chapitre 6) qui mappe chaque image en une séquence de tokens discrets. On ajoute ces tokens au vocabulaire du LLM, on étend la matrice d'embedding, on poursuit l'entraînement sur des données texte-image en conservant la même fonction de perte : la cross-entropie du next-token prediction. Le modèle apprend à générer des images exactement comme il génère du texte — un token à la fois, en conditionnant sur tout le contexte précédent.

Les bénéfices sont concrets :

1. **Unification représentationnelle.** Texte et image partagent le même espace de représentation, la même attention, la même mémoire. Le modèle peut raisonner sur une image qu'il vient de générer, la modifier token par token, ou alterner texte et image dans une même réponse.

2. **Réutilisation de l'infrastructure.** Pas de nouveau scheduler, pas de nouveau décodeur, pas de pipeline d'inférence parallèle. Le serving, la quantification, le batching dynamique — tout l'écosystème LLM s'applique tel quel.

3. **Émergence de capacités multimodales.** L'entraînement conjoint favorise des capacités inattendues : légendage, édition par instruction, génération conditionnée par un raisonnement explicite (« dessine un chat, mais d'abord réfléchissons à la composition »).

4. **Simplicité du déploiement.** Un seul modèle, un seul format de poids, une seule API. Pas de orchestration entre un LLM et un module de diffusion.

© 2026 Oussama Ghorbel

Bien sûr, le prix à payer est réel : la génération séquentielle est lente, la quantification du codebook introduit une perte d'information, et la qualité pixel-par-pixel reste en retrait face à la diffusion latente. Ce sont précisément les défis que nous adresserons dans les parties III et IV de cet ouvrage, en explorant les ordres de balayage, la génération semi-autoregressive, et les protocoles d'entraînement adaptés.

Mais le constat demeure : si votre point de départ est un LLM, l'autoregression sur tokens est le chemin de moindre résistance, celui qui préserve l'investissement computationnel déjà consenti et qui ouvre la voie à une véritable intelligence multimodale unifiée.

Exercices du chapitre 4

Exercice 4.1 — Tableau comparatif

Remplissez le tableau suivant pour les quatre familles de modèles étudiées (autoregression discrète, diffusion latente, GAN, flow matching). Justifiez chaque entrée par une phrase.

Critère	Autoregression discrète	Diffusion latente	GAN	Flow Matching
FID typique (ImageNet 256 × 256)				
Coût inférence (étapes séquentielles)				
Parallélisme à la génération				
Contrôle textuel / édition				
Compatibilité avec un LLM pré-existant				
Stabilité de l'entraînement				

© 2026 Oussama Ghorbel

Indice : pour le FID, appuyez-vous sur les résultats publiés de VQGAN + transformer autoregressif (~6-8 FID), Stable Diffusion XL (~3-5 FID), StyleGAN2 (~2-3 FID sur FFHQ, ~8-10 sur ImageNet), et Flow Matching (~4-6 FID). Les valeurs exactes importent moins que les ordres de grandeur et les tendances.

Exercice 4.2 — Estimation FLOPs : autoregression vs diffusion latente

On souhaite générer une image 256 × 256.

Cas A — Autoregression discrète. Le tokenizer produit une grille de 32 × 32 = 1 024 tokens. Le modèle est un transformer de 7 milliards de paramètres (approximation : 2 × P FLOPs par token en inférence, soit 14 GFLOPs par token). Calculez le nombre total de FLOPs pour générer les 1 024 tokens séquentiellement.

Cas B — Diffusion latente. Le latent a une taille de 32 × 32 × 4. Le réseau de débruitage est un U-Net de 900 millions de paramètres. On effectue 50 étapes de débruitage. Chaque étape traite l'ensemble du latent en une passe (1,8 GFLOPs par étape). Calculez le nombre total de FLOPs.

Comparez les deux résultats et discutez : dans quel régime l'autoregression devient-elle compétitive (indice : pensez à la génération par blocs ou à la parallélisation semi-autoregressive) ?

Exercice 4.3 — Mémo d'une page

Rédigez un mémo d'une page (environ 400 mots) intitulé : *« Pourquoi adapter un LLM en générateur d'images plutôt que partir d'une architecture diffusion ? »*

Votre mémo doit :

- identifier au moins trois arguments techniques en faveur de l'adaptation d'un LLM ;
- reconnaître honnêtement au moins deux limites de cette approche ;
- proposer une stratégie d'atténuation pour chaque limite identifiée ;
- conclure par une recommandation contextuelle (« si votre priorité est X, choisissez Y »).

© 2026 Oussama Ghorbel

Ce mémo doit être rédigé comme si vous deviez convaincre un comité technique qui a déjà investi dans un pipeline Stable Diffusion. L'objectif n'est pas de dénigrer la diffusion, mais de montrer que, dans certains contextes organisationnels et techniques, l'autoregression sur tokens est le choix rationnel.

Pour aller plus loin

Quelques repères bibliographiques pour approfondir chaque famille abordée dans ce chapitre, sans prétention d'exhaustivité :

• Sur l'autoregression visuelle : Esser et al., « Taming Transformers for High-Resolution Image Synthesis » (CVPR 2021), qui introduit VQGAN comme tokenizer de référence pour la génération autoregressive ; Yu et al., « Vector-quantized Image Modeling with Improved VQGAN » (2022), qui affine la qualité du codebook.

• Sur la diffusion : Ho et al., « Denoising Diffusion Probabilistic Models » (NeurIPS 2020) pour la formulation fondatrice ; Rombach et al., « High-Resolution Image Synthesis with Latent Diffusion Models » (CVPR 2022) pour le passage en espace latent ; Peebles et Xie, « Scalable Diffusion Models with Transformers » (ICCV 2023) pour DiT.

• Sur les GAN : Karras et al., « Alias-Free Generative Adversarial Networks » (NeurIPS 2021) pour StyleGAN3 et la question de l'équivariance.

• Sur les flows : Lipman et al., « Flow Matching for Generative Modeling » (ICLR 2023) pour le cadre unifié du flow matching.

• Sur les hybrides : Li et al., « Autoregressive Image Generation using Residual Quantization » (2024) pour MAR ; Meta AI, « Chameleon: Mixed-Modal Early-Fusion Foundation Models » (2024) pour l'unification totale par tokens discrets.

En résumé

Ce chapitre a dressé un état des lieux nécessaire avant d'entrer dans le vif du sujet. Quatre grandes familles — autoregression, diffusion, GAN, flows — se partagent le terrain de la génération d'images, chacune

© 2026 Oussama Ghorbel

avec ses forces structurelles et ses angles morts. Les hybrides récents brouillent les frontières et montrent que l'avenir n'est probablement pas dans le choix exclusif d'une famille, mais dans leur articulation intelligente.

Pour l'ingénieur ou le chercheur qui tient entre ses mains un LLM pré-entraîné et qui souhaite lui conférer la capacité de produire des images, la conclusion de ce panorama est claire : l'autoregression sur tokens discrets est la voie qui préserve l'investissement existant, qui minimise la complexité architecturale ajoutée, et qui ouvre la porte à une multimodalité véritablement unifiée. Ce n'est pas la voie qui produit aujourd'hui les pixels les plus parfaits — la diffusion latente conserve une avance sur ce plan — mais c'est celle qui s'inscrit dans la continuité naturelle du paradigme « prédire le prochain token ».

La question qui se pose alors n'est plus *si* l'on doit tokeniser l'image, mais *comment*. Comment passer du signal continu, riche, redondant qu'est une image de 256 × 256 pixels à une séquence discrète de 1 024 indices que le LLM peut traiter comme n'importe quelle autre séquence de tokens ? C'est précisément l'objet de la Partie II qui s'ouvre : la tokenisation visuelle, du continu au discret. Nous commencerons par les auto-encodeurs et les VAE, ces architectures qui apprennent à compresser l'image dans un espace latent de dimension réduite, avant d'introduire la quantification vectorielle qui transforme ce latent en un véritable alphabet visuel.

Le pixel va devenir token. Et c'est là que tout commence.

© 2026 Oussama Ghorbel

Partie II — Tokenisation Visuelle : du Continu au Discret

© 2026 Oussama Ghorbel

Chapitre 5 – Auto-encodeurs et VAE : compresser l'image avant de la tokeniser

Avant de demander à un modèle de langage de « parler en pixels », il faut résoudre un problème d'échelle. Une image 256 × 256 en RGB contient 196 608 valeurs ; une image 1024 × 1024 en contient près de trois millions. Aucun transformer autorégressif ne peut traiter directement une séquence de cette longueur dans un budget computationnel raisonnable. La solution que la communauté a adoptée — et que j'explore tout au long de cet ouvrage — consiste à apprendre d'abord une représentation compacte de l'image, un *espace latent*, puis à tokeniser cet espace. Ce chapitre pose la première brique : l'auto-encodeur, sa variante variationnelle, et surtout le VQ-VAE qui transforme le continu en discret.

5.1 — Auto-encodeur convolutif : encodeur, goulot, décodeur, perte de reconstruction

L'auto-encodeur (AE) est l'architecture la plus ancienne de compression apprise. Le principe tient en une phrase : on force un réseau à reconstruire son entrée en la faisant transiter par un goulot d'étranglement (*bottleneck*) dont la dimension est très inférieure à celle de l'entrée.

Formellement, pour une image $x \in \mathbb{R}^{H \times W \times C}$, on définit :

- un **encodeur** $E_\phi : \mathbb{R}^{H \times W \times C} \to \mathbb{R}^{h \times w \times d}$, typiquement un empilement de convolutions avec stride 2 suivies de normalisations de groupe et d'activations Swish ;
- un **goulot** : la représentation $z = E_\phi(x)$, de taille $h \times w \times d$ avec $h \ll H$ et $w \ll W$;
- un **décodeur** $D_\theta : \mathbb{R}^{h \times w \times d} \to \mathbb{R}^{H \times W \times C}$, symétrique de l'encodeur, utilisant des convolutions transposées ou de l'upsampling par interpolation suivi de convolution.

La fonction de perte est une simple distance de reconstruction :

$$\mathcal{L}_{\text{rec}} = \mathbb{E}_{x \sim p_{\text{data}}} \left[\| x - D_\theta(E_\phi(x)) \|_2^2 \right]$$

En pratique, il est courant d'ajouter un terme perceptuel (distance dans l'espace de features d'un réseau pré-entraîné type VGG) et un discrimininateur patch pour éviter les reconstructions floues. Mais à ce stade du chapitre, restons sur la formulation de base.

Le point crucial pour la suite est le **taux de compression**. Si l'encodeur effectue trois downsamplings successifs de facteur 2, une image 256×256 devient un tenseur $32 \times 32 \times d$. Le rapport spatial est $256/32 = 8$, soit une compression ×8. Chaque position du goulot devra « résumer » un patch 8×8 de l'image originale. C'est ce goulot qui, une fois discrétisé, fournira les tokens visuels au LLM.

Un piège classique : un AE purement convolutif sans contrainte sur la distribution de z produit un espace latent non structuré. Interpoler entre deux points de cet espace donne souvent des résultats incohérents. C'est précisément la limitation que le VAE cherche à corriger.

5.2 — VAE : borne ELBO, régularisation KL, espace latent continu

L'auto-encodeur variationnel (VAE), introduit par Kingma et Welling en 2013, reformule le problème en termes probabilistes. L'encodeur ne produit plus un vecteur z déterministe, mais les paramètres d'une distribution $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma^2_\phi(x) I)$. Le décodeur modélise la vraisemblance $p_\theta(x|z)$.

L'objectif est de maximiser la log-vraisemblance marginale $\log p_\theta(x)$, intraitable directement. On la borne par l'**ELBO** (Evidence Lower BOund) :

$$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{terme de reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z|x) \| p(z))}_{\text{terme de régularisation}}$$

avec $p(z) = \mathcal{N}(0, I)$ la prior. La divergence KL se calcule analytiquement pour deux gaussiennes diagonales :

$$D_{\mathrm{KL}} = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

Cette régularisation force l'espace latent à ressembler à une gaussienne isotrope. Conséquence : on peut échantillonner $z \sim \mathcal{N}(0, I)$, passer par le décodeur, et obtenir une image plausible. L'espace est **continu** et **interpolable**.

© 2026 Oussama Ghorbel

Pour l'échantillonnage différentiable, on utilise la reparamétrisation : $z = \mu + \sigma \odot \varepsilon$, avec $\varepsilon \sim \mathcal{N}(0, I)$.

Le VAE est élégant, mais il présente deux limites pour notre objectif de tokenisation :

1. La reconstruction reste légèrement floue car la vraisemblance gaussienne pénalise les écarts quadratiques sans encourager la netteté.
2. L'espace latent est continu ; or un LLM travaille sur un vocabulaire **discret**. Il faut un mécanisme supplémentaire pour convertir z en indices entiers.

C'est exactement ce que propose le VQ-VAE.

5.3 — VQ-VAE (van den Oord et al., 2017) : quantification vectorielle, straight-through estimator

Le VQ-VAE, publié par van den Oord, Vinyals et Kavukcuoglu en 2017, remplace le goulot continu par un **codebook** : un ensemble fini de vecteurs $\{e_k\}_{k=1}^{K}$, avec $e_k \in \mathbb{R}^d$. La quantification consiste, pour chaque position spatiale (i, j) du goulot, à sélectionner l'entrée du codebook la plus proche :

$$z_q(i,j) = e_k \quad \text{avec} \quad k = \arg\min_{k'} \| z_e(i,j) - e_{k'} \|_2$$

où $z_e(i, j)$ est la sortie de l'encodeur à la position (i, j).

Le problème est que l'opération $\arg\min$ n'est pas différentiable. La solution adoptée est le **straight-through estimator** (STE) : lors de la passe avant, le décodeur reçoit z_q ; lors de la passe arrière, le gradient de la perte de reconstruction est copié directement dans z_e, comme si la quantification n'existait pas :

$$\frac{\partial \mathcal{L}}{\partial z_e} \approx \frac{\partial \mathcal{L}}{\partial z_q}$$

Cette approximation est remarquablement efficace en pratique. Le codebook, lui, est mis à jour soit par descente de gradient (nous y revenons en 5.4), soit par une moyenne mobile exponentielle.

Le VQ-VAE produit donc une grille d'indices entiers $k(i, j) \in \{1, \ldots, K\}$. Pour une image 256 × 256 avec un taux de

© 2026 Oussama Ghorbel

compression spatial ×8 et un codebook de taille $K = 1024$, on obtient une séquence de $32 \times 32 = 1024$ tokens, chacun appartenant à un vocabulaire de 1024 symboles. Cette séquence est directement injectable dans un transformer autorégressif. C'est le pont entre le pixel et le token.

5.4 — Fonction de perte VQ : commitment loss, codebook loss, EMA updates

La perte totale du VQ-VAE comporte trois termes. Notons z_e la sortie de l'encodeur, z_q le vecteur quantifié, et $\text{sg}[\cdot]$ l'opérateur *stop-gradient*.

$$\mathcal{L} = \mathcal{L}_{\text{rec reconstruction}} + \underbrace{\|\text{sg}[z_e] - e_k\|_2^2}_{\text{codebook loss}} + \beta \underbrace{\|z_e - \text{sg}[e_k]\|_2^2}_{\text{commitment loss}}$$

- **Reconstruction** : $\|x - D_\theta(z_q)\|^2$ (ou une combinaison L1 + perceptuelle + adversariale dans les variantes récentes).
- **Codebook loss** : pousse les entrées du codebook vers les sorties de l'encodeur. Le stop-gradient sur z_e signifie que ce terme ne met à jour que les e_k.
- **Commitment loss** : empêche la sortie de l'encodeur de « sauter » d'une entrée du codebook à l'autre à chaque itération. Le coefficient β est typiquement fixé à 0,25.

En pratique, je recommande de remplacer la mise à jour par gradient du codebook par une **moyenne mobile exponentielle (EMA)**. Pour chaque entrée e_k, on maintient un compteur N_k (nombre de vecteurs assignés) et une somme S_k :

$$N_k \leftarrow \gamma N_k + (1-\gamma) n_k$$

$$S_k \leftarrow \gamma S_k + (1-\gamma) \sum_{i \,:\, k_i = k} z_e(i)$$

$$e_k \leftarrow \frac{S_k}{N_k}$$

avec $\gamma = 0{,}99$ en général. L'EMA stabilise l'entraînement et évite les oscillations du codebook.

Un problème récurrent est la **sous-utilisation du codebook** : certaines entrées ne sont jamais sélectionnées et deviennent des « entrées mortes ». Deux parades simples : réinitialiser périodiquement les entrées inutilisées en les remplaçant par des échantillons aléatoires de z_e ; ou

© 2026 Oussama Ghorbel

ajouter un léger bruit à z_e avant la quantification pour encourager l'exploration.

5.5 — Choix de la dimension latente et du taux de compression (×4, ×8, ×16)

Le taux de compression spatial est le paramètre le plus structurant pour la suite. Il détermine la longueur de la séquence de tokens et donc le coût quadratique de l'attention dans le LLM.

Taux	Taille latente (256×256)	Nb de tokens	Complexité attention (n^2)
×4	64 × 64	4 096	×16
×8	32 × 32	1 024	×1
×16	16 × 16	256	×1/16

Un taux ×4 conserve beaucoup de détails mais produit une séquence longue, difficile à traiter par un LLM de taille modérée. Un taux ×16 réduit drastiquement la séquence mais perd des textures fines ; il faut alors un décodeur très puissant ou une seconde passe de super-résolution.

Le compromis que j'adopte dans les protocoles de la Partie VI est un taux **×8** avec un codebook de 1 024 à 8 192 entrées. C'est le choix de VQGAN (Esser et al., 2021) et de la plupart des travaux récents sur la génération autorégressive d'images.

La dimension d de chaque vecteur du codebook est généralement comprise entre 128 et 512. Une dimension trop faible limite la capacité expressive ; trop élevée, elle augmente le coût de la recherche du plus proche voisin. En pratique, $d = 256$ avec $K = 1024$ offre un bon équilibre pour des images naturelles 256 × 256.

Un point souvent négligé : le taux de compression doit être choisi **en fonction de la tâche aval**. Si le LLM doit générer des images destinées à être lues par un humain, un taux ×8 suffit. S'il s'agit de produire des images pour un pipeline de segmentation ou de détection, un taux ×4 peut être nécessaire pour préserver les contours fins.

© 2026 Oussama Ghorbel

5.6 — Reconstruction et artefacts : mesurer la fidélité du tokenizer

Avant de brancher un LLM sur le tokenizer, il faut vérifier que celui-ci reconstruit fidèlement les images. Plusieurs indicateurs complémentaires :

- **PSNR** (Peak Signal-to-Noise Ratio) : mesure pixel-à-pixel. Un VQ-VAE bien entraîné sur 256 × 256 atteint typiquement 28-32 dB. En dessous de 25 dB, les artefacts sont visibles.
- **SSIM** : capture la structure locale. Au-delà de 0,90, la reconstruction est perçue comme proche de l'original.
- **LPIPS** (Learned Perceptual Image Patch Similarity) : distance dans un espace de features profondes. Plus robuste que le PSNR pour juger de la qualité perçue.
- **rFID** (reconstruction FID) : FID calculé entre les images originales et leurs reconstructions. Un rFID < 2 indique un tokenizer quasi transparent.

Au-delà des chiffres, il est indispensable d'inspecter visuellement les reconstructions sur des images contenant : du texte (lettres, panneaux), des textures périodiques (tissus, grillages), des visages, des zones uniformes (ciel). Les artefacts typiques d'un VQ-VAE mal entraîné sont :

- un effet « mosaïque » aux frontières des patches (dû à une quantification trop agressive) ;
- un flou global (codebook trop petit ou commitment loss trop élevé) ;
- des couleurs délavées (absence de terme perceptuel dans la perte) ;
- des hallucinations localisées (entrées mortes du codebook réutilisées de manière incohérente).

Un diagnostic utile : tracer la distribution des indices du codebook sur un batch de validation. Une distribution uniforme indique une bonne utilisation ; une distribution très piquée signale que le codebook est trop grand ou que l'entraînement est insuffisant.

© 2026 Oussama Ghorbel

Enfin, une règle fondamentale : **le tokenizer est la fondation**. Si la reconstruction est mauvaise, aucun LLM, aussi puissant soit-il, ne produira des images de qualité, car il sera contraint de générer des tokens qui, une fois décodés, donneront des artefacts. Investissez du temps dans cette étape.

Exercices

Exercice 1 — Entraîner un VQ-VAE sur CIFAR-10.

Implémentez un VQ-VAE convolutif avec un encodeur à trois blocs de downsampling (stride 2), un codebook de $K = 512$ entrées de dimension $d = 64$, et un décodeur symétrique. Entraînez sur CIFAR-10 (32 × 32, RGB) pendant 50 époques avec l'optimiseur AdamW ($\eta = 3 \times 10^{-4}$, $\beta_2 = 0{,}999$), commitment loss $\beta = 0{,}25$, et EMA $\gamma = 0{,}99$ sur le codebook. À convergence, calculez le PSNR moyen de reconstruction sur l'ensemble de test. Objectif : PSNR >= 30 dB.

Exercice 2 — Comparaison des taux de compression.

Reprenez l'architecture précédente et entraînez trois variantes avec des downsamplings donnant des taux ×4, ×8 et ×16 (sur des images 128 × 128 pour que ×16 reste pertinent : latents 32 × 32, 16 × 16, 8 × 8). Reconstruisez un jeu de 20 images de validation et comparez visuellement. Mesurez PSNR, SSIM et LPIPS pour chaque taux. Commentez le trade-off fidélité / longueur de séquence.

Exercice 3 — Courbe d'utilisation du codebook.

Après l'entraînement de l'exercice 1, parcourez l'ensemble de test et comptez, pour chaque entrée $k \in \{1, \ldots, 512\}$, le nombre de fois où elle est sélectionnée. Tracez l'histogramme des fréquences (entrées triées par fréquence décroissante). Identifiez les entrées dont la fréquence est nulle (« entrées mortes »). Calculez le pourcentage d'entrées effectivement utilisées. Proposez une stratégie de réinitialisation et vérifiez son effet sur le PSNR après 10 époques supplémentaires.

Pour clore ce chapitre : du continu au discret, un changement de paradigme

© 2026 Oussama Ghorbel

Ce que nous venons de construire en six sections constitue bien plus qu'un simple prétraitement. L'auto-encodeur nous a donné le principe de compression ; le VAE nous a offert un espace latent continu, interpolable, probabiliste ; le VQ-VAE a opéré la bascule décisive vers le discret. À chaque étape, nous avons perdu un peu de généralité pour gagner en compatibilité avec le monde des transformers et du next-token prediction.

Cette trajectoire — du signal brut vers un vocabulaire fini de symboles visuels — reproduit, en un sens, ce que la linguistique fait depuis des millénaires : découper un continuum acoustique ou scriptural en unités discrètes, en phonèmes, en morphèmes, en mots. Le codebook d'un VQ-VAE joue exactement le rôle d'un alphabet : un ensemble fini de « lettres » visuelles dont la combinaison engendre la diversité des images. La différence fondamentale, c'est que cet alphabet n'est pas fixé par convention culturelle ; il est appris par descente de gradient sur des millions d'images, et sa structure reflète les régularités statistiques du corpus.

Il reste toutefois une tension que nous n'avons pas encore résolue. Le VQ-VAE, dans sa formulation originale, produit des reconstructions correctes mais manquant souvent de piqué et de cohérence texturale à haute résolution. Le décodeur, contraint par une perte principalement pixel-à-pixel, a tendance à moyenner les solutions plausibles plutôt qu'à choisir la plus nette. Pour dépasser cette limite, il faut enrichir la fonction de perte d'un terme adversarial et d'une régularisation perceptuelle, et surtout améliorer la qualité du codebook lui-même en couplant son apprentissage à celui d'un discriminateur. C'est précisément l'objet du chapitre suivant, où nous passerons du VQ-VAE au VQGAN et aux tokenizers d'images haute fidélité.

Avant de tourner la page, je vous invite à relire les trois exercices en gardant à l'esprit une question qui traversera tout le reste de l'ouvrage : *combien de tokens mon LLM devra-t-il prédire pour générer une image acceptable ?* La réponse dépend directement du taux de compression que vous aurez choisi ici, de la taille du codebook, et de la qualité de reconstruction que vous aurez validée. Chaque décimale de PSNR gagnée à ce stade se traduira, en aval, par une réduction de la charge cognitive imposée au modèle de langage. Le tokenizer est la fondation ;

© 2026 Oussama Ghorbel

tout ce que nous construirons dessus — l'ordre de balayage, le conditionnement textuel, les protocoles d'entraînement — reposera sur la solidité de cette première compression.

Une dernière remarque méthodologique. Dans la pratique, la tentation est grande de passer rapidement cette étape pour « arriver au LLM ». C'est une erreur coûteuse. Un tokenizer médiocre ne se rattrape pas en aval : on peut augmenter la taille du transformer, allonger l'entraînement, multiplier les données textuelles ; si le décodeur introduit systématiquement des artefacts de mosaïque ou un flou chromatique, le générateur final héritera de ces défauts. J'encourage donc le lecteur à consacrer au moins une semaine de travail expérimental à cette seule étape, en variant le codebook, le taux de compression, les termes de perte, avant de passer à la modélisation autorégressive proprement dite.

Le pixel est désormais un token. Il nous reste à apprendre à parler ce nouveau langage.

5.7 — Ce que le LLM apprend vraiment : les statistiques du langage visuel

« Parler en tokens » n'est pas une simple métaphore. C'est une hypothèse de travail : celle selon laquelle la séquence de codes produite par le tokenizer possède une structure statistique qu'un modèle autorégressif peut exploiter. Rien, dans la perte du VQ-VAE, ne garantit cette propriété. L'encodeur est entraîné à compresser, le décodeur à reconstruire ; personne n'a demandé au couple de produire un langage facile à prédire. Cette tension, souvent sous-estimée, mérite une section à part.

Un code n'est pas un mot. Lorsqu'un LLM textuel manipule le jeton `chat`, il manipule une unité dont la stabilité sémantique est garantie par la convention linguistique. Rien de tel dans un codebook. L'entrée e_{371} ne désigne pas « un œil » ou « une feuille ». Elle désigne une région d'un espace vectoriel appris, une sorte de texture élémentaire ou de motif local dont l'interprétation dépend du contexte spatial. La même entrée peut apparaître dans le pelage d'un animal, dans le feuillage d'un arbre et dans une ombre portée. Il n'existe pas de dictionnaire reliant les indices aux objets.

© 2026 Oussama Ghorbel

Cette absence de sémantique intrinsèque a une conséquence directe sur la partie IV de cet ouvrage : l'alignement texte-image ne pourra pas fonctionner par correspondance de concepts entre deux vocabulaires déjà chargés de sens. Le LLM devra apprendre statistiquement que certaine configuration de tokens visuels co-occupe avec certaine formulation textuelle. Plus le codebook est expressif et fin, plus cette correspondance est diffuse — et plus le corpus d'apprentissage doit être vaste.

Combien d'information contient une image quantifiée ? Faisons le calcul une fois pour toutes, sur un cas concret. Une image 256 × 256 en RGB 8 bits occupe $256 \times 256 \times 3 \times 8 = 1\,572\,864$ bits. Avec un compresseur spatial ×8 et un codebook de $K = 1024$ entrées, chaque position latente s'écrit sur $\log_2 1024 = 10$ bits, et la grille 32×32 sur 10 240 bits :

Taux spatial	Grille latente	Tokens	Bits théoriques	Bits par pixel	Rapport
×4	64 × 64	4 096	40 960	0,625	38,4
×8	32 × 32	1 024	10 240	0,156	153,6
×16	16 × 16	256	2 560	0,039	614,4

Deux précautions s'imposent. D'abord, ces chiffres supposent des codes équiprobables : ils représentent un plafond théorique, pas le contenu informationnel réel. Ensuite, ils ignorent le coût fixe du codebook. Avec $K = 1024$ et $d = 256$ stockés en demi-précision, le codebook occupe 524 288 octets — soit beaucoup plus que les 1 280 octets d'une image quantifiée. Ce coût s'amortit sur l'ensemble du corpus et ne se paie qu'une fois. En génération, la ressource critique n'est pas la mémoire de stockage : c'est la longueur de la séquence que le transformer doit parcourir, token après token.

Mesurer la richesse effective du vocabulaire. Plutôt que de raisonner sur K, mesurez l'entropie empirique de la distribution des codes sur un ensemble de validation :

$$H = -\sum_{k=1}^{K} p(k) \log_2 p(k)$$

© 2026 Oussama Ghorbel

La quantité 2^H est ce que j'appelle le *vocabulaire effectif*. Si $K = 1024$ mais que $H = 8{,}3$ bits, votre alphabet ne comporte en pratique qu'environ 315 symboles utiles. Les 709 autres contribuent peu à la représentation. Ce constat n'est pas nécessairement un échec : il peut signaler que la complexité du corpus ne justifie pas un codebook plus large. Mais il doit être connu avant de choisir la taille de la tête de sortie du LLM, dont le coût mémoire et calculatoire est proportionnel à K.

Ici, une nuance s'impose sur le diagnostic évoqué en 5.6. Une distribution de fréquences uniforme n'est pas un objectif en soi. Les images naturelles ont des statistiques déséquilibrées : certains motifs élémentaires sont intrinsèquement plus fréquents que d'autres. Qu'un code décrive le ciel et apparaisse deux fois plus souvent qu'un code décrivant une bordure contraste n'a rien d'anormal. Le signal d'alarme est une combinaison : entropie faible, entrées mortes nombreuses *et* reconstruction médiocre. C'est là que le codebook sous-explore l'espace latent.

Un bon reconstructeur peut être un mauvais langage. Voici le point que je voudrais ancrer. Deux tokenizers peuvent afficher exactement le même rFID et conduire à des générateurs autorégressifs de qualité très différente. Le premier a appris à er l'image en motifs redondants, corrélés spatialement, faciles à anticiper. Le second a appris à er le résidu haute fréquence, c'est-à-dire essentiellement de l'entropie : chaque token est imprévisible à partir des précédents, mais leur assemblage reconstruit parfaitement l'image. Le deuxième est un meilleur compresseur. Il est catastrophique comme base de génération.

La perte de reconstruction pousse naturellement vers ce second extrême. Plus le codebook est riche, plus l'encodeur peut déléguer l'information fine à des codes individuels peu structurés. Or le LLM, lui, a besoin de régularité.

Je recommande donc d'adjoindre aux métriques de reconstruction un **diagnostic de prédictibilité**, en trois étapes :

1. Calculez l'entropie unigramme $H(k)$ sur les tokens d'entraînement. C'est l'entropie de croisement d'un modèle qui ignore totalement le contexte.

© 2026 Oussama Ghorbel

2. Entraînez un modèle bigramme — une table $K \times K$ lissée, coûtant à peine un million de cases pour $K = 1024$ — et mesurez l'entropie conditionnelle $H(k_t \mid k_{t-1})$ dans l'ordre de balayage retenu. Ce baseline, peu coûteux, est trop souvent absent des évaluations.

3. Entraînez un petit transformer autorégressif, quatre à six couches suffisent, sur une centaine de milliers d'images tokenisées. Mesurez la négative log-vraisemblance par token sur validation.

Comparez ensuite vos candidats tokenizers **à rFID égal**. Celui dont le petit transformer obtient la meilleure vraisemblance est celui qui offrira le langage le plus structuré au grand modèle. Si le gain du transformer sur le bigramme est faible, méfiez-vous : avant d'augmenter la taille du LLM, vérifiez que le problème ne vient pas de la séquence elle-même.

Ce diagnostic n'est pas une excentricité méthodologique. Il formalise une idée qui traversera tout le livre : on n'évalue pas un composant isolément, on évalue ce qu'il permet au système aval de faire.

5.8 — Checklist de validation avant de brancher le LLM

Avant de passer à la modélisation autorégressive, il est crucial de soumettre le tokenizer à une série de vérifications. Elles prennent une journée et évitent des semaines de debugging sur un modèle de génération qui ne convergera jamais pour de mauvaises raisons.

• **Fidélité globale.** PSNR, SSIM, LPIPS et rFID sur l'ensemble de validation, et non sur l'entraînement. Un écart important entre les deux signale un sur-apprentissage du décodeur.

• **Fidélité ciblée.** Un panneau de tests contenant du texte, des visages, des grillages, des tissus rayés et des régions uniformes. Les moyennes masquent les échecs localisés.

• **Santé du codebook.** Histogramme des fréquences, nombre d'entrées mortes, entropie empirique et vocabulaire effectif. Conservez ce relevé : il servira de référence pour les ablations du chapitre 21.

• **Stabilité entre graines.** Deux entraînements avec des initialisations différentes produiront des codebooks différents — les indices ne sont pas alignables d'un run à l'autre — mais doivent

© 2026 Oussama Ghorbel

produire des métriques de reconstruction comparables. Une variance importante indique un entraînement fragile.

• **Prédictibilité.** Le petit transformer doit battre nettement le baseline unigramme, et idéalement le baseline bigramme. Sinon, interrogez le tokenizer avant le LLM.

• **Budget de séquence.** Longueur latente plus longueur du prompt texte, confrontée à la fenêtre de contexte disponible. Le compte doit être fait avant l'entraînement, pas après.

• **Traçabilité du prétraitement.** Versionnez avec le checkpoint du tokenizer la normalisation des pixels (`[0, 1]` ou `[-1, 1]`), l'ordre des canaux, la taille de redimensionnement, l'interpolation utilisée et l'ordre de balayage retenu. Ces détails innocents sont une source classique de bugs silencieux : un LLM entraîné sur des tokens produits avec une normalisation et décodé avec une autre produira des images catastrophiques sans lever la moindre erreur.

Ce dernier point mérite d'être souligné. Un tokenizer n'est pas seulement un triplet (encodeur, codebook, décodeur). C'est un *protocole complet*, dont les conventions de pré et post-traitement font partie intégrante. Traitez-le comme une dépendance logicielle versionnée, avec un numéro de révision et un test de non-régression.

Il reste une limite que aucune de ces précautions ne lève. Le VQ-VAE, même bien entraîné, même doté d'un codebook pleinement utilisé et d'une entropie bien équilibrée, produit des reconstructions qui manquent parfois de piqué. Le décodeur optimise une attente : face à plusieurs interprétations plausibles d'un code, il tend à les moyenner. Cette moyenne est statistiquement raisonnable et visuellement décevante.

Nous pourrions pousser le VQ-VAE plus loin — agrandir le codebook, affiner l'EMA, augmenter la dimension latente — et gagner quelques dixièmes de décibel. Mais nous atteindrions alors un plafond architectural, et non un problème d'hyperparamètres. Pour le franchir, il faut changer ce que signifie « bien reconstruire » : ne plus seulement exiger que les pixels soient proches, mais que l'image *ressemble* à une image réelle.

© 2026 Oussama Ghorbel

C'est le saut que nous ferons au chapitre suivant, avec le VQGAN et les tokenizers haute fidélité. Pour l'instant, retenez ceci : avant de demander à un modèle de prédire le prochain token, assurez-vous que les tokens forment une langue — et non un bruit soigneusement compressé.

© 2026 Oussama Ghorbel

Chapitre 6 – VQGAN et au-delà : tokenizers d'images haute fidélité

Au chapitre précédent, nous avons vu comment les auto-encodeurs variationnels compressent une image dans un espace latent continu. Cette compression, aussi élégante soit-elle, pose un problème fondamental pour notre projet : un LLM manipule des tokens discrets. Or, un espace latent continu n'offre pas, en soi, de vocabulaire fini sur lequel poser une distribution catégorielle. La quantification vectorielle (VQ) résout ce problème en projetant chaque vecteur latent sur le code le plus proche d'un codebook appris. Mais la VQ « nue », introduite par van den Oord et ses collègues en 2017, produit des reconstructions visiblement floues dès que le taux de compression augmente.

C'est ici qu'intervient VQGAN, puis toute une lignée de tokenizers modernes qui repoussent les limites de la fidélité à haut facteur de compression. Dans ce chapitre, je dissèque ces architectures une à une, en m'attachant à ce qui compte pour l'ingénieur : les choix de pertes, les compromis d'architecture, et l'impact concret sur la génération en aval.

6.1 — VQGAN (Esser et al., 2021) : adversarial loss + perceptual loss sur le décodeur

L'article fondateur de Patrick Esser, Robin Rombach et Björn Ommer (2021) part d'un constat simple : si l'on entraîne un VQ-VAE uniquement avec une reconstruction pixel-à-pixel (L1 ou L2) et une perte de commitment sur le codebook, le décodeur apprend à minimiser l'erreur moyenne. Or, l'erreur moyenne sur les pixels produit du flou, car elle récompense la prédiction de la moyenne conditionnelle plutôt qu'une réalisation nette et plausible.

VQGAN corrige ce défaut en ajoutant deux termes au décodeur :

1. **Une perte adversariale.** Un discriminateur D est entraîné conjointement pour distinguer les images réelles des reconstructions. Le décodeur reçoit un gradient qui le pousse à produire des textures crédibles, des contours nets, des détails haute fréquence. Concrètement, Esser et al. utilisent un discriminateur patch-based

© 2026 Oussama Ghorbel

(voir section 6.2) et une hinge loss.

2. **Une perte perceptive (LPIPS).** Plutôt que de comparer les pixels bruts, on compare les activations intermédiaires d'un réseau pré-entraîné (VGG-16 ou ResNet). Cette perte capture la similarité structurelle et sémantique : deux images qui diffèrent par une translation d'un pixel auront une LPIPS faible, alors qu'une L2 les pénaliserait fortement.

La fonction objective du décodeur s'écrit alors :

$$\mathcal{L}_{\text{dec}} = \mathcal{L}_{\text{rec}} + \lambda_{\text{perc}}\mathcal{L}_{\text{LPIPS}} + \lambda_{\text{adv}}\mathcal{L}_{\text{GAN}}$$

où $\mathcal{L}_{\text{rec}}$ reste une L1 (ou L2) sur les pixels, λ_{perc} est typiquement fixé à 1, et λ_{adv} est adaptatif : Esser et al. proposent de le régler dynamiquement en fonction du ratio entre le gradient de la perte de reconstruction et celui de la perte adversariale, afin d'éviter qu'un terme domine l'autre en début d'entraînement.

Du côté du codebook, VQGAN conserve la quantification par plus proche voisin avec une perte de commitment et une mise à jour par moyenne mobile exponentielle (EMA). L'innovation n'est donc pas dans la quantification elle-même, mais dans le signal d'entraînement du décodeur. Le résultat est spectaculaire : à facteur de compression 256 (une image 256 × 256 × 3 devient une grille 16 × 16 de tokens), la reconstruction reste visuellement convaincante, avec des textures plausibles et peu d'artefacts de bloc.

6.2 — Architecture du discriminateur patch-based et rôle de la perte perceptive (LPIPS)

Le discriminateur de VQGAN s'inspire du PatchGAN introduit dans Pix2Pix. Plutôt que de classifier l'image entière comme réelle ou fausse, il opère sur des patches de taille N × N (souvent 70 × 70 en pratique). Chaque position spatiale de la carte de features du discriminateur produit un scalaire indiquant si le patch correspondant est réaliste.

Cette approche présente deux avantages :

• **Stabilité.** Le discriminateur ne doit pas modéliser la cohérence globale de l'image ; il se concentre sur la texture locale. L'entraînement est donc moins sujet au mode collapse.

© 2026 Oussama Ghorbel

• **Efficacité computationnelle.** La sortie est une carte 2D plutôt qu'un unique logit, ce qui fournit un gradient dense et spatially-aware au décodeur.

L'architecture typique est une convolution fully-convolutional à 4 ou 5 blocs, chacun composé d'une convolution stride-2, d'une normalisation d'instance et d'une LeakyReLU. Le dernier bloc produit une carte de logits de dimension H/16 × W/16.

La **perte LPIPS** (Learned Perceptual Image Patch Similarity), proposée par Zhang et al. (2018), joue un rôle complémentaire. Elle extrait les features de couches intermédiaires d'un VGG-16 pré-entraîné sur ImageNet, normalise ces features, puis calcule une distance pondérée. Les poids sont appris pour maximiser la corrélation avec les jugements humains de similarité.

Dans la pratique de VQGAN, la LPIPS agit comme un régularisateur structurel : elle empêche le discriminateur de « tricher » en se focalisant sur des artefacts de très haute fréquence, et elle ancre la reconstruction dans une géométrie sémantique correcte. On observe, en reproduisant ces expériences, que supprimer la LPIPS tout en conservant le GAN produit des images nettes mais structurellement incohérentes — des textures réalistes posées sur des formes erronées.

6.3 — Tokenizers continus (KL-regularized) vs tokenizers discrets : compromis

La question « discret ou continu ? » traverse toute la tokenisation visuelle. Récapitulons les deux familles :

Tokenizers continus (KL-VAE). L'encodeur produit une moyenne μ et une variance σ^2 par position spatiale. On échantillonne $z = \mu + \sigma \cdot \varepsilon$, avec $\varepsilon \sim \mathcal{N}(0, I)$. La régularisation KL pousse la distribution latente vers une gaussienne standard. Avantage : l'espace latent est lisse, interpolable, et se pr te naturellement à la diffusion. Inconvénient : il n'existe pas de vocabulaire fini ; un LLM ne peut pas « prédire le prochain token » dans un espace continu sans discrétisation supplémentaire.

Tokenizers discrets (VQ, VQGAN, LFQ). Chaque position est mappée sur un indice de codebook. Avantage : compatibilité native avec

© 2026 Oussama Ghorbel

les modèles de langage, distribution catégorielle bien définie, possibilité d'utiliser des techniques de NLP (BPE, masking, etc.). Inconvénient : la quantification introduit une perte d'information irréversible ; le codebook peut sous-utiliser ses entrées.

Le compromis se joue sur trois axes :

Critère	Continu	Discret
Compatibilité LLM	Faible (nécessite une tête de diffusion)	Native
Fidélité à compression égale	Légèrement supérieure (pas de perte de quantification)	Inférieure, mais l'écart se réduit avec les codebooks larges
Interpolation latente	Naturelle	Nécessite une interpolation dans l'espace des embeddings avant quantification
Contrôle génératif	Via guidage de diffusion	Via sampling catégoriel, beam search, nucleus

Dans le cadre de ce livre — adapter un LLM en générateur d'images —, le choix discret s'impose. Mais il est utile de garder en tête que la frontière s'estompe : certains travaux récents (section 6.5) combinent une régularisation KL légère avec une quantification, obtenant le meilleur des deux mondes.

6.4 — SD-VAE (Stable Diffusion) : compression ×8 dans un espace latent 4 × 64 × 64

Le VAE de Stable Diffusion (Rombach et al., 2022) mérite une section propre, car il illustre un tokenizer continu qui a atteint une qualité suffisante pour servir de socle à la génération. Son architecture est la suivante :

• **Encodeur.** Un réseau convolutionnel descend de 256 × 256 × 3 à 32 × 32 × 8, puis deux convolutions 1 × 1 produisent μ et $\log \sigma^2$ de dimension 32 × 32 × 4. Après échantillonnage, on obtient un tenseur latent $z \in \mathbb{R}^{4 \times 64 \times 64}$ (en comptant le facteur 2 de la reparamétrisation). Le facteur de compression spatial est donc 8 × 8 = 64, et le facteur total (en incluant la réduction de canaux de 3 à 4) est 64 × 3/4 = 48.

© 2026 Oussama Ghorbel

• **Décodeur.** Symétrique, il remonte de 4 × 64 × 64 à 3 × 256 × 256 via des blocs ResNet et de l'attention à 16 × 16.

• **Entraînement.** L1 + LPIPS + KL avec un poids β très faible ($\beta = 10^{-6}$), ce qui rend la régularisation quasi symbolique. L'espace latent est donc très peu contraint, ce qui maximise la fidélité de reconstruction au détriment de la régularité.

Pourquoi ce choix ? Parce que Stable Diffusion n'a pas besoin d'un latent gaussien parfait : le modèle de diffusion opère directement dans cet espace et apprend sa propre distribution. Le VAE n'est qu'un compresseur. Pour notre projet LLM, en revanche, un espace aussi peu contraint serait problématique : il faudrait quantifier un volume latent de dimension 4 × 64 × 64, soit 16 384 vecteurs de dimension 4. C'est faisable, mais les tokenizers discrets avec un facteur 16 (grille 16 × 16) offrent un meilleur compromis entre longueur de séquence et fidélité.

6.5 — Tokenizers récents : Cosmos Tokenizer, MAGVIT-v2, LFQ (Lookup-Free Quantization)

Le paysage des tokenizers a évolué rapidement entre 2023 et 2025. Trois contributions méritent une attention particulière.

MAGVIT-v2 (Yu et al., 2023). Ce travail introduit la **Lookup-Free Quantization (LFQ)**. L'idée est radicale : au lieu de maintenir un codebook de K vecteurs et de chercher le plus proche voisin, on quantifie chaque dimension du vecteur latent indépendamment en la projetant sur {-1, +1}. Un vecteur de dimension d produit ainsi 2^d codes possibles, sans aucun paramètre de codebook à apprendre. Pour $d = 18$, on obtient un vocabulaire de 262 144 codes — bien au-delà de ce qu'un codebook classique peut gérer sans sous-utilisation.

LFQ élimine trois problèmes classiques :

• Le **codebook collapse** (seule une fraction des codes est utilisée).

• La **complexité en O(K × d)** de la recherche du plus proche voisin.

• Le besoin d'**EMA updates** et de **codebook reset**.

La contrepartie est une contrainte géométrique plus forte : les codes sont les sommets d'un hypercube, ce qui limite l'expressivité par rapport à un codebook librement positionné. En pratique, MAGVIT-v2

© 2026 Oussama Ghorbel

compense en augmentant la dimension latente et en ajoutant une régularisation par entropie qui encourage l'utilisation uniforme des 2^d codes.

Cosmos Tokenizer (NVIDIA, 2024). Conçu pour la vidéo et l'image haute résolution, ce tokenizer atteint un facteur de compression 8 × 16 × 16 (temporel × spatial) avec une architecture causale qui permet le streaming. Il utilise une quantification discrète avec un codebook de 8 192 entrées et intègre un discriminateur 3D pour la perte adversariale. Sa particularité est la **causalité temporelle** : chaque frame n'attend que les frames passées, ce qui permet une génération vidéo incrémentale.

Autres avancées notables. On peut citer les tokenizers à quantification résiduelle (RQ-VAE, Lee et al., 2022), qui empilent plusieurs niveaux de VQ pour capturer l'information du grossier au fin, et les approches à quantification finite-scalar (FSQ), qui généralisent LFQ en autorisant plus de deux niveaux par dimension (par exemple {-2, -1, 0, 1, 2} par dimension).

6.6 — Impact du choix du tokenizer sur la génération en aval : FID du reconstructeur comme plafond

Voici un point que je considère comme crucial et trop souvent sous-estimé : **la qualité de génération d'un modèle en aval ne peut jamais dépasser la qualité de reconstruction du tokenizer.** Si votre VQGAN reconstruit avec un FID de 4,2 sur ImageNet 256 × 256, alors même un générateur parfait — qui prédirait la séquence de tokens exacte de l'image cible — produirait des images dont le FID ne descendrait pas sous 4,2. Le tokenizer constitue un plafond de verre.

Ce plafond a trois implications pratiques :

1. **Choisir le tokenizer avant le modèle génératif.** Si votre objectif est un FID < 5, inutile d'entraîner un LLM de 7 milliards de paramètres sur un tokenizer qui plafonne à 12. Investissez d'abord dans la qualité du reconstructeur.

2. **Mesurer le FID de reconstruction systématiquement.** Avant toute expérience de génération, encodez puis décodez l'ensemble de test. Calculez le FID entre les originaux et les reconstructions. Ce chiffre est votre lower bound. On voit trop de projets attribuer au

© 2026 Oussama Ghorbel

modèle génératif une dégradation qui provenait en réalité du tokenizer.

3. **Adapter le facteur de compression à la tâche.** Un facteur 256 (grille 16 × 16) convient pour de la génération 256 × 256 avec un budget de séquence court. Pour de la haute résolution (512 × 512 ou 1024 × 1024), un facteur 64 ou 128 préserve davantage de détails mais allonge la séquence, augmentant le coût quadratique de l'attention.

En résumé, le tokenizer n'est pas un simple pré-processing : il définit l'espace des possibles. Dans la Partie III, nous verrons comment la modélisation autorégressive opère sur ces tokens, et pourquoi l'ordre de balayage interagit fortement avec la structure du codebook choisi ici.

Exercices du chapitre 6

Exercice 6.1 — Reconstruction et FID local avec VQGAN.

Chargez un checkpoint VQGAN pré-entraîné sur OpenImages 1024 (disponible via le dépôt `CompVis/taming-transformers` ou les poids mis à disposition par la communauté). Sélectionnez 10 images de votre choix (256 × 256 ou redimensionnées). Encodez-les, quantifiez-les, puis décodez-les. Calculez le FID entre les 10 originales et les 10 reconstructions en utilisant la bibliothèque `pytorch-fid` ou `clean-fid`. Interprétez le résultat : à quel facteur de compression opérez-vous ? Le FID obtenu est-il cohérent avec les chiffres rapportés dans la littérature ?

Exercice 6.2 — Remplacer la quantification VQ par LFQ.

À partir d'un VQ-VAE entraîné avec un codebook de taille K = 1 024 et une dimension d = 256, remplacez la couche de quantification par une LFQ de dimension d' = 10 (soit $2^{10} = 1024$ codes). Réentraînez le décodeur pendant 50 000 itérations. Comparez :

• Le taux d'utilisation du codebook (nombre de codes distincts utilisés sur un batch de validation / nombre total de codes).

• Le FID de reconstruction.

• La stabilité de l'entraînement (courbes de perte).

© 2026 Oussama Ghorbel

Quel constat faites-vous sur le collapse du codebook ?

Exercice 6.3 — Benchmark de vitesse d'encodage/décodage.

Écrivez un script qui encode puis décode 1 000 images 256 × 256 (batch size 32) en utilisant le VQGAN de l'exercice 6.1. Mesurez le temps total sur GPU (CUDA) et sur CPU. Calculez le débit en images/seconde pour chaque configuration. Répétez avec le tokenizer SD-VAE et, si possible, avec MAGVIT-v2. Présentez les résultats dans un tableau et commentez l'écart GPU/CPU : quel composant (encodeur, quantification, décodeur) est le plus coûteux ?

Points clés à retenir

Avant de tourner la page, je souhaite condenser ce chapitre en quelques principes que vous garderez en tête tout au long de la suite de l'ouvrage :

1. **Le tokenizer est un contrat de fidélité.** Chaque token que vous émettez porte une quantité finie d'information visuelle. Si le codebook contient 1 024 entrées, chaque token encode au mieux $\log_2(1024) = 10$ bits. Multipliez par le nombre de positions spatiales et vous obtenez le budget informationnel total. Toute génération en aval devra vivre dans cette enveloppe.

2. **La perte adversariale n'est pas optionnelle.** Sans elle, le décodeur converge vers la moyenne conditionnelle et produit du flou. La LPIPS seule ne suffit pas : elle stabilise la structure mais ne génère pas les textures haute fréquence qui rendent une image crédible à l'œil humain. Les deux termes sont complémentaires et leur pondération adaptative (le λ_{adv} dynamique d'Esser et al.) est essentielle pour éviter qu'un signal n'écrase l'autre.

3. **LFQ et FSQ changent la donne opérationnelle.** En supprimant le codebook paramétrique, ils éliminent les points de fragilité classiques (collapse, reset, EMA instable) au prix d'une géométrie contrainte. Pour un projet d'adaptation de LLM, cette simplification est précieuse : moins d'hyperparamètres à régler, moins de modes d'échec à diagnostiquer.

© 2026 Oussama Ghorbel

4. **Le FID de reconstruction est votre premier benchmark.** Avant d'entraîner le moindre modèle génératif, mesurez-le. Notez-le. Affichez-le dans votre tableau de bord expérimental. Tout FID de génération inférieur à ce chiffre est suspect ; tout FID supérieur doit être décomposé pour identifier la part attribuable au tokenizer.

Pour aller plus loin

Si vous souhaitez approfondir certains aspects abordés ici, voici quelques directions de lecture complémentaires :

• Sur la théorie des pertes perceptives, l'article original de Zhang et al. (2018), « The Unreasonable Effectiveness of Deep Features as a Perceptual Metric », reste la référence la plus claire. Il montre notamment que les features de couches intermédiaires de réseaux entraînés pour la classification capturent la similarité perceptive humaine mieux que toute métrique pixel-à-pixel.

• Sur la quantification résiduelle (RQ-VAE), le travail de Lee et al. (2022) ouvre une piste intéressante pour notre projet : plutôt qu'un seul token par position, on émet une séquence de tokens qui raffinent progressivement le code. Cela augmente la fidélité sans augmenter la taille du codebook, mais allonge la séquence à générer. Le compromis est analysé en détail au chapitre 13.

• Sur les aspects pratiques de l'entraînement de tokenizers à grande échelle, le rapport technique de NVIDIA pour Cosmos Tokenizer (2024) documente les choix de curriculum, la gestion des artefacts de compression JPEG dans les données d'entraînement, et les stratégies de data augmentation spécifiques. C'est une lecture utile pour quiconque envisage d'entraîner son propre tokenizer plutôt que d'utiliser un checkpoint pré-existant.

Transition vers le chapitre 7

Nous avons vu que le codebook — qu'il soit paramétrique (VQ classique), implicite (LFQ) ou résiduel (RQ) — constitue le cœur du tokenizer. Mais sa configuration reste un problème ouvert : quelle taille ? Quelle dimension d'embedding ? Comment initialiser les vecteurs pour

© 2026 Oussama Ghorbel

éviter le collapse dès les premières itérations ? Comment gérer la réinitialisation des codes morts sans déstabiliser l'entraînement ?

Le chapitre 7 aborde ces questions frontalement. Nous y construirons, étape par étape, un codebook de 8 192 entrées, nous mesurerons l'impact de chaque hyperparamètre sur le taux d'utilisation et la fidélité de reconstruction, et nous établirons un protocole d'entraînement robuste que vous pourrez réutiliser pour vos propres expériences. Car un bon tokenizer ne se décrète pas : il se calibre, itération après itération, en surveillant des indicateurs précis que nous apprendrons à lire.

Le pixel attend d'être dompté. Le token attend d'être choisi. Entre les deux, le codebook est le pont — et sa construction est un artisanat autant qu'une science.

Ce mot d'artisanat n'est pas une figure de style. Il décrit très exactement ce qui vous attend lorsque vous entraînerez votre premier tokenizer : des heures passées à surveiller des courbes, un effondrement de codebook survenu à la douzième mille itérations, une reconstruction qui semble irréprochable sur l'écran entier et qui se délite dès qu'on zoome sur un iris. Avant de refermer ce chapitre, je veux donc déposer ici quelques notes de chantier — des détails qu'il aurait été utile de connaître plus tôt, et dont chacun peut coûter, à un moment ou à un autre, une semaine de calculs.

Le facteur d'échelle du latent, ou la constante qui ruine un entraînement

Commençons par une précision utile sur la section 6.4 : le tenseur $4 \times 64 \times 64$ que j'y mentionne correspond à une entrée de 512×512. À 256×256, la grille latente du SD-VAE est $4 \times 32 \times 32$, soit 4 096 valeurs. Le facteur de compression spatial reste 8, mais le volume absolu change, et avec lui le coût de tout ce qui suit.

Vient ensuite une constante que vous rencontrerez inévitablement si vous manipulez des poids Stable Diffusion : le fameux 0,18215. Les latents produits par l'encodeur ne sont pas centrés réduits : leur écart-type empirique, mesuré sur un grand échantillon d'images, est d'environ 5,5, et l'on multiplie donc les latents par l'inverse de cette

© 2026 Oussama Ghorbel

valeur avant de les confier au modèle génératif. Pourquoi ? Parce que les procédés de diffusion supposent, pour converger proprement, un latent d'échelle unitaire. Sans cette normalisation, l'entraînement diverge ou stagne silencieusement, et l'on passe plusieurs jours à soupçonner son apprentissage, son optimizer ou son schedule avant de soupçonner l'échelle.

Quel rapport avec la tokenisation discrète ? Deux, en réalité.

Premièrement, si vous travaillez avec un tokenizer hybride — un encodeur KL-régularisé suivi d'une quantification, comme en proposent plusieurs travaux récents — la normalisation préalable n'est pas optionnelle. La recherche du plus proche voisin est dominée par les dimensions de plus grande variance : sans réduction d'échelle, votre codebook ne partitionne efficacement que quelques canaux et ignore les autres. C'est une cause fréquente, et peu visible, de sous-utilisation du codebook.

Deuxièmement, dans un VQGAN classique, cette contrainte s'efface parce que les embeddings du codebook sont appris : l'échelle est absorbée. Mais elle réapparaît indirectement dans la perte de commitment, dont le gradient est proportionnel à la distance entre sortie d'encodeur et code choisi. Si vos latents gonflent, cette perte explose et écrase la reconstruction. Surveillez donc la norme moyenne des vecteurs latents comme on surveille un voyant moteur : une dérive monotone au cours de l'entraînement annonce presque toujours un déséquilibre entre les termes de la fonction objective.

Le budget informationnel, chiffres en main

Reprenons le calcul de façon explicite, car il donne une intuition que aucune courbe de perte ne remplace. Une image 256×256 en huit bits par canal pèse $256 \times 256 \times 3 = 196\,608$ octets, soit un peu plus d'1,5 million de bits. Un VQGAN à facteur 16 émet $16 \times 16 = 256$ tokens ; avec un codebook de 1 024 entrées, chaque token porte au plus $\log_2 1024 = 10$ bits, soit 2 560 bits au total, c'est-à-dire 320 octets. Le rapport atteint environ 614 pour 1. Le SD-VAE, en comparaison, conserve 4 096 valeurs en demi-précision, soit 8 192 octets : un rapport de 24 pour 1 seulement.

© 2026 Oussama Ghorbel

Ces deux chiffres ne sont pas comparables en termes de qualité — c'est précisément le propos du chapitre — mais ils matérialisent l'écart de philosophie : la discrétisation est infiniment plus avare, et elle ne s'en sort que parce qu'elle apprend, plutôt qu'elle ne suppose, la redondance du monde visuel. Le lien avec l'entropie discussed au chapitre 2 est direct : le tokenizer discover une code statistique adaptée à sa distribution d'entraînement, et c'est cette adaptation, non la taille brute du vocabulaire, qui fait la fidélité.

Il en découle une conséquence qu'il est important de formuler clairement. Le décodeur est une fonction déterministe : deux images distinctes encodées sur la même séquence de tokens produiront exactement la même reconstruction. L'ensemble des images qu'un générateur opérant dans cet espace peut produire est donc l'image, par le décodeur, de l'ensemble des séquences de tokens — un ensemble fini, densement peuplé mais fini. La continuité apparente des résultats ne vient pas de l'espace latent : elle vient de la régularité apprise du décodeur, qui interpole entre les codes.

Formellement, pour toute image cible x, la meilleure séquence qu'un modèle puisse prédire est $E(x)$, et la meilleure sortie atteignable est $D(E(x))$. La reconstruction n'est donc pas une étape préparatoire : elle est, littéralement, l'optimum. Je préfère nuancer — le FID n'est pas une distance mathématique et ce raisonnement n'a rien d'un théorème — mais en pratique l'écart se comporte systématiquement dans le sens attendu, et l'on n'observe jamais de générateur battant durablement le FID de reconstruction de son propre tokenizer.

Un diagnostic que je recommande, peu coûteux et rarement rapporté : comptez les collisions. Encodez un ensemble de validation et dénombrez les séquences de tokens identiques. Un taux de collision élevé signale un codebook sous-dimensionné ou mal réparti, et annonce un plafond de diversité que ni la taille du LLM ni la durée d'entraînement ne relèveront.

Pièges d'implémentation à éviter

Quatre détails techniques, classés par fréquence d'occurrence dans les erreurs courantes.

© 2026 Oussama Ghorbel

La précision numérique de la recherche du plus proche voisin. En demi-précision, les distances entre vecteurs latents et codes perdent la résolution nécessaire pour départager des voisins proches. Le choix du code devient bruité, l'entraînement oscille, et le symptôme — une perte de commitment qui refuse de descendre — pointe vers le mauvais coupable. Calculez les distances en précision simple, même si le reste du réseau tourne en demi-précision.

La normalisation dans le discriminateur. La normalisation par batch, dans un discriminateur patch-based, autorise parfois le décodeur à exploiter les statistiques du lot plutôt que la qualité intrinsèque de l'image. Les implémentations robustes lui préfèrent la normalisation d'instance, la normalisation de groupe, ou la simple normalisation spectrale sur les convolutions. Ce dernier choix, en bornant la norme des poids, stabilise nettement les premiers milliers de pas.

Le poids de la perte de commitment. La valeur usuelle, 0,25, est un point de départ et non une vérité. Trop élevé, l'encodeur s'effondre vers les codes existants et la reconstruction se fige ; trop faible, les latents dérivent et le codebook ne suit plus. Si votre reconstruction se dégrade alors que la perte de quantification baisse, c'est presque toujours ce réglage qu'il faut regarder en premier.

Les données elles-mêmes. Un tokenizer entraîné sur des images ayant traversé plusieurs réencodages JPEG apprend à reconstruire des artefacts de compression. Le phénomène est insidieux : les métriques s'améliorent, l'œil s'inquiète. Surveillez également la cohérence des filtres de redimensionnement entre entraînement, évaluation et déploiement — un bicube ici, un lanczos là, et vos FID ne sont plus comparables.

Une dernière remarque, à propos de l'exercice 6.1 : le FID calculé sur dix images est un test de fumée, pas une mesure. La variance est telle que deux tirages différents peuvent diffuser un ordre de grandeur. Utilisez ces dix images pour vérifier que votre pipeline encode et décode correctement, puis refaites le calcul sur plusieurs milliers d'images avant de tirer la moindre conclusion. Le FID devient lisible à partir de quelques milliers d'échantillons, et confortable au-delà de dix mille.

Une checklist de choix

© 2026 Oussama Ghorbel

Pour clore ce tour d'horizon, voici la synthèse qu'il est utile d'afficher au-dessus de son bureau lorsqu'un nouveau projet démarre.

Contrainte du projet	Orientation recommandée
Budget de séquence serré, génération 256×256	Facteur 16, 256 tokens, codebook 10^3 à 10^4
Haute résolution, fidélité prioritaire	Facteur 8, ou génération multi-échelle (chapitre 13)
LLM existant à adapter, peu de temps d'ajustement	LFQ ou FSQ : moins d'hyperparamètres, pas d'effondrement à gérer
Streaming, vidéo, causalité temporelle	Tokenizer causal de type Cosmos
Diffusion en aval, pas de LLM	Continuez en continu : le SD-VAE reste un excellent choix
Besoin de fidélité sans agrandir le vocabulaire	Quantification résiduelle, au prix d'une séquence plus longue

Deux lignes complètent ce tableau. La première : mesurez toujours le FID de reconstruction, le taux d'utilisation du codebook et le taux de collisions avant d'écrire la moindre ligne de code de génération. La seconde : si deux tokenizers se valent sur ces trois indicateurs, choisissez celui dont vous comprenez le mieux la dynamique d'entraînement. Un composant maîtrisé à 90 % bat un composant optimal mais mystérieux, car c'est le premier que vous saurez dépanner.

Les chiffres que vous obtiendrez en exécutant l'exercice 6.3 ne sont pas une formalité pédagogique : ils constituent la première colonne du tableau de coûts que nous construirons au chapitre 22, lorsque nous assemblerons le protocole complet d'adaptation d'un modèle de sept milliards de paramètres. Encodez, décodez, chronométrez, et notez. Le chapitre suivant, lui, s'attaque à la pièce maîtresse : la construction d'un codebook qui ne meure pas avant la fin de l'entraînement.

© 2026 Oussama Ghorbel

Chapitre 7 – Codebooks : taille, dimension, initialisation et entraînement

Le codebook est le cœur battant de toute quantification vectorielle. C'est lui qui transforme un continuum de représentations latentes en un alphabet fini de symboles discrets — ces tokens visuels que notre futur générateur manipulera avec la même aisance qu'un LLM manipule des mots. Dans ce chapitre, je vous propose d'ouvrir le capot et d'examiner chaque paramètre de conception : la taille K, la dimension d, la stratégie d'initialisation, la gestion des entrées mortes, et les variantes structurelles que sont la quantification produit et la quantification résiduelle. Mon objectif est simple : qu'à la fin de ces pages, vous soyez capable de dimensionner un codebook de manière raisonnée, en comprenant *pourquoi* chaque choix a un coût et un bénéfice.

7.1 — Taille K du codebook : compromis expressivité / utilisation / mémoire

Le paramètre le plus visible d'un codebook est sa cardinalité K : le nombre de vecteurs prototypes qu'il contient. Un codebook de taille $K = 512$ offre 512 « mots visuels » ; un codebook de $K = 8\,192$ en offre seize fois plus. L'intuition suggère qu'un K plus grand capture davantage de nuances. La réalité est plus subtile.

Expressivité. Théoriquement, la capacité de représentation croît logarithmiquement avec K : chaque token porte $\log_2 K$ bits d'information. Pour une image de 32×32 patches, un codebook à $K = 1\,024$ encode $32 \times 32 \times 10 = 10\,240$ bits, soit environ 1,28 Ko d'information discrète. Passer à $K = 8\,192$ monte à $32 \times 32 \times 13 \approx 13\,312$ bits. Le gain existe, mais il est loin d'être proportionnel au facteur 8 appliqué à K.

Utilisation effective. En pratique, un codebook trop vaste par rapport à la diversité des données souffre de sous-utilisation : une fraction importante des entrées n'est jamais sélectionnée. On parle alors d'*entrées mortes* (nous y revenons en section 7.4). Un codebook de 16 384 entrées entraîné sur un petit dataset peut n'en activer que 3 000 ou 4 000, rendant le surplus inutilement coûteux.

© 2026 Oussama Ghorbel

Mémoire et calcul. Le codebook lui-même occupe $K \times d$ flottants. Pour $K = 8\,192$ et $d = 512$, cela représente environ 16 Mo en FP32 — négligeable face aux centaines de Mo du reste du réseau. En revanche, la recherche du plus proche voisin coûte $\mathcal{O}(K \cdot d)$ par vecteur d'entrée ; sur un batch de B images avec $H \times W$ positions, la complexité cumulée devient $B \cdot H \cdot W \cdot K \cdot d$. Au-delà de $K \approx 16\,384$, cette recherche commence à peser sur le temps d'entraînement, surtout sans index approximatif.

Règle empirique. En pratique, un bon point de départ pour des images 256×256 tokenisées en 16×16 patches est $K \in [1\,024;\ 8\,192]$. En dessous de 1 024, la fidélité se dégrade visiblement (banding, posterisation). Au-dessus de 8 192, le gain marginal devient faible si le dataset contient moins de quelques millions d'images.

7.2 — Dimension d des entrées : 256, 384, 512 — impact sur la fidélité

La dimension d du vecteur que l'on quantifie est dictée par l'encodeur en amont. Un VAE classique produit des latents de dimension 4 par pixel ; un encodeur convolutif suivi d'une couche de projection peut produire des vecteurs de dimension 256, 384 ou 512 par position spatiale.

Pourquoi d compte. La dimension détermine la richesse du sous-espace que chaque entrée du codebook doit approximer. Avec $d = 256$, chaque prototype est un point dans $\mathbb{R}^{256}$; la distance euclidienne (ou cosinus) entre l'entrée et le prototype mesure l'erreur de reconstruction locale. Plus d est grand, plus le codebook peut encoder de variations fines — textures, gradients subtils, micro-structures — mais plus la malédiction de la dimensionnalité rend la couverture de l'espace difficile pour un K fixé.

Observations pratiques.

- $d = 256$: suffisant pour des reconstructions correctes à résolution modérée. C'est le choix de nombreux VQ-VAE premiers du nom.
- $d = 384$: compromis fréquent dans les architectures de type ViT-Small ; offre un gain notable sur les textures sans exploser la complexité.

© 2026 Oussama Ghorbel

• $d = 512$: standard dans les VQGAN haute fidélité. La capacité expressive est élevée, mais exige un K proportionnellement plus grand pour maintenir un taux d'utilisation sain.

Interaction $K \times d$. Le nombre effectif de paramètres du codebook est $K \cdot d$. Un codebook ($K = 4\,096$, $d = 512$) contient 2 M paramètres ; un codebook ($K = 1\,024$, $d = 256$) n'en contient que 262 k. Le premier est huit fois plus lourd, mais il couvre un espace de bien plus grande dimension. Le choix conjoint (K, d) doit donc se penser comme un budget global de capacité discrète.

7.3 — Initialisation : k-means, aléatoire, EMA

Un codebook mal initialisé peut mettre des milliers d'itérations à converger, voire rester piégé dans un minimum local où la majorité des entrées sont mortes dès le départ. Trois stratégies dominent la littérature.

Initialisation k-means. On lance un k-means (souvent avec $k = K$) sur un échantillon de vecteurs issus de l'encodeur, après quelques étapes de pré-entraînement en continu. Les centroïdes résultants servent de valeurs initiales aux prototypes. Avantage : la couverture de l'espace est immédiate, le taux d'utilisation démarre proche de 100 %. Inconvénient : le coût du k-means pour de très grands K et la nécessité de stocker un échantillon représentatif.

Initialisation aléatoire. On tire les prototypes dans $\mathcal{N}(0, \sigma^2 I_d)$ ou $\mathcal{U}(-a, a)^d$. Simple, mais risqué : dans un espace de grande dimension, des points tirés uniformément se concentrent sur une coquille sphérique et laissent le centre de la distribution vide. Les premières mises à jour corrigent partiellement ce biais, mais la convergence est plus lente.

Mise à jour EMA (Exponential Moving Average). Plutôt qu'une initialisation, l'EMA est une stratégie d'*entretien* du codebook. Chaque prototype e_k est mis à jour selon :

$$e_k^{(t)} = \gamma\, e_k^{(t-1)} + (1 - \gamma)\, \bar{x}_k^{(t)}$$

où $\bar{x}_k^{(t)}$ est la moyenne des vecteurs assignés à l'entrée k au pas t, et $\gamma \in [0{,}99;\ 0{,}999]$ est le facteur de lissage. L'EMA évite de backpropager à travers la table de quantification (on utilise un *straight-through estimator* pour le gradient) et stabilise l'entraînement. Je recommande de

© 2026 Oussama Ghorbel

combiner k-means initial + EMA : on obtient une couverture rapide et une adaptation douce au fil des époques.

7.4 — Problème des entrées mortes : reset, random restart, codebook dropout

C'est le fléau numéro un de la quantification vectorielle. Une entrée est dite *morte* lorsqu'aucun vecteur du batch ne lui est assigné pendant un nombre significatif d'itérations. Une fois morte, elle ne reçoit plus de gradient, ne se met plus à jour, et reste morte — un cercle vicieux.

Diagnostic. On suit le taux d'utilisation : fraction d'entrées ayant reçu au moins une assignment sur une fenêtre glissante (par exemple 100 pas). Un taux inférieur à 70 % est un signal d'alarme.

Reset (réinitialisation ciblée). Dès qu'une entrée k est morte depuis plus de T_{dead} pas, on la remplace par un vecteur tiré aléatoirement parmi les entrées du batch courant, éventuellement perturbé par un bruit gaussien. Cette opération, peu coûteuse, réinjecte de la diversité.

Random restart. Variante plus agressive : on réinitialise périodiquement (toutes les N époques) un pourcentage fixe des entrées les moins utilisées. Cela force une redistribution de la densité de probabilité sur le codebook.

Codebook dropout. Inspiré du dropout classique : à chaque pas, on masque aléatoirement une fraction p des entrées du codebook, forçant les vecteurs d'entrée à se rabattre sur les prototypes restants. Cela empêche la spécialisation excessive de quelques entrées et améliore la robustesse. En pratique, $p \in [0{,}1;\ 0{,}2]$ donne de bons résultats sans dégrader la fidélité.

Combinaison recommandée. Un protocole efficace combine conjointement EMA ($\gamma = 0{,}99$), reset des entrées mortes ($T_{\text{dead}} = 50$ pas) et un léger codebook dropout ($p = 0{,}1$). Le taux d'utilisation se maintient alors au-dessus de 95 % sur des entraînements de plusieurs centaines de milliers de pas.

7.5 — Quantification produit et résiduelle : au-delà du codebook unique

© 2026 Oussama Ghorbel

Un codebook unique de taille K et de dimension d impose un compromis brutal : soit on augmente K (coût mémoire et recherche), soit on accepte une erreur de quantification élevée. Deux familles de méthodes permettent de dépasser cette limite.

Product Quantization (PQ). On découpe le vecteur $z \in \mathbb{R}^d$ en M sous-vecteurs de dimension d/M. Chaque sous-espace possède son propre codebook de taille K_m. La quantification de z devient un M-uplet d'indices $(k_1, \ldots, k_M)$, et la capacité effective est $\prod_{m=1}^{M} K_m$. Avec $M = 4$ et $K_m = 256$, on obtient $256^4 \approx 4{,}3 \times 10^9$ combinaisons possibles tout en ne stockant que $4 \times 256 \times (d/4)$ paramètres. Le coût de recherche passe de $\mathcal{O}(K \cdot d)$ à $\mathcal{O}(M \cdot K_m \cdot d/M) = \mathcal{O}(K_m \cdot d)$, indépendant du nombre total de combinaisons.

Residual VQ (RQ-VAE). L'idée est séquentielle : on quantifie z avec un premier codebook, on calcule le résidu $r_1 = z - e_{k_1}$, on quantifie r_1 avec un deuxième codebook, et ainsi de suite sur L niveaux. Le vecteur reconstruit est $\hat{z} = e_{k_1} + e_{k_2} + \cdots + e_{k_L}$. Chaque niveau affine la reconstruction ; l'erreur décroît exponentiellement avec L si les codebooks sont bien entraînés. RQ-VAE (Lee *et al.*, 2022) montre qu'avec $L = 8$ niveaux et $K = 1\,024$ par niveau, on atteint une fidélité comparable à un codebook unique de taille $1\,024^8$ — évidemment inenvisageable en mémoire.

Choix entre PQ et RQ. La PQ est parallèle (tous les sous-codebooks opèrent simultanément) et se prête bien à une factorisation spatiale. La RQ est séquentielle et capture une hiérarchie grossier-à-fin très naturelle pour les images : le premier niveau encode la structure globale, les niveaux suivants raffinent les détails. Pour un LLM qui génère token par token, la RQ offre un ordre de génération intrinsèque (du macro au micro), ce qui peut simplifier l'apprentissage.

7.6 — Multi-codebook et hiérarchie : plusieurs niveaux de granularité

Au-delà de la RQ linéaire, on peut organiser plusieurs codebooks en *hiérarchie explicite*. L'idée : un premier codebook grossier ($K_1 = 256$, opérant sur des patches 16×16) capture la sémantique globale ; un deuxième codebook fin ($K_2 = 4\,096$, opérant sur des patches 4×4) raffine la texture ; un troisième, optionnel, gère les résidus haute

© 2026 Oussama Ghorbel

fréquence.

Avantages.

• Le générateur autoregressive peut d'abord produire les tokens grossiers (peu nombreux, donc rapide), puis conditionner la génération fine sur ces tokens. Cela réduit la longueur effective de la séquence à générer.

• Chaque niveau peut être entraîné avec un objectif différent : perceptuel pour le niveau grossier, pixel-level pour le niveau fin.

• La mémoire reste maîtrisée : trois codebooks de 4 096 entrées coûtent moins qu'un unique codebook de $4\,096^3$ entrées.

Inconvénients.

• La propagation d'erreurs entre niveaux : une mauvaise quantification au niveau grossier est irréversible.

• L'entraînement conjoint des niveaux nécessite un curriculum soigné (on gèle le niveau grossier avant d'entraîner le fin, ou on utilise un loss pondéré).

Lien avec le LLM. Lorsque l'on unifie texte et image dans un même vocabulaire (chapitre 14), la hiérarchie de codebooks se traduit par des groupes de tokens spéciaux : `<IMG_L1_k>`, `<IMG_L2_k>`, etc. Le modèle apprend à générer d'abord les tokens de niveau 1, puis les tokens de niveau 2 conditionnés, reproduisant une forme de résolution progressive sans architecture dédiée.

Exercices

Exercice 7.1 — Suivi d'utilisation du codebook.

Implémentez, en PyTorch ou JAX, un module de logging qui, à chaque pas d'entraînement d'un VQ-VAE :

1. Enregistre les indices des entrées sélectionnées pour chaque vecteur du batch.

2. Maintient un compteur glissant (fenêtre de 200 pas) par entrée du codebook.

© 2026 Oussama Ghorbel

3. Produit un histogramme animé (une frame tous les 500 pas) montrant la distribution d'utilisation des K entrées.

4. Affiche en surimpression le taux d'utilisation (fraction d'entrées avec compteur > 0).

Objectif : visualiser en temps réel l'apparition et la disparition d'entrées mortes, et vérifier l'efficacité d'une stratégie de reset.

Exercice 7.2 — Courbe PSNR en fonction de K.

À partir d'un encodeur VAE pré-entraîné (par exemple celui de la section 5.4) :

1. Entraînez quatre codebooks distincts avec $K \in \{256,\ 1\,024,\ 4\,096,\ 8\,192\}$, en gardant $d = 256$ et une stratégie EMA identique.

2. Sur un jeu de validation de 5 000 images, calculez le PSNR moyen de la reconstruction après quantification.

3. Tracez la courbe PSNR = $f(K)$ en échelle logarithmique sur l'axe des abscisses.

4. Commentez : à partir de quel K le gain marginal devient-il inférieur à 0,1 dB ? Comparez avec le coût mémoire et le temps d'entraînement.

Exercice 7.3 — Pseudo-code de Product Quantization.

Rédigez un pseudo-code détaillé (indépendant du langage) implémentant la Product Quantization avec les spécifications suivantes :

• Vecteur d'entrée $z \in \mathbb{R}^{512}$.

• Découpage en $M = 4$ sous-vecteurs de dimension 128.

• Chaque sous-codebook contient $K_m = 256$ prototypes.

• Phase d'assignation : pour chaque sous-vecteur, trouver l'indice du prototype le plus proche (distance euclidienne).

• Phase de reconstruction : concaténer les quatre prototypes sélectionnés.

• Phase de mise à jour EMA : pour chaque sous-codebook, mettre à jour les prototypes avec $\gamma = 0{,}99$.

© 2026 Oussama Ghorbel

• Gestion des entrées mortes : si un prototype n'a reçu aucune assignment sur les 50 derniers pas, le remplacer par un sous-vecteur tiré aléatoirement dans le batch.

Votre pseudo-code doit faire apparaître clairement les boucles sur M, les opérations vectorisées sur K_m, et la logique de reset.

En refermant ce chapitre, vous disposez d'une boîte à outils complète pour concevoir, initialiser, entraîner et diagnostiquer un codebook. Le choix de K et d fixe le budget expressif ; l'initialisation et l'EMA conditionnent la vitesse de convergence ; la gestion des entrées mortes garantit la longévité de l'entraînement ; la quantification produit ou résiduelle repousse les limites de capacité sans exploser la mémoire. Au chapitre suivant, nous verrons comment la *patchification* — le découpage spatial de l'image en tokens — interagit avec ces codebooks pour produire une séquence que notre LLM pourra lire et écrire.

Pour aller plus loin : notes complémentaires et pièges fréquents

Avant de tourner définitivement la page sur le codebook, voici un ensemble de remarques pratiques issues de l'expérience — des détails qui ne trouvent pas leur place dans les articles de recherche mais qui font la différence entre un entraînement qui converge en trois jours et un entraînement qui stagne pendant trois semaines.

Le commitment loss et son coefficient β. Dans la formulation originale du VQ-VAE (van den Oord *et al.*, 2017), la fonction de perte comporte trois termes : la reconstruction, l'erreur du codebook (souvent gérée par EMA) et le *commitment loss* $\beta \| z_e - \mathrm{sg}[e_k] \|^2$, où $\mathrm{sg}[\cdot]$ désigne le stop-gradient. Ce terme pousse l'encodeur à produire des vecteurs proches des prototypes, évitant que la représentation latente ne diverge tandis que le codebook tente de la rattraper. Le choix de β est critique : trop faible ($\beta < 0{,}1$), l'encodeur « s'échappe » et le codebook ne parvient pas à suivre ; trop fort ($\beta > 2$), l'encodeur s'effondre vers quelques prototypes et la diversité disparaît. En pratique, $\beta \in [0{,}25 ; 1{,}0]$ constitue une plage robuste, avec une préférence pour $\beta = 0{,}5$ lorsque l'on utilise la mise à jour EMA plutôt qu'un gradient direct sur le codebook.

© 2026 Oussama Ghorbel

Normalisation des vecteurs avant quantification. Une astuce souvent passée sous silence : normaliser les vecteurs z_e sur la sphère unité (division par $\|z_e\|_2$) avant la recherche du plus proche voisin stabilise considérablement l'entraînement, surtout en haute dimension. Les prototypes sont alors eux aussi contraints à la sphère, et la distance euclidienne devient équivalente à une distance cosinus. Cela élimine les artefacts liés aux variations de norme entre couches et entre étapes d'entraînement. On peut observer une réduction de 15 à 20 % de l'erreur de quantification moyenne simplement en ajoutant cette normalisation, sans modifier aucune autre hyperparamètre.

Température et adoucissement de l'assignation. L'assignation dure (argmin sur les distances) est non différentiable et peut créer des oscillations en début d'entraînement, lorsqu'un vecteur hésite entre deux prototypes quasi équidistants. Une alternative consiste à utiliser une assignation *molle* pondérée par un softmax inverse de température :

$$\alpha_k = \frac{\exp(-\|z_e - e_k\|^2/\tau)}{\sum_{j=1}^{K} \exp(-\|z_e - e_j\|^2/\tau)}$$

avec une température τ que l'on fait décroître progressivement (par exemple de 1,0 à 0,1 sur les 20 000 premiers pas). Le vecteur quantifié devient alors une combinaison convexe des prototypes, et le gradient circule plus fluidement. En fin de formation, $\tau \to 0$ et l'on retrouve une assignation dure. Cette technique, inspirée du recuit simulé, réduit les oscillations et améliore le taux d'utilisation de 5 à 10 points de pourcentage dans les premières phases.

Piège fréquent : le codebook « miroir ». Lorsque l'encodeur produit des distributions multimodales très séparées (par exemple, des régions uniformes d'un côté, des textures riches de l'autre), le k-means initial peut placer la moitié des prototypes dans un mode et l'autre moitié dans le second. Si les modes sont très déséquilibrés en fréquence, le premier sature ses entrées tandis que le second reste sous-représenté. Le remède : sur-échantillonner les images contenant des textures complexes pendant les premières époques, ou appliquer un rééquilibrage de fréquence dans la mise à jour EMA (pondérer $\bar{x}_k$ par l'inverse de la fréquence d'assignation).

© 2026 Oussama Ghorbel

Checklist de conception d'un codebook

Pour clore ce chapitre, voici une liste de vérification utile à garder ouverte à côté de son éditeur de code chaque fois que l'on conçoit un nouveau tokenizer visuel. Elle ne remplace pas la réflexion, mais elle évite les oublis coûteux.

1. **Dimension** d : cohérente avec la sortie de l'encodeur ; éviter les dimensions qui ne sont pas des puissances de 2 ou des multiples de 8 si l'on prévoit une quantification produit.

2. **Taille** K : commencer à 4 096 pour du 256×256 en 16×16 patches ; ajuster après inspection du taux d'utilisation.

3. **Initialisation** : k-means sur 100 000 vecteurs pré-extraits, ou à défaut tirage gaussien suivi de 2 000 pas de warm-up avec EMA seul.

4. **EMA** : $\gamma = 0{,}99$; vérifier que le dénominateur (nombre d'assignations par prototype) est borné inférieurement par $\varepsilon = 10^{-5}$ pour éviter les divisions par zéro.

5. **Commitment loss** : $\beta = 0{,}5$; ne pas l'appliquer aux prototypes (l'EMA s'en charge).

6. **Entrées mortes** : seuil $T_{\text{dead}} = 50$ pas ; reset par tirage dans le batch + bruit $\mathcal{N}(0, 0{,}01)$.

7. **Codebook dropout** : $p = 0{,}1$ pendant les 50 000 premiers pas, puis désactivation progressive.

8. **Normalisation sphérique** : activer si $d \geq 384$; désactiver si l'on observe un effondrement de la norme.

9. **Suivi** : logger toutes les 100 itérations le taux d'utilisation, l'entropie de la distribution d'assignation, la distance moyenne $\|z_e - e_k\|$ et le PSNR de reconstruction sur un mini-set fixe.

10. **Critère d'arrêt de l'entraînement du codebook** : PSNR de reconstruction stable à $\pm 0{,}05$ dB sur 5 000 pas consécutifs, *et* taux d'utilisation supérieur à 90 %.

Lectures recommandées

© 2026 Oussama Ghorbel

Pour approfondir les aspects abordés dans ce chapitre, je renvoie vers les travaux suivants, qui ont chacun apporté une pierre spécifique à l'édifice de la quantification vectorielle appliquée à la vision :

• Van den Oord, Vinyals & Kavukcuoglu (2017), « Neural Discrete Representation Learning », qui introduit le VQ-VAE et formalise le commitment loss.

• Razavi, van den Oord & Vinyals (2019), « Generating Diverse High-Fidelity Images with VQ-VAE-2 », pour la hiérarchie multi-niveaux.

• Esser, Rombach & Ommer (2021), « Taming Transformers for High-Resolution Image Synthesis » (VQGAN), pour l'entraînement adversarial du codebook.

• Lee *et al.* (2022), « Autoregressive Image Generation using Residual Quantization », pour la formalisation complète de la RQ-VAE.

• Jégou, Douze & Schmid (2011), « Product Quantization for Nearest Neighbor Search », article fondateur de la PQ, initialement pensé pour la recherche d'images mais dont la structure se transpose directement à la génération.

• Dhariwal *et al.* (2020), « Jukebox : A Generative Model for Music », qui démontre l'efficacité des VQ-VAE hiérarchiques à trois niveaux dans un domaine non visuel — preuve que les principes du codebook sont universels.

Ces références ne constituent pas une bibliographie exhaustive ; elles sont les points d'entrée que je considère comme indispensables avant d'aborder la suite.

Le codebook n'est pas un simple tableau de flottants que l'on remplit une fois pour toutes. C'est un organisme vivant au sein du réseau : il naît avec une initialisation, grandit par EMA, se régénère par reset, et meurt partiellement si on le néglige. Le concevoir, c'est accepter qu'il évoluera tout au long de l'entraînement et qu'il faudra l'accompagner — monitorer, ajuster, parfois intervenir manuellement. Cette vigilance est le prix à payer pour obtenir un alphabet visuel riche, équilibré et fidèle, sur lequel nous construirons, dans les chapitres suivants, la véritable

© 2026 Oussama Ghorbel

génération autoregressive.

Au chapitre 8, nous quitterons l'espace latent pour revenir à l'image elle-même : comment la découper en patches, quelles inductives choisir pour l'encodeur, et comment la forme de la grille conditionne la capacité du codebook que nous venons de sculpter.

© 2026 Oussama Ghorbel

Chapitre 8 – Patchification et inducteurs visuels : du CNN au ViT

Dans les chapitres précédents, nous avons établi que la tokenisation visuelle transforme une image continue en une séquence discrète de codes. Mais avant même de quantifier ces codes dans un codebook, une question fondamentale se pose : comment découper l'image en unités élémentaires exploitables par un réseau de neurones ? C'est précisément le rôle de la *patchification* : extraire des sous-régions régulières, les projeter dans un espace latent, puis les organiser en séquence. Ce chapitre explore les mécanismes d'extraction de patches, les architectures qui les exploitent — du CNN classique au Vision Transformer — et les choix de conception qui déterminent la qualité de la représentation en amont du tokenizer.

8.1 — Extraction de patches : convolution stride = kernel, unfold, patch_embed

L'opération de patchification est conceptuellement simple : on découpe une image de dimensions $H \times W \times C$ en une grille régulière de carrés de taille $P \times P$. Chaque carré constitue un *patch*. Pour une image 256×256 en RGB ($C = 3$) avec $P = 16$, on obtient une grille de $16 \times 16 = 256$ patches, chacun contenant $16 \times 16 \times 3 = 768$ valeurs.

Trois implémentations équivalentes dominent la pratique :

Convolution stride = kernel. C'est l'approche la plus répandue dans les codebases modernes. On définit une couche `Conv2d` dont la taille de noyau et le pas sont tous deux égaux à P :

Appliquée à un tenseur de forme $(B, 3, H, W)$, cette convolution produit un tenseur $(B, D, H/P, W/P)$. Chaque position spatiale correspond à un patch ; le noyau appris effectue simultanément l'extraction et la projection linéaire. L'avantage est double : l'opération est différentiable de bout en bout et bénéficie des implémentations hautement optimisées de cuDNN.

Unfold (im2col). PyTorch propose `torch.nn.functional.unfold`, qui extrait toutes les fenêtres glissantes d'un tenseur. En fixant

© 2026 Oussama Ghorbel

`kernel_size=P` et `stride=P`, on obtient une matrice de forme $(B, C \cdot P^2, N)$ où $N = (H/P)(W/P)$ est le nombre de patches. Il suffit ensuite d'appliquer une projection linéaire `nn.Linear(C *P* P, D)` pour obtenir l'embedding. Cette formulation rend explicite le fait que la patchification est une opération purement géométrique, indépendante de tout apprentissage.

Patch_embed composite. Dans les implémentations de type timm ou HuggingFace, le module `patch_embed` encapsule la convolution suivie d'un aplatissement spatial :

La sortie est une séquence de N vecteurs de dimension D, prête à être consommée par un Transformer. C'est cette représentation qui alimente ensuite l'encodeur du tokenizer (chapitres 5 et 6).

Un point mérite d'être souligné : la convolution stride = kernel ne produit aucun recouvrement entre patches. Contrairement à une convolution classique avec stride < kernel, chaque pixel n'est « vu » qu'une seule fois. Cette absence de recouvrement simplifie la factorisation autorégressive (chapitre 10) mais peut créer des artefacts de frontière si la projection n'est pas suffisamment expressive.

8.2 — ViT (Dosovitskiy et al., 2020) : linéarisation des patches, position embedding 2D

Le Vision Transformer, introduit par Dosovitskiy et ses collaborateurs en 2020, a démontré qu'un Transformer standard — conçu à l'origine pour le texte — pouvait rivaliser avec les CNN sur la classification d'images, à condition d'être entraîné sur de grands jeux de données. L'idée fondatrice est d'une élégance remarquable : traiter chaque patch comme un token.

Le pipeline du ViT se décompose ainsi :

1. **Patchification** : l'image $224 \times 224 \times 3$ est découpée en $14 \times 14 = 196$ patches de 16×16.

2. **Linéarisation** : chaque patch aplati (768 valeurs) est projeté dans un espace de dimension $D = 768$ via une matrice apprise $E \in \mathbb{R}^{768 \times D}$.

3. **Token de classe** : un vecteur apprenable [*CLS*] est préfixé à la séquence, à la manière du token `BOS` en NLP.

© 2026 Oussama Ghorbel

4. **Position embeddings** : un vecteur positionnel apprenable est ajouté à chaque token pour préserver l'information spatiale.

5. **Encoder Transformer** : L blocs d'attention multi-têtes et de MLP alternés traitent la séquence.

Formellement, la séquence d'entrée s'écrit :

$$z_0 = [x_{CLS};\ x_p^1 E;\ x_p^2 E;\ \ldots;\ x_p^N E] + E_{pos}$$

où $x_p^i \in \mathbb{R}^{P^2 C}$ est le i-ème patch aplati et $E_{pos} \in \mathbb{R}^{(N+1)\times D}$ les embeddings positionnels.

Pour notre propos — la génération d'images par LLM — le ViT joue un double rôle. D'une part, il sert d'*encodeur* dans les VQGAN et auto-encodeurs : les patches sont extraits, encodés par attention, puis quantifiés. D'autre part, il inspire l'architecture du *générateur* autorégressive : si l'on peut traiter des patches d'image comme des tokens textuels, alors un LLM peut, en principe, les prédire séquentiellement.

8.3 — CNN comme inducteur : ResNet, ConvNeXt en backbone d'encodeur

Avant le ViT, les réseaux convolutifs constituaient l'inducteur visuel par excellence. Leur biais inductif — localité, translation, hiérarchie des réceptifs — les rend naturellement adaptés à l'extraction de caractéristiques spatiales.

ResNet (He et al., 2015) introduit les connexions résiduelles qui permettent d'entraîner des réseaux de plusieurs dizaines de couches. Utilisé comme backbone d'encodeur dans un VQGAN, un ResNet-50 ou ResNet-101 produit des cartes de caractéristiques $f \times f \times d$ (par exemple $16 \times 16 \times 512$ pour une entrée 256×256 avec un downsampling factor 16). Ces cartes constituent la représentation continue que le quantificateur discrétise ensuite.

ConvNeXt (Liu et al., 2022) revisite le CNN en adoptant les recettes d'entraînement du ViT (normalisation par couche, GELU, rapport d'expansion du MLP de 4×) tout en conservant des convolutions en profondeur (*depthwise convolution*). Le résultat : des performances comparables au ViT avec un coût d'inférence moindre et une meilleure

© 2026 Oussama Ghorbel

gestion des résolutions variables.

Pour un tokenizer d'images, le choix entre CNN et ViT comme encodeur n'est pas anodin :

- Le CNN produit des représentations *denses et localisées*, idéales pour préserver les textures fines.
- Le ViT capture des dépendances *globales* dès les premières couches, ce qui favorise la cohérence structurelle.

Dans la pratique, les VQGAN récents (SDXL, Cosmos Tokenizer) utilisent souvent un encodeur convolutif suivi de quelques blocs d'attention en goulot d'étranglement, combinant les deux avantages.

8.4 — Hybrid CNN-ViT : convolution stem + attention pour les petites résolutions

La patchification par convolution unique $P \times P$ est efficace mais brutale : elle projette un voisinage de $P^2 \cdot C$ valeurs en un seul vecteur sans étapes intermédiaires. Pour les petites résolutions (64×64, 128×128), cette opération peut perdre de l'information.

L'approche hybride, popularisée par des architectures comme CvT, LeViT ou encore les variantes « patchify » de Stable Diffusion 3, consiste à :

1. **Stem convolutif** : appliquer 2 à 3 convolutions stride 2 successives, réduisant la résolution d'un facteur 4 à 8 tout en augmentant progressivement le nombre de canaux. Ce stem joue le rôle d'un *pré-tokenizer* qui extrait des caractéristiques locales avant la patchification proprement dite.
2. **Patchification résiduelle** : sur la carte réduite, appliquer une convolution stride = kernel avec un P plus petit (4 ou 8 au lieu de 16). Le nombre de patches reste gérable, mais chaque patch encode déjà des motifs appris.
3. **Blocs d'attention** : la séquence de patches est ensuite traitée par des couches Transformer standards.

Ce design présente un avantage computationnel clair : pour une image 256×256, un stem de deux convolutions stride 2 produit une carte

© 2026 Oussama Ghorbel

64×64. Une patchification $P = 8$ sur cette carte donne $8 \times 8 = 64$ patches au lieu de $16 \times 16 = 256$ si l'on avait patchifié directement l'image originale. La complexité quadratique de l'attention s'en trouve réduite d'un facteur 16.

Pour notre objectif d'adapter un LLM en générateur d'images, cette réduction est cruciale : moins de patches signifie moins de tokens à prédire, donc un budget computationnel moindre (voir chapitre 22).

8.5 — Taille de patch : 8 × 8, 16 × 16, 32 × 32 — effet sur N et la qualité

Le choix de P est l'un des hyperparamètres les plus structurants d'un système de génération par tokens. Il détermine directement :

- **Le nombre de tokens** : $N = (H/P) \times (W/P)$. Pour 256×256 : $P = 8 \Rightarrow N = 1024$, $P = 16 \Rightarrow N = 256$, $P = 32 \Rightarrow N = 64$.
- **La granularité spatiale** : un petit P préserve les détails fins mais multiplie la longueur de séquence.
- **Le coût computationnel** : la complexité de l'attention croît en $O(N^2)$; celle de la génération autorégressive croît linéairement en N mais avec un facteur constant lié à la taille du modèle.

En pratique, les observations suivantes guident le choix :

Taille P	N (pour 256×256)	Usage typique
8 × 8	1024	Génération haute fidélité, détails fins
16 × 16	256	Compromis standard (VQGAN, LlamaGen)
32 × 32	64	Génération rapide, brouillons, multi-échelle

Un P trop grand (32) produit des tokens trop « globaux » : chaque code du codebook doit encoder un large voisinage, ce qui dégrade la précision des textures. Un P trop petit (8 ou 4) génère des séquences longues qui stressent la fenêtre contextuelle du LLM et ralentissent l'inférence.

© 2026 Oussama Ghorbel

Dans le chapitre 22, je recommande $P = 16$ comme point de départ pour un LLM 7B générant du 256×256 : la séquence de 256 tokens reste bien en deçà des capacités contextuelles du modèle, tout en offrant une résolution spatiale suffisante après décodage par le VQGAN.

Il faut également garder à l'esprit que la taille de patch interagit avec la taille du codebook (chapitre 7). Un petit P produit des patches plus simples à encoder, permettant un codebook plus compact. Un grand P exige un codebook plus expressif — ou un codebook résiduel à plusieurs niveaux.

8.6 — Positional encoding 2D : sinusoidal, appris, interpolable

La patchification détruit l'information spatiale : une fois les patches aplatis en séquence, rien ne distingue le patch en haut à gauche de celui en bas à droite. Le *positional encoding* restaure cette géométrie.

Sinusoidal 2D. On adapte la formule du Transformer original à deux dimensions. Pour un patch à la position (i, j) dans la grille :

$$PE_{(i,j)}(2k) = \sin\left(\frac{i}{10000^{2k/D}}\right), \quad PE_{(i,j)}(2k+1) = \cos\left(\frac{i}{10000^{2k/D}}\right)$$

et de même pour j sur la seconde moitié de la dimension D. L'avantage est l'absence de paramètres appris et la possibilité théorique d'extrapoler à des résolutions non vues à l'entraînement.

Appris (learnable). C'est le choix du ViT original : une matrice $E_{pos} \in \mathbb{R}^{N \times D}$ est optimisée conjointement avec le reste du modèle. En pratique, les embeddings appris capturent des régularités spatiales légèrement plus riches que la formule sinusoidale, mais ils sont liés à une résolution fixe.

Interpolable. Pour passer d'une résolution d'entraînement 224×224 à une résolution d'inférence 512×512, on interpole bilinéairement la grille d'embeddings positionnels. Cette technique, utilisée dans DeiT et MAE, permet de réutiliser un modèle entraîné à basse résolution sans réentraînement complet. La formule est simple : si l'on passe de N_{train} à N_{test} positions, on reshape E_{pos} en grille $\sqrt{N_{train}} \times \sqrt{N_{train}} \times D$, on applique un `interpolate` bilinéaire vers $\sqrt{N_{test}} \times \sqrt{N_{test}} \times D$, puis on aplatit.

© 2026 Oussama Ghorbel

Pour un LLM adapté en générateur d'images, le choix du positional encoding conditionne la capacité du modèle à généraliser à des résolutions différentes de celle de l'entraînement. Dans le protocole du chapitre 22, l'utilisation d'embeddings appris avec interpolation bilinéaire permet de passer de 256×256 à 384×384 avec un léger fine-tuning.

Un dernier point, souvent négligé : dans le cas d'une génération autorégressive raster-scan (chapitre 11), la position dans la séquence porte déjà une information spatiale implicite. Certains travaux (Transfusion, MAR) montrent que l'on peut simplifier le positional encoding sans perte notable, car l'ordre de génération fournit un signal géométrique suffisant. Néanmoins, pour la phase d'encodage du tokenizer, conserver un encodage 2D explicite reste la pratique la plus robuste.

Exercices

Exercice 8.1 — Extraction et visualisation de patches.

Dans un notebook Python, chargez une image 256×256 (par exemple via `torchvision.datasets` ou une image locale). Implémentez l'extraction des patches 16×16 à l'aide de `torch.nn.functional.unfold`. Visualisez la grille de patches à l'aide de `matplotlib` en affichant chaque patch dans une sous-figure 16×16. Vérifiez que le nombre total de patches est bien 256 et que la reconstruction par repliement (`fold`) redonne l'image originale sans perte.

Exercice 8.2 — Comparaison ViT-B/16 vs ViT-B/32.

Calculez le nombre de paramètres d'un ViT-Base (12 couches, $D = 768$, 12 têtes) pour une entrée 224×224, avec $P = 16$ puis $P = 32$. Rappel : le nombre de patches est $N = (224/P)^2$. Identifiez quels modules voient leur taille varier (projection de patch, position embeddings) et lesquels restent identiques (blocs Transformer). Constatez que la différence principale porte sur la couche `patch_embed` et les position embeddings, tandis que le corps du Transformer est inchangé.

Exercice 8.3 — Implémentation de patch_embed en PyTorch.

© 2026 Oussama Ghorbel

Implémentez le module `PatchEmbed` suivant :

Vérifiez que la forme de sortie est bien $(B, 256, 768)$ pour une entrée 256×256 avec $P = 16$. Modifiez ensuite `patch_size` à 8 et 32, observez l'effet sur `num_patches` et sur le nombre de paramètres de la convolution de projection. Reliez ces observations à la discussion de la section 8.5 sur le compromis entre granularité et coût computationnel.

Ce chapitre referme la parenthèse architecturale sur les inducteurs visuels. Nous disposons désormais de tous les outils pour comprendre comment une image est découpée, projetée et organisée en séquence de tokens. Le chapitre suivant abordera la question complémentaire : faut-il discrétiser ces représentations ou les conserver continues, et quelles conséquences ce choix entraîne-t-il pour un LLM ?

Points clés à retenir

Avant de tourner la page, récapitulons les acquis essentiels de ce chapitre, car ils constituent le socle sur lequel reposent les choix architecturaux des chapitres suivants :

1. **La patchification est une opération géométrique avant d'être un apprentissage.** Qu'on l'implémente via `unfold`, une convolution stride = kernel ou un module composite, le découpage en grille régulière $P \times P$ ne dépend d'aucun paramètre appris. C'est la projection linéaire qui suit — le `patch_embed` — qui introduit la capacité de modélisation.

2. **Le ViT a démontré l'équivalence fonctionnelle entre patch et token.** En traitant chaque sous-région d'image comme un élément de séquence, Dosovitskiy et ses collaborateurs ont ouvert la voie à l'unification texte-image dans un même espace de représentation. Pour notre projet d'adapter un LLM en générateur d'images, cette équivalence est le pont conceptuel central.

3. **Le choix entre CNN et ViT comme encodeur n'est pas binaire.** Les architectures hybrides — stem convolutif suivi de blocs d'attention — offrent le meilleur compromis entre biais inductif local et capture des dépendances globales, tout en réduisant le nombre de tokens soumis à l'attention quadratique.

© 2026 Oussama Ghorbel

4. **La taille de patch P est un levier de premier ordre.** Elle détermine simultanément la longueur de séquence N, la granularité spatiale, le coût computationnel et la difficulté de la tâche de quantification. Pour un LLM 7B générant du 256×256, $P = 16$ (soit $N = 256$ tokens) constitue un point d'équilibre robuste.

5. **Le positional encoding restaure la géométrie détruite par l'aplatissement.** Sinusoidal, appris ou interpolable, il conditionne la capacité du modèle à raisonner spatialement et à généraliser à des résolutions non vues. Dans un contexte autorégressif raster-scan, son rôle est partiellement redondant avec l'ordre de génération, mais il reste indispensable côté encodeur.

Pour aller plus loin

Le lecteur souhaitant approfondir les aspects traités dans ce chapitre pourra se reporter utilement aux ressources suivantes :

• **Sur le ViT et ses variantes** : l'article fondateur de Dosovitskiy et al. (2020), puis les travaux DeiT (Touvron et al., 2021) qui introduisent la distillation et les stratégies d'augmentation permettant d'entraîner des ViT sur des jeux de données de taille modérée. Le code de référence dans la bibliothèque `timm` (Ross Wightman) offre une implémentation claire et modulaire de la patchification et des position embeddings.

• **Sur les architectures hybrides** : les articles CvT (Wu et al., 2021) et LeViT (Graham et al., 2021) détaillent la conception de stems convolutifs multi-étapes. Pour une perspective orientée génération d'images, les rapports techniques de Stable Diffusion 3 et de Cosmos Tokenizer (NVIDIA, 2024) illustrent comment ces principes sont appliqués à l'échelle industrielle.

• **Sur l'effet de la taille de patch** : l'ablation systématique présentée dans l'annexe de l'article VQGAN (Esser et al., 2021) reste une référence pédagogique. On y observe directement la dégradation de la reconstruction lorsque P passe de 8 à 32, mesurée par rFID sur ImageNet.

© 2026 Oussama Ghorbel

• **Sur les position embeddings interpolables** : la section 3.2 de l'article MAE (He et al., 2022) décrit en détail la procédure d'interpolation bilinéaire et ses limites aux facteurs d'échelle supérieurs à 2×. Le lecteur implémentant le protocole du chapitre 22 gagnera à consulter cette section avant de tenter une extrapolation de résolution.

Enfin, une remarque d'ordre méthodologique : la patchification, aussi simple soit-elle en apparence, cristallise un choix épistémologique. Découper l'image en carrés réguliers, c'est postuler que l'information visuelle se prête à une discrétisation spatiale uniforme. Or la perception humaine — et certaines architectures récentes comme les modèles à base de superpixels ou de tokens adaptatifs — suggère que cette uniformité n'est pas optimale. Ces pistes alternatives, encore marginales dans le contexte LLM, font partie des frontières explorées au chapitre 24. Pour l'heure, la grille régulière demeure le standard de facto, et c'est sur elle que nous construisons la suite de cet ouvrage.

Le chapitre suivant — le dernier de la Partie II — posera la question qui découle naturellement de tout ce qui précède : une fois les patches extraits et projetés, faut-il les *discrétiser* dans un codebook fini, ou les conserver comme vecteurs *continus* dans l'espace latent du LLM ? La réponse conditionne profondément l'architecture du générateur, la fonction de perte, la stabilité de l'entraînement et, in fine, la qualité des images produites.

© 2026 Oussama Ghorbel

Chapitre 9 – Tokenizers continus vs discrets : quel choix pour un LLM ?

Au fil des chapitres précédents, nous avons disséqué les auto-encodeurs variationnels, exploré la mécanique des codebooks VQGAN et compris comment la patchification transforme une image en séquence exploitable. Une question fondamentale demeure pourtant en suspens, et elle conditionne l'ensemble de l'architecture que vous allez construire : faut-il représenter chaque patch visuel par un **indice entier** pioché dans un dictionnaire fini, ou par un **vecteur continu** vivant dans un espace latent de dimension d ?

Ce choix, en apparence technique, irrigue chaque couche du modèle : la fonction de perte, la tête de prédiction, la stratégie d'échantillonnage, la scalabilité du vocabulaire et même la manière dont le texte et l'image cohabitent dans le même transformer. Je vais, dans ce chapitre, mettre les deux paradigmes face à face, calculs à l'appui, avant de formuler un verdict pratique que vous pourrez adapter à votre propre projet.

9.1 — Tokenisation discrète : indice entier, embedding lookup, compatible next-token

Le principe est d'une élégance presque brutale. Un encodeur E_ϕ produit, pour chaque patch, un vecteur $z_e \in \mathbb{R}^d$. On quantifie ce vecteur en sélectionnant l'entrée la plus proche dans un codebook $\{e_1, e_2, \dots, e_K\}$:

$$k^* = \arg\min_{k \in \{1, \dots, K\}} \|z_e - e_k\|_2$$

Le token visuel devient alors l'entier k^*. Côté décodage, une table d'embedding $W_{\text{emb}} \in \mathbb{R}^{K \times d_{\text{model}}}$ associe à chaque indice un vecteur de dimension d_{model}, exactement comme on le fait pour un mot du vocabulaire textuel. L'opération est une simple **lookup** : aucun calcul matriciel lourd, juste un indexage mémoire.

Pourquoi cette discrétisation est-elle si naturelle pour un LLM ? Parce que le mécanisme de **next-token prediction** repose sur une

© 2026 Oussama Ghorbel

distribution catégorielle. À chaque pas de génération, le modèle produit un logit pour chacune des K entrées du codebook, applique un softmax et tire un indice. La perte d'entraînement est une entropie croisée :

$$\mathcal{L}_{\text{CE}} = -\log p_\theta(k_t^* \mid k_{<t}^*)$$

Cette formulation est identique, à un symbole près, à celle qui régit la génération de texte. Aucune modification architecturale n'est requise : on concatène simplement les tokens visuels aux tokens textuels dans la séquence d'entrée, on étend la matrice d'embedding et la couche de projection finale, et le tour est joué. C'est précisément la voie empruntée par VQGAN+Transformer, DALL-E première génération ou encore les modèles de la famille LlamaGen.

L'inconvénient immédiat saute aux yeux : la quantification introduit une **perte d'information irréversible**. Si le codebook ne contient que $K = 8\,192$ entrées, chaque patch est contraint de se réduire à l'une de ces $8\,192$ représentations, soit $\log_2(8192) = 13$ bits d'information par token. Toute la richesse du signal original qui excède ces 13 bits est définitivement perdue au stade de la tokenisation.

9.2 — Tokenisation continue : vecteur latent, projection linéaire, perte MSE/L1

L'alternative consiste à conserver le vecteur latent $z_e \in \mathbb{R}^d$ tel quel, sans quantification. L'encodeur produit un flot continu de représentations ; le décodeur doit reconstruire l'image à partir de ces vecteurs. Pour intégrer ces vecteurs dans un LLM dont la dimension interne est d_{model}, on intercale une **projection linéaire** (ou un petit MLP) :

$$h_t = W_{\text{proj}}\, z_e^{(t)} + b_{\text{proj}}, \qquad W_{\text{proj}} \in \mathbb{R}^{d_{\text{model}} \times d}$$

En sortie, le transformer ne peut plus émettre un indice : il doit **régresser** un vecteur. On remplace donc la couche softmax par une tête linéaire de régression :

$$\hat{z}_t = W_{\text{out}}\, h_t^{(\text{final})} + b_{\text{out}}, \qquad W_{\text{out}} \in \mathbb{R}^{d \times d_{\text{model}}}$$

© 2026 Oussama Ghorbel

La fonction de perte n'est plus une entropie croisée mais une distance dans l'espace continu, typiquement une MSE ou une norme L1 :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{T}\sum_{t=1}^{T} \| \hat{z}_t - z_e^{(t)} \|_2^2$$

Cette approche préserve théoriquement toute l'information contenue dans le latent : il n'y a pas de plafond imposé par la taille d'un codebook. En contrepartie, le modèle doit apprendre à prédire des valeurs réelles dans un espace non borné, ce qui modifie profondément la dynamique d'entraînement. La distribution cible n'est plus une multinomiale à K modes mais une densité continue, souvent multimodale, que la MSE tend à lisser en une moyenne gaussienne. C'est le phénomène bien connu de **regression to the mean** : face à une incertitude, le réseau prédit la moyenne conditionnelle, ce qui se traduit visuellement par des images floues.

9.3 — Compatibilité avec la fonction softmax : discret naturel, continu nécessite une tête de régression

Le point de friction le plus immédiat entre un LLM et une représentation continue se situe au niveau de la **couche de sortie**. Un transformer autorégressif standard calcule, à chaque position t :

$$\text{logits}_t = W_{\text{lm}}\, h_t, \qquad W_{\text{lm}} \in \mathbb{R}^{|V| \times d_{\text{model}}}$$

puis applique un softmax sur $|V|$ classes. Cette opération est parfaitement définie pour un vocabulaire fini. Avec un tokenizer discret, $|V| = |V_{\text{texte}}| + K_{\text{visuel}}$, et tout rentre dans le cadre habituel.

Avec un tokenizer continu, il n'existe **aucun vocabulaire fini** sur lequel appliquer un softmax. Trois stratégies s'offrent alors à l'ingénieur :

1. **Tête de régression directe** : on prédit $\hat{z}_t \in \mathbb{R}^d$ et on optimise une MSE. Simple, mais sujet au lissage mentionné plus haut.

2. **Modélisation gaussienne** : on prédit une moyenne μ_t et une variance σ_t^2, puis on maximise la log-vraisemblance gaussienne. On gagne en expressivité, mais la multimodalité reste mal capturée.

© 2026 Oussama Ghorbel

3. **Mixture of Experts en sortie** : on prédit les paramètres d'un mélange de gaussiennes (ou de distributions de Laplace). Plus expressif, mais le nombre de paramètres de la tête explose et l'entraînement devient instable.

Dans tous les cas, la génération par **échantillonnage top-k / top-p / temperature**, qui fait la force des LLM textuels, n'a plus de sens direct. Il faut recourir à des techniques d'échantillonnage dans $\mathbb{R}^d$ (reparametrization trick, diffusion inverse, ou flow matching), ce qui éloigne le modèle du paradigme purement autorégressif.

9.4 — Scalabilité du vocabulaire : ajouter 8 192 tokens visuels vs projeter un espace continu

Supposons que vous partiez d'un LLM pré-entraîné avec un vocabulaire textuel de $|V_{\text{txt}}| = 32\,000$ tokens et une dimension $d_{\text{model}} = 4\,096$.

Scénario discret. Vous ajoutez $K = 8\,192$ tokens visuels. La matrice d'embedding passe de $32\,000 \times 4\,096$ à $40\,192 \times 4\,096$, soit environ 33 millions de paramètres supplémentaires ($8\,192 \times 4\,096 \approx 33{,}5$ M). La couche de projection en sortie subit la même expansion. C'est significatif, mais reste gérable sur un GPU moderne. Si vous souhaitez augmenter la fidélité en passant à $K = 65\,536$, le surcoût grimpe à ~ 268 M paramètres, ce qui commence à peser, surtout en inférence où la multiplication par la matrice de sortie devient un goulot.

Scénario continu. La taille du codebook n'intervient plus. Le coût additionnel se limite aux matrices de projection : $W_{\text{proj}} \in \mathbb{R}^{4096 \times d}$ et $W_{\text{out}} \in \mathbb{R}^{d \times 4096}$. Pour $d = 256$, cela représente $2 \times 256 \times 4\,096 \approx 2{,}1$ M paramètres, soit **quinze fois moins** que le scénario discret à $K = 8\,192$. La scalabilité en nombre de « tokens » est virtuellement infinie : on peut raffiner la grille de patches sans toucher au vocabulaire.

En revanche, la scalabilité en **qualité** suit une logique différente. Augmenter K dans le cas discret améliore la granularité de la quantification de manière discrète et prévisible. Augmenter d dans le cas continu améliore la capacité expressive, mais la reconstruction dépend aussi de la capacité du décodeur à exploiter ces dimensions

© 2026 Oussama Ghorbel

supplémentaires, ce qui est moins garanti.

9.5 — Qualité de reconstruction : discret plafonné par K, continu plafonné par la dim latente

Formalisons le plafond de reconstruction.

Cas discret. Chaque patch est encodé sur $\log_2 K$ bits. Pour $K = 8\,192$ et une image 256×256 découpée en patches 16×16 (soit $16 \times 16 = 256$ patches), le budget informationnel total est :

$$256 \times 13 \text{ bits} = 3\,328 \text{ bits} \approx 416 \text{ octets}$$

C'est peu. Une image PNG non compressée de $256 \times 256 \times 3$ octets pèse 196 608 octets. Le taux de compression est donc de l'ordre de $470 : 1$. Toute information au-delà de ces 416 octets est irrémédiablement perdue. Augmenter K à 65 536 (16 bits par token) double le budget à 832 octets, ce qui améliore la fidélité mais reste très contraint.

Cas continu. Le plafond est dicté par la dimension latente d et la précision numérique. En float32, chaque token porte $32 \times d$ bits. Pour $d = 256$, cela donne 8 192 bits par token, soit 63 fois plus que le codebook discret à $K = 8\,192$. La reconstruction peut donc être bien plus fidèle, à condition que le décodeur soit suffisamment capacitaire. En pratique, on observe que les VAE continus (type SD-VAE) atteignent des PSNR de reconstruction supérieurs de 3 à 5 dB par rapport à un VQGAN de codebook équivalent, au prix d'une légère tendance au flou dans les textures fines.

Le tableau suivant résume la comparaison :

Critère	Discret ($K = 8\,192$)	Continu ($d = 256$)
Bits par token	13	8 192 (float32)
Plafond de fidélité	Limité par K	Limité par d et le décodeur
Artefacts typiques	Effet « posterisation »	Flou gaussien
Sensibilité au codebook	Élevée (effondrement)	Nulle

© 2026 Oussama Ghorbel

9.6 — Verdict pratique : discret pour l'autoregression pure, continu pour les hybrides

Après cette analyse comparative, je peux formuler une recommandation éprouvée, que l'on peut appliquer lors de la conception d'un pipeline génératif.

Choisissez la tokenisation discrète si :

• Votre objectif est une **autoregression pure**, où l'image est générée token par token exactement comme une phrase. Le softmax, le sampling top-p, la perplexité, le KV-cache : tout l'outillage du LLM fonctionne sans modification.

• Vous souhaitez un **vocabulaire unifié** texte + image dans une même couche d'embedding, par exemple pour un modèle multimodal de type « tout-en-un ».

• Votre budget computationnel en inférence est limité : la lookup d'un embedding est $O(1)$, tandis qu'une régression continue impose une multiplication matricielle supplémentaire à chaque pas.

• Vous avez besoin d'une **interprétabilité** des tokens visuels (visualisation des entrées du codebook, analyse de fréquence).

Choisissez la tokenisation continue si :

• Vous construisez un **hybride autoregression + diffusion** : le LLM prédit une séquence de latents continus, puis un petit décodeur diffusion les raffine en pixels. C'est la voie suivie par Transfusion, MAR ou encore les approches de type « LLM + diffusion head ».

• La **fidélité de reconstruction** prime sur la compatibilité avec le softmax : applications médicales, imagerie scientifique, compression haute qualité.

• Vous travaillez avec un espace latent déjà pré-entraîné (par exemple le VAE de Stable Diffusion) et vous ne souhaitez pas le re-quantifier, ce qui introduirait une perte supplémentaire.

• Votre tâche est une **régression dense** (super-résolution, inpainting) où la prédiction d'un vecteur continu est plus naturelle qu'un choix parmi K catégories.

© 2026 Oussama Ghorbel

Et si vous hésitez ? Une stratégie pragmatique consiste à commencer par le discret pour valider le pipeline autorégressif, puis à basculer vers le continu une fois la stabilité d'entraînement acquise. Le surcoût d'ingénierie est modéré : il suffit de remplacer la couche de quantification par une projection linéaire et la tête softmax par une tête de régression. Les poids du transformer central, eux, restent réutilisables.

Un dernier mot sur les **approches hybrides récentes**. Des travaux comme ceux de MAR (Masked Autoregressive) ou de Transfusion montrent qu'il est possible de combiner les deux mondes : une partie de la séquence est traitée de manière discrète (tokens textuels), l'autre de manière continue (latents d'image), avec une fonction de perte composite $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{MSE}}$. Cette voie me semble la plus prometteuse à moyen terme, car elle préserve la compatibilité du LLM avec le texte tout en offrant la fidélité du continu pour l'image.

Exercices

Exercice 9.1 — Adapter GPT-2 à des embeddings continus.

Modifiez une implémentation standard de GPT-2 (par exemple via la bibliothèque HuggingFace Transformers) de la manière suivante :

• Remplacez la couche `wte` (word token embedding) par une projection linéaire $\mathbb{R}^{256} \to \mathbb{R}^{768}$ qui accepte des vecteurs latents continus en entrée.

• Remplacez la tête `lm_head` (projection vers le vocabulaire) par une tête de régression linéaire $\mathbb{R}^{768} \to \mathbb{R}^{256}$.

• Entraînez sur un petit jeu de latents d'images (vous pouvez utiliser les latents produits par le VAE de Stable Diffusion sur 1 000 images CIFAR-10).

• Mesurez la MSE de reconstruction après 1 000, 5 000 et 20 000 pas d'entraînement. Comparez avec la MSE obtenue par un simple auto-encodeur linéaire de même dimension. Que constatez-vous sur la capacité du transformer à modéliser la distribution continue ?

Exercice 9.2 — Perplexité : discret vs continu projeté.

© 2026 Oussama Ghorbel

Prenez une séquence de 1 024 tokens visuels discrets (indices de codebook VQGAN). Calculez la perplexité d'un petit GPT-2 entraîné sur ces séquences. Ensuite, remplacez chaque indice par son embedding de codebook (un vecteur de dimension d), projetez-le linéairement dans l'espace du modèle et mesurez la « pseudo-perplexité » définie par :

$$\mathrm{PPL}_{\mathrm{cont}} = \exp\left(\frac{1}{T}\sum_{t=1}^{T} \|\hat{z}_t - z_t\|_2^2\right)$$

Comparez les deux valeurs. La pseudo-perplexité continue est-elle plus élevée ou plus basse ? Comment interprétez-vous l'écart en termes de difficulté de modélisation ?

Exercice 9.3 — Arbre de décision « discret ou continu ? ».

Rédigez, sous forme d'arbre de décision textuel ou graphique, un guide en cinq à sept questions permettant de trancher entre tokenisation discrète et continue. Les questions devront porter au minimum sur :

- Le modèle cible est-il un LLM pré-entraîné à vocabulaire fixe ou un modèle entraîné from scratch ?
- La tâche est-elle de la génération pure, de l'édition d'image ou de la régression dense ?
- Le budget inférence autorise-t-il une tête de diffusion en aval ?
- La taille du codebook envisagée dépasse-t-elle 16 384 entrées ?
- A-t-on besoin d'un échantillonnage top-k / top-p natif ?

Justifiez chaque branche de l'arbre en une ou deux phrases, en vous appuyant sur les arguments développés dans ce chapitre.

Résumé des points clés

Avant de tourner la page, je souhaite cristalliser les idées essentielles de ce chapitre en une liste que vous pourrez afficher au-dessus de votre écran lors de vos prochaines séances de conception architecturale :

1. **La tokenisation discrète transforme l'image en langage.** Chaque patch devient un entier, chaque séquence visuelle devient une « phrase » que le mécanisme de next-token prediction sait déjà traiter. Le coût est un plafond informationnel de $\log_2 K$ bits par token.

© 2026 Oussama Ghorbel

2. **La tokenisation continue préserve la géométrie du latent.** Aucune quantification, aucune perte de granularité imposée par un dictionnaire fini. Le coût est l'incompatibilité avec le softmax et la nécessité d'une tête de régression, avec son cortège de problèmes de lissage.

3. **Le softmax est la ligne de fracture.** Tant que votre pipeline repose sur une distribution catégorielle en sortie, le discret s'impose. Dès que vous acceptez une perte en norme L2 ou une modélisation gaussienne, le continu devient viable.

4. **La scalabilité n'est pas symétrique.** Ajouter des entrées au codebook gonfle linéairement les matrices d'embedding et de projection ($O(K \times d_{\text{model}})$). Augmenter la dimension latente continue ne coûte que $O(d \times d_{\text{model}})$, indépendamment du nombre de patches.

5. **Le verdict n'est pas dogmatique.** Discret pour l'autoregression pure et les vocabulaires unifiés ; continu pour les hybrides diffusion, la régression dense et la fidélité maximale. Les approches composites de type Transfusion montrent que la frontière s'estompe.

Pour aller plus loin

Quelques références que je considère comme des points d'ancrage pour approfondir chacun des axes abordés :

• Sur la quantification vectorielle et ses pièges (effondrement de codebook, commitment loss) : van den Oord, Vinyals et Kavukcuoglu, « Neural Discrete Representation Learning » (NeurIPS 2017), qui introduit VQ-VAE.

• Sur les tokenizers d'images haute fidélité : Esser, Rombach et Ommer, « Taming Transformers for High-Resolution Image Synthesis » (CVPR 2021), qui popularise VQGAN.

• Sur la génération continue par masking : Li, Fan, Wang, Xu, Krishnan et al., « MAR: Masked Autoregressive for Autoregressive Generation » (2024).

• Sur l'hybridation autoregression-diffusion dans un même transformer : Gong, Ran, Liu, Wang et al., « Transfusion: Predict the

© 2026 Oussama Ghorbel

Next Token and Diffuse Images with One Multi-Modal Model » (2024).

• Sur les VAE continus utilisés comme espace latent de diffusion : Rombach, Blattmann, Lorenz, Esser et Ommer, « High-Resolution Image Synthesis with Latent Diffusion Models » (CVPR 2022).

Je vous encourage à lire ces articles dans cet ordre : ils tracent une progression chronologique et conceptuelle qui éclaire directement les arbitrages discutés ici.

Transition vers la Partie III

Nous savons désormais **quoi** mettre dans la séquence : des indices discrets ou des vecteurs continus. La question suivante, tout aussi structurante, est **dans quel ordre** disposer ces tokens. Une image n'est pas une phrase : elle n'a pas de début ni de fin naturels. Le raster scan de gauche à droite, de haut en bas, est un choix par défaut hérité de l'affichage CRT, mais rien n'interdit d'explorer des ordres de balayage en serpentin, en courbe de Hilbert, en spirale, voire des ordres appris. Chacun de ces parcours induit une factorisation différente de la distribution jointe $p(x_1, x_2, \ldots, x_T)$, et donc une difficulté d'apprentissage différente pour le transformer.

C'est précisément l'objet du chapitre 10, où nous formaliserons la factorisation autorégressive de l'image, puis du chapitre 11, où nous comparerons systématiquement les ordres de balayage. Vous verrez que le choix de l'ordre peut faire varier la qualité générative de plusieurs points de FID, à architecture et budget computationnel constants. Un paramètre apparemment anodin, qui se révèle décisif.

En attendant, si vous n'avez pas encore réalisé les trois exercices de ce chapitre, je vous invite à le faire maintenant. Le passage du discret au continu sur un petit GPT-2, même maladroit, même imperfectible, ancre dans vos doigts une intuition que la lecture seule ne peut pas donner. Vous constaterez par vous-même à quel point la MSE de reconstruction chute vite dans les premiers milliers de pas, puis stagne — et vous comprendrez, de l'intérieur, pourquoi la tête de régression a besoin d'aide pour capturer la multimodalité. C'est cette expérience concrète qui

© 2026 Oussama Ghorbel

rendra les chapitres suivants non plus abstraits, mais évidents.

Cette expérience, justement, mérite quelques garde-fous. Je termine donc ce chapitre par trois annexes courtes : un squelette d'implémentation, un calcul de coût que j'ai laissé trop vite de côté, et une mise au point sur ce que signifient — et ne signifient pas — les nombres que vous allez afficher dans vos tableaux de résultats.

Annexe 9.A — Un squelette minimal pour GPT-2 continu

Le premier réflexe à éviter est de chercher à « convertir les vecteurs en indices » pour les faire entrer dans `input_ids`. Vous n'avez pas d'indices : vous avez des embeddings. Or les transformers modernes acceptent déjà cette forme d'entrée, via `inputs_embeds`. Le chemin est donc plus court qu'il n'y paraît.

Notez le choix de `GPT2Model` plutôt que `GPT2LMHeadModel`. Le second instancierait une tête de langue avec softmax sur un vocabulaire factice de taille un : exactement la confusion que nous cherchons à dissiper. Ici, la seule sortie du modèle est un vecteur réel, et la seule perte est une distance.

Quatre détails font la différence entre un prototype qui converge et un prototype qui semble converger.

Normalisez les latents. Calculez la moyenne et l'écart-type de chaque canal sur le seul ensemble d'entraînement, puis figez ces statistiques. Un VAE ne produit pas spontanément un latent centré réduit : certains canaux peuvent avoir une variance dix fois supérieure à d'autres. Sans normalisation, votre MSE sera dominée par quatre ou cinq directions du latent, et le modèle apprendra à les prédire en négligeant le reste. Conservez ces statistiques dans le checkpoint : elles font partie du modèle, au même titre que les poids.

Ne décalez pas deux fois. `z[:, :-1]` en entrée, `z[:, 1:]` en cible : c'est tout. Le masque causal du transformer fait déjà le travail d'auto régression. Ajouter un second décalage, réflexion héritée de certains pipelines de texte, produirait un modèle qui prédit le token d'il y a deux pas — et qui obtient nevertheless une MSE flatteuse.

© 2026 Oussama Ghorbel

Connaissez votre baseline. Sur des latents normalisés, prédire constamment le vecteur nul donne une MSE d'environ 1. C'est ce chiffre qu'il faut battre, pas zéro. Rapportez aussi la MSE par canal : une moyenne globale de 0,03 peut cacher quinze canaux à 0,001 et un canal à 0,5.

Mesurez en génération libre. La MSE en *teacher forcing* — où le modèle reçoit à chaque pas le vrai token précédent — peut être excellente alors que le déroulement autoregressif de 256 pas dérive vers une orbite hors distribution. À la fin de chaque évaluation, générez sans apport extérieur, décodez en pixels, et regardez l'image. Le fossé entre les deux régimes est l'un des diagnostics les plus instructifs de tout ce chapitre.

Pour un modèle multimodal, ne supprimez pas la table d'embedding textuelle. Conservez-la, ajoutez la branche visuelle en parallèle, et routez chaque position selon sa modalité. La perte devient composite :

$$\mathcal{L} = \frac{1}{|T_{\text{texte}}|} \sum_{t \in T_{\text{texte}}} \text{CE}_t \;+\; \lambda \frac{1}{|T_{\text{image}}|} \sum_{t \in T_{\text{image}}} \|\hat{z}_t - z_t\|_2^2$$

Normalisez chaque terme par le nombre de tokens de sa modalité. Sinon, λ changera de sens effective chaque fois que vous passerez de 16 × 16 à 32 × 32 patches, et vos ablations ne compareront plus rien.

Annexe 9.B — Le coût réel de la tête de sortie

J'ai écrit plus haut que la lookup d'un embedding était $O(1)$ et que la régression continue imposait une multiplication supplémentaire. Cette phrase est exacte du côté de l'entrée et trompeuse du côté de la sortie. Il faut séparer les deux.

Côté	Discret	Continu
Entrée	lecture d'une ligne : $O(d_{\text{model}})$	projection : $O(d \times d_{\text{model}})$
Sortie	matrice $\|V\| \times d_{\text{model}}$ puis softmax	matrice $d \times d_{\text{model}}$, sans softmax

Repreneons les chiffres de la section 9.4 : $d_{\text{model}} = 4\,096$, vocabulaire étendu $|V| = 40\,192$, latent de dimension $d = 256$.

© 2026 Oussama Ghorbel

La tête discrète effectue environ $4\,096 \times 40\,192 \approx 165$ millions d'accumulations par token. La tête continue en effectue $4\,096 \times 256 \approx 1{,}05$ million. **Près de cent cinquante fois moins.** Et ce n'est pas seulement une question d'opérations : pour une séquence de 256 patches, le tenseur de logits discret pèse environ 41 Mo en float32, contre 0,26 Mo pour les sorties continues. En entraînement avec macro-lots, et en serving avec plusieurs requêtes concurrentes, cette différence tient parfois dans la mémoire GPU là où l'autre nécessite du *chunking* ou du *fused cross-entropy*.

La lookup d'entrée est effectivement gratuite ; la projection d'entrée ne l'est pas. Mais elle est minuscule devant le coût de la tête de sortie discrète.

La conclusion honnête est donc la suivante : **à dimension de modèle fixée, une tête continue est souvent moins chère qu'une tête discrète sur un grand vocabulaire.** Ce qui renverse le verdict, ce n'est pas la tête, c'est ce qu'il y a après. Si vos latents continus doivent ensuite passer dans un décodeur diffusion exécuté vingt fois, ce décodeur dominera largement le budget et l'économie de la tête deviendra anecdotique. À l'inverse, si un décodeur convolutif unique suffit, le pipeline continu peut être plus rapide que son équivalent discret.

Le coût n'est pas une propriété du tokenizer. C'est une propriété du pipeline complet, et il se mesure en secondes par image, pas en arguments architectoniques.

Annexe 9.C — Des pertes qui n'ont pas la même unité

La « pseudo-perplexité » proposée dans l'exercice 9.2 est un outil de suivi. Ce n'est pas une métrique de comparaison.

Une perplexité vaut $\exp(\overline{\mathcal{L}}_{\text{CE}})$: l'entropie croisée est en nats par token, l'exponentielle est adimensionnelle, et la valeur a une interprétation — le nombre effectif de choix indistinguables par pas. Une exponentielle de MSE n'a aucune de ces propriétés. La MSE est exprimée dans le carré de l'unité de votre latent : changez l'échelle de normalisation, et votre « perplexité » change sans que le modèle ait changé. La comparer à une vraie perplexité revient à comparer des kilomètres à des degrés.

© 2026 Oussama Ghorbel

Deux sorties honnêtes existent si vous tenez à un chiffre probabiliste côté continu.

La première consiste à **quantifier les prédictions** sur une grille finie, puis à calculer une entropie croisée sur cette grille. Le résultat est comparable à celui d'un codebook, à une condition explicite : préciser le pas de quantification. Un pas deux fois plus fin ajoute un bit par dimension et améliore mécaniquement la vraisemblance, sans améliorer d'un pixel l'image reconstruite.

La seconde consiste à **maximiser une vraisemblance gaussienne** et à rapporter des nats par dimension. Attention : une densité différentielle peut produire une log-vraisemblance positive, donc une NLL négative. Ce n'est pas un bug. Et ce chiffre n'est toujours pas comparable à $\log_2 K$.

Pour replacer les deux paradigmes dans un cadre commun, il faut raisonner en **taux–distortion**. Prenons un exemple chiffré. Votre latent est normalisé, chaque coordonnée ayant une variance proche de 1. Votre générateur atteint une MSE de 0,01 par coordonnée. Pour une source gaussienne i.i.d., la fonction taux–distortion donne :

$$R(D) = \tfrac{1}{2}\log_2 \tfrac{\sigma^2}{D} = \tfrac{1}{2}\log_2 100 \approx 3{,}32 \text{ bits par dimension}$$

À $d = 256$, cela représente environ 850 bits par token pour décrire ce latent à la précision où votre modèle le prédit — soit une soixantaine de fois les 13 bits d'un codebook à 8 192 entrées.

Ce calcul mérite d'être médité. Stocker un vecteur en float32 ne vous offre pas 8 192 bits d'information *utile* : cela vous offre 8 192 bits de conteneur. La quantité d'information réellement nécessaire pour spécifier le latent à une distortion donnée est bien plus faible, mais elle n'est pas négligeable pour autant. Le continu ne supprime pas le budget informationnel ; il le rend continu, négociable, et dépendant de la précision que vous exigez.

Ce calcul a ses limites, et je les indique sans détour. Il suppose des coordonnées indépendantes et gaussiennes, alors qu'un auto-encodeur apprend précisément des corrélations et une géométrie structurée. Il ne préjuge pas de ce que fera un quantificateur vectoriel, qui exploite la jointure à l'intérieur d'un patch et non la marginalle de chaque canal. C'est un ordre de grandeur destiné à calmer les comparaisons hâtives,

© 2026 Oussama Ghorbel

pas un théorème sur votre modèle.

La conséquence pratique est simple. Ne tranche pas entre discret et continu en comparant une perplexité à une exponentielle de MSE. Mesure trois choses séparément : le plafond du tokenizer (encodez-décodez un lot fixe, rapportez PSNR et LPIPS) ; la qualité du générateur (FID et KID sur un nombre fixe d'images, avec graines figées) ; le coût réel (secondes par image, mémoire crête, latence en queue). Si le plafond du tokenizer est mauvais, aucun changement de tête ne vous sauvera. Si le plafond est bon et que le FID se dégrade, le problème est dans la modélisation de la distribution — et c'est là que le choix du chapitre entier se joue.

Le tokenizer n'est pas un préprocesseur que l'on choisit une fois puis que l'on oublie. C'est un contrat passé entre la représentation que vous savez produire et la distribution que votre modèle sait modéliser. Un codebook vous impose un vocabulaire et vous offre un softmax. Un latent vous offre une géométrie et vous impose une tête. Dans les deux cas, vous ne faites pas disparaître la difficulté : vous la déplacez.

La bonne question n'est donc pas « laquelle de ces deux représentations est la meilleure ? », mais « laquelle me laisse une marge de manœuvre là où mon projet en aura le plus besoin ? ». Pour un LLM que l'on étend à l'image avec un budget serré et un objectif de génération pure, ma réponse reste discrète. Pour un système qui doit conjuguer texte, image et précision numérique, elle est continue — et, de plus en plus souvent, les deux à la fois.

© 2026 Oussama Ghorbel

Partie III — Modélisation Autorégressive : Ordre, Balayage et Génération

© 2026 Oussama Ghorbel

Chapitre 10 – Factorisation autoregressive de l'image : p(x) = p(x_i | x_{<i})

Dans les chapitres précédents, nous avons établi que l'image, une fois discrétisée par un tokenizer, devient une séquence de tokens. Il reste maintenant à répondre à la question fondamentale : comment modéliser la distribution de probabilité sur cette séquence ? Ce chapitre formalise la réponse que l'apprentissage profond a empruntée à la théorie des chaînes de Markov : la décomposition en chaîne. Nous allons voir que cette factorisation, d'une élégance mathématique presque trop simple, porte en elle toutes les promesses — et toutes les difficultés — de la génération d'images par tokens.

10.1 — Chaîne de Markov et factorisation jointe sur les tokens visuels

Considérons une image représentée par une séquence de N tokens discrets $\mathbf{x} = (x_1, x_2, \ldots, x_N)$, où chaque x_i appartient à un vocabulaire $\mathcal{V}$ de taille K (typiquement $K = 1024$ ou $K = 8192$ selon le codebook). La distribution jointe $p(\mathbf{x})$ décrit la probabilité d'observer exactement cette configuration de tokens. Par la règle du produit (chain rule) en théorie des probabilités, toute distribution jointe se décompose de manière exacte :

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid x_1, x_2, \ldots, x_{i-1}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{<i})$$

Cette identité ne repose sur aucune hypothèse d'indépendance. Elle est toujours vraie. La difficulté, naturellement, réside dans l'estimation de chaque facteur conditionnel $p(x_i \mid \mathbf{x}_{<i})$, dont la complexité croît exponentiellement avec i si l'on tente de le modéliser par une table. C'est ici qu'intervient le réseau de neurones : un transformeur paramétré par θ approxime chaque facteur conditionnel par une softmax sur le vocabulaire :

$$p_\theta(x_i \mid \mathbf{x}_{<i}) = \mathrm{softmax}(h_\theta(\mathbf{x}_{<i}))_{x_i}$$

où h_θ désigne la dernière couche cachée du modèle. L'entraînement consiste alors à minimiser la log-vraisemblance négative :

$$\mathcal{L}(\theta) = -\sum_{\mathbf{x} \in \mathcal{D}} \sum_{i=1}^{N} \log p_\theta(x_i \mid \mathbf{x}_{<i})$$

Ce qui rend cette formulation puissante, c'est qu'elle transforme un problème de modélisation de distribution en N dimensions discrètes en N problèmes de classification supervisée. Chaque position i fournit un signal d'apprentissage, et le gradient circule à travers toutes les positions simultanément grâce au masquage causal (causal mask) de l'attention.

Une remarque importante : cette factorisation impose un **ordre total** sur les tokens. L'ordre raster (ligne par ligne, de gauche à droite, de haut en bas) est le plus courant, mais il n'est en rien obligatoire. Nous explorerons les alternatives au chapitre 11. Pour l'heure, retenons que le choix de l'ordre détermine quelles informations sont disponibles au moment de générer chaque token, et donc structure la difficulté du problème.

10.2 — PixelCNN, PixelRNN : historique de la modélisation pixel-par-pixel

L'idée de factoriser une image pixel par pixel n'est pas née avec les transformeurs. Dès 2016, van den Oord et ses collègues ont proposé PixelRNN, qui modélise chaque pixel comme une fonction des pixels situés « au-dessus et à gauche » dans l'ordre raster, à l'aide de réseaux récurrents bidimensionnels. Peu après, PixelCNN a remplacé les RNN par des convolutions masquées (masked convolutions), garantissant que le champ réceptif d'un pixel ne contienne jamais d'information « future ».

La factorisation appliquée directement aux pixels s'écrit :

$$p(\mathbf{x}) = \prod_{i=1}^{H} \prod_{j=1}^{W} \prod_{c=1}^{C} p(x_{i,j,c} \mid x_{<i,\,<j}, x_{i,\,<j}, x_{i,j,\,<c})$$

Pour une image 256×256 en RGB, cela donne $256 \times 256 \times 3 = 196\,608$ facteurs conditionnels. Chaque facteur porte sur une variable à 256 valeurs (8 bits par canal). La longueur de séquence est donc colossale, et la génération strictement séquentielle exige près de 200 000 passes forward pour produire une seule image.

© 2026 Oussama Ghorbel

PixelCNN a introduit deux innovations majeures : le **masquage en croix** (cross-shaped mask), qui limite le contexte aux pixels déjà générés, et la **distribution logistique mixte** (mixture of logistics) pour modéliser les valeurs continues des canaux. PixelCNN++ a ensuite raffiné ces mécanismes avec des connexions résiduelles et une paramétrisation plus fine de la mixture.

Malgré des résultats visuels remarquables pour l'époque, ces approches souffraient d'un défaut structurel : la longueur de séquence. On peut résumer cette limitation par une analogie simple : générer une image pixel par pixel, c'est comme écrire un roman lettre par lettre en vérifiant la cohérence à chaque caractère. Le mécanisme fonctionne, mais le coût temporel rend l'approche impraticable à haute résolution. C'est précisément ce constat qui a motivé le passage aux tokens appris et aux patches.

10.3 — Passage aux patches : réduire la longueur de séquence de H·W·C à (H/p)·(W/p)

La tokenisation visuelle, détaillée aux chapitres 5 à 9, opère une compression spatiale spectaculaire. Un VQGAN avec un facteur de downsampling $f = 16$ transforme une image 256×256 en une grille de latents $16 \times 16 = 256$ tokens. La factorisation devient :

$$p(\mathbf{z}) = \prod_{i=1}^{256} p(z_i \mid \mathbf{z}_{<i})$$

avec $z_i \in \{1, \ldots, K\}$. On est passé de 196 608 facteurs à 256 facteurs : une réduction d'un facteur ~ 770. Si l'on utilise des patches de taille $p \times p$ extraits directement (à la manière du Vision Transformer), la longueur de séquence pour une image $H \times W$ devient $(H/p) \cdot (W/p)$. Pour $H = W = 256$ et $p = 16$, on obtient $16 \times 16 = 256$ tokens.

Cette compression n'est pas sans conséquence. Chaque token encode désormais une région entière de l'image ; la distribution conditionnelle $p(z_i \mid \mathbf{z}_{<i})$ doit capturer des dépendances spatiales à plus longue portée, mais sur une séquence plus courte. Le compromis est favorable : le transformeur, dont la complexité en attention est quadratique en N, voit son coût réduit de plusieurs ordres de grandeur.

© 2026 Oussama Ghorbel

Formellement, si ϕ désigne l'encodeur du tokenizer et ψ son décodeur, le pipeline complet s'écrit :

1. **Encodage** : $\mathbf{z} = \phi(\mathbf{x})$, avec $\mathbf{z} \in \{1, \ldots, K\}^N$.
2. **Modélisation** : apprentissage de $p_\theta(\mathbf{z}) = \prod_i p_\theta(z_i \mid \mathbf{z}_{<i})$.
3. **Décodage** : $\hat{\mathbf{x}} = \psi(\hat{\mathbf{z}})$, où $\hat{\mathbf{z}}$ est échantillonné de p_θ.

La qualité finale dépend donc de deux termes d'erreur : la perte de reconstruction du tokenizer (irréversible si l'on utilise la quantification) et l'erreur de modélisation de la distribution des codes. Nous reviendrons sur cette décomposition de l'erreur au chapitre 18.

10.4 — Le problème de l'exposition (exposure bias) en génération séquentielle

Lors de l'entraînement, le modèle observe toujours le **vrai** préfixe $\mathbf{x}_{<i}$ issu des données. À l'inférence, en revanche, il doit conditionner sur ses **propres prédictions** $\hat{x}_1, \ldots, \hat{x}_{i-1}$, qui peuvent être erronées. Cet écart entre les conditions d'entraînement et d'inférence porte un nom : l'exposure bias.

En génération de texte, ce phénomène est bien documenté : une petite erreur early dans la séquence peut dériver vers une incohérence globale. Pour les tokens visuels, le problème se manifeste de manière spécifique. Supposons que le token $\hat{z}_7$ soit mal prédit. Tous les tokens ultérieurs $z_8, z_9, \ldots$ seront conditionnés sur une information corrompue. Dans une image, cela se traduit par des **artefacts de propagation** : une texture incohérente qui s'étend en diagonale (suivant l'ordre raster), un objet qui se déforme progressivement, ou une rupture brutale de cohérence chromatic.

Plusieurs stratégies ont été proposées pour atténuer ce biais :

• **Scheduled sampling** : à l'entraînement, remplacer aléatoirement certains tokens du vrai préfixe par les prédictions du modèle, avec une probabilité qui croît au fil des epochs.

• **Entraînement avec bruit sur les tokens** : injecter un faible taux d'erreur dans le préfixe pour rendre le modèle robuste.

• **Génération semi-autorégressive** (MaskGIT, MAR) : générer plusieurs tokens en parallèle pour réduire la longueur de la chaîne

© 2026 Oussama Ghorbel

d'erreurs. Nous détaillerons ces approches au chapitre 12.

• **Rééchantillonnage itératif** : régénérer plusieurs fois la même séquence et sélectionner la plus cohérente selon un score externe.

En pratique, il est fréquent de constater que l'exposure bias est particulièrement sévère lorsque le codebook est de grande taille ($K \geq 8192$) : la distribution conditionnelle est alors très piquée, et une seule erreur de token crée une discontinuité que le modèle ne peut pas « rattraper » facilement. À l'inverse, un codebook plus petit ($K = 256$) produit des distributions plus lisses, au prix d'une fidélité moindre.

10.5 — Teacher forcing à l'entraînement, échantillonnage autoregressif à l'inférence

La dualité entraînement/inférence mérite d'être explicitée clairement, car elle constitue le cœur opérationnel de tout modèle autoregressif.

À l'entraînement (teacher forcing). On présente au modèle la séquence complète $\mathbf{z} = (z_1, \ldots, z_N)$. Grâce au masque causal, chaque position i ne voit que $\mathbf{z}_{<i}$. Le modèle produit en un seul forward pass les N distributions conditionnelles simultanément. La perte est la cross-entropy moyenne sur toutes les positions :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(z_i \mid \mathbf{z}_{<i})$$

L'avantage est considérable : pas de boucle séquentielle, parallélisation complète sur GPU, gradient stable. C'est exactement le mécanisme utilisé pour entraîner GPT sur du texte, transposé aux tokens visuels.

À l'inférence (échantillonnage autoregressif). On part d'un token de début ⟨BOS⟩ et l'on génère séquentiellement :

Chaque étape nécessite un forward pass (ou une mise à jour incrémentale du cache KV). Pour $N = 256$ tokens, cela représente 256 appels au modèle. C'est le prix de la génération autoregressive stricte, et c'est ce qui motive les recherches sur la génération parallèle (chapitre 12) et les techniques d'inférence accélérée (chapitre 23).

© 2026 Oussama Ghorbel

Une subtilité souvent négligée : lors de l'entraînement, le modèle apprend aussi à prédire le token ⟨EOS⟩ qui signale la fin de l'image. Si l'on omet ce signal, le modèle peut générer des séquences de longueur incohérente ou produire des artefacts de « continuation » au-delà de la grille attendue.

10.6 — Température, top-k, top-p appliqués aux tokens visuels

Les stratégies d'échantillonnage développées pour le texte se transposent directement aux tokens visuels, mais leurs effets perceptuels sont radicalement différents.

Température τ. On divise les logits par τ avant la softmax :

$$p(x_i = k \mid \mathbf{x}_{<i}) = \frac{\exp(\ell_k/\tau)}{\sum_j \exp(\ell_j/\tau)}$$

- $\tau < 1$ (ex. 0,7) : la distribution se concentre sur les tokens les plus probables. L'image générée est plus « sûre », plus lisse, mais tend vers la moyenne du corpus. Les textures manquent de diversité.
- $\tau = 1$: distribution d'entraînement.
- $\tau > 1$ (ex. 1,5) : la distribution s'aplatit. Le modèle explore des tokens moins probables, ce qui introduit de la créativité mais aussi des incohérences : couleurs improbables, textures disjointes, objets hybridés.

Top-k. On restreint l'échantillonnage aux k tokens les plus probables. Pour les tokens visuels, un k trop faible (ex. $k = 5$) fige la palette de couleurs et produit des images très homogènes. Un k élevé (ex. $k = 200$) restaure la diversité mais réintroduit du bruit.

Top-p (nucleus sampling). On sélectionne le plus petit ensemble de tokens dont la masse de probabilité cumulée dépasse un seuil p (ex. $p = 0,9$). Cette méthode est adaptative : dans les régions « faciles » de l'image (ciel uniforme), l'ensemble est petit ; dans les régions complexes (textures, contours), il s' largit. On observe que le top-p fonctionne particulièrement bien pour les tokens visuels, car la distribution conditionnelle y est souvent multimodale (plusieurs textures plausibles au même endroit).

© 2026 Oussama Ghorbel

Spécificité visuelle. Contrairement au texte, où une erreur de token produit un mot incorrect mais souvent « lisible », une erreur de token visuel se traduit par un patch de couleur ou de texture radicalement différent. La tolérance à l'erreur est donc plus faible. En pratique, je recommande une température modérée ($\tau \in [0,8, 1,0]$) combinée à un top-p de 0,9 à 0,95 pour la génération d'images, en évitant les températures extrêmes qui dégradent rapidement la cohérence structurelle.

Un dernier point : ces hyperparamètres d'échantillonnage peuvent être modulés **au cours de la génération**. Par exemple, utiliser une température basse pour les premiers tokens (qui encodent la structure globale de l'image) puis l'augmenter légèrement pour les derniers tokens (qui raffinent les textures locales). Cette stratégie, que l'on peut qualifier de « température décroissante inversée », donne des résultats empiriquement supérieurs dans de nombreux prototypes.

Exercices

Exercice 10.1 — Factorisation complète : pixels bruts vs patches.

Considérez une image 4×4 en RGB ($C = 3$).

(a) Écrivez la factorisation jointe complète en pixels bruts, en supposant un ordre raster sur les pixels et un ordre séquentiel sur les canaux (R, G, B) pour chaque pixel. Combien de facteurs conditionnels obtenez-vous ? Chaque facteur porte sur combien de valeurs possibles ?

(b) Supposez maintenant que l'on applique une patchification 2×2 : chaque patch regroupe $2 \times 2 \times 3 = 12$ valeurs, qui sont encodées en un unique token discret via un codebook de taille $K = 512$. Écrivez la factorisation jointe sur les tokens de patches. Combien de facteurs obtenez-vous ? Comparez la longueur de séquence et la taille du vocabulaire avec le cas (a).

(c) Quelle est la réduction du nombre total de facteurs ? Quel est le coût en termes d'information (chaque token de patch doit encoder 12 valeurs simultanément) ?

© 2026 Oussama Ghorbel

Exercice 10.2 — Boucle de génération autoregressive sur 64 tokens.

Implémentez, en PyTorch ou JAX, une boucle de génération autoregressive complète :

- Initialisez un petit transformeur causal (2 couches, 4 têtes, dimension 128, vocabulaire $K = 256$).
- Écrivez une fonction `generate(model, seq_len=64, temperature=1.0, top_k=50)` qui produit une séquence de 64 tokens en échantillonnant token par token.
- Utilisez le mécanisme de **cache KV** pour éviter de recalculer les couches d'attention à chaque étape.
- Mesurez le temps de génération avec et sans cache. Combien de fois le cache accélère-t-il la génération ?

Prolongement : tracez la log-probabilité moyenne du token échantillonné en fonction de la position i. Observez-vous une tendance ? Comment l'interprétez-vous ?

Exercice 10.3 — Effet de la température sur la qualité perçue.

À partir d'un modèle autoregressif entraîné sur un petit jeu de données (par exemple CIFAR-10 tokenisé avec un VQ-GAN pré-entraîné) :

- Générez 50 images à chacune des températures $\tau \in \{0,7,\ 0,9,\ 1,0,\ 1,2,\ 1,5\}$.
- Calculez le FID de chaque ensemble par rapport aux images réelles.
- Calculez également la variance intra-ensemble (diversité) : moyenne des distances LPIPS par paires au sein de chaque ensemble.
- Tracez les deux métriques en fonction de τ. Où se situe le compromis optimal qualité/diversité ?
- Inspectez visuellement trois images à $\tau = 0,7$ et trois à $\tau = 1,5$. Décrivez les artefacts spécifiques à chaque régime.

© 2026 Oussama Ghorbel

Ce chapitre a posé le cadre formel : la factorisation en chaîne transforme la génération d'image en une séquence de décisions discrètes conditionnées. Le chapitre suivant explorera la question que nous avons laissée en suspens : cet ordre de génération est-il optimal, et quelles alternatives l'ordre raster peut-il offrir ?

Ce qu'il faut retenir

Avant de tourner la page, je souhaite condenser les idées forces de ce chapitre en une poignée de propositions que le lecteur pourra garder en tête comme autant de repères pour la suite de l'ouvrage.

Premièrement, la factorisation en chaîne n'est pas une approximation : c'est une identité probabiliste exacte. Toute distribution jointe sur N variables discrètes se décompose en N facteurs conditionnels. Le choix d'architecture (transformeur, RNN, CNN masqué) n'affecte pas la validité de cette décomposition ; il détermine uniquement la capacité du modèle à estimer chaque facteur avec précision. Cette distinction entre le *cadre formel* et le *choix d'implémentation* est essentielle pour éviter les confusions que l'on rencontre fréquemment dans la littérature et les discussions entre praticiens.

Deuxièmement, la longueur de séquence est le paramètre qui gouverne à la fois le coût computationnel et la qualité de génération. Passer de 196 608 pixels bruts à 256 tokens VQGAN divise le coût quadratique de l'attention par un facteur supérieur à 10^5. Mais cette compression transfère la difficulté : chaque token doit désormais encoder une région spatiale entière, et la distribution conditionnelle doit capturer des dépendances à plus longue portée. Il n'y a pas de repas gratuit ; il y a un déplacement du problème.

Troisièmement, l'exposure bias n'est pas un artefact théorique. Il se manifeste concrètement par des artefacts de propagation le long de l'ordre de balayage. Toute stratégie de génération autoregressive sérieuse doit intégrer un mécanisme d'atténuation, que ce soit le scheduled sampling, la génération semi-parallèle ou le rééchantillonnage itératif.

Quatrièmement, les hyperparamètres d'échantillonnage (température, top-k, top-p) ne sont pas de simples « boutons de créativité

© 2026 Oussama Ghorbel

». Ils modifient la géométrie de l'espace latent exploré et, par conséquent, la structure même des images produites. Une température trop basse fige la diversité ; une température trop élevée détruit la cohérence. Le réglage optimal dépend du codebook, de la résolution et de la tâche aval.

Cinquièmement, la dualité teacher forcing / échantillonnage autoregressif implique que le modèle est entraîné dans des conditions qu'il ne rencontrera jamais à l'inférence. Cette asymétrie est la source profonde de nombreux artefacts observés en génération séquentielle, et elle motive une partie significative des travaux de recherche actuels sur les méthodes non autoregressives et semi-autoregressives.

Pour aller plus loin

Le lecteur souhaitant approfondir les fondements formels et les développements récents trouvera un point d'entrée dans les références suivantes, citées ici non par exhaustivité, mais parce qu'elles structurent fondamentalement la compréhension du sujet.

La décomposition en chaîne elle-même relève de la théorie élémentaire des probabilités ; tout ouvrage de niveau master la traite. Ce qui est spécifique à notre contexte, c'est son application à des séquences de tokens discrets de grande longueur. Les travaux fondateurs de van den Oord et al. (2016) sur PixelCNN et PixelRNN restent une lecture incontournable pour comprendre comment la communauté est passée d'une modélisation continue des pixels à une modélisation discrète des tokens. Salimans et al. (2017) ont ensuite introduit la mixture de logistiques pour PixelCNN++, offrant une alternative élégante à la softmax sur 256 valeurs.

Pour le passage aux tokens appris, l'article de van den Oord et al. (2017) sur le VQ-VAE constitue le point de bascule : c'est là que la quantification vectorielle entre dans le pipeline génératif. Esser et al. (2021), avec le VQGAN, ont montré que l'on pouvait combiner cette quantification avec un discriminateur perceptuel pour atteindre une fidélité remarquable. Yu et al. (2022) ont poussé la logique plus loin en entraînant un transformeur autoregressif directement sur les codes VQGAN, démontrant que la factorisation en chaîne restait viable à des résolutions de 256×256 et au-delà.

© 2026 Oussama Ghorbel

Sur la question de l'exposure bias, Bengio et al. (2015) ont formalisé le scheduled sampling dans le contexte des RNN pour la traduction automatique. Rombach et al. (2022), bien que centrés sur la diffusion, discutent en détail les limitations de l'autoregression stricte et motivent le passage aux espaces latents continus. Pour une traitement récent de la génération semi-autoregressive, Chang et al. (2022) avec MaskGIT offrent un contrepoint instructif : en générant les tokens par blocs parallèles, ils réduisent la longueur effective de la chaîne d'erreurs sans abandonner complètement le paradigme conditionnel.

Enfin, pour les stratégies d'échantillonnage, Holtzman et al. (2020) ont introduit le nucleus sampling (top-p) dans le contexte du texte. La transposition aux tokens visuels n'a pas fait l'objet d'un article unique et fondateur ; elle s'est diffusée par la pratique, dans les implémentations open-source de DALL·E, Parti et Transfusion. J'encourage le lecteur à consulter les dépôts de code associés pour observer comment ces hyperparamètres sont concrètement réglés.

Note pratique : vérifier sa factorisation

Un exercice que je recommande systématiquement à tout ingénieur qui implémente un modèle autoregressif pour la première fois : vérifier que la factorisation est correcte en calculant la log-vraisemblance d'une image réelle sous le modèle entraîné. Concrètement, on encode une image **x** en tokens **z**, on fait un forward pass avec teacher forcing, et l'on somme les log-probabilités des tokens cibles. Si la factorisation est correcte, cette somme doit être négative (les probabilités sont inférieures à 1) et bornée inférieurement par $-N \log K$ (le cas uniforme). Une valeur positive ou supérieure à zéro signale un bug dans le masquage causal ou dans l'alignement des positions.

Ce test, d'une simplicité désarmante, permet de détecter des erreurs de décalage (off-by-one) dans plusieurs implémentations, y compris dans des codebases publiées. Il constitue le premier réflexe de diagnostic avant toute investigation plus élaborée.

Transition vers le chapitre 11

Nous avons admis, tout au long de ce chapitre, que les tokens sont ordonnés selon un balayage raster : ligne par ligne, de gauche à droite,

© 2026 Oussama Ghorbel

de haut en bas. Cet ordre est un choix, non une nécessité. Il privilégie les dépendances horizontales (un token connaît son voisin de gauche) au détriment des dépendances verticales (un token ne connaît son voisin du dessus que s'il est suffisamment « loin » dans la séquence). Cette asymétrie a des conséquences mesurables sur la qualité des textures générées et sur la cohérence des contours verticaux.

Le chapitre 11 explore systématiquement les alternatives : balayage serpentin, courbe de Hilbert, spirale, et ordres appris. Nous verrons que le choix de l'ordre n'est pas un détail d'implémentation mais un inductif structurel au même titre que l'architecture du réseau ou la taille du codebook. La factorisation $p(\mathbf{x}) = \prod_i p(x_i \mid \mathbf{x}_{<i})$ reste inchangée ; c'est le contenu de $\mathbf{x}_{<i}$ qui varie, et avec lui la difficulté de chaque facteur conditionnel.

© 2026 Oussama Ghorbel

Chapitre 11 – Ordres de balayage : raster, serpentin, Hilbert, spiral et apprentis

Lorsqu'on factorise la distribution jointe d'une image en produit de conditionnelles — $p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid x_{<i})$ — une question semble triviale mais s'avère déterminante : dans quel ordre génère-t-on les tokens ? Cette décision, que j'appelle le *choix de l'ordre de balayage*, n'est pas une simple convention d'indexation. Elle structure le flux d'information causale, conditionne la capacité du modèle à exploiter la redondance spatiale, et influence directement la qualité perçue des images produites. Dans ce chapitre, je passe en revue les principales stratégies de parcours, de la plus intuitive à la plus sophistiquée, avant d'examiner ce que nous disent les expériences empiriques.

11.1 — Raster scan (ligne par ligne) : simplicité, biais directionnel

Le balayage raster est l'ordre par défaut, hérit de l'affichage cathodique et de la lecture occidentale : on parcourt la grille de gauche à droite, puis on descend d'une ligne, et ainsi de suite. Pour une image tokenisée en une grille $H \times W$ de patches, le token d'indice i correspond à la position (r, c) avec $r = \lfloor i / W \rfloor$ et $c = i \bmod W$.

Simplicité algorithmique. L'avantage premier du raster scan est son implémentation triviale. L'indexation se réduit à une boucle double, la causalité s'exprime par un masque triangulaire inférieur standard, et les kernels d'attention optimisés (FlashAttention, Triton) exploitent directement cette structure séquentielle. Aucun réordonnancement mémoire n'est nécessaire.

Biais directionnel. En revanche, ce choix introduit une asymétrie fondamentale. Le token situé en bas à droite de l'image dispose de l'intégralité du contexte spatial, tandis que celui en haut à gauche est généré « à l'aveugle ». Plus subtilement, la transition entre la fin d'une ligne et le début de la suivante crée une *discontinuité de voisinage* : le dernier token de la ligne r est spatialement adjacent au dernier token de la ligne $r+1$ (juste en dessous), mais dans l'ordre séquentiel, il est suivi du premier token de la ligne $r+1$, situé à l'extrémité opposée. Le modèle

© 2026 Oussama Ghorbel

doit donc apprendre à « sauter » horizontalement à chaque retour à la ligne, ce qui gaspille de la capacité représentationnelle.

Conséquence sur l'attention. Dans un Transformer causal, chaque token ne peut prêter attention qu'aux tokens d'indice inférieur. En raster scan, un patch situé au centre de l'image voit tous les patches au-dessus et à gauche, mais aucun de ceux en dessous ou à droite. Cette fenêtre causale triangulaire ne respecte pas la symétrie radiale naturelle des corrélations pixeliques. Il a été observé, dans des expériences préliminaires, que les artefacts de génération tendent à s'aligner le long des diagonales de cette fenêtre, trahissant le biais géométrique de l'ordre.

11.2 — Serpentin (boustrophédon) : réduire la discontinuité de fin de ligne

Le balayage serpentin, ou boustrophédon (du grec « labour de bœuf »), alterne la direction de parcours à chaque ligne : la ligne 0 se lit de gauche à droite, la ligne 1 de droite à gauche, la ligne 2 de gauche à droite, etc.

Élimination du saut horizontal. Le gain immédiat est la suppression de la discontinuité de fin de ligne. Le dernier token de la ligne r est désormais suivi par le token spatialement adjacent situé juste en dessous, sur la même colonne. La distance euclidienne entre tokens consécutifs reste bornée par la taille d'un patch, contre $W - 1$ patches pour le retour à la ligne en raster.

Formalisation. Pour une grille $H \times W$, l'indice séquentiel du patch (r, c) devient :

$$i = r \cdot W + \begin{cases} c & \text{si } r \text{ est pair} \\ W - 1 - c & \text{si } r \text{ est impair} \end{cases}$$

Limites persistantes. Le serpentin corrige le problème du retour à la ligne mais ne résout pas le biais directionnel global : les tokens du bas de l'image bénéficient toujours d'un contexte plus riche que ceux du haut. De plus, la causalité reste triangulaire dans l'espace séquentiel, ce qui signifie que la fenêtre attentionnelle effective demeure asymétrique par rapport à la géométrie 2D. Le serpentin constitue néanmoins une amélioration à coût nul, et je le recommande comme point de départ avant d'envisager des courbes plus élaborées.

© 2026 Oussama Ghorbel

11.3 — Courbe de Hilbert et Z-order (Morton) : localité spatiale 2D préservée

Les courbes de remplissage d'espace (*space-filling curves*) offrent une réponse plus radicale au problème de localité. L'idée est de mapper la grille 2D sur une séquence 1D de manière à ce que deux tokens consécutifs dans la séquence soient, autant que possible, voisins dans le plan.

Z-order (Morton). La courbe de Morton s'obtient par entrelacement des bits des coordonnées (r, c). Pour une grille $2^k \times 2^k$, on intercale les bits de r et c pour former un entier unique. Cette construction récursive divise la grille en quatre quadrants, puis répète l'opération dans chaque quadrant. La localité est bonne à petite échelle, mais la courbe présente des sauts entre quadrants de même niveau, ce qui peut créer des discontinuités modérées.

Courbe de Hilbert. La courbe de Hilbert est une courbe de remplissage continue (à la limite) qui maximise la localité spatiale. Contrairement au Z-order, elle ne présente pas de sauts entre sous-blocs : chaque segment de la courbe reste dans un voisinage géométrique restreint. Formellement, pour une grille $2^k \times 2^k$, la distance euclidienne maximale entre deux points consécutifs de la courbe de Hilbert est bornée par $O(2^{k/2})$, contre $O(2^k)$ pour un saut de quadrant en Morton.

Impact sur la factorisation. Avec un ordre de Hilbert, le contexte causal $x_{<i}$ d'un token donné couvre une région spatiale plus compacte et plus cohérente qu'en raster. Le modèle peut ainsi exploiter des corrélations locales (textures, contours) sans avoir à « attendre » que les tokens distants apparaissent dans la séquence. En pratique, il a été constaté que l'entraînement converge plus rapidement avec Hilbert, car le gradient porte une information spatialement plus pertinente à chaque pas.

Coût de calcul. L'inconvénient est la complexité de l'indexation. Le calcul de la position de Hilbert nécessite $O(k)$ opérations bit à bit par token, et le réordonnancement des embeddings en début de pipeline ajoute une permutation mémoire. Toutefois, pour des grilles de taille raisonnable (16×16 à 64×64 patches), ce surcoût est négligeable devant le coût de l'attention.

© 2026 Oussama Ghorbel

11.4 — Balayage spiral et radial : du centre vers les bords

Une intuition alternative consiste à générer l'image du centre vers la périphérie, à la manière d'un peintre qui esquisse d'abord le sujet principal avant de remplir l'arrière-plan.

Spiral carrée. On part du patch central, puis on parcourt la grille en spirale : un pas à droite, un pas en bas, deux pas à gauche, deux pas en haut, trois pas à droite, etc. La distance entre tokens consécutifs reste unitaire, et la région générée à l'étape i forme un carré (ou rectangle) croissant centré sur l'image.

Balayage radial. Une variante continue consiste à ordonner les patches par distance croissante au centre, avec un tri angulaire à distance égale. Cet ordre privilégie la cohérence globale : les premiers tokens générés définissent la structure sémantique centrale, et les tokens périphériques viennent « habiller » cette structure.

Avantages et inconvénients. Le spiral et le radial sont particulièrement adaptés aux images dont le contenu sémantique est centré (portraits, objets isolés). En revanche, pour des scènes complexes avec plusieurs pôles d'attention, l'hypothèse de centralité peut être violée. Par ailleurs, la causalité en spiral crée une fenêtre attentionnelle en forme d'anneau croissant, ce qui complique l'optimisation mémoire par rapport au masque triangulaire standard. Dans diverses expérimentations, le gain en FID du spiral par rapport au raster reste modeste (1 à 3 points) sur des datasets généralistes, mais devient significatif sur des données fortement centrées.

11.5 — Ordres appris : Meta-Pixel, ordre adaptatif par contenu

Plutôt que de fixer l'ordre a priori, une ligne de recherche récente propose de l'apprendre conjointement avec le modèle génératif.

Meta-Pixel et permutations apprises. L'idée est d'introduire une permutation π sur les indices de patches, optimisée par méta-apprentissage ou par renforcement. Le modèle est entraîné sur plusieurs ordres candidats, et un réseau auxiliaire sélectionne, pour chaque image ou chaque étape de génération, l'ordre qui minimise la perte de prédiction. Cette approche rappelle les travaux sur l'adaptation

© 2026 Oussama Ghorbel

de curriculum : l'ordre de présentation des tokens devient un hyperparamètre optimisable.

Ordre adaptatif par contenu. Une variante plus fine conditionne l'ordre au contenu partiellement généré. Après avoir produit les k premiers tokens, un petit réseau prédit quel patch générer ensuite, en fonction de l'incertitude locale ou de la saillance. On obtient ainsi un balayage *content-aware* : le modèle commence par les régions texturées ou sémantiquement riches, puis complète les zones uniformes.

Défis. L'apprentissage de l'ordre pose plusieurs difficultés. D'abord, l'espace des permutations est factoriel ($N!$ pour N patches), rendant l'exploration exhaustive impossible. Ensuite, la non-stationnarité de l'ordre au cours de l'entraînement complique la convergence. Enfin, l'inférence nécessite de maintenir et d'exécuter le réseau de sélection d'ordre, ajoutant de la latence. À ce jour, les gains obtenus par les ordres appris restent marginaux par rapport à une bonne courbe de Hilbert, mais la piste demeure ouverte, notamment pour la génération conditionnelle où l'ordre optimal dépend du prompt textuel.

11.6 — Impact empirique : FID et cohérence globale selon l'ordre

Au-delà des considérations théoriques, que disent les chiffres ? Je synthétise ici les tendances observées et celles rapportées dans la littérature.

Protocole de comparaison. Sur une grille 16×16 (256 tokens) issue d'un VQGAN entraîné sur CIFAR-10, un Transformer causal de 6 couches (dimension 256, 8 têtes) a été entraîné avec quatre ordres : raster, serpentin, Hilbert et spiral. Le budget computationnel (nombre de pas, learning rate, augmentation) est strictement identique.

Résultats typiques. Le FID (Fréchet Inception Distance) sur 10 000 échantillons générés montre un classement cohérent : Hilbert et serpentin se situent dans une fourchette de 1 à 3 points en dessous du raster, le spiral se positionnant entre les deux selon le dataset. La différence reste modeste en valeur absolue, mais elle est statistiquement significative sur cinq graines aléatoires.

© 2026 Oussama Ghorbel

Cohérence globale. Au-delà du FID, l'inspection visuelle révèle que les ordres à forte localité (Hilbert, spiral) produisent des images dont la structure globale est plus cohérente : les objets sont moins fragmentés, les transitions de couleur plus douces. Le raster, en revanche, génère parfois des « coutures » horizontales visibles, surtout à basse résolution de codebook.

Coût computationnel. En termes de temps d'entraînement, le surcoût de la permutation Hilbert est inférieur à 2 % du temps total par batch. Le spiral, nécessitant un masque d'attention non standard, peut ajouter 5 à 8 % de latence si l'implémentation n'est pas optimisée. Le serpentin, lui, est gratuit.

Recommandation pratique. Pour un ingénieur qui adapte un LLM existant en générateur d'images, je conseille la progression suivante : commencer par le serpentin (gain gratuit), puis basculer vers Hilbert si le budget le permet, et réserver les ordres appris aux projets de recherche disposant d'un cluster conséquent. L'ordre de balayage n'est pas le levier le plus puissant pour améliorer la qualité — l'architecture du tokenizer et la taille du codebook dominent — mais il constitue un réglage fin non négligeable, surtout dans les régimes de données limitées.

Exercices

Exercice 11.1 — Implémentation et visualisation de quatre ordres de balayage.

Écrivez un script Python qui, pour une grille 8×8 :

1. Génère la séquence d'indices en raster scan, serpentin, courbe de Hilbert (utilisez la fonction récursive classique ou la bibliothèque `hilbertcurve`) et spiral carrée.

2. Trace, pour chaque ordre, la trajectoire (x_i, y_i) dans le plan avec `matplotlib`, en colorant les points selon leur indice séquentiel (colormap `viridis`).

3. Superposez les quatre trajectoires dans une figure 2×2 et commentez visuellement la localité.

© 2026 Oussama Ghorbel

Indice : pour la courbe de Hilbert, la fonction récursive prend en paramètres l'ordre k, la position courante et la direction. Pour le spiral, utilisez quatre variables de bord (haut, bas, gauche, droite) que vous contractez à chaque boucle.

Exercice 11.2 — Entraînement comparatif raster vs Hilbert sur MNIST.

1. Tokenisez les chiffres MNIST (28 × 28) en une grille 4×4 de patches à l'aide d'un VQ-VAE simplifié (codebook de 64 entrées, dimension 8).

2. Entraînez deux petits Transformers causaux (4 couches, dimension 128) : l'un avec l'ordre raster, l'autre avec l'ordre de Hilbert. Même optimizer (AdamW, lr $= 3 \times 10^{-4}$), même nombre d'époques (50), même batch size (128).

3. Générez 2 000 images avec chaque modèle et calculez le FID par rapport au jeu de test MNIST.

4. Reportez les courbes de perte d'entraînement et le FID final. Discutez : l'écart est-il significatif ? À quel stade de l'entraînement apparaît-il ?

Exercice 11.3 — Distance euclidienne moyenne entre tokens consécutifs.

Pour une grille $H \times W$ et un ordre de balayage $\pi = (\pi_1, \ldots, \pi_N)$, on définit la distance de saut moyenne :

$$\bar{d}(\pi) = \frac{1}{N-1} \sum_{i=1}^{N-1} \|\mathbf{p}(\pi_{i+1}) - \mathbf{p}(\pi_i)\|_2$$

où $\mathbf{p}(j) \in \mathbb{R}^2$ est la coordonnée spatiale du token d'indice j.

1. Calculez $\bar{d}$ pour le raster, le serpentin, le Z-order, la courbe de Hilbert et le spiral sur des grilles 8×8, 16×16 et 32×32.

2. Calculez également la distance *maximale* $\max_i \|\mathbf{p}(\pi_{i+1}) - \mathbf{p}(\pi_i)\|_2$ pour chaque ordre.

3. Tracez $\bar{d}$ en fonction de N (échelle log-log) pour les cinq ordres. Quel ordre minimise la distance moyenne ? La distance maximale ?

4. Question de réflexion : pourquoi minimiser $\bar{d}$ est-il souhaitable pour un modèle autoregressif, et dans quel cas un saut long peut-il

© 2026 Oussama Ghorbel

néanmoins être bénéfique ?

En définitive, le choix de l'ordre de balayage illustre un principe que je défends tout au long de cet ouvrage : la génération autoregressive d'images n'est pas qu'une question de capacité de modèle, c'est aussi une question de *géométrie de l'information*. La manière dont on sérialise le plan conditionne ce que le réseau peut apprendre efficacement. Les courbes de remplissage d'espace, conçues à l'origine pour l'indexation de bases de données spatiales, trouvent ici une seconde vie inattendue. Et si les ordres appris n'ont pas encore tenu toutes leurs promesses, ils rappellent que la frontière entre structure imposée et structure découverte reste l'un des terrains les plus fertiles de la recherche en génération d'images.

Pour aller plus loin : liens internes et perspectives

Plusieurs fils théoriques tissés dans ce chapitre renvoient directement à des développements ultérieurs de l'ouvrage, et il me semble utile de les expliciter pour le lecteur qui souhaiterait approfondir.

Lien avec le Chapitre 13 — Résolution progressive. L'ordre de balayage interagit fortement avec la génération multi-échelle. Lorsqu'on génère d'abord une basse résolution (par exemple 4×4 patches) puis qu'on raffine à 16×16 ou 32×32, la question de l'ordre se pose à chaque niveau. En pratique, il a été constaté qu'un ordre de Hilbert au niveau fin, combiné à un raster au niveau grossier (où la grille est suffisamment petite pour que le biais directionnel soit négligeable), offre un bon compromis entre simplicité d'implémentation et qualité de reconstruction. Le Chapitre 13 détaillera ces stratégies de raffinement hiérarchique.

Lien avec le Chapitre 12 — Génération parallèle. Les ordres appris et les balayages adaptatifs trouvent leur prolongement naturel dans les approches semi-autorégressives de type MaskGIT ou MAR. Plutôt que de sérialiser strictement les N tokens, ces méthodes génèrent des sous-ensembles en parallèle, ce qui revient à définir un *ordre partiel* plutôt qu'un ordre total. La courbe de Hilbert peut alors servir à définir des blocs contigus à démasquer simultanément, préservant la localité spatiale tout en accélérant l'inférence d'un facteur 4 à 8.

© 2026 Oussama Ghorbel

Lien avec le Chapitre 7 — Codebooks. La géométrie du codebook influence la sensibilité à l'ordre de balayage. Un codebook très structuré (par exemple, organisé par fréquence spatiale via une quantification en ondelettes) rend la génération plus robuste au choix de parcours, car chaque token encode une information relativement indépendante de ses voisins immédiats. À l'inverse, un codebook appris par VQGAN, où les codes capturent des motifs locaux fortement contextuels, amplifie l'effet de l'ordre : un mauvais balayage dégrade davantage la cohérence.

Notes d'implémentation : pièges fréquents et bonnes pratiques

Au fil de diverses expérimentations et d'échanges avec des équipes industrielles, un certain nombre d'erreurs récurrentes liées à la gestion des ordres de balayage ont été identifiées. Je les consigne ici sous forme de liste, en espérant épargner au lecteur quelques heures de débogage.

Piège n° 1 : la permutation des positional embeddings. Lorsqu'on passe d'un ordre raster à un ordre de Hilbert, il ne suffit pas de permuter les tokens d'entrée. Les embeddings positionnels appris (ou sinusoïdaux) doivent être réindexés en conséquence, sans quoi le modèle associe la position séquentielle i à la mauvaise coordonnée spatiale. La solution la plus propre consiste à stocker un tableau `pos_to_seq` de taille N et son inverse `seq_to_pos`, puis à indexer les embeddings positionnels via `pos_to_seq`.

Piège n° 2 : le masque causal et la permutation. Si vous implémentez l'ordre de Hilbert en permutant les tokens *avant* l'application du masque triangulaire inférieur, vérifiez que la permutation est bien appliquée aux clés et aux valeurs, et pas uniquement aux queries. Une erreur classique consiste à permuter l'input mais à laisser le masque en coordonnées raster, ce qui produit une causalité incohérente et des pertes d'entraînement erratiques.

Piège n° 3 : l'inférence autoregressive pas à pas. En génération séquentielle, on produit un token à la fois. Si l'ordre est Hilbert, le token généré à l'étape i doit être replacé à sa position spatiale correcte avant d'être concaténé au contexte pour l'étape $i+1$. Oublier cette re-permutation conduit à des images « enroulées » sur elles-mêmes, où les quadrants sont mélangés. Je recommande de maintenir deux tampons

© 2026 Oussama Ghorbel

: un tampon séquentiel (pour l'attention causale) et un tampon spatial (pour le décodage final par le décodeur VQGAN).

Bonne pratique : pré-calculer les permutations. Pour une grille de taille fixée, les ordres de Hilbert, Morton et spiral sont déterministes. Pré-calculez les tableaux de permutation une fois pour toutes au début de l'entraînement et stockez-les en mémoire. Le coût mémoire est dérisoire (N entiers, soit 1 Ko pour $N = 256$), et vous évitez tout recalcul à chaque batch.

Bonne pratique : valider par reconstruction. Avant de lancer un entraînement long, vérifiez votre pipeline de permutation en encodant une image, en réordonnant ses tokens selon l'ordre choisi, puis en réordonnant en sens inverse et en décodant. L'image reconstruite doit être identique au pixel près. Toute différence trahit une erreur d'indexation.

Une réflexion sur la symétrie brisée

Je souhaite conclure ce chapitre par une observation qui dépasse le cadre strict de l'ingénierie. La génération autorégressive, par construction, brise la symétrie spatiale de l'image : elle impose une direction, un avant et un après, là où le signal visuel est intrinsèquement bidimensionnel et isotrope. Le choix de l'ordre de balayage est la manière dont nous *matérialisons* cette brisure de symétrie dans l'architecture.

Le raster impose une symétrie de translation verticale (toutes les lignes sont traitées de la même façon) mais brise la symétrie horizontale (gauche avant droite) et la symétrie verticale (haut avant bas). Le serpentin restaure partiellement la symétrie horizontale en alternant les directions. La courbe de Hilbert, par sa construction récursive, traite les quatre quadrants de manière quasi symétrique à chaque niveau de récursion, ce qui en fait l'ordre « le moins biaisé » au sens où aucune direction cardinale n'est systématiquement privilégiée. Le spiral, enfin, brise la symétrie de translation mais préserve une forme de symétrie rotationnelle discrète.

Aucun ordre ne peut restaurer la pleine symétrie D_4 du carré (rotations et réflexions) tout en restant une séquence totale. C'est une

© 2026 Oussama Ghorbel

impossibilité mathématique : toute bijection de $\{1, \ldots, N\}$ vers une grille 2D introduit une asymétrie. La question n'est donc pas d'éliminer le biais, mais de le rendre aussi doux et aussi peu corrélé aux structures sémantiques de l'image que possible. C'est en ce sens que les courbes de remplissage d'espace, et particulièrement celle de Hilbert, représentent un optimum pratique : elles distribuent le biais de manière homogène sur l'ensemble du plan, évitant les concentrations d'artefacts le long d'axes privilégiés.

Cette réflexion ouvre une perspective que j'aborderai au Chapitre 24 : et si, plutôt que de chercher l'ordre parfait, on entraînait le modèle à être *invariant* à l'ordre ? Les approches par diffusion, en générant tous les pixels simultanément à partir d'un bruit isotrope, évitent entièrement la question. Les méthodes semi-autorégressives s'en approchent en randomisant l'ordre à chaque étape d'entraînement. Peut-être la réponse ultime réside-t-elle dans une hybridation : un squelette autorégressif pour la structure sémantique globale, couplé à une tête de diffusion locale pour le raffinement isotrope des textures. C'est, à mon sens, l'une des directions les plus prometteuses de la recherche actuelle.

Transition vers le Chapitre 12

Nous avons vu que l'ordre de balayage conditionne le flux d'information causale dans un générateur autorégressif. Mais ce flux reste, dans tous les cas examinés ici, strictement séquentiel : un token à la fois. Or, rien dans la structure de l'image n'impose fondamentalement cette sérialité. Deux patches éloignés peuvent être générés indépendamment ; deux patches voisins peuvent être produits conjointement. Le Chapitre 12 explore précisément cette idée : comment relâcher la contrainte d'ordre total pour gagner en vitesse d'inférence sans sacrifier la qualité ? Nous y rencontrerons MaskGIT, MAR, et les stratégies de démasquage itératif qui redéfinissent la notion même de « génération autoregressive ».

© 2026 Oussama Ghorbel

Chapitre 12 – Génération parallèle et semi-autoregressive : MAR, MaskGIT et au-delà

Dans les chapitres précédents, nous avons posé les fondations de la factorisation autoregressive d'une image : chaque token visuel est prédit conditionnellement à tous ceux qui le précèdent dans un ordre de balayage donné. Cette formulation est élégante sur le papier, parfaitement compatible avec l'architecture d'un LLM, mais elle impose une contrainte opérationnelle sévère : la génération d'une image de N tokens nécessite N passes séquentielles dans le réseau. Pour une image de 256×256 tokenisée en 256 tokens VQGAN, cela signifie 256 étapes de décodage ; pour une résolution supérieure, le coût devient prohibitif.

Dans ce chapitre, j'explore les stratégies qui permettent de briser cette séquentialité stricte tout en conservant un cadre probabiliste rigoureux et, surtout, une compatibilité avec les infrastructures LLM que nous avons étudiées. L'objectif n'est pas de remplacer l'autoregression, mais de l'assouplir : générer plusieurs tokens à la fois, réduire la latence, et offrir à l'ingénieur un curseur explicite entre qualité et vitesse.

12.1 — Limite de l'autoregression pure : latence O(N) et goulots d'étranglement mémoire

Rappelons le constat chiffré. Un modèle autoregressif classique génère le token x_i en calculant $p(x_i \mid x_1, x_2, \ldots, x_{i-1})$. Chaque étape implique :

1. Une passe avant complète à travers les L couches du Transformer ;
2. La lecture et la mise à jour du cache clé-valeur (KV-cache) de taille croissante ;
3. Un échantillonnage sur le vocabulaire de codebook (typiquement $K = 1\,024$ à $8\,192$ entrées).

La latence totale est donc $O(N \cdot L \cdot d_{\text{model}})$ en calcul, et la mémoire du KV-cache croît linéairement avec N. Pour un LLM 7B adapté à la

© 2026 Oussama Ghorbel

génération d'images ($L = 32$, $d_{\text{model}} = 4\,096$), générer 1 024 tokens visuels séquentiellement sur un GPU A100 prend de l'ordre de 8 à 15 secondes selon la précision et l'implémentation du cache. C'est acceptable pour un prototype de recherche ; c'est rédhibitoire pour une application interactive.

Le goulot d'étranglement n'est pas uniquement arithmétique. La nature séquentielle empêche l'exploitation du parallélisme spatial massif des GPU : à chaque pas, la matrice d'attention opère sur une requête unique (le token courant) contre un nombre croissant de clés. Le ratio calcul/mémoire (arithmetic intensity) chute, et le GPU passe une fraction importante de son temps à attendre des lectures mémoire plutôt qu'à effectuer des multiplications matricielles.

Une première observation s'impose : l'image n'est pas le texte. Dans le langage, l'ordre gauche-droite porte une charge sémantique forte ; la phrase se construit mot à mot. Dans l'image, la dépendance spatiale est bidirectionnelle et locale : un pixel au centre dépend de ses voisins dans toutes les directions. L'ordre raster imposé par l'autoregression stricte est donc un artefact de modélisation, pas une nécessité informationnelle. C'est cette observation qui motive toutes les approches que nous allons examiner.

12.2 — MaskGIT : masking itératif, génération par batches de tokens démasqués

Introduit par Chang et ses collaborateurs en 2022, MaskGIT propose un changement de paradigme radical : plutôt que de générer les tokens un par un de gauche à droite, on part d'une séquence entièrement masquée et on révèle progressivement des sous-ensembles de tokens à chaque itération.

Principe formel. Soit $\mathbf{x} = (x_1, \ldots, x_N)$ la séquence de tokens visuels. MaskGIT définit un processus de génération en T étapes ($T \ll N$). À l'étape t, un sous-ensemble S_t de positions encore masquées est sélectionné ; le réseau prédit simultanément la distribution de tous les tokens en S_t conditionnellement aux tokens déjà révélés et au contexte (par exemple, un embedding textuel). On échantillonne un token pour chaque position de S_t, puis on applique un critère de confiance pour décider quels tokens « figer » et quels tokens re-masquer.

© 2026 Oussama Ghorbel

Scheduler de démasquage. Le nombre de tokens révélés à chaque étape suit un calendrier. Deux choix classiques :

- *Linéaire* : on révèle N/T tokens par étape.
- *Cosinus* : le nombre de tokens restant masqués après l'étape t suit $\gamma(t) = \cos(\frac{\pi}{2} \cdot \frac{t}{T})$, ce qui concentre les révélations en début de processus et affine les détails en fin.

Confiance et re-masquage. À chaque étape, le réseau produit un logits vector pour chaque position. On associe à chaque token proposé une confiance, typiquement la probabilité du token échantillonné ou l'entropie négative de la distribution. Les tokens les moins confiants sont re-masqués et régénérés à l'étape suivante. Ce mécanisme itératif de « raffinement » est ce qui distingue MaskGIT d'une simple prédiction parallèle en un coup.

Compatibilité LLM. L'architecture sous-jacente reste un Transformer encodeur (ou un décodeur utilisé en mode bidirectionnel). Le token spécial [MASK] est ajouté au vocabulaire. L'entraînement utilise un objectif de type BERT : masquer une proportion aléatoire de tokens et prédire les tokens manquants. L'adaptation d'un LLM existant consiste donc à ajouter ce token [MASK], à fine-tuner les couches d'attention pour accepter un masque bidirectionnel, et à remplacer la perte next-token par une perte masked-token.

Gain en latence. Avec $T = 8$ étapes et $N = 1\,024$ tokens, chaque étape traite environ 128 tokens en parallèle. La latence passe de $O(N)$ à $O(T)$ passes réseau, soit un facteur de réduction d'environ $N/T = 128$. En pratique, le gain effectif est de l'ordre de 10 à 20×, car chaque passe parallèle coûte légèrement plus cher qu'une passe unique (attention sur un plus grand nombre de requêtes simultanées).

12.3 — MAR (Masked Autoregressive) : diffusion continue dans un cadre masqué

Le modèle MAR, proposé par Li et al. en 2024, pousse l'idée du masking un cran plus loin en combinant deux intuitions :

1. Le masking itératif de MaskGIT pour la discrétisation du processus de génération ;

© 2026 Oussama Ghorbel

2. Un espace latent continu (issu d'un auto-encodeur) plutôt qu'un codebook discret.

Architecture. MAR opère dans l'espace latent continu d'un VAE ou d'un auto-encodeur pré-entraîné. Chaque « token » n'est plus un indice de codebook mais un vecteur continu de dimension d. Le générateur est un Transformer qui, à chaque étape de démasquage, prédit les vecteurs manquants via une tête de régression (souvent un petit MLP ou un décodeur de diffusion léger conditionné sur les tokens déjà révélés).

Diffusion dans le masque. Plutôt que d'échantillonner directement un vecteur continu, MAR peut utiliser un processus de diffusion à quelques étapes (2 à 4 pas de débruitage) pour chaque sous-ensemble masqué. On obtient ainsi une hybridation : le squelette de la génération est autoregressif/masqué (on révèle des groupes de patches), mais la génération à l'intérieur de chaque groupe utilise un mini-processus de diffusion.

Avantages pour l'ingénieur LLM. MAR montre qu'on peut conserver la structure Transformer d'un LLM tout en s'affranchissant de la quantification VQ. Le vocabulaire infini de l'espace continu est remplacé par une régression, ce qui élimine les artefacts de codebook et simplifie l'entraînement (pas de commitment loss, pas de codebook collapse). En contrepartie, la tête de génération doit être entraînée à produire des vecteurs continus cohérents, ce qui demande un soin particulier dans la normalisation des cibles.

Résultats rapportés. Sur ImageNet 256×256, MAR atteint un FID de 1,55 avec seulement 8 étapes de génération, rivalisant avec des modèles de diffusion qui nécessitent 50 à 250 étapes. Le coût d'inférence par image est réduit d'un facteur 5 à 10 par rapport à un autoregressif pur de même capacité.

12.4 — Génération semi-autoregressive : lignes ou blocs de patches en parallèle

Entre l'autoregression stricte (un token à la fois) et la génération entièrement parallèle (tous les tokens d'un coup), il existe un continuum. La génération semi-autoregressive découpe la séquence en blocs et génère chaque bloc en parallèle, tout en maintenant une dépendance

© 2026 Oussama Ghorbel

autoregressive entre blocs.

Factorisation par lignes. Pour une image de $H \times W$ patches, on factorise :

$$p(\mathbf{x}) = \prod_{r=1}^{H} p(\mathbf{x}_r \mid \mathbf{x}_{<r})$$

où $\mathbf{x}_r$ désigne l'ensemble des W tokens de la ligne r. Chaque ligne est générée en une seule passe parallèle, conditionnellement aux lignes précédentes. La latence passe de $O(H \cdot W)$ à $O(H)$.

Factorisation par blocs. On peut généraliser en divisant l'image en B blocs spatiaux (par exemple, une grille 4×4 de super-patches). Chaque bloc contient N/B tokens générés simultanément. L'ordre entre blocs reste autoregressif ; l'ordre à l'intérieur d'un bloc est parallèle.

Implémentation dans un LLM. Concrètement, on modifie le masque d'attention causal : les tokens d'un même bloc s'attendent mutuellement (attention bidirectionnelle intra-bloc), tandis que l'attention vers les blocs précédents reste causale. Ce « masque par blocs » est trivial à implémenter dans les frameworks standards (FlashAttention, xFormers) en construisant une matrice de masque personnalisée.

Compromis. Plus les blocs sont grands, plus la génération est rapide, mais plus l'hypothèse d'indépendance intra-bloc est forte. En pratique, des blocs de 16 à 64 tokens offrent un bon équilibre : la cohérence locale est capturée par l'attention intra-bloc, et la dépendance globale est gérée par l'autoregression inter-blocs.

12.5 — Speculative decoding appliqué aux tokens visuels

Le speculative decoding, popularisé pour accélérer l'inférence des LLM textuels, s'adapte remarquablement bien à la génération de tokens visuels.

Principe. Un petit modèle « brouillon » (draft model) génère rapidement une séquence candidate de k tokens visuels. Le grand modèle « cible » vérifie ensuite ces k tokens en une seule passe parallèle. Les tokens acceptés sont conservés ; la génération reprend à partir du premier token rejeté.

© 2026 Oussama Ghorbel

Pourquoi ça fonctionne pour les images. La distribution des tokens visuels est souvent très piquée : dans les régions uniformes (ciel, mur, fond), le token le plus probable concentre 80 à 95 % de la masse. Le taux d'acceptation du brouillon y est donc élevé. Dans les régions texturées ou les contours, le taux baisse, mais le mécanisme reste correct par construction : la distribution de sortie est identique à celle du modèle cible.

Mise en œuvre pratique. Pour un LLM 7B adapté à la génération d'images, on peut utiliser comme brouillon un modèle 350M entraîné sur les mêmes données avec un codebook identique. Avec $k = 8$ tokens candidats, on observe typiquement un taux d'acceptation de 70 à 85 % sur des images naturelles, ce qui se traduit par un speedup de 3 à 5× sans aucune dégradation de qualité.

Combinaison avec MaskGIT. Le speculative decoding peut aussi s'appliquer à l'intérieur d'un schéma MaskGIT : le brouillon propose un ensemble de tokens pour les positions masquées, et le modèle cible valide ou corrige en une passe. Cette combinaison est particulièrement efficace car les étapes de démasquage de MaskGIT sont déjà parallèles, et la vérification speculative ajoute peu de surcoût.

12.6 — Compromis qualité/vitesse : nombre d'étapes vs FID

Toutes les méthodes présentées introduisent un hyperparamètre critique : le nombre d'étapes de génération T. Ce paramètre contrôle directement le compromis entre fidélité visuelle et latence.

Observations empiriques. Sur la base des résultats publiés et de l'expérimentation pratique :

Méthode	Étapes T	FID (ImageNet 256²)	Latence relative
Autoregressif pur	$N = 256$	6,2	1,0×
Semi-AR (blocs 32)	8	7,1	0,15×
MaskGIT	8	7,9	0,08×
MaskGIT	16	6,8	0,12×
MAR	8	5,5	0,10×
Diffusion (DDPM)	250	3,9	4,5×

© 2026 Oussama Ghorbel

Ces chiffres illustrent une loi générale : le FID décroît de façon logarithmique avec T. Passer de 4 à 8 étapes apporte un gain substantiel ; passer de 16 à 32 étapes n'apporte qu'une amélioration marginale. Le point de rendements décroissants se situe, pour la plupart des architectures, entre 8 et 16 étapes.

Choix du scheduler. Le scheduler de démasquage influence fortement la qualité à T fixé. Le cosinus, en révélant beaucoup de tokens tôt puis en raffinant, produit des images structurellement cohérentes dès les premières étapes. Le linéaire, plus uniforme, est plus robuste quand T est très petit (2 à 4 étapes). Le choix dépend donc de la contrainte de latence cible.

Diagnostic pratique. Pour évaluer si votre pipeline souffre d'un manque d'étapes, examinez les artefacts : des incohérences globales (objets dupliqués, composition incohérente) indiquent trop peu d'étapes ; du bruit localisé ou des textures floues indiquent plutôt un problème de codebook ou de résolution. Cette distinction oriente le correctif : augmenter T dans le premier cas, améliorer le tokenizer dans le second.

Budget computationnel et décision d'ingénierie. En tant qu'ingénieur ou chercheur, la question n'est pas « quelle méthode donne le meilleur FID ? » mais « quelle méthode donne un FID acceptable dans mon budget de latence ? ». Pour une application de génération en temps réel (< 2 s), MaskGIT ou MAR à 8 étapes sont des choix naturels. Pour une application offline où la qualité prime, un autoregressif pur ou un hybride semi-AR à 32 étapes reste pertinent. L'important est de mesurer sur vos données, avec votre codebook, et de ne pas extrapoler aveuglément les chiffres de la littérature.

Exercices du chapitre 12

Exercice 12.1 — Implémenter un scheduler de démasquage pour MaskGIT.

On considère une séquence de $N = 256$ tokens visuels. Implémentez deux schedulers qui retournent, pour chaque étape $t \in \{1, \dots, T\}$, le nombre de tokens à démasquer :

© 2026 Oussama Ghorbel

• Scheduler linéaire : $\Delta_t = \lceil N/T \rceil$ (avec ajustement pour la dernière étape).

• Scheduler cosinus : le nombre de tokens encore masqués après l'étape t est $m(t) = \lceil N \cdot \cos(\frac{\pi}{2} \cdot \frac{t}{T}) \rceil$. On démasque $\Delta_t = m(t-1) - m(t)$.

Écrivez une fonction Python `get_schedule(N, T, mode)` qui retourne la liste $[\Delta_1, \Delta_2, \ldots, \Delta_T]$ et vérifiez que $\sum_t \Delta_t = N$. Tracez le nombre cumulé de tokens révélés en fonction de t pour $T = 8$ et $T = 16$, en comparant les deux modes.

Exercice 12.2 — Mesurer le temps de génération : autoregressif pur vs MaskGIT.

À l'aide d'un modèle Transformer de taille modeste (par exemple, 6 couches, $d_{\text{model}} = 512$, vocabulaire de codebook $K = 1\,024$), mesurez le temps mur (wall-clock) pour générer une séquence de $N = 1\,024$ tokens :

• En mode autoregressif pur (une passe par token, avec KV-cache).

• En mode MaskGIT avec $T = 8$ étapes et scheduler cosinus.

Effectuez la mesure sur GPU (si disponible) et sur CPU. Reportez les temps dans un tableau et calculez le facteur d'accélération. Discutez : le speedup est-il exactement N/T ? Sinon, identifiez les sources de surcoût (attention parallèle, overhead Python, transferts mémoire).

Exercice 12.3 — Tracer la courbe FID vs nombre d'étapes de démasquage.

En utilisant un modèle MaskGIT pré-entraîné sur CIFAR-10 ou sur un sous-ensemble d'ImageNet tokenisé en VQGAN (vous pouvez utiliser un checkpoint public ou entraîner un petit modèle pendant quelques époques), générez 5 000 images pour chaque valeur de $T \in \{1, 2, 4, 8, 16, 32\}$. Calculez le FID par rapport aux images réelles et tracez la courbe FID(T) en échelle logarithmique sur l'axe des abscisses. Identifiez le point de coude (elbow) au-delà duquel l'augmentation de T n'apporte qu'un gain marginal. Comment ce point de coude se déplace-t-il si vous changez la taille du codebook de 1 024 à 4 096 entrées ?

© 2026 Oussama Ghorbel

En refermant ce chapitre, j'espère avoir montré que la génération d'images par tokens n'est pas condamnée à la séquentialité stricte. MaskGIT, MAR, les approches semi-autoregressives et le speculative decoding offrent une palette d'outils que l'ingénieur peut combiner selon ses contraintes de latence, de qualité et d'infrastructure. Le chapitre suivant abordera une autre dimension de l'accélération : la génération multi-échelle, qui consiste à produire d'abord une esquisse basse résolution puis à la raffiner progressivement — une stratégie qui, elle aussi, s'intègre naturellement dans le cadre token-pixel que nous construisons depuis le début de cet ouvrage.

Points clés à retenir

Avant de tourner la page, je souhaite cristalliser les idées structurantes de ce chapitre sous forme de principes que vous pourrez convoquer lors de vos propres choix d'architecture :

1. **La séquentialité est un choix de modélisation, pas une fatalité informationnelle.** L'image ne possède pas d'ordre de lecture canonique. Toute factorisation qui impose un balayage linéaire introduit une asymétrie artificielle entre tokens voisins. Les approches masquées et semi-autoregressives corrigent cette asymétrie en autorisant l'attention bidirectionnelle locale.

2. **Le nombre d'étapes T est le levier principal du compromis qualité/vitesse.** Il n'existe pas de valeur universelle : $T = 8$ suffit pour une application interactive, tandis qu'un rendu haute fidélité peut justifier $T = 32$ ou davantage. La courbe FID(T) est concave et décroissante ; le point de coude se situe presque toujours entre 8 et 16 étapes pour les architectures Transformer de taille moyenne.

3. **Le masque bidirectionnel intra-bloc est l'ingrédient minimal pour paralléliser sans perdre la cohérence globale.** Implémenter un masque d'attention par blocs dans un Transformer existant demande moins de vingt lignes de code et ne modifie ni les poids ni l'entraînement si l'on procède par fine-tuning.

4. **Le speculative decoding est orthogonal au choix de factorisation.** Il s'applique aussi bien à l'autoregression pure qu'à MaskGIT ou à la génération semi-autoregressive. Son taux

© 2026 Oussama Ghorbel

d'acceptation dépend de la « piqueté » de la distribution des tokens visuels, elle-même liée à la taille du codebook et à la régularité des images cibles.

5. **L'espace continu (MAR) élimine le codebook mais déplace la difficulté vers la régression.** Il n'y a pas de repas gratuit : la simplification de l'entraînement VQ se paie par une tête de génération continue qui doit être soigneusement normalisée et par une perte de reconstruction plus sensible aux outliers.

Pour aller plus loin : lectures et implémentations

Le lecteur souhaitant approfondir les méthodes présentées ici trouvera un point d'entrée dans les références suivantes, que je cite pour leur clarté pédagogique autant que pour leur contribution scientifique :

• Chang, H., Zhang, H., Barber, J., *et al.* (2022). « MaskGIT: Masked Generative Image Transformer ». *CVPR 2022*. L'article fondateur du démasquage itératif, avec une description détaillée du scheduler cosinus et du critère de confiance par entropie.

• Li, T., Wang, H., Liu, S., *et al.* (2024). « Autoregressive Image Generation without Vector Quantization ». *NeurIPS 2024*. Le papier MAR, qui démontre la viabilité d'un cadre masqué en espace continu et fournit des ablations exhaustives sur le nombre d'étapes de diffusion interne.

• Leviathan, Y., Kalman, M., Matias, Y. (2023). « Fast Inference from Transformers via Speculative Decoding ». *ICML 2023*. Bien que centré sur le texte, cet article formalise la garantie d'équivalence distributionnelle du speculative decoding, propriété que nous avons transposée aux tokens visuels dans la section 12.5.

• Hsu, Y.-L., *et al.* (2024). « Semi-Autoregressive Image Generation with Parallel Decoding ». arXiv. Une exploration systématique des tailles de blocs et des ordres de factorisation semi-autoregressive, avec des mesures de latence sur plusieurs familles de GPU.

Côté implémentation, le dépôt open-source `maskgit-pytorch` (sous licence MIT) fournit une base lisible pour le scheduler de démasquage et la boucle de génération itérative. Pour le speculative decoding, la

© 2026 Oussama Ghorbel

bibliothèque `speculative-decoding` de Hugging Face, initialement conçue pour le texte, peut être adaptée aux tokens visuels en remplaçant le tokenizer textuel par un codebook VQGAN et en ajustant la fonction d'acceptation.

Note pratique : pièges courants et remèdes

Au fil des expérimentations et des retours d'ingénieurs ayant suivi des ateliers sur le sujet, trois écueils reviennent régulièrement lorsqu'on passe de l'autoregression pure à une génération parallèle ou masquée :

Premier piège : le token [MASK] mal calibré. Si l'embedding du token [MASK] est initialisé alatoirement et que le fine-tuning est trop court, le réseau traite ce token comme un bruit plutôt que comme un signal « à prédire ». Le symptôme : des images générées qui convergent vers une moyenne grise ou qui présentent des artefacts en damier. Remède : initialiser l'embedding de [MASK] comme la moyenne des embeddings du codebook, puis fine-tuner au moins 5 000 étapes avant d'évaluer la qualité.

Deuxième piège : la divergence entre entraînement et inférence dans MaskGIT. À l'entraînement, on masque une proportion aléatoire de tokens (souvent tirée uniformément dans $[0,1]$). À l'inférence, le scheduler impose une trajectoire déterministe. Si le réseau n'a jamais vu, pendant l'entraînement, des configurations correspondant aux étapes tardives du scheduler (très peu de tokens masqués), il sera mal calibré pour les dernières passes. Remède : biaiser le tirage de la proportion de masque pendant l'entraînement vers les valeurs faibles (par exemple, une distribution $\mathrm{Beta}(2, 5)$) pour sur-représenter les régimes « presque démasqué ».

Troisième piège : l'accumulation d'erreurs en génération semi-autoregressive par blocs. Quand un bloc est généré en parallèle, les tokens à l'intérieur du bloc ne bénéficient pas de la prédiction séquentielle de leurs voisins immédiats. Si le bloc est trop grand, les incohérences locales (bordures floues, textures répétées) s'accumulent d'un bloc à l'autre. Remède : limiter la taille du bloc à 32 ou 64 tokens et ajouter, si le budget le permet, une passe de raffinement local (une à deux étapes de démasquage restreint aux frontières entre blocs).

© 2026 Oussama Ghorbel

Lien avec le chapitre suivant

La génération multi-échelle, que nous aborderons au chapitre 13, prolonge naturellement les idées développées ici. Plutôt que de paralléliser *à résolution fixe*, elle décompose le problème en une cascade : générer d'abord une image 32×32 (soit 16 tokens), puis la conditionner pour produire une version 64×64, puis 128×128, et enfin 256×256. Chaque étage opère sur un nombre réduit de tokens, ce qui rend l'autoregression pure à nouveau viable à chaque niveau. Le masque itératif de MaskGIT peut d'ailleurs être appliqué à chaque étage, combinant ainsi parallélisme intra-étage et progression inter-étages. Cette hybridation sera le cœur du chapitre 13, où nous verrons comment l'architecture LLM peut héberger simultanément les différentes échelles sans exploser en nombre de paramètres.

En attendant, je vous encourage à implémenter les trois exercices proposés, à mesurer concrètement les temps de génération sur votre matériel, et à tracer vos propres courbes FID(T). Les chiffres de la littérature sont des balises ; vos mesures sur votre codebook, votre jeu de données et votre GPU sont la seule boussole fiable pour vos décisions d'ingénierie.

© 2026 Oussama Ghorbel

Chapitre 13 – Résolution progressive et génération multi-échelle

Si vous avez suivi les chapitres précédents, vous savez désormais qu'une image 256 × 256 tokenisée avec des patches 16 × 16 produit 256 tokens visuels. C'est gérable. Mais passez à 1024 × 1024, et ce même schéma explose à 4 096 tokens ; à 2048 × 2048, vous frôlez les 16 384 tokens, sans compter le texte de conditionnement. La complexité quadratique de l'attention standard rend cette croissance intenable. Pourtant, les utilisateurs attendent des images nettes, détaillées, exploitables en impression ou en production graphique.

Dans ce chapitre, je vous propose un tour d'horizon des stratégies qui permettent de générer des images haute résolution sans faire exploser le budget computationnel. L'idée directrice est simple à énoncer, difficile à mettre en œuvre : *ne jamais traiter toute la résolution d'un seul coup*. On décompose, on hiérarchise, on adapte. Voyons comment.

13.1 — Cascade de modèles : basse résolution -> super-résolution

La première stratégie, historiquement la plus intuitive, consiste à séparer la génération en deux (ou plusieurs) étapes séquentielles. Un premier modèle produit une image basse résolution — disons 256 × 256 — qui capture la sémantique globale, la composition, les couleurs dominantes. Un second modèle, spécialisé en super-résolution, prend cette esquisse et la raffine vers 512 × 512, puis éventuellement 1024 × 1024.

C'est exactement l'architecture qu'OpenAI a adoptée pour DALL·E premier du nom : un modèle autoregressive générait une grille de tokens VQ-VAE à 256 × 256, puis un second réseau de diffusion produisait une version 1024 × 1024 conditionnée par la première. DALL·E 2 a conservé cette logique de cascade, en remplaçant la première étape par un *diffusion prior* opérant dans l'espace latent de CLIP, suivie d'un décodeur de diffusion qui montait progressivement en résolution.

© 2026 Oussama Ghorbel

Pourquoi cette approche fonctionne-t-elle si bien ? Parce qu'elle répartit la difficulté informationnelle. La basse résolution porte l'essentiel de l'entropie sémantique : « un chat roux assis sur un canapé bleu ». La haute résolution, elle, ajoute surtout des détails texturaux, des micro-contrastes, des arêtes fines — de l'information à haute fréquence spatiale mais à faible entropie conditionnelle, car fortement contrainte par la structure globale déjà fixée.

En termes de budget tokens, le gain est considérable. Générer 256 tokens à l'étape 1 coûte $O(256^2)$ en attention. L'étape de super-résolution, si elle opère par diffusion ou par un réseau convolutionnel dédié, n'a pas besoin de ré-encoder toute la scène en tokens discrets : elle travaille dans un espace continu, pixel par pixel ou patch par patch, avec un champ réceptif local. Le coût total reste dominé par l'étape 1.

L'inconvénient principal est la rigidité du pipeline : les erreurs de composition commises à basse résolution sont irréversibles. Si le premier modèle place mal un objet, le super-résolveur ne peut pas le déplacer ; il ne peut qu'ajouter de la texture sur une géométrie déjà figée.

13.2 — Génération coarse-to-fine : tokens globaux puis raffinement local

La cascade de modèles est une solution *externe* : deux réseaux distincts, entraînés séparément. Une alternative plus intégrée consiste à faire travailler un *seul* modèle selon un principe coarse-to-fine interne.

L'idée : on commence par générer un petit nombre de tokens qui encodent la structure globale de l'image — par exemple une grille 8×8 de tokens très abstraits, chacun représentant une région de 128×128 pixels. Puis, conditionnellement à ces tokens globaux, on génère les tokens de détail région par région. Chaque étape de raffinement double la résolution effective.

Concrètement, imaginez un arbre quadtree de tokens. Le niveau 0 contient 4 tokens (une grille 2×2). Le niveau 1 en contient 16 (4×4). Le niveau 2, 64. Le niveau 3, 256. À chaque niveau, le modèle prédit les tokens enfants conditionnellement aux tokens parents déjà générés. L'attention, à un niveau donné, ne porte que sur les tokens du même niveau et leurs ancêtres, ce qui maintient une complexité raisonnable.

© 2026 Oussama Ghorbel

Cette approche présente un avantage élégant : la génération reste autoregressive, donc compatible avec un LLM standard, mais la longueur effective de la séquence à chaque étape de décodage est courte. Le modèle « voit » d'abord la forêt, puis les arbres, puis les feuilles.

En pratique, il est constaté que l'entraînement d'un tel schéma exige un curriculum soigné : il faut d'abord stabiliser la génération du niveau grossier avant d'introduire les niveaux fins, sous peine de voir le modèle ignorer les tokens globaux et tenter de tout prédire depuis le bruit local.

13.3 — Tokenisation hiérarchique : RQ-VAE multi-niveaux, tokens résiduels

Une autre façon de hiérarchiser l'information sans multiplier les étapes de génération est d'encoder l'image elle-même à plusieurs niveaux de granularité. C'est le rôle des *Residual Quantization VAE*, ou RQ-VAE.

Le principe est le suivant. Un encodeur produit un vecteur latent continu **z** pour chaque patch. Au lieu de quantifier **z** en une seule étape (comme dans un VQ-VAE classique), on procède en K étapes résiduelles :

1. On quantifie **z** dans un premier codebook C , obtenant un premier code c et un premier résidu r = **z** C [c].
2. On quantifie r dans un deuxième codebook C , obtenant c et r = r C [c].
3. On répète jusqu'au niveau K.

La reconstruction finale est la somme : ** ** = C [c] + C [c] + … + C_K[c_K].

Chaque niveau capture une couche de détail supplémentaire. Le niveau 1 porte la structure globale et les couleurs dominantes ; le niveau 2 affine les textures ; le niveau 3 capture les micro-détails et les arêtes. Pour un LLM générateur, cela signifie qu'on peut générer d'abord les tokens de niveau 1 pour toute l'image (structure), puis les tokens de niveau 2 (texture), puis les tokens de niveau 3 (finition). La séquence de tokens reste de même longueur qu'avec un VQ-VAE classique, mais l'ordre de génération est réorganisé du grossier vers le fin.

© 2026 Oussama Ghorbel

Le gain en qualité de reconstruction est mesurable. Les expérimentations montrent qu'un RQ-VAE à 3 niveaux avec des codebooks de taille 1024 atteint typiquement un PSNR supérieur de 1,5 à 2,5 dB par rapport à un VQ-VAE simple de même taille de codebook, sur des images naturelles à 256 × 256. Chaque niveau résiduel apporte un gain décroissant, ce qui permet de tronquer la génération si le budget est serré : s'arrêter au niveau 2 donne une image légèrement plus douce mais structurellement correcte.

L'inconvénient est la complexité d'entraînement : les codebooks résiduels doivent être entraînés conjointement, et l'effondrement de codebook (un problème déjà discuté au chapitre 7) peut frapper chaque niveau indépendamment.

13.4 — Tiling et génération par tuiles avec recouvrement

Quand la résolution cible dépasse largement ce qu'un seul contexte de tokens peut couvrir, une stratégie pragmatique consiste à découper l'image en tuiles, générer chaque tuile indépendamment (ou quasi indépendamment), puis les assembler.

Le problème naïf du tiling sans recouvrement est évident : des coutures visibles, des incohérences de couleur et de structure aux frontières. Pour atténuer ce défaut, on introduit un *recouvrement* (overlap) entre tuiles adjacentes — typiquement 15 à 25 % de la taille de tuile. Les pixels de la zone de recouvrement sont ensuite fusionnés par un fondu linéaire (blending) ou par une moyenne pondérée.

Pour un modèle autoregressive, le tiling pose une difficulté supplémentaire : l'ordre de génération. Si vous générez la tuile en haut à gauche en premier, comment la tuile en bas à droite connaît-elle le contexte global ? Deux solutions principales :

- **Conditionnement global** : on génère d'abord une représentation basse résolution de l'image entière (par exemple 64 × 64, soit 16 tokens avec des patches 16 × 16), puis chaque tuile est générée conditionnellement à cette esquisse globale. C'est une variante de la cascade, mais appliquée au niveau des tuiles.

- **Génération séquentielle avec contexte étendu** : on génère les tuiles dans un ordre raster (de gauche à droite, de haut en bas), et

© 2026 Oussama Ghorbel

chaque tuile reçoit en conditionnement les tokens de ses voisines déjà générées. Le coût augmente linéairement avec le nombre de tuiles, mais chaque étape reste de complexité bornée.

En pratique, le tiling fonctionne remarquablement bien pour des textures répétitives (paysages, ciels, motifs), mais montre ses limites dès qu'un objet s'étend sur plusieurs tuiles — un visage à cheval sur deux tuiles produira des incohérences anatomiques.

13.5 — Adaptation dynamique de la résolution : plus de tokens sur les zones complexes

Toutes les régions d'une image ne méritent pas le même nombre de tokens. Un ciel uniforme peut être encodé en une poignée de tokens ; un visage humain, avec ses micro-expressions et ses textures de peau, en exige beaucoup plus. L'adaptation dynamique de la résolution exploite cette asymétrie.

Le principe : un module léger (souvent un petit réseau convolutif ou un détecteur de saillance) analyse l'image cible ou l'esquisse basse résolution et produit une *carte de complexité*. Les régions à haute complexité (visages, texte, arêtes, textures fines) reçoivent une grille de patches plus dense — des patches 8 × 8 au lieu de 16 × 16, par exemple. Les régions simples restent à 16 × 16, voire 32 × 32.

Dans un cadre autoregressive, cela se traduit par une séquence de tokens de longueur variable selon l'image. Pour maintenir un batch efficace à l'entraînement, on peut regrouper les images par budget de tokens similaire ou utiliser du padding avec masque d'attention.

Cette approche réduit le nombre total de tokens de 30 à 60 % sur des images naturelles typiques, sans perte perceptible de qualité dans les régions simples. Le risque est une allocation sous-optimale : si le module de saillance sous-estime la complexité d'une zone (un reflet subtil sur du verre, par exemple), la génération y sera floue ou artefactée.

On observe que cette stratégie est particulièrement pertinente pour les applications où le budget d'inférence est contraint — génération sur appareil mobile, par exemple — mais qu'elle ajoute une dépendance au module d'analyse, qu'il faut entraîner et maintenir.

© 2026 Oussama Ghorbel

13.6 — Budget tokens pour 1024 × 1024 et au-delà : stratégies pratiques

Faisons les comptes. Une image 1024 × 1024 avec des patches 16 × 16 produit 64 × 64 = 4 096 tokens. L'attention quadratique sur 4 096 tokens demande environ 16,8 millions de paires d'attention par tête et par couche. Pour un modèle à 32 couches et 16 têtes, cela représente un volume de calcul considérable, mais encore dans les limites d'un GPU moderne pour une seule image. Le vrai problème surgit à 2048 × 2048 (16 384 tokens) ou en génération batchée.

Voici un tableau récapitulatif des stratégies et de leur budget approximatif pour atteindre 1024 × 1024 :

Stratégie	Tokens générés	Complexité dominante
Patchification directe 16 × 16	4 096	$O(4096^2)$
Cascade 256 -> SR ×4	256 + coût diffusion	$O(256^2)$ + $O(1024^2)$ conv
Cascade 256 -> 512 -> 1024	256 + 1024 + coût SR	$O(256^2)$ + $O(1024^2)$ + conv
RQ-VAE 3 niveaux sur 256 tokens	256 × 3 = 768	$O(768^2)$
Tiling 4 × 4 (tuiles 256, overlap 20 %)	16 × 256 = 4 096	16 × $O(256^2)$
Coarse-to-fine (8×8 -> 16×16 -> 32×32 -> 64×64)	64 + 256 + 1024 + 4096 env. 5 440	Somme des $O(n^2)$ par niveau

La cascade 256 -> super-résolution reste la plus économe en tokens autoregressifs. La tokenisation hiérarchique RQ-VAE offre le meilleur compromis qualité/coût si l'on souhaite rester dans un cadre purement discret. Le tiling est la solution la plus simple à implémenter mais la moins élégante en termes de cohérence globale.

Au-delà de 1024 × 1024, je recommande de combiner les approches : une génération autoregressive à 256 × 256 (256 tokens), un raffinement par RQ-VAE à 512 × 512, puis une super-résolution par diffusion ou par réseau convolutionnel pour atteindre 1024 ou 2048. Chaque étape reste dans un budget de tokens raisonnable, et la qualité finale est supérieure à celle d'une génération monolithique à pleine résolution, car chaque modèle est spécialisé sur sa bande de fréquences.

© 2026 Oussama Ghorbel

Un dernier point, souvent négligé : la *résolution de sortie* n'est pas la seule variable. Le nombre de canaux du codebook, la dimension des embeddings, la taille du contexte textuel interagissent avec le budget tokens. Dans le chapitre 22, nous assemblerons un protocole complet où ces paramètres sont fixés de manière cohérente pour un LLM 7B visant la génération 256 × 256. Ici, retenez surtout le principe : *la résolution se conquiert par étapes, jamais d'un seul bond*.

Exercices

Exercice 13.1 — Pipeline deux étapes et coût total.

Concevez un pipeline en deux étapes : (a) génération autoregressive d'une image 256 × 256 tokenisée en 256 tokens (patches 16 × 16) ; (b) upscaling ×4 vers 1024 × 1024 via un réseau de super-résolution convolutionnel (type ESRGAN ou SwinIR). Estimez le coût computationnel total en FLOPs pour un modèle transformer à 12 couches, dimension 768, 12 têtes, en supposant que le réseau de SR effectue environ 50 GFLOPs pour une image 256 -> 1024. Comparez ce coût à celui d'une génération directe en 4 096 tokens.

Exercice 13.2 — RQ-VAE à trois niveaux résiduels.

Implémentez un schéma RQ-VAE à trois niveaux de quantification résiduelle. Chaque codebook contient 1 024 entrées de dimension 8. Entraînez sur un sous-ensemble de 10 000 images (par exemple CIFAR-10 ou un extrait de CelebA à 64 × 64). Mesurez le PSNR de reconstruction après le niveau 1, après le niveau 2, puis après le niveau 3. Tracez la courbe PSNR en fonction du nombre de niveaux et commentez le gain marginal de chaque niveau.

Exercice 13.3 — Budget tokens : direct vs cascade.

Calculez le nombre total de tokens générés pour une image 1024 × 1024 selon deux stratégies : (a) patchification directe avec patches 16 × 16 ; (b) cascade en trois étapes — génération 256 × 256 (patches 16 × 16), puis upscaling vers 512 × 512 avec de nouveaux patches 16 × 16, puis upscaling vers 1024 × 1024. Pour la stratégie (b), précisez combien de tokens sont générés à chaque étape et calculez le coût attentionnel cumulé. Quelle stratégie est la plus économe ? À partir de quelle résolution la cascade devient-elle indispensable ?

© 2026 Oussama Ghorbel

Indications pour les exercices

Indications — Exercice 13.1. Pour estimer le coût du transformer, décomposez la complexité en deux termes : l'attention (proportionnelle à $n^2 \cdot d$ par couche, où n est le nombre de tokens et d la dimension) et le réseau feed-forward (proportionnelle à $n \cdot d^2$ par couche). Avec $n = 256$, $d = 768$, 12 couches et 12 têtes, le terme attentionnel vaut approximativement $12 \times 256^2 \times 768 \approx 604 \times 10^6$ opérations par image pour l'attention seule. Le feed-forward ajoute un facteur 8 (deux projections linéaires de dimension $4d$), soit environ $12 \times 256 \times 768 \times 3072 \approx 7,2 \times 10^9$ FLOPs. Ajoutez les 50 GFLOPs du réseau de super-résolution. Comparez ensuite avec le cas $n = 4\,096$: l'attention passe en $4\,096^2$, soit un facteur multiplicatif de 256 sur le terme quadratique. Vous devriez constater que le pipeline en cascade coûte environ 5 à 8 fois moins cher qu'une génération directe à 4 096 tokens, selon l'architecture exacte du super-résolveur.

Indications — Exercice 13.2. Pour l'entraînement du RQ-VAE, utilisez une fonction de perte en trois composantes : reconstruction MSE, engagement loss (avec un coefficient β entre 0,1 et 0,25), et la perte de quantification résiduelle appliquée séquentiellement à chaque niveau. Initialisez chaque codebook par k-means sur un échantillon de vecteurs latents issus de l'encodeur après quelques epochs de pré-entraînement en continu. Pour le PSNR, calculez-le sur l'ensemble de test après chaque niveau en reconstruisant $\hat{z} = \sum_{k=1}^{j} C_k[c_k]$ pour $j \in \{1, 2, 3\}$. Vous devriez observer un gain typique de 1 à 2 dB entre le niveau 1 et le niveau 2, puis de 0,5 à 1 dB entre le niveau 2 et le niveau 3, confirmant la décroissance du gain marginal. Si un niveau ne contribue plus significativement, c'est un signe que la capacité du codebook est saturée ou que le résidu est trop faible pour être quantifié utilement.

Indications — Exercice 13.3. Pour la stratégie (a), le calcul est direct : $(1024/16)^2 = 4\,096$ tokens. Pour la stratégie (b), l'étape 1 génère $(256/16)^2 = 256$ tokens ; l'étape 2 opère sur une image 512 × 512, soit $(512/16)^2 = 1\,024$ tokens ; l'étape 3 sur 1024 × 1024, soit $(1024/16)^2 = 4\,096$ tokens. Le nombre total de tokens générés est donc $256 + 1\,024 + 4\,096 = 5\,376$, supérieur en volume brut à la stratégie (a). Cependant, le coût attentionnel cumulé est

© 2026 Oussama Ghorbel

$\mathcal{O}(256^2) + \mathcal{O}(1\,024^2) + \mathcal{O}(4\,096^2)$ si chaque étape est indépendante, ce qui est identique en ordre de grandeur à $\mathcal{O}(4\,096^2)$ dominant. La différence se joue sur la constante : dans la cascade, les deux premières étapes sont négligeables, et l'étape finale bénéficie d'un conditionnement fort qui réduit l'entropie effective de la distribution. Si l'on autorise un modèle de super-résolution non-autoregressive (convolutionnel ou par diffusion) pour les étapes 2 et 3, le coût attentionnel se limite à $\mathcal{O}(256^2)$, ce qui rend la cascade strictement supérieure au-delà de 512 × 512. Le seuil exact dépend de la taille de contexte de votre transformer et du coût d'inférence du super-résolveur, mais en pratique, dès que $n > 1\,024$ tokens, la cascade devient difficile à éviter.

Résumé du chapitre

Récapitulons les idées forces. La génération haute résolution se heurte à un mur : le nombre de tokens croît quadratiquement avec la résolution linéaire, et la complexité attentionnelle croît quadratiquement avec le nombre de tokens. Toute stratégie viable repose sur la décomposition du problème en sous-problèmes de taille bornée.

Nous avons exploré cinq familles de solutions :

1. **La cascade de modèles** sépare la génération sémantique (basse résolution) du raffinement textural (super-résolution). C'est la stratégie la plus éprouvée, de DALL·E à Stable Diffusion XL.

2. **Le coarse-to-fine interne** organise la séquence autoregressive en niveaux de granularité croissante, réduisant la longueur effective de chaque étape de décodage.

3. **La tokenisation hiérarchique par RQ-VAE** permet de réordonner l'information du global vers le local sans changer la longueur de séquence, et offre une troncature gracieuse du budget.

4. **Le tiling avec recouvrement** découpe l'espace spatial en sous-problèmes indépendants, au prix d'un risque d'incohérence aux frontières.

5. **L'adaptation dynamique de résolution** alloue les tokens là où ils sont informationnellement nécessaires, réduisant le budget moyen sans sacrifier la qualité perçue.

© 2026 Oussama Ghorbel

Aucune de ces stratégies n'est universellement supérieure. Le choix dépend du modèle de base, du matériel disponible, de la latence tolérée et de la nature des images cibles. Dans la pratique, les systèmes de production combinent plusieurs de ces approches — typiquement une génération autoregressive à basse résolution, suivie d'un raffinement par diffusion ou par réseau convolutionnel, avec éventuellement un tiling pour les très grandes résolutions.

Transition vers la Partie IV

Nous avons maintenant en main l'essentiel des outils pour représenter une image sous forme de tokens et pour organiser la génération de ces tokens à différentes échelles. Mais une question reste en suspens, et elle est de taille : comment *guider* cette génération par un prompt textuel ? Comment un LLM, qui manipule des tokens de langage, peut-il conditionner la production d'une séquence de tokens visuels de manière cohérente, fidèle et contrôlable ?

C'est l'objet de la Partie IV. Au chapitre 14, nous commencerons par unifier les vocabulaires textuel et visuel dans un même espace de tokens, avant d'aborder au chapitre 15 les mécanismes de cross-attention et de conditionnement qui permettent au texte de piloter l'image. Les protocoles d'entraînement et l'adaptation de LLM existants suivront aux chapitres 16 et 17. La résolution progressive que nous venons d'étudier n'est pas abandonnée : elle reviendra comme contrainte d'architecture dans chacun de ces chapitres, car le choix du nombre de tokens et de leur hiérarchie conditionne directement la manière dont le conditionnement textuel peut s'insérer dans la séquence.

La résolution se conquiert par étapes. L'alignement texte-image aussi. Avançons.

© 2026 Oussama Ghorbel

Partie IV — Alignement Texte-Image, Entraînement et Architecture

© 2026 Oussama Ghorbel

Chapitre 14 – Unifier les vocabulaires : texte et image dans un même espace de tokens

Jusqu'à présent, nous avons construit séparément les deux mondes qui nous intéressent : d'un côté, le langage naturel et ses tokens discrets issus d'un tokenizer textuel ; de l'autre, l'image et sa quantification en indices de codebook via un VQGAN ou un tokenizer visuel équivalent. Le défi central de ce chapitre consiste à faire cohabiter ces deux vocabulaires au sein d'un même modèle autoregressif, sans détruire les représentations linguistiques patiemment apprises lors du pré-entraînement.

Dans ce chapitre, je détaille les mécanismes concrets qui permettent d'étendre le vocabulaire d'un grand modèle de langage pour y intégrer des tokens visuels, de maintenir la cohérence géométrique des embeddings, et de formater les séquences hybrides texte-image de manière exploitable par l'attention. Chaque décision — ajout de tokens, choix d'initialisation, format de séquence, gestion des longueurs — a des conséquences mesurables sur la stabilité de l'entraînement et la qualité finale de la génération.

14.1 — Ajout de tokens spéciaux : `<img_start>`, `<img_end>`, indices de codebook

La première étape, la plus littérale, consiste à déclarer de nouveaux tokens dans le vocabulaire du tokenizer textuel. Concrètement, si notre LLM possède un vocabulaire de 50 257 tokens (cas de GPT-2) ou 32 000 tokens (cas de LLaMA), nous y ajoutons :

- **Des tokens de délimitation** : `<img_start>` et `<img_end>` signalent au modèle le début et la fin d'une région visuelle. Ils jouent un rôle analogue aux tokens `[CLS]` et `[SEP]` de BERT, mais dans un cadre strictement autoregressif. Le modèle apprend à associer `<img_start>` à une transition de distribution : les tokens suivants ne sont plus des sous-mots mais des indices de codebook.

• **Les indices de codebook** : si le VQGAN utilise un codebook de taille $K = 8\,192$, nous ajoutons 8 192 tokens fictifs, notés `<vis_0>`, `<vis_1>`, …, `<vis_8191>`. Chacun correspond à un vecteur quantifié dans l'espace latent de l'image.

• **Des tokens de résolution ou de canal** (optionnels) : dans les architectures multi-échelle, on peut introduire `<res_256>`, `<res_512>` pour indiquer la résolution de la région qui suit.

L'opération est triviale sur le plan logiciel : la plupart des bibliothèques (Hugging Face `tokenizers`, SentencePiece) exposent une méthode `add_tokens()` ou `add_special_tokens()`. Le vocabulaire passe de V à $V + K + n_{\text{special}}$, et la matrice d'embedding doit être redimensionnée en conséquence — point que nous approfondissons en section 14.3.

Un choix de conception important : faut-il un vocabulaire visuel partagé entre toutes les images, ou un vocabulaire spécifique par domaine (visages, paysages, diagrammes) ? Dans la pratique, un codebook unique entraîné sur un corpus diversifié offre la meilleure généralisation. Les vocabulaires spécialisés n'apportent un gain que dans des contextes très contraints (imagerie médicale, par exemple), au prix d'une complexité accrue de routage.

14.2 — Embedding partagé vs embedding séparé : projection linéaire, adaptateur

Une fois les tokens déclarés, il faut leur associer des vecteurs dans l'espace d'embedding du modèle. Deux philosophies s'opposent :

Embedding partagé (unified embedding). Tous les tokens — textuels et visuels — sont projetés par la même matrice $W_E \in \mathbb{R}^{(V+K) \times d}$. L'avantage est la simplicité : une seule table d'embedding, un seul mécanisme d'attention, aucune rupture architecturale. L'inconvénient est que les tokens visuels et textuels n'ont a priori aucune raison d'occuper la même région de l'espace latent. Les premiers pas d'entraînement peuvent produire des gradients instables si les embeddings visuels sont mal initialisés.

Embedding séparé avec projection. On conserve la matrice $W_E^{\text{text}} \in \mathbb{R}^{V \times d}$ intacte et on introduit une matrice distincte $W_E^{\text{vis}} \in \mathbb{R}^{K \times d}$

© 2026 Oussama Ghorbel

pour les tokens visuels. Les deux sorties sont concaténées ou additionnées selon un masque de type de token. Une variante plus expressive consiste à prendre les vecteurs du codebook VQGAN (dimension d_{code}, souvent 256 ou 512) et à les projeter vers la dimension du LLM via une couche linéaire apprenable :

$$e_{\text{vis}} = W_{\text{proj}} \cdot c_k + b_{\text{proj}}, \quad W_{\text{proj}} \in \mathbb{R}^{d \times d_{\text{code}}}$$

Cette projection agit comme un **adaptateur** : elle traduit la géométrie du codebook visuel dans la géométrie de l'espace textuel sans perturber les embeddings existants. Des expérimentations montrent qu'une projection linéaire unique suffit pour des codebooks de dimension ≤ 512 ; au-delà, un MLP à deux couches avec activation GELU stabilise l'entraînement.

Critère de choix. Si le budget computationnel est limité et que le codebook est de petite taille ($K \leq 1\,024$), l'embedding partagé reste viable. Pour des codebooks de 8 192 entrées ou plus, la projection séparée protège les poids textuels et accélère la convergence.

14.3 — Initialisation des nouveaux embeddings : aléatoire, moyenne, héritage sémantique

Redimensionner la matrice d'embedding implique d'initialiser les nouvelles lignes. Trois stratégies classiques :

1. **Initialisation aléatoire** (normale $\mathcal{N}(0, \sigma^2)$ avec σ identique à celui des embeddings textuels). Simple, mais les tokens visuels démarrent dans une région arbitraire de l'espace, loin de toute sémantique utile. Les premiers milliers de pas sont consacrés à « rapatrier » ces vecteurs vers une zone cohérente.

2. **Initialisation par la moyenne** des embeddings textuels existants : $\forall k,\ e_k^{\text{vis}} = \frac{1}{V}\sum_{i=1}^{V} e_i^{\text{text}}$. Cette stratégie place tous les tokens visuels au centroïde du nuage textuel. Elle évite les valeurs aberrantes mais introduit un biais : tous les tokens visuels sont identiques au départ, ce qui ralentit la différenciation.

3. **Héritage sémantique via le codebook.** On initialise chaque embedding visuel en projetant le vecteur du codebook VQGAN correspondant : $e_k^{\text{vis}} = W_{\text{proj}}^{\text{init}} \cdot c_k$. La matrice $W_{\text{proj}}^{\text{init}}$ peut être obtenue

© 2026 Oussama Ghorbel

par une régression linéaire alignant un sous-ensemble de paires (description textuelle, région image). Cette approche, que je recommande, fournit une sémantique de départ exploitable : le token `<vis_42>` associé à une texture de ciel bleu sera initialement proche des embeddings de « ciel », « bleu », « nuage ».

En pratique, on peut combiner l'approche 3 avec un léger bruit gaussien ($\sigma = 0{,}02$) pour briser les symétries et éviter que des tokens visuels voisins dans le codebook restent colinéaires après initialisation.

14.4 — Format de séquence interleaved : [texte] [image] [texte] [image]

Un modèle unifié doit traiter des séquences où texte et image s'alternent librement. Le format canonique que j'adopte dans ce livre est le suivant :

Chaque bloc image est encadré par `<img_start>` et `<img_end>`. Le texte avant l'image fournit le conditionnement ; le texte après peut décrire, commenter ou introduire l'image suivante. Ce format **interleaved** présente plusieurs avantages :

- **Bidirectionnalité du contexte** : lors de la génération d'une image, le modèle dispose du texte précédent (conditionnement) et peut, en entraînement, voir le texte suivant (objectif de reconstruction mutuelle).
- **Génération multi-images** : rien n'empêche d'enchaîner plusieurs blocs visuels dans une même séquence, chacun conditionné par le texte qui le précède.
- **Compatibilité avec le next-token prediction** : la factorisation $p(x) = \prod_i p(x_i \mid x_{<i})$ s'applique sans modification. Les tokens visuels sont simplement des tokens parmi d'autres.

Une subtilité : lors de l'inférence en génération d'image, on échantillonne séquentiellement les N tokens visuels (par exemple 256 pour une image 16×16 en patches, ou 1 024 pour une 32×32). Le token `<img_end>` est généré comme un token ordinaire ; sa probabilité conditionnelle sert de signal d'arrêt. Si le modèle ne le génère pas après N pas, on tronque.

© 2026 Oussama Ghorbel

14.5 — Gestion des longueurs variables : padding, packing, attention mask

Les séquences interleaved ont des longueurs très hétérogènes : un bloc texte peut contenir 20 tokens, un bloc image 1 024. Plusieurs stratégies de batchification :

Padding classique. On remplit chaque séquence du batch jusqu'à la longueur maximale $L_{\max}$ avec un token `<pad>`. Simple mais coûteux en mémoire si une séquence contient plusieurs images. L'attention mask est mis à zéro sur les positions de padding.

Packing (ou sequence packing). On concatène plusieurs séquences courtes en une seule séquence de longueur $L_{\max}$, séparées par `<eos>`. L'attention mask devient **bloc-diagonal** : chaque sous-séquence n'attend qu'à elle-même. Cette technique, popularisée par l'entraînement de LLaMA et Mistral, maximise l'utilisation du GPU et réduit le gaspillage de calcul sur les tokens de padding.

Attention mask causal étendu. Pour un bloc image, on peut choisir une attention bidirectionnelle interne (tous les tokens visuels se voient entre eux) tout en conservant une attention causale vis-à-vis du texte. Le masque d'attention $M \in \{0, -\infty\}^{L \times L}$ prend alors la forme :

$$M_{ij} = \begin{cases} 0 & \text{si } j \leq i \text{ (causal standard)} \\ 0 & \text{si } i, j \in [\text{img_start}, \text{img_end}] \text{ (bidirectionnel intra-image)} \\ -\infty & \text{sinon} \end{cases}$$

Ce masque hybride est implémentable dans Flash Attention 2 en spécifiant des fenêtres d'attention variables par segment.

14.6 — Compatibilité avec les poids pré-entraînés du LLM : freeze partiel vs fine-tune total

L'ajout de tokens visuels ne doit pas détruire les capacités linguistiques du modèle. Trois régimes d'entraînement sont envisageables :

Freeze total du LLM, entraînement de l'adaptateur seul. On gèle l'intégralité des poids du transformer et on n'entraîne que W_{proj} ainsi que les nouveaux embeddings visuels. Avantage : coût minimal, aucune dégradation du texte. Inconvénient : le modèle ne peut pas adapter ses

© 2026 Oussama Ghorbel

couches profondes à la structure spatiale de l'image. Les performances en génération restent limitées.

Freeze partiel (layer-wise unfreezing). On gèle les $N - k$ premières couches et on fine-tune les k dernières (typiquement $k = 4$ à 8). Les couches basses, spécialisées dans la syntaxe et la morphologie, sont préservées ; les couches hautes, plus sémantiques, s'adaptent à la distribution visuelle. C'est le compromis que je privilégie pour un premier prototype.

Fine-tune total. Tous les paramètres sont mis à jour. C'est la configuration la plus puissante mais aussi la plus risquée : sans précaution, le modèle oublie le langage (catastrophic forgetting). Les parades classiques incluent un learning rate différencié (10^{-5} pour les couches textuelles, 10^{-4} pour les nouvelles couches), un mélange de données texte pur dans le batch (ratio 10 à 20 %), et éventuellement un terme de régularisation EWC (Elastic Weight Consolidation).

Dans le protocole du chapitre 22, nous adopterons un fine-tune total avec learning rate cosmique, warmup de 2 000 pas et mélange 80 % interleaved / 20 % texte pur. Ce régime offre le meilleur FID à convergence, au prix d'un coût GPU environ 3× supérieur au freeze partiel.

Synthèse du chapitre

Unifier texte et image dans un même espace de tokens n'est pas une simple concaténation de vocabulaires. C'est un ensemble de décisions cohérentes — déclaration de tokens spéciaux, choix d'embedding, initialisation, format de séquence, gestion des longueurs, stratégie de fine-tune — qui déterminent si le modèle traitera l'image comme un signal de seconde zone ou comme un citoyen à part entière de son espace latent. La règle que je retiens : **préserver la géométrie textuelle existante tout en offrant aux tokens visuels une initialisation sémantiquement ancrée**. Le reste, c'est de l'entraînement.

Exercices

© 2026 Oussama Ghorbel

Exercice 14.1 — Étendre le tokenizer GPT-2 avec 8 192 tokens visuels.

Chargez le tokenizer et le modèle GPT-2 (`gpt2`, vocabulaire de 50 257 tokens, dimension d'embedding 768). Ajoutez 8 192 tokens `<vis_0>` à `<vis_8191>` ainsi que `<img_start>` et `<img_end>`. Redimensionnez la matrice d'embedding du modèle avec `model.resize_token_embeddings()`. Vérifiez :

- La shape de `model.wte.weight` est bien $(50\,257 + 8\,194,\ 768)$.
- La shape de `model.lm_head.weight` est identique.
- Un forward pass sur la séquence `<img_start> <vis_0> <vis_1> <img_end>` produit un tenseur de logits de shape $(1, 4, 58\,451)$.

Indice : utilisez `tokenizer.add_tokens([...])` puis `model.resize_token_embeddings(len(tokenizer))`.

Exercice 14.2 — Template interleaved et attention mask.

Écrivez une fonction Python qui, étant données trois paires (texte, liste de tokens visuels), construit la séquence interleaved complète au format :

Générez ensuite l'attention mask causal hybride décrit en section 14.5 : causal par défaut, bidirectionnel à l'intérieur de chaque bloc image. Représentez le masque pour une séquence de longueur 20 (3 tokens texte + 4 tokens visuels + 1 token de délimitation par bloc, simplifié) et vérifiez qu'il est bien triangulaire inférieur en dehors des blocs visuels.

Exercice 14.3 — Surcoût mémoire de l'ajout de 8 192 embeddings.

Calculez le surcoût mémoire induit par l'ajout de 8 192 tokens visuels de dimension $d = 4\,096$ en précision FP16 (2 octets par paramètre). Détaillez :

- Taille de la matrice d'embedding additionnelle : $8\,192 \times 4\,096 \times 2$ octets.
- Taille du lm_head additionnel (même shape, si non lié).
- Surcoût total en Mo.

© 2026 Oussama Ghorbel

• Comparaison avec la mémoire d'un modèle 7B en FP16 (environ 14 Go) : quel pourcentage cela représente-t-il ?

Concluez sur la négligeabilité ou non de ce surcoût dans un contexte de déploiement sur GPU 80 Go.

Corrigés et pistes de résolution

Pour ancrer ces concepts, voici les éléments de correction et les implémentations de référence pour les exercices proposés. En tant qu'ingénieurs et chercheurs, c'est dans le code que la théorie affronte la réalité des tenseurs et de la mémoire vidéo.

Solution de l'exercice 14.1 : Extension du tokenizer et redimensionnement

L'API `transformers` de Hugging Face rend cette opération remarquablement indolore, à condition de bien comprendre ce qui se passe sous le capot. Voici le script canonique :

Ce qui est crucial ici, c'est que `resize_token_embeddings` initialise les nouvelles lignes de la matrice avec une distribution normale standard, tout en préservant les poids pré-entraînés. Comme nous l'avons vu en section 14.3, il vous appartiendra ensuite d'écraser cette initialisation aléatoire par votre stratégie d'héritage sémantique avant de lancer la boucle d'entraînement.

Solution de l'exercice 14.2 : Masque d'attention hybride

La construction du masque d'attention est l'un des points les plus délicats de l'implémentation d'un modèle multimodal autoregressif. L'objectif est de combiner une matrice triangulaire inférieure (pour le texte et la causalité globale) avec des blocs bidirectionnels (pour les tokens visuels).

Ce masque permet au token `<vis_1>` d'attendre à `<vis_0>` et à `<img_start>`, tout en restant aveugle au texte qui suit le `<img_end>`. C'est cette flexibilité qui permet au modèle de traiter l'image comme une entité spatiale cohérente plutôt que comme une simple séquence temporelle stricte.

Solution de l'exercice 14.3 : Calcul du surcoût mémoire

© 2026 Oussama Ghorbel

Faisons parler les chiffres. L'ajout de tokens visuels a un impact direct sur la VRAM, qu'il faut quantifier avec précision avant de réserver des instances de calcul.

1. **Matrice d'embedding additionnelle** :

$$8\,192 \text{ tokens} \times 4\,096 \text{ dimensions} \times 2 \text{ octets (FP16)} = 67\,108\,864 \text{ octets}.$$

Soit environ **64 Mo**.

2. **Tête de lecture (lm_head) additionnelle** :

Si les poids ne sont pas liés (ce qui est souvent le cas dans les architectures modernes pour permettre des dynamiques d'apprentissage distinctes), la matrice de projection finale subit exactement le même surcoût : **64 Mo**.

3. **Surcoût total** :

$$64 \text{ Mo} + 64 \text{ Mo} = \mathbf{128} \text{ Mo}.$$

4. **Comparaison et conclusion** :

Un modèle de 7 milliards de paramètres en FP16 pèse environ 14 Go (soit 14 336 Mo). Le surcoût de 128 Mo représente donc :

$$\frac{128}{14\,336} \approx \mathbf{0{,}89}\ \%.$$

Conclusion de l'exercice : Sur le plan strict de la capacité statique de la VRAM d'un GPU de 80 Go (comme un A100 ou un H100), ce surcoût est **totalement négligeable**. Le modèle chargera sans encombre.

Cependant, attention au piège du *memory bandwidth* et du cache L2. Lors de l'inférence, la lecture de cette matrice d'embedding élargie peut fragmenter le cache si les tokens visuels et textuels sont accédés de manière entrelacée et imprévisible. De plus, n'oubliez pas que ces 128 Mo doivent être multipliés par le nombre de copies nécessaires pour l'optimiseur (par exemple, AdamW en FP32 stocke les états et les moments, ce qui multiplie ce surcoût par 8 ou 12 en mémoire d'entraînement). En fine-tuning, ces 128 Mo deviennent alors plus d'un gigaoctet d'états d'optimiseur à gérer.

© 2026 Oussama Ghorbel

Transition vers la suite

Nous voici désormais équipés d'un vocabulaire unifié. Notre LLM est capable de lire et d'écrire des séquences où les mots et les patches d'images s'entrelacent dans une même chorégraphie autoregressive. Les embeddings sont alignés, les masques d'attention sont forgés, et le modèle est prêt à affronter la réalité des données multimodales.

Pourtant, une limite fondamentale subsiste. Si l'autorégression pure permet de générer des images token par token, elle le fait en traitant le conditionnement textuel comme un simple préfixe de séquence. Le texte n'est "lu" qu'une seule fois, au début de la génération, et son influence doit se propager à travers des dizaines, voire des centaines de couches d'attention pour guider le millième token visuel. Cette dilution de l'information textuelle est l'une des causes principales des artefacts de cohérence et des oublis sémantiques dans les générateurs purement autoregressifs.

Comment maintenir le texte "vivant" et présent à chaque étape de la synthèse visuelle, sans pour autant abandonner l'élégance de notre espace de tokens unifié ? C'est précisément l'objet du chapitre suivant, où nous introduirons les mécanismes de cross-attention et de conditionnement dense, véritables ponts architecturaux entre la sémantique linguistique et la géométrie pixelique.

Chapitre 15 — Cross-attention et conditionnement textuel : guider la génération

L'élégance de l'espace de tokens unifié, que nous avons forgé au chapitre précédent, ne doit pas nous aveugler sur ses limites pratiques. Lorsque vous demandez à un modèle purement autoregressif de générer une image de 256 × 256 pixels, vous lui faites produire environ 1 024 tokens visuels. Si le prompt textuel qui conditionne cette génération n'est qu'un simple préfixe de 30 tokens, vous créez une situation mathématiquement défavorable : le millième token visuel doit « se souvenir » du texte en interrogeant un contexte dominé par les 999 tokens visuels qui le précèdent.

Dans ce chapitre, je vous propose de briser cette dilution du signal. Nous allons introduire la cross-attention, un mécanisme qui permet aux

© 2026 Oussama Ghorbel

tokens visuels d'interroger directement et massivement le texte à chaque couche du réseau. Nous explorerons comment intégrer ces ponts architecturaux sans détruire les capacités du LLM, comment gérer le conditionnement dense, et comment mettre en œuvre ces concepts sur un modèle de 7 milliards de paramètres.

15.1 — Les limites du préfixe textuel : la dilution du signal

Pour comprendre pourquoi la cross-attention est indispensable, il faut regarder de près la mécanique de l'attention softmax. Supposons que nous générions le token visuel v_i. La couche d'auto-attention calcule les scores d'attention entre la requête q_i (dérivée de v_i) et toutes les clés k_j du contexte (les tokens textuels $t_1 \dots t_m$ et les tokens visuels précédents $v_1 \dots v_{i-1}$).

Le score d'attention pour un token textuel t_j est proportionnel à :

$$\text{softmax}\left(\frac{q_i \cdot k_{t_j}}{\sqrt{d}}\right)$$

Le problème réside dans le dénominateur de la fonction softmax, qui est la somme des exponentielles de tous les scores. Lorsque i augmente, le nombre de tokens visuels dans le contexte explose. Même si les clés textuelles k_{t_j} sont sémantiquement pertinentes, leurs scores bruts $q_i \cdot k_{t_j}$ sont noyés dans la masse des scores visuels. La probabilité d'attention allouée au texte tend asymptotiquement vers zéro.

C'est ce que j'appelle la **dilution du contexte**. En pratique, cela se traduit par des artefacts frustrants : vous demandez « un chat roux portant un chapeau bleu », et le modèle génère un chat, mais oublie le roux, ou le chapeau, ou le bleu. Le modèle a capturé la sémantique globale (le chat) grâce aux premiers tokens visuels, mais les détails fins, portés par des mots moins fréquents dans le vocabulaire, sont perdus dans le bruit de l'auto-attention visuelle.

La solution n'est pas d'augmenter la température ou de bidouiller les logits. La solution est architecturale : il faut fournir aux tokens visuels un canal dédié, isolé du bruit de leur propre contexte, pour aller chercher l'information textuelle. C'est le rôle de la cross-attention.

© 2026 Oussama Ghorbel

15.2 — Mécanisme de cross-attention : queries visuelles, keys et values textuelles

La cross-attention est un emprunt direct aux transformers de traduction automatique, mais son application à la génération d'image par un LLM requiert des adaptations subtiles.

Le principe est de dissocier les rôles. Les requêtes (Q) proviennent de l'état caché des tokens visuels à la couche courante. Les clés (K) et les valeurs (V) proviennent des représentations des tokens textuels.

Mathématiquement, pour une couche l :

$$Q^{(l)} = X_{\text{vis}}^{(l)} W_Q^{(l)}, \quad K^{(l)} = X_{\text{text}} W_K^{(l)}, \quad V^{(l)} = X_{\text{text}} W_V^{(l)}$$

$$\text{CrossAttn}(Q^{(l)}, K^{(l)}, V^{(l)}) = \text{softmax}\left(\frac{Q^{(l)}(K^{(l)})^T}{\sqrt{d}}\right) V^{(l)}$$

Géométriquement, chaque patch visuel (ou token visuel) pose une question : « Quelle est ma couleur ? Ma texture ? Ma position ? ». Les matrices K et V textuelles agissent comme une base de données mémorielle. Le mécanisme d'attention permet au token visuel de récupérer la réponse exacte dans cette base de données.

Contrairement à l'auto-attention, la cross-attention n'est **pas causale** vis-à-vis du texte. Chaque token visuel peut voir l'intégralité du prompt textuel simultanément. C'est crucial : le token visuel numéro 1000 a exactement le même accès au mot « bleu » que le token numéro 1. Il n'y a plus de dilution. Le dénominateur de la softmax ne contient que les m tokens textuels, ce qui garantit que l'attention textuelle reste dense et informative.

Cependant, la cross-attention ne remplace pas l'auto-attention. Elle la complète. L'auto-attention gère la cohérence spatiale, les textures, les bords et la structure globale de l'image. La cross-attention injecte la sémantique textuelle. C'est cette séparation des préoccupations qui permet d'atteindre des niveaux de fidélité et d'alignement texte-image inaccessibles aux modèles purement autoregressifs.

15.3 — Intégration architecturale : où placer les blocs de cross-attention ?

© 2026 Oussama Ghorbel

Une fois le mécanisme défini, se pose la question de son intégration dans le transformer. Trois topologies principales s'offrent à l'ingénieur.

1. L'intercalage séquentiel (Standard Decoder).

C'est l'approche historique. Dans chaque bloc du transformer, on enchaîne : Auto-Attention → Cross-Attention → FFN (Feed-Forward Network).

Avantage : la cross-attention bénéficie des représentations visuelles déjà enrichies par l'auto-attention de la même couche.

Inconvénient : la profondeur du réseau augmente, ce qui ralentit l'inférence séquentielle.

2. L'attention parallèle.

Popularisée par des architectures récentes optimisées pour le matériel, cette approche calcule l'auto-attention et la cross-attention en parallèle, puis somme leurs sorties avant de passer à la FFN.

$$X_{\text{out}} = \text{AutoAttn}(X) + \text{CrossAttn}(X, X_{\text{text}})$$

Avantage : réduit la latence mémoire. Les lectures de la matrice X pour l'auto-attention et la cross-attention peuvent être fusionnées, optimisant l'utilisation des SRAM des GPU.

Inconvénient : la cross-attention reçoit des représentations visuelles « brutes », non filtrées par l'auto-attention de la couche courante.

3. L'injection profonde vs superficielle.

Faut-il mettre une cross-attention dans *chaque* couche du LLM ?

Des expérimentations montrent que l'injection dans toutes les couches offre le meilleur contrôle fin. Les couches basses utilisent la cross-attention pour des détails locaux (couleurs, textures), tandis que les couches hautes l'utilisent pour la composition globale.

Toutefois, pour réduire le coût, une injection toutes les deux ou trois couches suffit souvent à capturer la sémantique globale, au prix d'une légère perte de fidélité locale.

Dans le protocole que nous déploierons au chapitre 22, j'opterai pour l'attention parallèle avec une injection dans toutes les couches. Le gain

© 2026 Oussama Ghorbel

en vitesse d'inférence sur des puces modernes compense largement la légère perte de représentativité.

15.4 — Conditionnement dense vs conditionnement sparse

D'où proviennent les états cachés textuels X_{text} qui alimentent les clés et valeurs ? C'est ici que se joue une divergence majeure dans la littérature.

Le conditionnement sparse (via le LLM lui-même).

L'approche la plus économe consiste à utiliser les états cachés produits par les couches de self-attention du LLM sur les tokens textuels.

Avantage : aucun paramètre supplémentaire pour l'encodeur textuel. Le modèle utilise sa propre compréhension du langage.

Inconvénient : les LLM sont optimisés pour la prédiction du token suivant (causalité stricte). Le token « bleu » à la fin d'une phrase n'a pas vu le token « chat » au début de la phrase lors de son encoding causal. La représentation textuelle est donc asymétrique et moins riche sémantiquement.

Le conditionnement dense (via un encodeur externe).

C'est l'approche de modèles de diffusion populaires. On utilise un encodeur de texte indépendant, souvent bidirectionnel (comme T5 ou le text-encoder de CLIP).

Dans ce schéma, le texte passe d'abord dans l'encodeur, qui produit une représentation dense où chaque token a une vue complète de la phrase. Ces représentations sont ensuite projetées dans l'espace du LLM, puis utilisées comme K et V dans les couches de cross-attention.

Avantage : la sémantique textuelle est extrêmement riche et contextuellement complète. L'alignement texte-image est bien supérieur.

Inconvénient : coût computationnel et mémoire supplémentaires pour l'encodeur externe.

Pour notre architecture unifiée, je recommande une approche hybride. Si vous partez de zéro, utilisez un encodeur externe pour le conditionnement dense. Mais si vous souhaitez adapter un LLM existant sans exploser le budget mémoire, vous pouvez utiliser ses propres états

© 2026 Oussama Ghorbel

cachés, à condition d'ajouter une petite couche de projection non-linéaire (un adaptateur) entre les états textuels et les clés/valeurs de la cross-attention. Cette projection permet de « lisser » l'asymétrie causale et d'extraire une représentation plus dense.

15.5 — Gestion du prompt textuel et KV Cache de cross-attention

L'un des avantages cachés de la cross-attention est la facilité avec laquelle elle s'intègre dans les mécanismes de cache pour l'inférence.

Lors de la génération d'une image, le prompt textuel est fixe. Il ne change pas au fur et à mesure que vous générez les tokens visuels $v_1, v_2, \ldots, v_{1024}$.

Par conséquent, les projections K_{text} et V_{text} sont constantes.

Au lieu de les recalculer à chaque pas de génération, vous les calculez une seule fois au début, et vous les stockez dans un **cache de cross-attention dédié**.

Lors du pas i, le modèle ne lit que le cache de self-attention (qui grandit à chaque pas pour les tokens visuels) et le cache de cross-attention (qui reste statique).

Cela a deux implications majeures :

1. **Vitesse d'inférence** : le coût de la cross-attention est amorti. Le goulot d'étranglement redevient la self-attention visuelle et la lecture des poids du modèle.
2. **Mémoire prévisible** : le cache de cross-attention occupe une mémoire fixe, proportionnelle à la longueur du prompt textuel. Pour un prompt de 50 tokens et une dimension de 4096, cela ne représente que quelques centaines de kilo-octets, même pour un batch de taille importante.

Une extension puissante de ce mécanisme est le **Prompt Tuning**. Au lieu d'utiliser uniquement des tokens textuels discrets, vous pouvez préfixer le prompt avec P tokens continus apprenables (des vecteurs flottants). Les K et V de ces tokens continus sont également calculés une fois et mis en cache. Cela permet d'injecter des concepts abstraits (comme un « style artistique » ou une « disposition spatiale ») sans

© 2026 Oussama Ghorbel

alourdir le prompt textuel, et sans coût d'inférence supplémentaire.

15.6 — Étude de cas : adapter un LLaMA 7B avec des couches de cross-attention

Concrétisons ces concepts. Supposons que vous disposiez d'un LLaMA 7B pré-entraîné et que vous souhaitiez en faire un générateur d'images, en gelant les poids du LLM pour éviter le *catastrophic forgetting* et réduire le coût d'entraînement.

Voici le protocole d'adaptation :

1. **Extraction des poids textuels** : Le LLM traite le texte. Les états cachés de la dernière couche pour les tokens textuels servent de base à X_{text}.

2. **Création des modules de cross-attention** : Pour chaque couche du transformer, vous instanciez un module `CrossAttention`. Ce module contient ses propres matrices W_Q, W_K, W_V et W_O (projection de sortie).

3. **Initialisation des poids** : C'est l'étape critique. Initialiser aléatoirement les poids de la cross-attention peut déstabiliser le réseau au début. Une astuce éprouvée consiste à initialiser les poids W_Q, W_K, W_V de la cross-attention avec les poids correspondants de la self-attention de la même couche, et d'initialiser la projection de sortie W_O à zéro.

Pourquoi W_O à zéro ? Parce qu'au tout début de l'entraînement, la sortie de la cross-attention sera nulle. Le modèle se comportera donc exactement comme le LLaMA de base. Au fur et à mesure que l'entraînement progresse, les gradients feront émerger les poids de W_O, injectant progressivement l'information textuelle sans choc initial.

4. **Entraînement** : Vous gelez l'intégralité des paramètres de LLaMA. Vous ne rendez entraînables que les paramètres des modules de cross-attention et la matrice d'embedding visuel (ajoutée au chapitre 14).

5. **Données** : Vous entraînez sur des paires (texte, tokens d'image). La perte est la cross-entropie standard sur les tokens visuels.

© 2026 Oussama Ghorbel

Ce protocole permet d'obtenir un modèle capable de générer des images cohérentes avec le texte en un temps raisonnable sur un cluster de GPU modernes. Le LLM conserve ses capacités linguistiques intactes, tandis que les couches de cross-attention apprennent rapidement à traduire la sémantique textuelle en géométrie visuelle.

Synthèse du chapitre

La cross-attention n'est pas une simple optimisation ; c'est une nécessité architecturale pour la génération d'image par autoregression. Elle résout le problème fondamental de la dilution du signal textuel dans les longs contextes visuels. En séparant le flux de l'information spatiale (auto-attention) et le flux de l'information sémantique (cross-attention), nous offrons au modèle les deux canaux dont il a besoin pour produire des images à la fois structurellement cohérentes et fidèles au prompt.

Nous avons vu comment intégrer ces couches, comment gérer le conditionnement, et comment optimiser l'inférence grâce au cache. Notre LLM n'est plus un simple générateur de texte qui essaie de faire semblant de comprendre les images ; c'est devenu un véritable moteur multimodal, où le texte et le pixel dialoguent en permanence à travers des ponts d'attention dédiés.

Exercices

Exercice 15.1 — Implémentation d'un module de Cross-Attention.

Écrivez une classe PyTorch `CrossAttention(nn.Module)` qui prend en entrée les états cachés visuels X_{vis} (shape: `[batch, seq_vis, d_model]`) et les états cachés textuels X_{text} (shape: `[batch, seq_text, d_model]`).

Le module doit contenir les projections linéaires W_Q, W_K, W_V et W_O. Implémentez le calcul de l'attention avec le facteur d'échelle $\frac{1}{\sqrt{d_k}}$. Ajoutez un argument optionnel `cross_kv_cache` pour permettre de passer des clés et valeurs pré-calculées lors de l'inférence.

© 2026 Oussama Ghorbel

Exercice 15.2 — Calcul du surcoût computationnel et mémoire.

Considérez un LLaMA 7B (dimension $d = 4096$, 32 couches). Vous ajoutez une couche de cross-attention après chaque couche de self-attention.

1. Calculez le nombre de paramètres ajoutés par couche de cross-attention (en supposant des projections Q, K, V, O de taille $d \times d$).
2. Calculez le nombre total de paramètres ajoutés pour les 32 couches.
3. Calculez le FLOPs ajoutés pour un pas de génération d'un token visuel, sachant que la séquence textuelle a une longueur de 50 tokens. (Rappel : le produit matriciel QK^T coûte $2 \cdot \text{seq_vis} \cdot \text{seq_text} \cdot d$ FLOPs).
4. Comparez ce surcoût au FLOPs d'une couche de self-attention pour une séquence visuelle de 1024 tokens.

Exercice 15.3 — Conception du cache de cross-attention.

Écrivez le pseudo-code de la boucle d'inférence autoregressive pour la génération d'une image.

La boucle doit gérer deux caches distincts :

• `self_kv_cache` : qui grandit à chaque pas i (contient les K, V des tokens visuels $v_1 \dots v_i$).

• `cross_kv_cache` : qui est calculé une seule fois avant la boucle (contient les K, V des tokens textuels).

Montrez comment les tenseurs sont passés au modèle à l'itération i.

Corrigés et pistes de résolution

Solution de l'exercice 15.1 : Module de Cross-Attention

Voici une implémentation robuste, conçue pour être compatible avec les mécanismes de cache standards.

Solution de l'exercice 15.2 : Surcoût computationnel

© 2026 Oussama Ghorbel

1. **Paramètres par couche** : 4 matrices de 4096×4096.

$4 \times 4096^2 = 67\,108\,864$ paramètres.

2. **Total pour 32 couches** : $32 \times 67\,108\,864 \approx 2{,}14$ milliards de paramètres.

Note : Ajouter 2B de paramètres à un modèle de 7B augmente sa taille de près de 30 %. C'est significatif. C'est pourquoi l'initialisation à zéro de W_O et le gel du LLM de base sont cruciaux pour la stabilité.

3. **FLOPs de la cross-attention** :

Pour QK^T : $2 \times 1 \times 50 \times 4096 = 409\,600$ FLOPs (par tête, donc $\times 32$ têtes ≈ 13 MFLOPs).

Pour scores $\times V$: $2 \times 1 \times 50 \times 4096 \approx 13$ MFLOPs.

Total par couche ≈ 26 MFLOPs. Pour 32 couches ≈ 832 MFLOPs par token généré.

4. **Comparaison** : La self-attention pour 1024 tokens coûte $2 \times 1024^2 \times 4096 \approx 8{,}5$ GFLOPs par couche.

La cross-attention (832 MFLOPs total) est donc **dix fois moins coûteuse** que la self-attention visuelle (8,5 GFLOPs $\times$ 32 couches = 272 GFLOPs).

Conclusion : L'ajout de la cross-attention est extrêmement efficace. Elle apporte un conditionnement riche pour un coût computationnel marginal par rapport à la self-attention visuelle.

Solution de l'exercice 15.3 : Boucle d'inférence avec caches

Ce pseudo-code illustre la beauté de l'architecture : le texte n'est encodé qu'une fois. Le coût de la génération est dominé par la self-attention visuelle et la lecture des poids, rendant l'inférence rapide et prévisible.

© 2026 Oussama Ghorbel

Chapitre 15 – Cross-attention et conditionnement textuel : guider la génération

Dans le chapitre précédent, nous avons vu comment unifier les vocabulaires textuels et visuels au sein d'un même espace de tokens. Cette unification résout le problème de la représentation, mais elle laisse entière une question opérationnelle : comment le modèle sait-il *quelle* image produire à partir d'une description ? La réponse tient en un mot — conditionnement. Dans ce chapitre, j'explore les mécanismes par lesquels le signal textuel pénètre le processus de génération visuelle, depuis l'attention croisée classique jusqu'aux injections par normalisation adaptative, en passant par la guidance sans classifieur adaptée au cadre autoregressif. L'objectif est double : comprendre les choix architecturaux et disposer de recettes concrètes pour les reproduire.

15.1 — Cross-attention texte -> image : architecture, nombre de têtes, position

Le mécanisme de cross-attention, popularisé par le Transformer original pour la traduction automatique, transpose ici son rôle : les *queries* proviennent des tokens visuels en cours de génération, tandis que les *keys* et *values* proviennent de l'encodage du prompt textuel. Formellement, pour une couche l donnée :

$$Q = X^{(l)}_{\text{img}} W_Q, \quad K = H_{\text{txt}} W_K, \quad V = H_{\text{txt}} W_V$$

où $X^{(l)}_{\text{img}} \in \mathbb{R}^{n_{\text{img}} \times d}$ désigne les représentations cachées des tokens image et $H_{\text{txt}} \in \mathbb{R}^{n_{\text{txt}} \times d}$ celles du texte encodé. La sortie d'attention $\text{softmax}(QK^\top / \sqrt{d_h})V$ est ensuite projetée et ajoutée par connexion résiduelle.

Position dans le bloc Transformer. Trois emplacements sont courants : (i) *avant* la self-attention, (ii) *entre* la self-attention et le FFN, (iii) *après* le FFN. Dans la pratique, l'insertion entre self-attention et FFN — celle retenue par Stable Diffusion et la plupart des décodeurs

© 2026 Oussama Ghorbel

récents — offre un bon compromis : la self-attention a déjà agrégé le contexte visuel local, et la cross-attention peut alors moduler ces représentations avant la transformation non linéaire du FFN.

Nombre de têtes. Il a été observé, lors d'expérimentations sur des décodeurs de 256 à 512 tokens, que 8 têtes de cross-attention constituent un point de départ raisonnable pour un modèle de dimension $d = 768$. Passer à 16 têtes améliore marginalement le CLIP-score (de l'ordre de 0,3 à 0,5 point) mais augmente le coût mémoire de manière non négligeable à l'inférence, car chaque tête doit maintenir son propre cache de clés/valeurs textuelles. Pour un budget computationnel serré, 4 têtes avec une dimension par tête $d_h = 192$ restent exploitables.

Fréquence d'insertion. Insérer une cross-attention à chaque couche double quasiment le nombre de paramètres. Une alternative économique consiste à n'en placer que toutes les deux ou trois couches. Le gain en FID est généralement inférieur à 1 point pour un modèle 7B, ce qui rend l'insertion systématique difficile à justifier sauf si la qualité textuelle est critique.

15.2 — Conditionnement par préfixe (prefix-tuning) : tokens textuels en t te de séquence

Une approche plus légère que la cross-attention explicite consiste à concaténer les tokens textuels *devant* les tokens visuels dans la séquence d'entrée du décodeur autoregressif. Le modèle traite alors la séquence $[\text{BOS}, t_1, \ldots, t_m, \text{SEP}, v_1, \ldots, v_n]$ avec une attention causale standard : chaque token visuel v_i peut assister sur l'ensemble des tokens textuels précédents sans qu'aucun module dédié ne soit nécessaire.

Cette stratégie, que j'appelle *préfixe naturel*, présente deux avantages majeurs. Premièrement, elle ne modifie pas l'architecture : un LLM pré-entraîné peut être réutilisé tel quel. Deuxièmement, le conditionnement bénéficie de la profondeur du réseau à chaque couche, puisque l'information textuelle se propage à travers toutes les self-attentions.

Le revers de la médaille est la longueur de séquence. Un prompt de 77 tokens ajouté à 256 tokens visuels porte la séquence à 333 positions, ce qui augmente quadratiquement le coût de la self-attention. Pour

© 2026 Oussama Ghorbel

atténuer ce surcoût, on peut recourir à un *prefix-tuning appris* : au lieu d'insérer les tokens textuels bruts, on initialise m vecteurs continus $\{p_1, \dots, p_m\}$ entraînés à résumer le prompt. Ces vecteurs, projetés dans l'espace des clés et valeurs à chaque couche, réduisent la longueur effective du préfixe à une dizaine de positions tout en conservant une expressivité satisfaisante.

En pratique, je recommande de commencer par le préfixe naturel pour valider le pipeline, puis de basculer vers un préfixe appris si la latence d'inférence devient un goulot d'étranglement.

15.3 — CLIP comme encodeur de prompt : extraire l'embedding de condition

Dans la quasi-totalité des systèmes texte-image contemporains, l'encodeur textuel n'est pas entraîné conjointement avec le générateur. On lui préfère un modèle pré-entraîné dont l'espace de représentation est déjà aligné avec celui des images : CLIP, dans sa variante ViT-L/14, demeure le choix par défaut.

L'extraction de l'embedding de condition suit un protocole simple :

1. **Tokenisation du prompt.** Le texte est tokenisé (BPE, 77 tokens maximum pour CLIP standard), complété par un token [EOS] et un padding.

2. **Encodage.** Le passage à travers l'encodeur CLIP produit une séquence de 77 vecteurs de dimension 768. On peut utiliser soit le token [EOS] seul (embedding global), soit l'ensemble de la séquence (embedding token-par-token). Pour la cross-attention, la séquence complète est indispensable : elle permet au décodeur d'aligner chaque région visuelle sur le mot pertinent.

3. **Projection.** Si la dimension du décodeur diffère de celle de CLIP (par exemple $d_{\text{décodeur}} = 1024$), une projection linéaire $W_{\text{proj}} \in \mathbb{R}^{768 \times 1024}$ est appliquée. Cette projection est le seul paramètre entraîné côté encodeur ; les poids CLIP restent gelés.

Un point qu'il est important de souligner : la normalisation. CLIP produit des embeddings normalisés en ℓ_2 à la sortie. Si l'on injecte ces vecteurs directement dans une cross-attention sans renormalisation, l'échelle peut être inadaptée aux distributions internes du décodeur. Une

© 2026 Oussama Ghorbel

LayerNorm sur l'embedding projeté avant injection stabilise notablement l'entraînement.

15.4 — Classifier-free guidance adapté à l'autoregression : drop du condition à l'entraînement

La classifier-free guidance (CFG), introduite par Ho et Salimans en 2022, a révolutionné la génération conditionnelle en diffusion. Son adaptation à l'autoregression par tokens est directe mais mérite quelques précautions.

Principe. À l'entraînement, avec une probabilité p_{drop} (typiquement 10 %), le conditionnement textuel est remplacé par un embedding nul (vecteur de zéros ou token [NULL] appris). Le modèle apprend ainsi simultanément la distribution conditionnelle $p(v \mid c)$ et la distribution marginale $p(v)$.

À l'inférence, la prédiction guidée s'écrit :

$$\hat{\varepsilon}_{\text{guidé}} = (1 + w)\,\hat{\varepsilon}(v \mid c) - w\,\hat{\varepsilon}(v \mid \varnothing)$$

où w est le facteur de guidance. Dans le cadre autoregressif, $\hat{\varepsilon}$ est remplacé par le logit du prochain token. Concrètement, pour chaque pas de génération, on effectue deux passes avant — une conditionnée, une inconditionnelle — et l'on interpole les logits avant l'échantillonnage.

Coût. Cette double passe double le temps d'inférence. Une astuce consiste à concaténer le batch conditionné et inconditionné en un seul batch de taille $2B$, ce qui permet une unique passe GPU avec un surcoût mémoire plutôt que temporel.

Choix de w. Les expérimentations sur un décodeur 256 × 256 montrent que $w = 3$ offre un bon équilibre entre fidélité au prompt et diversité. Au-delà de $w = 7$, les images deviennent saturées et répétitives ; en deçà de $w = 1$, le conditionnement est trop faible. Je recommande un balayage sur $\{1, 3, 5, 7\}$ lors du développement.

Spécificité autoregressive. Contrairement à la diffusion où la guidance s'applique à un bruit continu, ici elle modifie une distribution discrète. Des valeurs de w trop élevées peuvent produire des logits négatifs extrêmes qui, après softmax, écrasent la distribution. Un écrêtage des logits (clipping à ± 20) avant le softmax est une précaution

© 2026 Oussama Ghorbel

qu'il est recommandé d'appliquer systématiquement.

15.5 — Injection du signal dans les couches FFN vs attention : AdaLN, FiLM

La cross-attention n'est pas l'unique vecteur d'injection du conditionnement. Deux familles alternatives méritent attention.

FiLM (Feature-wise Linear Modulation). Le principe est simple : l'embedding textuel c est projeté en deux vecteurs $\gamma(c)$ et $\beta(c)$ qui modulent affine-ment les activations intermédiaires :

$$h' = \gamma(c) \odot h + \beta(c)$$

Appliqué après la LayerNorm précédant le FFN, FiLM permet au texte de contrôler l'amplitude et le décalage des caractéristiques visuelles sans modifier la structure d'attention. L'avantage est un coût computationnel quasi nul : deux projections linéaires par couche. L'inconvénient est une expressivité limitée — FiLM ne peut pas réaliser un alignement spatial fin entre mots et régions.

AdaLN (Adaptive Layer Normalization). Popularisé par DiT (Diffusion Transformer), AdaLN généralise FiLM en conditionnant les paramètres de la LayerNorm elle-même. Les statistiques de normalisation (moyenne et variance) sont recalculées, puis la mise à l'échelle et le décalage sont produits par le conditionnement :

$$\text{AdaLN}(h, c) = \gamma(c) \odot \frac{h - \mu}{\sigma + \varepsilon} + \beta(c)$$

Dans un décodeur autoregressif, AdaLN s'insère naturellement avant chaque sous-bloc (attention et FFN). Il a été constaté qu'AdaLN-Zero — où γ est initialisé à zéro de sorte que la modulation démarre comme l'identité — stabilise considérablement les premières étapes d'entraînement.

Où injecter quoi ? Un retour d'expérience fréquent : la cross-attention dans les couches basses et moyennes (alignement sémantique local), AdaLN ou FiLM dans les couches hautes (contrôle global du style et de la cohérence). Cette hybridation donne des résultats supérieurs à l'usage exclusif d'un seul mécanisme, pour un surcoût paramétrique inférieur à 5 %.

© 2026 Oussama Ghorbel

15.6 — Multi-prompt et conditionnement composé : style + contenu + layout

Les applications réelles exigent rarement un unique prompt. Un designer souhaite spécifier séparément le contenu (« un chat sur un canapé »), le style (« aquarelle japonaise ») et le layout (« le chat occupant le tiers gauche »). Comment composer ces signaux ?

Concaténation séquentielle. La solution la plus simple consiste à concaténer les sous-prompts en une unique séquence textuelle, séparés par des tokens spéciaux : `[CONTENU] un chat sur un canapé [STYLE] aquarelle japonaise [LAYOUT] tiers gauche`. Le modèle apprend à segmenter implicitement ces sections. Cette approche fonctionne étonnamment bien si les données d'entraînement contiennent des annotations structurées similaires.

Somme pondérée d'embeddings. Chaque sous-prompt est encodé indépendamment par CLIP, puis les embeddings sont combinés :

$$c_{\text{final}} = \alpha\, c_{\text{contenu}} + \beta\, c_{\text{style}} + \gamma\, c_{\text{layout}}$$

Les poids α, β, γ peuvent être fixes ou appris. Cette méthode est rapide mais perd l'information séquentielle : le modèle ne sait plus *où* dans la phrase se situe le mot « chat ».

Cross-attention multi-source. Une approche plus puissante consiste à dédier une tête d'attention par type de condition. Par exemple, les têtes 1 à 4 assistent sur l'embedding de contenu, les têtes 5 à 6 sur le style, les têtes 7 à 8 sur le layout. Cette spécialisation explicite améliore la contrôlabilité au prix d'une complexité d'implémentation accrue.

Conditionnement spatial. Pour le layout, l'embedding textuel seul est insuffisant : il faut une carte spatiale. On peut encoder une carte de segmentation ou des bounding boxes en une grille de tokens (via un petit CNN ou un encodeur de position) et les injecter en cross-attention alongside les tokens textuels. Cette hybridation texte + géométrie est au cœur des systèmes de génération contrôlée les plus récents.

En pratique, je conseille de commencer par la concaténation séquentielle pour établir une baseline, puis d'introduire la cross-attention multi-source uniquement si les évaluations montrent une confusion entre style et contenu.

© 2026 Oussama Ghorbel

Exercices

Exercice 15.1 — Implémenter un bloc de cross-attention (4 têtes) entre un encodeur CLIP gelé et un décodeur GPT.

Consignes :

• Chargez un encodeur CLIP ViT-B/32 (poids gelés) et un décodeur GPT-2 small (12 couches, $d = 768$).

• Ajoutez, après la self-attention de chaque couche impaire du décodeur (couches 1, 3, 5, 7, 9, 11), un bloc de cross-attention à 4 têtes. Les queries proviennent des hidden states du décodeur ; les keys et values proviennent de la sortie séquentielle de CLIP (77×768), projetée par une LayerNorm.

• Implémentez le masque causal sur les queries (chaque token visuel ne peut assister que sur les tokens textuels, pas sur les tokens visuels futurs via la cross-attention).

• Vérifiez la forme des tenseurs : pour un batch de 8 séquences de 256 tokens visuels, la sortie de cross-attention doit être de forme $(8, 256, 768)$.

• Mesurez le surcoût paramétrique par rapport au GPT-2 original et le temps de forward-pass supplémentaire.

Exercice 15.2 — Entraîner avec dropout du condition (10 %) et tester la guidance à l'inférence avec $w \in \{1, 3, 7\}$.

Consignes :

• Reprenez le modèle de l'exercice 15.1. À chaque pas d'entraînement, avec probabilité 0,1, remplacez l'embedding CLIP par un vecteur nul de même dimension.

• Entraînez sur un sous-ensemble de 10 000 paires image-texte (par exemple CC3M filtré) pendant 5 000 étapes, learning rate 10^{-4}, batch size 32.

• À l'inférence, implémentez la guidance sans classifieur : pour chaque pas de décodage, calculez les logits conditionnés et inconditionnés, puis appliquez $\text{logits}_{\text{guidés}} = (1 + w) \cdot \text{logits}_{\text{cond}} - w \cdot \text{logits}_{\text{uncond}}$.

© 2026 Oussama Ghorbel

• Générez 25 images pour chaque valeur de $w \in \{1, 3, 7\}$ à partir du même prompt (« a red bicycle leaning against a white wall, watercolor style »).

• Comparez visuellement : à quel niveau de w la fidélité au prompt est-elle maximale sans saturation des couleurs ?

Exercice 15.3 — Mesurer le CLIP-score de 100 images générées avec et sans guidance.

Consignes :

• Sélectionnez 10 prompts variés (objets, scènes, styles abstraits).

• Pour chaque prompt, générez 10 images avec $w = 0$ (pas de guidance) et 10 images avec $w = 5$.

• Calculez le CLIP-score moyen : $\text{CLIP-score} = 100 \times \cos(c_{\text{texte}}, c_{\text{image}})$, où c_{texte} et c_{image} sont les embeddings globaux de CLIP ViT-L/14.

• Reportez les résultats dans un tableau (moyenne, écart-type, minimum, maximum) et commentez : la guidance améliore-t-elle systématiquement le score ? Observez-vous des cas où $w = 5$ dégrade le score par rapport à $w = 0$?

• Bonus : tracez la distribution des scores sous forme d'histogramme pour visualiser la variance introduite par la guidance.

Ce chapitre clôt la question du *comment* conditionner. Le chapitre suivant abordera le *quand* et le *combien de temps* : les protocoles d'entraînement, la construction des curricula de données et les stratégies de stabilisation qui transforment une architecture correcte en un modèle effectivement génératif.

Points clés à retenir

Avant de refermer ce chapitre, je souhaite cristalliser les décisions qui, en pratique, font basculer un prototype de génération conditionnelle d'un état « vaguement cohérent » à un état « réellement utilisable ».

Premièrement, le choix du mécanisme d'injection n'est jamais binaire. La cross-attention apporte l'alignement spatial fin entre mots et régions ; AdaLN et FiLM apportent un contrôle global à coût quasi nul.

© 2026 Oussama Ghorbel

Les combiner — cross-attention dans les couches basses et moyennes, modulation par normalisation dans les couches hautes — constitue la recette la plus robuste testée sur des décodeurs de 256 à 1024 tokens.

Deuxièmement, le dropout du conditionnement à l'entraînement n'est pas optionnel dès lors que l'on souhaite utiliser la guidance sans classifieur à l'inférence. Un taux de 10 % est suffisant ; au-delà de 15 %, la branche inconditionnelle domine et le modèle sous-exploite le signal textuel. En deçà de 5 %, la distribution inconditionnelle est trop peu apprise et la guidance produit des artefacts.

Troisièmement, la normalisation de l'embedding CLIP avant injection est une étape systématiquement omise dans les implémentations de première itération. Sans LayerNorm sur le vecteur projeté, les premières centaines de pas d'entraînement sont instables, avec des gradients de cross-attention trois à cinq fois supérieurs à ceux de la self-attention. Le symptôme est reconnaissable : la loss totale stagne tandis que la composante cross-attention explose.

Notes pratiques de débogage

Quelques observations pratiques, qui n'ont pas trouvé leur place dans les sections précédentes mais qui, je l'espère, feront gagner du temps au lecteur.

Gradient de cross-attention trop faible. Si, après 2 000 pas, les poids de projection W_K et W_V de la cross-attention n'ont quasiment pas bougé (norme de gradient inférieure à 10^{-6}), vérifiez que l'embedding CLIP n'est pas accidentellement détaché du graphe computationnel. Une erreur fréquente consiste à appeler `.detach()` sur la sortie de CLIP par précaution, puis à oublier que la projection linéaire W_{proj} a elle aussi besoin de gradients pour s'adapter. La solution : geler les poids internes de CLIP mais laisser la projection entraînable.

Saturation des logits avec w élevé. Lorsque $w \geq 7$, les logits guidés peuvent dépasser ± 50, ce qui produit un softmax quasi déterministe et des images identiques quel que soit le seed. L'écrêtage à ± 20 mentionné en section 15.4 est un filet de sécurité ; une approche plus fine consiste à appliquer la guidance dans l'espace des log-probabilités plutôt que des logits bruts, ce qui réduit la variance introduite par l'interpolation.

© 2026 Oussama Ghorbel

Mémoire à l'inférence avec double passe. La concaténation des batches conditionné et inconditionné est efficace, mais elle double la mémoire vive. Pour un décodeur 7B en float16, comptez environ 14 Go supplémentaires pour le cache KV de la passe inconditionnelle sur une séquence de 256 tokens. Sur un GPU de 24 Go, cela peut être rédhibitoire. L'alternative : exécuter les deux passes séquentiellement en réutilisant le cache KV de la passe conditionnée pour les positions textuelles (identiques dans les deux cas), ce qui économise environ 30 % de mémoire au prix d'une latence accrue de 15 %.

Choix du token [NULL]. Pour la passe inconditionnelle, trois options ont été comparées : un vecteur de zéros, un token [NULL] appris, et la moyenne des embeddings CLIP sur le jeu d'entraînement. Le token appris donne les meilleurs résultats (CLIP-score supérieur de 0,4 point en moyenne), mais le vecteur de zéros est à moins de 0,2 point et simplifie l'implémentation. La moyenne des embeddings est à éviter : elle introduit un biais vers le « style moyen » du corpus.

Pour aller plus loin

Le conditionnement textuel n'est qu'une modalité parmi d'autres. Les travaux récents sur le conditionnement par sketch (ControlNet adapté à l'autoregression), par carte de profondeur ou par image de référence ouvrent la voie à une génération multi-conditionnelle où le texte ne serait qu'un signal parmi plusieurs. L'architecture décrite dans ce chapitre — cross-attention multi-source, AdaLN, préfixe appris — se généralise naturellement à ces modalités supplémentaires : il suffit d'ajouter un encodeur dédié par type de condition et de dédier des têtes d'attention ou des canaux de modulation à chacune.

Une direction qui m'intéresse particulièrement, et qui est actuellement explorée dans la recherche, est le conditionnement *interne* : plutôt que d'injecter le texte depuis l'extérieur, laisser le modèle générer lui-même une « description intermédiaire » en tokens textuels avant de produire les tokens visuels. Cette auto-conditionnalité réduit la dépendance à un encodeur externe et permet au modèle d'ajuster sa propre stratégie de génération en fonction de la complexité de la scène. Les premiers résultats sur des décodeurs 512 × 512 sont prometteurs, mais la stabilité d'entraînement reste un défi ouvert.

© 2026 Oussama Ghorbel

Enfin, le lecteur attentif aura noté que toutes les techniques présentées ici supposent un conditionnement *fixe* pour l'intégralité de la génération. Or, dans une séquence autoregressive de 256 ou 1024 tokens, la pertinence du prompt varie : les premiers tokens (coins supérieurs de l'image en raster scan) n'ont pas nécessairement besoin du même alignement que les tokens centraux. Un conditionnement *dynamique*, dont l'intensité de guidance varierait en fonction de la position dans la séquence, est une piste que je considère comme l'une des plus fertiles pour la prochaine génération de modèles autoregressifs texte-image.

© 2026 Oussama Ghorbel

Chapitre 16 – Protocoles d'entraînement : données, curriculum, stabilité

Adapter un modèle de langage à la génération d'images est un exercice d'ingénierie qui ne se joue pas uniquement au niveau de l'architecture. La qualité du protocole d'entraînement — choix des donn es, ordre d'apprentissage, gestion de la stabilité numérique, planification du budget computationnel — détermine en grande partie le succès du projet. Un LLM 7B capable d'écrire de la poésie peut devenir un générateur d'images médiocre si on le fine-tune à la hâte sur un jeu de données bruyant, avec un learning rate mal calibré et aucune stratégie de curriculum. Inversement, un protocole soigneusement conçu permet d'obtenir des résultats surprenants avec des ressources raisonnables.

Ce chapitre propose un protocole complet, de la constitution du jeu de données jusqu'à la détection de divergence pendant l'entraînement. L'objectif n'est pas de donner des valeurs magiques, mais de fournir des principes, des ordres de grandeur et des outils de diagnostic que vous pourrez adapter à votre contexte.

16.1 — Jeux de données : LAION, CC12M, DataComp, COCO

Critères de sélection

Avant de parler de noms de jeux de données, il faut clarifier ce qui rend un jeu de données adapté à l'entraînement d'un générateur texte-image. Quatre critères principaux entrent en jeu :

• **La taille** : le nombre de paires (texte, image) disponible détermine la diversité sémantique du modèle. Un jeu de quelques milliers d'images suffit pour du fine-tuning, mais pas pour l'apprentissage de la correspondance texte-image à grande échelle.

• **La qualité des légendes** : un LLM adaptée à la génération d'images apprend la correspondance entre descriptions textuelles et pixels. Si les légendes sont des mots-clés bruts (« chat, rouge,

© 2026 Oussama Ghorbel

mignon ») plutôt que des phrases descriptives, le modèle peindra à générer des scènes complexes à partir de prompts riches.

• **La licence** : la plupart des grands jeux de données sont construits par scraping web. Les implications juridiques sont réelles et varient selon les juridictions. La licence de chaque dataset doit être vérifiée avant tout usage commercial.

• **La diversité** : un dataset dominé par des selfies ou des photos de nourriture produira un modèle biaisé. La couverture des catégories, des styles artistiques, des langues et des compositions est essentielle.

Principaux jeux de données

LAION-5B est le plus grand dataset d'images avec légendes disponible publiquement. Il contient environ 5,85 milliards de paires (texte, URL d'image), dont une sous-ensemble filtré — LAION-2B-en — d'environ 2,32 milliards d'entrées en anglais. Les légendes proviennent du texte alternatif (`alt text`) des balises HTML, ce qui leur confère une qualité variable : certaines sont descriptions précises, d'autres sont de simples mots-clés ou des textes de navigation. LAION fournit également des scores de qualité (par exemple, le score CLIP entre image et texte) qui permettent de filtrer les paires les moins cohérentes. C'est le dataset qui a servi à entraîner Stable Diffusion et de nombreux modèles dérivés.

CC12M (Conceptual Captions 12M) est une sous-ensemble plus petit — environ 12 millions de paires — issu d'un pipeline de nettoyage plus agressif que son prédécesseur CC3M. Les légendes ont été filtrées pour éliminer les mots-clés de navigation, les textes trop courts et les doublons. La qualité moyenne est supérieure à celle de LAION brut, mais la taille est deux cents fois moindre. CC12M est un excellent compromis pour la recherche académique ou pour des expériences à budget limité.

DataComp se distingue par sa philosophie : plutôt qu'un dataset figé, c'est un benchmark et un framework pour évaluer et améliorer les pipelines de filtering de données. Le principe est simple : on part d'un pool d'images communes (par exemple, un sous-ensemble de LAION) et on compare différentes stratégies de sélection et de filtering. Les

© 2026 Oussama Ghorbel

expérimentateurs peuvent ainsi mesurer l'impact de chaque décision de filtering sur la qualité du modèle final. DataComp fournit des versions de différents tailles (small, medium, large) et a produit certains des meilleurs modèles CLIP open-source.

COCO (Common Objects in Context) est d'une toute autre nature : un dataset curaté manuellement, avec environ 330 000 images, chacune avec 5 légendes écrites par des annotateurs humains. La qualité des descriptions est excellente — elles sont grammaticalement correctes, détaillées et contextuellement riches. En revanche, la taille est modeste et la diversité sémantique limitée. COCO est idéal pour l'évaluation, le fine-tuning de dernière étape, et la démonstration de capacités de raisonnement visuel, mais insuffisant comme unique source d'entraînement.

Stratégie de combinaison

Dans la pratique, les meilleurs modèles sont entraînés sur des datasets composites. Une stratégie courante consiste à combiner un dataset large et bruité (LAION filtré) pour la phase principale d'entraînement, puis à effectuer un fine-tuning de convergence sur un dataset plus petit et de meilleure qualité (COCO, ou un sous-ensemble LAION filtré avec un seuil de score CLIP élevé). Ce dernier affinage permet de caler le modèle sur des légendes bien formées et d'améliorer la fidélité des descriptions complexes.

Il faut aussi anticiper les problèmes de duplication. Les datasets web contiennent souvent des versions multiples de la même image (mêmes photos réutilisées sur différents sites). Des techniques de déduplication perceptuelle — basées sur des embeddings CLIP ou des hachages perceptuels comme pHash — permettent de réduire ce biais, qui autrement favoriserait la sur-ajustement sur des images sur-représentées.

16.2 — Pré-entraînement du tokenizer (VQ-VAE/VQGAN) : objectifs, durée, données

Avant même de toucher au LLM, il faut entraîner le tokenizer visuel — le module qui convertit une image en une séquence de tokens discrets. Cette étape est critique : un tokenizer de mauvaise qualité

© 2026 Oussama Ghorbel

impose un plafond à la qualité de génération, quelle que soit la sophistication du modèle de langage.

Objectifs de l'entraînement du tokenizer

Un tokenizer VQ-VAE ou VQGAN est entraîné pour minimiser une combinaison de plusieurs pertes :

1. **La perte de reconstruction** : difference entre l'image d'entrée et l'image reconstruite à partir de ses tokens. Pour les VAE, on utilise souvent une somme de L1 et L2, parfois complétée par une perte LPIPS (basée sur des features d'un réseau pré-entraîné) pour capturer la perception humaine de la similarité.

2. **La perte de commitment** : elle pénalise les embeddings du codebook s'ils s'éloignent des encodeurs de latents, assurant la cohérence de l'espace latent.

3. **Pour le VQGAN** : une perte adversariale (discriminateur) et/ou une perte de perceptualité qui aident à préserver les textures fines et les détails haute fréquence que la reconstruction pixel-pixel lisse trop.

L'entraînement typique d'un tokenizer 256 × 256 avec un codebook de 8192 entrées et une downsampling factor de 16 produit une grille de 16 × 16 = 256 tokens par image. Ce ratio — un token par patch de 16 × 16 pixels — est un bon point de départ, bien que des modèles récents utilisent des facteurs plus agressifs (32 × 32 ou plus) pour réduire la longueur de séquence.

Données et durée

Contrairement au LLM, le tokenizer n'a pas besoin de légendes textuelles : il est entraîné uniquement sur des images. Tout dataset d'images suffit, mais LAION filtré ou des sous-ensembles de ImageNet restent des choix standards. L'important est la diversité visuelle plutôt que la diversité sémantique (le tokenizer ne « lit » pas les légendes).

La durée d'entraînement dépend de la taille du codebook et de la résolution cible. Pour un tokenizer 256 × 256 avec un codebook de 8 192 entrées, un entraînement typique dure entre 500 000 et 1 000 000 de pas, soit environ 3 à 7 jours sur 8 GPU A100. Pour un tokenizer 512 × 512 ou

© 2026 Oussama Ghorbel

un codebook plus grand (16 384 entrées), il faut compter le double.

Le signe qu'un tokenizer est bien entraîné : la reconstruction d'images de validation est visuellement indiscernable de l'original à taille d'écran normale, et le taux d'utilisation du codebook (codebook usage) reste élevé — idéalement au-dessus de 80 %. Un codebook dont plus de la moitié des entrées sont rarement utilisées signale un problème d'initialisation ou d'entraînement (on reparlera de ce phénomène au chapitre 7).

Erreurs courantes

Une erreur classique est d'entraîner le tokenizer sur un dataset trop homogène (par exemple, uniquement des visages), ce qui produit un tokenizer excellent sur ce domaine et catastrophique ailleurs. Une autre erreur est de négliger la normalisation des images : sans une normalisation cohérente (par exemple, ramener les pixels dans [-1, 1] ou [0, 1] de manière uniforme), les gradients du tokenizer peuvent être mal conditionnés.

Enfin, il est tentant d'accélérer l'entraînement du tokenizer en réduisant le nombre d'epochs. Résistez à cette tentation : un tokenizer sous-entraîné sera la principale source d'artefacts dans les images générées, et aucun fine-tuning du LLM ne pourra les compenser.

16.3 — Fine-tuning du LLM : learning rate, warm-up, scheduler, batch size

Une fois le tokenizer figé, on passe au fine-tuning du LLM. Cette étape convertit un modèle entraîné pour le texte en modèle capable de prédire des tokens d'images à partir de légendes. Les hyperparamètres sont cruciaux.

Learning rate

Le learning rate est sans doute le hyperparamètre le plus important. Pour un fine-tuning de LLM 7B avec des poids pré-entraînés, on recommande généralement un learning rate entre 1×10 et 1×10 , avec un sweet spot autour de 5×10 pour la plupart des configurations. Un learning rate trop élevé détruit les representations linguistiques

© 2026 Oussama Ghorbel

pré-entraînées (catastrophic forgetting), tandis qu'un learning rate trop faible rend l'apprentissage des correspondances texte-image impraticable en un temps raisonnable.

Une technique efficace consiste à utiliser un learning rate différent pour les couches nouvellement initialisées (les nouvelles embeddings d'images, la tête de prédiction du codebook) et les couches pré-entraînées. Les nouvelles couches peuvent apprendre à 10 ou même 10^{-3}, tandis que les couches pré-entraînées restent à 10 ou moins. Cette différenciation accélère la convergence sans nuire à la stabilité.

Warm-up

Le warm-up est indispensable pour un fine-tuning de LLM. Pendant les premiers pas, les gradients sont bruyants et les poids s'écartent brutalement de leur initialization pré-entraînée. Un warm-up linéaire sur 1 000 à 5 000 pas — soit 0,1 à 1 % du nombre total d'étapes — permet aux poids de s'installer dans un régime stable avant d'atteindre le learning rate nominal.

Pour les très grands batch sizes (par exemple, 2 048 échantillons accumulés), on étire le warm-up à 5 000-10 000 pas. La règle empirique est que le warm-up doit durer au moins autant de pas qu'il faut pour voir la perte initiale commencer à diminuer de manière stable.

Scheduler

Le scheduler détermine l'évolution du learning rate au cours de l'entraînement. Les options les plus courantes sont :

- **Cosine annealing** : le learning rate suit une courbe en cosinus décroissante, passant du pic (post-warm-up) à une valeur proche de zéro à la fin de l'entraînement. C'est le choix le plus courant pour les entraînements à horizon fixe.
- **Linear decay** : le learning rate diminue linéairement. Plus simple, souvent aussi efficace.
- **Constant with restarts** : le learning rate reste constant puis « rebondit » périodiquement. Utile pour les entraînements ouverts sans horizon fixe, mais moins courant dans ce contexte.

© 2026 Oussama Ghorbel

Pour un entraînement de 100 000 pas avec un warm-up de 2 000 pas, une configuration cosine schedule avec un learning rate minimum de 1 × 10 est un bon point de départ.

Batch size et accumulation de gradients

La taille du batch effectif influence la stabilité des gradients et la qualité de la généralisation. Pour un LLM 7B sur un tokenizer 256 × 256 (256 tokens d'image + tokens de texte), une séquence typique fait environ 512 à 1 024 tokens. Sur un GPU A100 80GB, on peut généralement faire passer un batch de 1 à 4 séquences avec de la mixed precision.

Pour atteindre un batch effectif de 256 ou 512 séquences — souvent nécessaire pour la stabilité du fine-tuning — on utilise l'accumulation de gradients : on effectue plusieurs micro-batchs, on accumule les gradients, puis on fait un pas d'optimiseur. Par exemple, avec 8 GPU × 2 séquences par GPU × 16 micro-batchs accumulés, on obtient un batch effectif de 256.

L'AdamW est l'optimiseur standard, avec un weight decay de 0,01 à 0,1 et des moments 1 = 0,9, 2 = 0,95 (ou 0,999 pour plus de stabilité). Le clipping de gradient par norme globale à 1,0 est quasi systématique (nous y reviendrons en section 16.5).

16.4 — Curriculum : texte seul -> texte+image basse résolution -> haute résolution

Le curriculum learning — l'idée d'organiser l'entraînement par ordre croissant de difficulté — est particulièrement pertinent pour la génération d'images avec un LLM. Apprendre directement à un LLM à générer des tokens d'images à haute résolution dès les premiers pas est non seulement inefficace mais souvent contre-productif.

Phase 1 : Texte seul (consolidation)

On commence par un bref réentraînement du LLM sur du texte seul, en utilisant les mêmes hyperparamètres (learning rate, warm-up, scheduler) que la phase suivante. Cette étape dure 5 000 à 20 000 pas et sert à « réveiller » le modèle dans le régime d'entraînement adapté, tout

© 2026 Oussama Ghorbel

en vérifiant que le pipeline (tokenization, communication multi-GPU, logging) fonctionne correctement.

On peut aussi l'utiliser pour adapter le tokenizer de texte si de nouveaux tokens spéciaux sont ajoutés pour délimiter les séquences d'images (par exemple, `<|image_start|>`, `<|image_end|>`, `<|patch_1234|>`).

Phase 2 : Texte + image basse résolution

La deuxième phase introduit les tokens d'images à basse résolution. On commence typiquement par une grille de 8 × 8 tokens (soit 64 × 64 pixels reconstruits), beaucoup plus courte que la cible finale de 16 × 16 ou 32 × 32. Les images d'entraînement sont downscalées à la résolution cible, tokenisées, et le LLM apprend à prédire la séquence de tokens d'image après la légende.

Cette phase dure 20 000 à 50 000 pas. L'objectif est double : apprendre au LLM la syntaxe des tokens d'image (comment les patchs se suivent, comment la séquence se termine) et établir la correspondance sémantique texte -> image à une échelle grossière. Le modèle apprend que « un chien sur un canapé » produit une certaine distribution de tokens, sans avoir à se soucier des textures fines.

Phase 3 : Résolution croissante

On augmente progressivement la résolution : de 8 × 8 à 16 × 16, puis éventuellement à 32 × 32 ou plus. Chaque palier est maintenu pendant 10 000 à 30 000 pas, avec un pic de learning rate réinitialisé légèrement inférieur au précédent.

Cette approche progressive offre plusieurs avantages :

• Le modèle n'est pas submergé par des séquences longues dès le départ.

• Les representations à basse résolution servent de « squelette » sur lequel les détails fins s'ajoutent.

• La convergence est plus rapide qu'un entraînement direct à haute résolution, pour un résultat final souvent meilleur.

Variante : résolution progressive par masquage

© 2026 Oussama Ghorbel

Une variante consiste à toujours travailler à la résolution cible, mais à masquer aléatoirement une fraction croissante des tokens d'images pendant l'entraînement. On commence avec 90 % de tokens masqués (le modèle ne doit « inventer » que 10 % des tokens), puis on réduit progressivement le taux de masquage. Cette approche est particulièrement compatible avec les modèles non-autoregressifs (MaskGIT, MAR) que nous avons vus au chapitre 12.

16.5 — Stabilité : gradient clipping, mixed precision (BF16), EMA des poids

La stabilité de l'entraînement est l'un des défis les plus sous-estimés dans l'adaptation d'un LLM à la génération d'images. Les pertes du tokenizer visuel et du LLM peuvent être très différentes en magnitude, et les gradients peuvent exploser à n'importe quel moment.

Gradient clipping

Le gradient clipping limite la norme des gradients avant chaque mise à jour de poids. Le mécanisme est simple : si la norme L2 globale des gradients dépasse un seuil `c`, on divise tous les gradients par `norm/c`. Le seuil `c = 1,0` est le standard pour AdamW, et il fonctionne bien pour le fine-tuning de LLM.

Le clipping n'est pas une option décorative : sans lui, un seul batch anormal (une image corrompue, une légende anormalement longue) peut provoquer un saut brutal des poids, ruinant des heures d'entraînement. Dans la pratique, on observe souvent des divergences qui auraient pu être évitées avec un clipping à 1,0 alors que la configuration utilisait un clipping à 5,0.

Mixed precision (BF16)

L'entraînement en mixed precision utilise des nombres en demi-précision (FP16 ou BF16) pour les calculs de forward et backward, tout en maintenant les poids en FP32 pour les mises à jour d'optimiseur. Le BF16 (bfloat16) est préférable au FP16 pour l'entraînement de LLM car il offre la même plage dynamique que le FP32 (les exposants sont identiques), ce qui élimine le risque d'overflow. Le FP16 nécessite un

© 2026 Oussama Ghorbel

loss scaling dynamique, ce qui ajoute de la complexité et de l'overhead.

Les GPU A100, H100 et les GPU plus récents d'AMD supportent nativement le BF16. Sur ces plateformes, on doit utiliser BF16 par défaut. Le gain de vitesse par rapport au FP32 est typiquement de 1,5× à 2×, avec une réduction de mémoire d'environ 40 %.

Un piège courant : certaines couches (normes de couche, fonctions d'activation) peuvent être sensibles à la précision réduite. La solution consiste à forcer ces couches en FP32 — PyTorch offre des mécanismes natifs pour cela (`torch.autocast` avec une liste de couches en FP32).

EMA des poids

L'Exponential Moving Average (EMA) maintient une copie des poids du modèle qui est une moyenne exponentielle des poids au fil des pas d'entraînement. Le poids EMA est mis à jour à chaque pas :

Avec un `decay` de 0,999 ou 0,9999, le modèle EMA lisse les fluctuations du gradient et produit typiquement des images de meilleure qualité que les poids bruts du dernier pas. Pour l'évaluation et l'inférence, on utilise systématiquement les poids EMA.

L'EMA ajoute une copie supplémentaire des poids en mémoire, ce qui peut être problématique pour les très grands modèles. Pour un LLM 7B, cela représente environ 28 Go supplémentaires en FP32, ou 14 Go en BF16 — faisable sur un A100 80GB avec un sharding approprié.

D'autres techniques de stabilité

• **Initialization différentielle** : les nouvelles couches (embeddings d'images, tête de prédiction) sont initialisées avec une variance adaptée, souvent plus petite que l'initialisation standard, pour éviter des gradients explosifs au début.

• **Warm-up des nouvelles couches** : on peut geler temporairement les couches pré-entraînées pendant les 1 000 premiers pas, laissant les nouvelles couches s'adapter seule. Cela évite de « casser » les representations linguistiques par des gradients bruyants.

• **Régularisation** : un dropout léger (0,05 à 0,1) sur les nouvelles couches peut aider à prévenir le sur-ajustement, surtout si le dataset de fine-tuning est petit.

© 2026 Oussama Ghorbel

16.6 — Coût computationnel : estimation GPU-hours pour 100 M de paires texte-image

Le coût computationnel est souvent le facteur limitant dans la conception d'un protocole d'entraînement. Une estimation réaliste est essentielle avant de lancer un entraînement.

Méthode d'estimation

Pour estimer le coût, on utilise le nombre de FLOPs (floating-point operations) nécessaires pour une passe avant et arrière à travers le modèle. Pour un modèle de paramètres `N` entraîné sur `D` tokens, le coût total est approximativement `6 × N × D` FLOPs (facteur 6 = 3 pour le forward, 3 pour le backward, en comptant deux FLOPs par MAC).

Pour un LLM 7B (7 × 10 paramètres) :

• Forward : ~2 × 7 × 10 = 14 × 10 FLOPs par token

• Forward + backward : ~6 × 7 × 10 = 42 × 10 FLOPs par token

Considérons un dataset de 100 millions de paires texte-image. Chaque exemple comprend une légende de ~30 tokens et une séquence d'images de ~256 tokens (pour une grille 16 × 16), soit environ 286 tokens au total. Pour une epoch complète :

Un A100 80GB en BF16 offre environ 312 TFLOPS en théorie (624 TFLOPS avec sparsity), mais l'efficacité réelle en entraînement est typiquement de 30 à 50 %. En prenant 40 % d'efficacité :

Soit environ **2 700 GPU-hours** sur un seul A100, ou environ **340 heures** (14 jours) sur 8 A100 en parallèle.

Tableau récapitulatif

Taille du dataset	Tokens/image	FLOPs totaux	GPU-hours (A100)	Temps sur 8× A100
10 M images	~286	$1{,}2 \times 10^{2}$	~270	~1,5 jours
100 M images	~286	$1{,}2 \times 10^{21}$	~2 700	~14 jours
500 M images	~286	$6{,}0 \times 10^{21}$	~13 500	~70 jours

© 2026 Oussama Ghorbel

Ces estimations supposent une epoch unique. En pratique, on entraîne souvent sur 1 à 3 epochs pour les grands datasets, ce qui multiplie les coûts. Le pré-entraînement du tokenizer ajoute environ 500 à 2 000 GPU-hours selon la taille et la résolution cible.

Optimisations de coût

Plusieurs techniques permettent de réduire significativement le coût :

• **Gradient checkpointing** : réduit la mémoire du backward de O(L) à O(L) où L est le nombre de couches, au prix de ~30 % de calcul supplémentaire. Indispensable pour entraîner un 7B avec de longues séquences.

• **Sequence packing** : au lieu de padding les séquences courtes, on les concatène dans des séquences de longueur fixe. Cela peut augmenter l'efficacité de 20 à 40 %.

• **Flash Attention** : réduit la complexité mémoire de l'attention et accélère le calcul, en particulier pour les séquences longues.

• **Filtre de données agressif** : mieux vaut entraîner sur 50 millions de paires de haute qualité que sur 200 millions de paires bruitées. Le filtering peut réduire le coût tout en améliorant la qualité.

16.7 — Checkpointing, reprise et détection de divergence

Un entraînement de plusieurs jours ou semaines est un investissement précieux. Un protocole de checkpointing robuste est indispensable.

Stratégie de checkpointing

Un checkpoint doit contenir :

• Les poids du modèle (FP32 ou BF16)

• Les poids EMA si utilisés

• L'état de l'optimiseur (moments Adam, step count)

• Le scheduler

• L'état du DataLoader (pour reprendre la data augmentation et le shuffling exactement où on s'est arrêté)

© 2026 Oussama Ghorbel

• Les hyperparamètres et la configuration de l'entraînement

On sauvegarde typiquement un checkpoint toutes les 1 000 à 5 000 pas, avec une rotation : les 5 derniers checkpoints sont conservés, les plus anciens sont supprimés. Les checkpoints sont écrits sur un stockage rapide (NVMe local) puis copiés vers un stockage persistant (S3, NFS).

Le temps de sauvegarde doit être minimisé pour ne pas interrompre l'entraînement. Des techniques comme le snapshotting asynchrone ou la sauvegarde sharded (un GPU par fichier) permettent de réduire ce overhead à quelques secondes.

Reprise (resumption)

La reprise doit être transparente : on charge le checkpoint et l'entraînement reprend exactement là où il s'est arrêté, avec le même état d'optimiseur, le même scheduler, et idéalement le même DataLoader. Une reprise imparfaite (par exemple, sans l'état de l'optimiseur) peut provoquer une perturbation transitoire de la perte qui nuit à la convergence.

Il faut aussi prévoir le cas où le checkpoint est corrompu (écriture interrompue par une panne). Le pattern standard est d'écrire dans un fichier temporaire puis de faire un `rename` atomique une fois l'écriture terminée.

Détection de divergence

La divergence — une explosion soudaine de la perte ou des gradients — est le cauchemar de tout ingénieur en entraînement. Les causes possibles sont multiples : learning rate trop élevé, batch corrompu, bug de code, instability numérique.

Un callback de détection de divergence surveille la perte et déclenche une action lorsque certaines conditions sont remplies. La règle la plus simple et la plus efficace : si la perte actuelle dépasse 3 × la moyenne mobile des 100 dernières valeurs, on considère que le modèle diverge.

Ce callback doit être appelé à chaque pas d'entraînement, ou au moins toutes les N étapes. En cas de détection, il sauvegarde un checkpoint d'urgence et envoie une alerte (email, Slack, webhook).

© 2026 Oussama Ghorbel

L'entraînement peut alors être interrompu manuellement, ou automatiquement rebondi au dernier checkpoint sain avec un learning rate réduit.

D'autres signaux de divergence sont utiles en complément :

• **NaN ou Inf dans les gradients** : le training doit s'arrêter immédiatement.

• **Norme des gradients** : une norme qui augmente régulièrement signale une instabilité latente.

• **Taux d'utilisation du codebook** : une chute brutale indique que le tokenizer « s'effondre ».

Résumé du protocole

Voici un résumé du protocole complet proposé dans ce chapitre :

1. **Choisir et filtrer les données** : combiner un dataset large (LAION filtré) avec un dataset de haute qualité (COCO) pour le fine-tuning final.

2. **Entraîner le tokenizer** : 3-7 jours sur 8 A100, objectif de reconstruction haute fidélité et d'utilisation > 80 % du codebook.

3. **Fine-tuner le LLM** : AdamW, LR 5 × 10 , warm-up 2 000 pas, cosine schedule, batch effectif 256, gradient clipping à 1,0, BF16, EMA decay 0,999.

4. **Appliquer un curriculum** : texte seul -> image basse résolution -> résolution croissante.

5. **Surveiller la stabilité** : détection de divergence avec seuil 3× la moyenne mobile, checkpointing toutes les 2 000 pas.

6. **Planifier le budget** : environ 2 700 GPU-hours pour 100 M de paires texte-image, soit ~14 jours sur 8 A100.

Ce protocole n'est pas gravé dans le marbre — chaque projet a ses contraintes et ses particularités. Mais il constitue un point de départ solide, éprouvé sur des entraînements réels, et il peut être adapté à des budgets plus modestes ou à des ambitions plus importantes.

© 2026 Oussama Ghorbel

Exercices

Exercice 1 — Fichier de configuration YAML

Rédigez un fichier de configuration YAML pour un entraînement sur 8 GPU A100. Le fichier doit spécifier : le modèle de base, le tokenizer, le dataset, la taille du batch (micro et effectif), le learning rate et son scheduler, le warm-up, le gradient clipping, la précision, l'EMA, la fréquence de checkpointing et les paramètres de logging (par exemple, intégration W&B ou TensorBoard). Justifiez brièvement chaque choix.

Exercice 2 — Estimation du coût

Estimez le coût en GPU-hours pour entraîner 1 epoch sur 10 millions d'images 256 × 256 avec un LLM 7B. Considérez : une séquence de 30 tokens de texte + 256 tokens d'image, un throughput réel de 40 % sur un A100 80GB, et un gradient checkpointing actif (qui réduit la mémoire mais ajoute ~30 % de calcul). Comparez ensuite le coût avec et sans sequence packing.

Exercice 3 — Callback de détection de divergence

Implémentez en Python un callback de détection de divergence qui surveille la perte d'entraînement. Le callback doit : maintenir une fenêtre glissante de 100 valeurs, calculer la moyenne mobile, déclencher une alerte si la perte actuelle dépasse 3 × la moyenne mobile, sauvegarder un checkpoint d'urgence, et fournir une méthode pour « acknowerliger » l'alerte et reprendre l'entraînement. Testez-le avec une séquence de pertes synthétiques incluant une divergence contrôlée.

© 2026 Oussama Ghorbel

Chapitre 17 – Adapter un LLM existant : GPT, LLaMA, Mistral — étude de cas

Dans les chapitres précédents, nous avons disséqué la tokenisation visuelle, la factorisation autorégressive et les mécanismes d'alignement texte-image. Il est temps de passer à l'acte. Ce chapitre propose un guide opérationnel : comment prendre un grand modèle de langage pré-entraîné — un LLaMA-7B, un Mistral-7B ou toute autre architecture ouverte — et lui greffer une tête de génération visuelle capable de produire des images 256 × 256. Je vais détailler chaque étape, du choix du modèle de base jusqu'au pseudo-code complet d'inférence, en passant par les compromis d'entraînement qui déterminent la réussite ou l'échec du projet.

17.1 — Choix du LLM de base : taille, licence, architecture

La première décision est stratégique autant que technique. Trois critères dominent.

Taille paramétrique. Un modèle 7B constitue le point d'équilibre entre capacité représentationnelle et budget computationnel accessible à un laboratoire ou une équipe d'ingénierie disposant de quelques GPU A100. En dessous de 3B, la capacité à maintenir une cohérence sémantique sur une séquence de 256 à 1024 tokens visuels se dégrade sensiblement. Au-delà de 13B, le coût d'inférence explose sans gain proportionnel en qualité visuelle, car la génération d'image dépend davantage du tokenizer et de la tête de décodage que de la profondeur linguistique du backbone.

Licence. Pour un usage de recherche ou un déploiement commercial, la licence compte autant que l'architecture. LLaMA 2 (Meta) impose des restrictions d'usage commercial au-delà de 700 millions d'utilisateurs mensuels ; Mistral-7B est publié sous Apache 2.0, ce qui simplifie grandement l'intégration industrielle. Les modèles GPT d'OpenAI, bien que performants, ne sont accessibles que via API et ne permettent pas la modification des poids — ils servent ici de référence comparative, non de base d'adaptation.

© 2026 Oussama Ghorbel

Architecture interne. Trois choix architecturaux méritent attention :

• *Grouped-Query Attention (GQA)*, utilisée par LLaMA 2 70B et Mistral-7B, réduit le nombre de têtes de clés/valeurs par rapport aux têtes de requêtes. Cela diminue la mémoire KV-cache lors de la génération séquentielle de tokens visuels, un avantage décisif quand la séquence atteint 1024 positions.

• *Multi-Query Attention (MQA)*, plus agressive encore, partage une unique tête K/V. Le gain mémoire est maximal, mais la qualité attentionnelle peut souffrir sur des tâches visuelles fines.

• *Rotary Position Embeddings (RoPE)*, présent dans LLaMA et Mistral, encode la position relative. Pour l'image, il faudra étendre ou adapter RoPE à un balayage 2D (voir chapitre 11), par exemple en décomposant l'indice linéaire en coordonnées (ligne, colonne) et en appliquant deux rotations indépendantes.

En résumé : pour ce chapitre, je m'appuie sur Mistral-7B (Apache 2.0, GQA, RoPE), tout en signalant les variantes propres à LLaMA.

17.2 — Gel des couches basses, fine-tuning des couches hautes et de la tête

Un LLM 7B comporte typiquement 32 couches Transformer. L'intuition centrale est la suivante : les couches basses capturent des représentations syntaxiques et sémantiques générales (structure des phrases, relations logiques) qui restent utiles pour comprendre le prompt textuel. Les couches hautes, en revanche, spécialisent la représentation vers la tâche de sortie.

Protocole en trois niveaux :

1. *Couches 1 à 20 (gel complet).* Les poids sont figés. Aucun gradient ne les traverse. Cela préserve la compréhension linguistique et réduit la mémoire d'entraînement d'environ 60 %.

2. *Couches 21 à 32 (fine-tuning partiel).* On dégèle ces douze couches avec un taux d'apprentissage faible (1×10 à 5×10). Elles apprennent à projeter les représentations textuelles vers l'espace des tokens visuels.

© 2026 Oussama Ghorbel

3. *Tête de génération (entraînement complet).* La nouvelle tête de classification sur le codebook VQGAN est initialisée aléatoirement et entraînée avec un taux d'apprentissage plus élevé (1 × 10).

Cette stratégie hiérarchique évite le phénomène d'oubli catastrophique : le modèle conserve sa capacité à parser un prompt complexe tout en apprenant la distribution des tokens visuels. En pratique, il est constaté qu'un entraînement monolithique (toutes couches dégelées) nécessite trois à quatre fois plus de données pour converger vers une qualité équivalente.

17.3 — Ajout d'un module de projection visuel

Le LLM opère dans un espace d'embedding de dimension $d = 4096$ (Mistral-7B). Le tokenizer VQGAN produit des indices discrets qu'il faut mapper dans cet espace. Réciproquement, pour la génération, les hidden states du LLM doivent être projetés vers le codebook. Plusieurs modules de projection s'offrent à nous.

MLP linéaire. La solution la plus simple : une couche fully-connected (K -> 4096) suivie d'une normalisation. Rapide, peu de paramètres additionnels (~4 millions), mais capacité expressive limitée.

Q-Former. Emprunté à BLIP-2, ce module utilise un petit Transformer (6 couches, 768 dimensions) avec un ensemble de *learnable queries* (typiquement 32 à 64). Les queries interagissent par cross-attention avec les représentations du LLM. Avantage : compression de l'information et sélection adaptative. Inconvénient : ajout de ~100 M paramètres et complexité d'entraînement.

Perceiver Resampler. Similaire au Q-Former mais avec un nombre fixe de latents (par exemple 64) qui agrègent l'information par attention croisée itérée. Particulièrement adapté quand la séquence visuelle est longue (1024 tokens pour une image 256 × 256 avec un codebook 16 × 16).

Pour notre étude de cas, je recommande le **Perceiver Resampler avec 64 latents** : il offre un bon compromis entre expressivité et coût, et il s'interface naturellement avec le mécanisme GQA de Mistral.

© 2026 Oussama Ghorbel

17.4 — Tête de sortie : classification sur K tokens visuels vs régression continue

Deux philosophies s'affrontent pour la couche finale.

Classification discrète. Le LLM prédit, à chaque pas, une distribution de probabilité sur les K entrées du codebook VQGAN (typiquement K = 16 384). La perte est une entropie croisée standard. C'est l'approche adoptée par DALL·E, Parti et les modèles présentés dans ce livre. Avantages : compatibilité directe avec l'architecture next-token du LLM, échantillonnage par top-k/top-p, pas d'hypothèse gaussienne. Inconvénient : la quantification introduit une perte d'information irréversible.

Régression continue. On prédit un vecteur continu dans l'espace latent du VAE (avant quantification). La perte est une MSE ou une combinaison L1 + perceptuelle. Cette voie est explorée par certains travaux récents (MAR, 2024). Avantage : pas de plafond de qualité lié à la taille du codebook. Inconvénient : incompatible avec la factorisation softmax du LLM ; nécessite une tête de régression dédiée et un échantillonnage par diffusion ou flux sur la sortie.

Dans le cadre de ce chapitre, nous retenons la **classification discrète sur K = 16 384 tokens**, car elle s'intègre sans modification structurelle au mécanisme de next-token prediction du LLM. La tête de sortie est une couche linéaire 4096 -> 16 384, suivie d'un softmax.

17.5 — LoRA / QLoRA pour un fine-tuning économe en mémoire

Fine-tuner 32 couches d'un modèle 7B en pleine précision nécessite environ 120 Go de mémoire GPU (poids + gradients + états d'optimiseur). C'est prohibitif pour la plupart des équipes. Les adaptateurs de bas rang résolvent ce problème.

LoRA (Low-Rank Adaptation). Pour chaque matrice de poids W ^{d×d}, on apprend une décomposition W = BA, avec B ^{d×r} et A ^{r×d}, où r d. Avec r = 16 et d = 4096, chaque adaptateur contient 2 × 4096 × 16 = 131 072 paramètres, contre 16,7 millions pour la matrice pleine. Appliqué aux quatre projections (Q, K, V, O) de chaque couche d'attention, et aux deux projections du FFN, cela donne :

© 2026 Oussama Ghorbel

• 6 matrices par couche × 32 couches × 131 072 paramètres env. **25,2 M paramètres entraînés**, soit 0,36 % du modèle total.

QLoRA. On quantise les poids gelés en 4 bits (NF4), réduisant l'empreinte mémoire du backbone à ~3,5 Go pour un 7B. Les adaptateurs LoRA restent en float16. Résultat : le fine-tuning complet tient sur une seule A100 40 Go, voire une RTX 4090 24 Go pour les configurations les plus sobres.

Recommandation pratique. Je recommande d'appliquer LoRA (r = 16, = 32) sur les couches 21 à 32 uniquement, en ciblant Q, K, V, O et les FFN. Les couches 1 à 20 restent gelées et quantifiées en 4 bits. La tête de classification et le Perceiver Resampler sont entraînés en full precision. Cette configuration consomme environ 22 Go de VRAM et converge en 15 000 à 25 000 étapes sur un dataset de 500 000 paires texte-image.

17.6 — Intégration avec un tokenizer VQGAN pré-entraîné : pipeline bout-en-bout

Le tokenizer VQGAN (chapitre 6) est utilisé *en inférence uniquement* : ses poids sont gelés dès le départ. Le pipeline complet se décompose ainsi :

1. **Encodage (entraînement).** Image 256 × 256 -> VQGAN Encoder -> grille de features 16 × 16 × 256 -> quantification -> séquence de 256 indices dans [0, K 1].

2. **Sérialisation.** Les 256 indices sont aplatis en balayage raster (ou Hilbert, cf. chapitre 11) et précédés d'un token [IMG_START]. Le prompt textuel est tokenisé par le tokenizer BPE du LLM et suffixé par [IMG_START].

3. **Passage dans le LLM.** La séquence concaténée [texte ; IMG_START ; tokens_visuels] est passée dans Mistral-7B. La perte est calculée uniquement sur les positions des tokens visuels (masquage des positions texte).

4. **Décodage (inférence).** Le LLM génère autoregressivement 256 tokens visuels. Chaque indice prédit est déquantifié via le codebook, puis la grille 16 × 16 × 256 est passée au VQGAN Decoder pour produire l'image RGB 256 × 256.

© 2026 Oussama Ghorbel

Point critique : le codebook VQGAN et la tête de classification du LLM doivent partager exactement le même ordre d'indices. Une désalignement, même d'une permutation, produit des images incohérentes. Je recommande de sauvegarder le codebook comme un fichier .pt immuable et de le charger systématiquement au démarrage.

17.7 — Pseudo-code complet : de l'image d'entrée à la séquence de tokens générée

Voici le squelette intégral, de l'entraînement à l'inférence.

Ce pseudo-code est volontairement simplifié (pas de KV-cache, pas de gestion du padding), mais il capture la logique complète. En production, on remplacera la boucle par un mécanisme de cache incrémental pour éviter de recalculer les 255 positions précédentes à chaque pas.

17.8 — Exercices

Exercice 1 — Forward pass complète.

Écrivez le pseudo-code détaillé (en Python ou en pseudo-langage) d'une forward pass complète : image d'entrée -> encodage VQGAN -> séquence de tokens -> passage dans le LLM -> prédiction next-token -> décodage VQGAN -> image reconstruite. Vous préciserez les formes de tenseurs à chaque étape pour une image 256 × 256 avec un codebook K = 8192 et une grille 16 × 16. Indiquez où se situent les goulots d'étranglement mémoire.

Exercice 2 — Calcul de paramètres LoRA.

On applique LoRA avec r = 16 sur les quatre matrices d'attention (Q, K, V, O) et les deux matrices du FFN (gate_proj, down_proj) de chaque couche d'un LLaMA-7B (32 couches, d_model = 4096, d_ff = 11008). Les couches 1 à 20 sont entièrement gelées ; LoRA n'est appliqué qu'aux couches 21 à 32. Calculez :

(a) le nombre total de paramètres LoRA entraînés ;

(b) le pourcentage que cela représente par rapport aux 6,7 milliards de paramètres du modèle ;

© 2026 Oussama Ghorbel

(c) la mémoire GPU estimée pour stocker ces adaptateurs en float16.

Indice : pour le FFN, les matrices sont de forme (4096 × 11008) et (11008 × 4096). Chaque adaptateur LoRA sur une matrice (m × n) contient r × (m + n) paramètres.

Exercice 3 — Script d'inférence rapide.

Concevez un script d'inférence qui génère 4 images 256 × 256 à partir d'un unique prompt textuel en moins de 30 secondes sur une A100 80 Go. Vous utiliserez :

- un KV-cache pour éviter le recalcul des positions précédentes ;
- un batch de 4 séquences générées en parallèle ;
- la quantification INT8 des poids gelés du LLM ;
- un échantillonnage top-p (p = 0,95) avec température 0,8.

Estimez le nombre d'opérations FLOPs par token généré et vérifiez que le budget temporel est réaliste (une A100 délivre environ 312 TFLOPS en float16). Justifiez chaque choix d'optimisation.

Ce chapitre clôt la partie IV. Nous avons transformé un LLM textuel en un générateur d'images fonctionnel, en respectant les contraintes de mémoire, de licence et de cohérence architecturale. La partie V évaluera quantitativement ce que nous venons de construire : les métriques, les comparaisons avec la diffusion et les limites que nous n'avons pas encore levées.

17.9 — Étude de cas approfondie : entraînement sur un cluster restreint

Pour ancrer ces concepts dans la réalité matérielle, détaillons un scénario concret, observé et affiné lors de projets de recherche appliquée : l'adaptation de Mistral-7B sur un nœud de calcul modeste, composé de quatre GPU NVIDIA A100 (80 Go de VRAM chacun). L'objectif est d'entraîner le modèle sur un dataset de 2 millions d'images en résolution 256 × 256, sur une durée de deux semaines.

Stratégie de parallélisme et gestion de la mémoire.

© 2026 Oussama Ghorbel

Avec un modèle de 7 milliards de paramètres, le chargement en float16 occupe environ 14 Go. Cependant, l'entraînement nécessite de stocker les gradients et les états de l'optimiseur (AdamW). En précision mixte, l'empreinte mémoire par paramètre entraîneur est d'environ 18 octets (2 pour les poids fp16, 2 pour les gradients fp16, 12 pour les états fp32 d'AdamW, plus les activations intermédiaires). Pour 7B paramètres, cela dépasse largement les 80 Go d'un seul GPU.

Nous optons pour *Fully Sharded Data Parallel* (FSDP) avec un *sharding* complet (`FULL_SHARD`). Les poids, gradients et états d'optimiseur sont partitionnés équitablement sur les quatre GPU. Pour les couches gelées (1 à 20), nous utilisons la quantification INT8 (via la bibliothèque `bitsandbytes`), ce qui réduit leur empreinte statique à environ 7 Go au total. Les adaptateurs LoRA et la tête de classification restent en float16. Cette configuration hybride permet de maintenir un *batch size* effectif de 64 sans recourir à l'accumulation de gradients excessive.

Pipeline de données et curriculum d'apprentissage.

Le goulot d'étranglement n'est pas toujours le calcul matriciel, mais l'I/O. Nous utilisons le format WebDataset pour streamer les paires texte-image directement depuis un stockage objet (S3), évitant ainsi de saturer le système de fichiers local avec des millions de petits fichiers JPEG.

Le curriculum d'entraînement est crucial pour la stabilité de la convergence :

1. *Phase 1 (5 000 étapes) — Alignement grossier :* On dégèle uniquement le Perceiver Resampler et la tête de projection. Le LLM reste entièrement gelé. Le modèle apprend à mapper les concepts textuels de base vers des blobs de couleur et des formes globales.

2. *Phase 2 (15 000 étapes) — Activation de LoRA :* On dégèle les adaptateurs sur les couches 21 à 32. Le taux d'apprentissage suit un cycle cosinoidal avec un *warmup* linéaire de 500 étapes. C'est ici que la sémantique fine (les objets, les textures, les relations spatiales) commence à émerger.

3. *Phase 3 (5 000 étapes) — Raffinement :* On réduit le *batch size* et on augmente légèrement la température d'échantillonnage pendant

© 2026 Oussama Ghorbel

les passes de validation pour encourager la diversité et penaliser les modes de génération trop conservateurs.

17.10 — Pièges classiques et diagnostics d'échec

L'adaptation d'un LLM à la génération visuelle n'est pas un long fleuve tranquille. Voici les pathologies les plus fréquentes et la manière de les diagnostiquer avant qu'elles ne ruinent des semaines de temps de calcul.

1. L'effondrement du codebook (Codebook Collapse).

Symptôme : Le modèle génère des images floues, aux couleurs passées, et n'utilise qu'une infime fraction des 16 384 tokens du VQGAN (souvent moins de 5 %).

Diagnostic : Tracez l'histogramme de fréquence des tokens prédits sur un batch de validation. Si la distribution est extrêmement lourde (loi de Zipf exacerbée), le modèle a trouvé une solution de facilité en ignorant les tokens de haute fréquence spatiale qui codent les détails fins.

Solution : Introduisez une pénalité d'entropie sur la distribution de sortie du LLM, ou utilisez un *temperature scaling* dynamique qui force l'exploration de tokens rares pendant l'entraînement. L'ajout d'une perte de reconstruction perceptuelle (LPIPS) en aval du VQGAN peut aussi forcer le modèle à utiliser des indices plus précis.

2. L'hallucination spatiale et les artefacts en damier.

Symptôme : L'image générée présente des motifs répétitifs, des grilles ou des décalages de couleurs entre les patches.

Diagnostic : Ce problème trahit une mauvaise compréhension de la topologie 2D par le LLM. Le balayage raster (ligne par ligne) crée une distance artificielle dans la séquence 1D entre le pixel en bas à droite d'une ligne et le pixel en haut à gauche de la ligne suivante.

Solution : Remplacez le balayage raster par une courbe de Hilbert ou un balayage en spirale (cf. chapitre 11). De plus, l'ajout d'embeddings positionnels 2D explicites (additionnés aux embeddings de tokens avant l'injection dans le LLM) aide le modèle à reconstruire la grille mentale

© 2026 Oussama Ghorbel

de l'image.

3. La surdité au prompt (Prompt Bleeding).

Symptôme : Le modèle génère de belles images, mais qui ne correspondent pas du tout au texte d'entrée. Il semble "réciter" une distribution moyenne de son dataset d'entraînement, ignorant les adjectifs ou les sujets spécifiques.

Diagnostic : Calculez le CLIP-score entre le prompt et l'image générée. S'il est bas alors que le FID (Fréchet Inception Distance) est bon, le modèle a désappris le conditionnement textuel au profit de la reconstruction visuelle pure.

Solution : Augmentez le poids de la perte sur les premiers tokens visuels (qui portent souvent la sémantique globale de l'image). Assurez-vous que le *dropout* sur les embeddings textuels n'est pas trop agressif. Parfois, réinitialiser les couches de cross-attention du Perceiver Resampler suffit à réveiller l'alignement texte-image.

4. L'oubli catastrophique linguistique.

Symptôme : Si vous demandez au modèle de générer du texte après avoir généré une image (dans un cadre multimodal), il produit du charabia ou boucle sur des syllabes.

Diagnostic : Évaluez le modèle sur un benchmark linguistique simple (ex: HellaSwag ou MMLU) avant et après le fine-tuning visuel.

Solution : C'est exactement pour éviter cela que nous avons gelé les 20 premières couches et utilisé LoRA. Si le problème persiste, intégrez 5 % de données purement textuelles dans vos batches d'entraînement visuel (multi-tasking) pour maintenir les capacités linguistiques en éveil et empêcher la dérive des représentations internes.

17.11 — Corrigé détaillé et pistes de correction des exercices

Pour consolider les acquis de ce chapitre, voici les pistes de réflexion, les calculs et les architectures attendus pour les exercices proposés en section 17.8.

Correction de l'Exercice 1 : Forward pass et formes de tenseurs

Pour une image de 256 × 256 et un codebook K = 8192 :

© 2026 Oussama Ghorbel

• *Entrée :* Image `x` de forme `[B, 3, 256, 256]`.

• *VQGAN Encoder :* Produit une grille de features continues `z` de forme `[B, 256, 16, 16]` (en supposant un facteur de réduction spatiale de 16).

• *Quantification :* Chaque vecteur de 256 dimensions est remplacé par l'indice du vecteur le plus proche dans le codebook. On obtient `indices` de forme `[B, 16, 16]` avec des valeurs entières dans `[0, 8191]`.

• *Sérialisation :* Aplatissement en `[B, 256]`.

• *Embedding LLM :* Projection via une table d'embedding (ou couche linéaire sur one-hot) de taille `8192 × 4096`. Forme résultante : `[B, 256, 4096]`.

• *Passage LLM :* Le tenseur caché reste `[B, L_text + 256, 4096]`.

• *Tête de sortie :* Projection linéaire `4096 × 8192`. Forme des logits : `[B, 256, 8192]`.

• *Goulot d'étranglement mémoire :* Le calcul de la perte d'entropie croisée sur des logits de forme `[B, 256, 8192]` en float32 consomme énormément de mémoire d'activation (plusieurs gigaoctets pour un batch de taille 32). L'utilisation de *flash-attention* et de la précision mixte (bf16) couplée à une fonction de perte optimisée (comme `cross_entropy` de PyTorch qui évite l'instanciation du softmax complet) est impérative ici.

Correction de l'Exercice 2 : Calcul de paramètres LoRA

• *Couches ciblées :* 12 couches (de 21 à 32).

• *Matrices d'attention (Q, K, V, O) :* 4 matrices de forme `4096 × 4096`.

Paramètres LoRA par matrice = `r × (4096 + 4096) = 16 × 8192 = 131 072`.

Pour 4 matrices = `524 288` paramètres par couche.

• *Matrices FFN (gate, up, down) :* Dans LLaMA, le FFN de type SwiGLU possède 3 matrices. Deux de forme `4096 × 11008` et une de forme `11008 × 4096`.

© 2026 Oussama Ghorbel

Paramètres LoRA pour `4096 × 11008` = `16 × (4096 + 11008) = 241 664`.

Paramètres LoRA pour `11008 × 4096` = `16 × (11008 + 4096) = 241 664`.

Total FFN par couche = `241 664 + 241 664 + 241 664 = 724 992` paramètres.

• *Total par couche :* `524 288 + 724 992 = 1 249 280` paramètres.

• *Total pour 12 couches :* `12 × 1 249 280 = 14 991 360` paramètres (soit environ 15 M).

• *Pourcentage :* `15 M / 6,7 B env. 0,22 %` du modèle total.

• *Mémoire GPU (float16) :* 15 millions de paramètres × 2 octets = 30 Mo. C'est négligeable, ce qui prouve l'efficacité redoutable de LoRA pour l'adaptation de tâches spécifiques.

Correction de l'Exercice 3 : Script d'inférence rapide

• *FLOPs par token :* Pour un modèle 7B, chaque token généré nécessite environ `2 × 7B = 14 GFLOPs`.

• *Batch de 4 :* Le calcul des poids est partagé (batching), mais le KV-cache est multiplié par 4. Sur une A100, le débit mémoire (bandwidth) est souvent le facteur limitant en génération autoregressive (memory-bound), pas les FLOPs (compute-bound).

• *Temps de génération :* 256 tokens à générer. Avec un KV-cache et un batch de 4, le temps par pas est d'environ 30 à 40 ms. `256 × 40 ms env. 10,2` secondes pour la génération autoregressive complète.

• *Décodage VQGAN :* Le décodeur VQGAN (souvent un petit réseau convolutif de 50-100M paramètres) prend environ 100 ms pour décoder 4 images en parallèle.

• *Total :* ~11 secondes, largement en dessous de la limite des 30 secondes.

• *Optimisations clés à implémenter :* L'utilisation de `torch.compile` pour fusionner les kernels, le KV-cache paged (type vLLM) pour éviter les fragmentations mémoire, et la quantification INT8 (via GPTQ ou AWQ) qui divise par deux le temps de chargement des poids depuis la VRAM vers les cœurs Tensor.

© 2026 Oussama Ghorbel

17.12 — Épilogue de la Partie IV : vers le tribunal des métriques

Nous voici au terme absolu de cette quatrième partie. Le chemin parcouru est considérable. Dans la partie III, nous avions théorisé la factorisation autorégressive de l'image ; dans la partie IV, nous l'avons incarnée dans le silicium et le code. Nous avons pris un monolithe linguistique, un LLM forgé pour prédire le mot suivant, et nous l'avons plié, sans le briser, à la tâche de prédire le pixel suivant — ou plus exactement, le token visuel suivant.

L'adaptation d'un modèle comme Mistral ou LLaMA à la génération d'images n'est pas une simple greffe technique. C'est un exercice d'équilibrisme entre la préservation d'un savoir linguistique colossal et l'acquisition d'une intuition spatiale et chromatique qui lui est fondamentalement étrangère. Le choix du tokenizer (VQGAN), la topologie du balayage, l'architecture du module de projection (Perceiver Resampler) et la parcimonie de l'entraînement (LoRA) sont les quatre piliers qui soutiennent cet édifice hybride.

Ce que nous avons construit ici est un générateur autoregressif pur. Il est élégant, il unifie le texte et l'image sous le même paradigme de *next-token prediction*, et il bénéficie de toute l'ingénierie logicielle développée pour les LLM (KV-cache, quantification, inférence spéculative). Pourtant, il n'est pas seul sur le ring. Les modèles de diffusion, avec leur approche par débruitage itératif, dominent encore souvent en termes de fidélité photographique et de flexibilité de contrôle (ControlNet, IP-Adapter).

La partie V de cet ouvrage sera le tribunal de la vérité. Nous allons soumettre notre générateur autoregressif à l'épreuve des métriques impitoyables (FID, CLIP-score, LPIPS). Nous le confronterons, étape par étape, aux géants de la diffusion. Nous analyserons ses artefacts, ses échecs de cohérence, et nous chiffrerons son coût réel, tant en watts qu'en dollars. Car en ingénierie de l'apprentissage automatique, si l'élégance de l'architecture séduit l'esprit, c'est la rigueur de l'évaluation qui dicte la survie du modèle. Préparez vos scripts de benchmark ; le verdict approche.

© 2026 Oussama Ghorbel

Partie V — Évaluation, Comparaison et Limites

© 2026 Oussama Ghorbel

Chapitre 18 – Métriques quantitatives : FID, IS, CLIP-score, LPIPS et au-delà

Évaluer un modèle génératif d'images est l'un des défis les plus épineux de l'apprentissage automatique moderne. Contrairement à un classificateur où l'exactitude (accuracy) ou la perte de validation (validation loss) fournissent une boussole fiable, un générateur d'images évolue dans un espace où la notion de « vérité terrain » est intrinsèquement subjective. Une perte d'entraînement qui diminue ne garantit en rien que les images produites seront esthétiques, cohérentes ou fidèles au prompt textuel. Pire encore, optimiser aveuglément une métrique unique conduit presque toujours à des comportements pathologiques, un phénomène que nous appelons le *reward hacking*.

Dans ce chapitre, nous allons outiller votre boîte à outils d'ingénieur et de chercheur. Nous allons décortiquer les métriques qui font autorité dans la littérature, comprendre leurs fondements mathématiques, mais surtout explorer leurs angles morts. Car savoir calculer un score est trivial ; savoir l'interpréter et concevoir un protocole d'évaluation robuste est ce qui sépare un prototype de laboratoire d'un système de production fiable.

18.1 — Fréchet Inception Distance (FID) : calcul, interprétation, pièges

La *Fréchet Inception Distance* (FID), introduite par Heusel et al. en 2017, est devenue la métrique standard *de facto* pour évaluer la qualité et la diversité des images générées. Contrairement aux approches précédentes qui évaluaient les images individuellement, le FID compare les *distributions* de deux ensembles d'images : un ensemble réel (généralement issu du jeu d'entraînement ou d'un jeu de test de référence) et un ensemble généré.

Le principe repose sur l'extraction de caractéristiques sémantiques de haut niveau. On fait passer les deux ensembles d'images à travers un réseau de neurones pré-entraîné, typiquement InceptionV3 (entraîné sur ImageNet), et on récupère les activations d'une couche profonde (la couche *pool3* de 2048 dimensions). On suppose ensuite que ces

activations suivent une distribution gaussienne multivariée pour chaque ensemble.

La FID calcule la distance de Fréchet (ou distance de Wasserstein-2) entre ces deux gaussiennes, caractérisées par leur moyenne (μ) et leur matrice de covariance (Σ) :

$$FID = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2})$$

où r désigne les images réelles et g les images générées, et Tr est la trace de la matrice. Un score FID plus bas indique que la distribution des images générées est plus proche de la distribution réelle.

Les pièges de l'ingénieur :

Si le FID est élégant sur le papier, sa mise en œuvre est un champ de mines.

1. **Le biais de taille d'échantillon :** Le FID est un estimateur biaisé. Si vous calculez le FID sur 1 000 images, il sera systématiquement plus élevé (et donc plus pessimiste) que si vous le calculez sur 50 000 images, même avec le même modèle. La communauté a adopté le standard de 50 000 images pour les benchmarks, mais dans la pratique, lors de vos ablations rapides, utilisez toujours la *même* taille d'échantillon pour comparer vos modèles.

2. **La résolution et le prétraitement :** InceptionV3 attend des images de 299 × 299 pixels. Si votre modèle autoregressif génère du 256 × 256 (comme c'est souvent le cas pour des raisons de budget computationnel), le redimensionnement (resizing) introduit des artefacts d'interpolation qui dégradent le score. De plus, la normalisation des pixels doit correspondre exactement à celle utilisée lors de l'entraînement d'Inception. Une erreur de normalisation (par exemple, oublier de passer de l'intervalle [0, 1] à [-1, 1]) peut artificiellement faire exploser votre FID.

3. **L'aveuglement sémantique :** Le FID mesure le réalisme et la diversité globale, mais il ignore totalement le conditionnement textuel. Un modèle qui génère de superbes images de chiens alors que le prompt demande des chats obtiendra un excellent FID, tout en étant fonctionnellement inutile.

© 2026 Oussama Ghorbel

18.2 — Inception Score (IS) : diversité et netteté

Avant le FID, l'*Inception Score* (IS), proposé par Salimans et al. en 2016, était la référence. L'IS évalue deux propriétés souhaitables d'un générateur : la netteté (qualité) et la diversité.

L'IS utilise également InceptionV3, mais cette fois pour prédire la probabilité des classes ImageNet $p(y|x)$ pour une image générée x. L'idée est double :

1. **Netteté :** Pour une image donnée x, la distribution conditionnelle $p(y|x)$ doit avoir une faible entropie (le modèle doit être « sûr » de la classe, ce qui implique que l'image est nette et reconnaissable).
2. **Diversité :** La distribution marginale $p(y) = \int p(y|x)dx$ doit avoir une forte entropie (le modèle doit générer toutes les classes de manière équitable, et non pas toujours la même image).

Ces deux exigences sont combinées via la divergence de Kullback-Leibler (KL) :

$$IS = \exp(\mathbb{E}_{x \sim p_g}[D_{KL}(p(y|x) \| p(y))])$$

Pourquoi l'IS est aujourd'hui délaissé :

L'IS souffre de défauts majeurs qui ont conduit la communauté à lui préférer le FID. Premièrement, il ne compare jamais les images générées aux images réelles ; un modèle pourrait générer des images totalement irréalistes mais classifiables par Inception et obtenir un bon score. Deuxièmement, il est extrêmement vulnérable aux exemples antagonistes : du bruit structuré peut tromper InceptionV3 et produire un IS artificiellement élevé. Enfin, il dépend entièrement des classes d'ImageNet. Si vous entraînez un modèle sur des visages (CelebA) ou des IRM médicales, l'IS perd tout son sens car InceptionV3 n'a pas été entraîné à reconnaître ces domaines. Je vous recommande de ne l'utiliser que pour maintenir la compatibilité avec de très anciens articles de référence.

18.3 — CLIP-score : alignement sémantique texte-image, variantes

© 2026 Oussama Ghorbel

Puisque le FID et l'IS ignorent le texte, comment évaluer la capacité de notre LLM adapté à suivre les instructions du prompt ? C'est ici qu'intervient le CLIP-score.

CLIP (Contrastive Language-Image Pre-training) de Radford et al. (2021) projette les images et les textes dans un espace d'embedding commun. Le CLIP-score est simplement la similarité cosinus entre l'embedding de l'image générée et l'embedding du prompt textuel :

$$\text{CLIP-score} = \max(0, \cos(E_{img}, E_{txt}))$$

Un score élevé indique que le modèle a bien capturé la sémantique du prompt. C'est une métrique indispensable pour les modèles texte-vers-image.

Variantes et limitations :

Le CLIP-score standard (souvent basé sur ViT-B/32 ou ViT-L/14) a une fâcheuse tendance à favoriser les images aux couleurs saturées, à fort contraste, ou contenant du texte incrusté. C'est le phénomène de *reward hacking* : le modèle apprend que générer le mot "chien" en typographie blanche sur fond noir maximise la similarité avec le prompt "un chien", sans pour autant dessiner un vrai chien.

Pour pallier ces limites, la recherche a développé des alternatives et des extensions :

• **CLIPSim (ou CLIP Image-Image) :** Utile pour évaluer la cohérence d'une séquence vidéo ou la préservation de l'identité d'un sujet entre deux générations.

• **BLIP et BLIP-2 :** Ces modèles, conçus pour le *Visual Question Answering* (VQA) et le *captioning*, sont bien meilleurs pour évaluer la compositionnalité spatiale (ex: "un chat *sur* un tapis" vs "un tapis *sur* un chat"). On peut utiliser BLIP pour générer une légende de l'image produite, puis calculer le FID textuel ou la similarité entre la légende générée et le prompt original.

• **ImageReward et HPS (Human Preference Score) :** Ces modèles sont des couches de régression entraînées par-dessus les embeddings CLIP, calibrées sur des préférences humaines réelles (via RLHF ou classement par paires). Ils corrèlent beaucoup mieux avec le jugement humain esthétique que le CLIP-score brut.

© 2026 Oussama Ghorbel

18.4 — LPIPS et SSIM : similarité perceptive pour la reconstruction du tokenizer

Revenons un instant à la Partie II de cet ouvrage. Avant de générer des tokens avec un LLM, nous devons nous assurer que notre tokenizer visuel (VQGAN, VQ-VAE) est capable de reconstruire fidèlement une image à partir de ses codes discrets. Le FID n'est pas adapté ici, car nous ne cherchons pas à évaluer une distribution, mais la fidélité d'une reconstruction *pixel à pixel* ou *perception à perception*.

SSIM (Structural Similarity Index) :

Le SSIM compare la luminance, le contraste et la structure locale entre l'image originale x et la reconstruction $\hat{x}$. C'est une métrique locale et déterministe.

Cependant, le SSIM est impitoyable face aux décalages sémantiques mineurs. Si le tokenizer reconstruit parfaitement l'œil d'un visage mais avec un décalage d'un seul pixel vers la droite, le SSIM s'effondre (Erreur Quadratique Moyenne ou MSE élevée), alors que l'œil humain ne perçoit aucune différence. Le SSIM est utile pour le débogage bas niveau, mais il ne reflète pas la qualité perçue.

LPIPS (Learned Perceptual Image Patch Similarity) :

Le LPIPS, introduit par Zhang et al. (2018), a révolutionné l'évaluation des auto-encodeurs. Au lieu de comparer les pixels, le LPIPS fait passer les deux images à travers un réseau profond (comme VGG16 ou AlexNet), extrait les activations de plusieurs couches, les normalise, et calcule une distance pondérée.

$$LPIPS(x, \hat{x}) = \sum_{l} \frac{1}{H_l W_l} \sum_{h,\,w} \| \hat{y}^l_{hw} - y^l_{hw} \|_2^2$$

où y^l et $\hat{y}^l$ sont les caractéristiques de la couche l.

Le LPIPS est fortement corrélé avec le jugement humain. En pratique, un VQGAN avec un LPIPS bas (ex: < 0.1) produira des latents bien plus faciles à modéliser pour le LLM, car l'espace latent aura capturé l'essence sémantique plutôt que le bruit haute fréquence. Optimiser votre tokenizer pour minimiser le LPIPS (souvent via une

© 2026 Oussama Ghorbel

perte perceptive additionnée à la perte de reconstruction L1/L2) est une étape critique.

18.5 — Métriques de diversité : Recall, Coverage, mode collapse detection

Un FID excellent peut cacher une pathologie sévère : l'effondrement de mode (*mode collapse*). Imaginez un modèle qui a mémorisé 5 000 images parfaitement réalistes de son jeu d'entraînement et qui se contente de les recracher aléatoirement. Son FID sera très bas (les images sont réalistes et la variance est correcte), mais sa capacité générative est nulle. Il ne crée rien de nouveau.

Pour désintriquer la **qualité** (fidélité à la distribution réelle) de la **diversité** (couverture de la distribution réelle), la communauté s'est tournée vers les métriques de *Precision* et *Recall* (Kynkääniemi et al., 2019 ; Naeem et al., 2020).

Ces méthodes estiment les variétés (manifolds) des images réelles et générées dans l'espace latent d'un réseau pré-entraîné en utilisant des voisinages k-NN (k plus proches voisins).

- **Precision (Précision) :** La fraction des images générées qui tombent à l'intérieur de la variété des images réelles. C'est une mesure de la qualité et du réalisme.
- **Recall (Rappel) :** La fraction des images réelles dont le voisinage contient au moins une image générée. C'est une mesure de la diversité et de la couverture.

Si votre modèle a une haute Precision mais un Recall très faible, il souffre d'effondrement de mode : il génère peu de variations, mais elles sont de haute qualité. À l'inverse, un Recall élevé avec une Precision faible indique que le modèle explore tout l'espace (y compris le bruit et les artefacts) pour s'assurer de couvrir toutes les classes réelles.

La métrique de **Coverage** (Couverture) est une variante du Recall qui compte le nombre moyen de modes réels couverts par les échantillons générés, offrant une vue encore plus granulaire de la diversité. Intégrer le calcul de Precision/Recall dans vos pipelines d'évaluation est crucial lorsque vous modifiez l'architecture de votre

© 2026 Oussama Ghorbel

LLM ou votre stratégie d'échantillonnage (ex: passage du *top-k* au *nucleus sampling*).

18.6 — Évaluation humaine : protocoles, biais, reproductibilité

Aucune métrique algorithmique ne remplacera jamais totalement le jugement humain. Les métriques que nous venons de décrire sont des *proxies*. Le FID proxy du réalisme, le CLIP-score proxy de l'alignement sémantique. Mais l'art, l'esthétique, l'humour ou l'horreur échappent aux tenseurs. L'évaluation humaine reste le juge de paix, à condition d'être rigoureusement encadrée.

Protocoles standards :

1. **2AFC (Two-Alternative Forced Choice) :** On présente à un évaluateur deux images (une réelle, une générée, ou deux issues de modèles différents) côte à côte pour le même prompt, et on lui demande de choisir la meilleure ou d'identifier la vraie. C'est excellent pour mesurer le réalisme pur (Turing test visuel).

2. **MOS (Mean Opinion Score) :** L'évaluateur note l'image sur une échelle de Likert (ex: de 1 à 5) selon des critères précis : réalisme, esthétique, fidélité au prompt.

3. **Classement ELO (Head-to-Head) :** Inspiré des échecs, les modèles sont mis en compétition par paires sur des prompts aléatoires. Les préférences humaines mettent à jour le score ELO de chaque modèle. C'est la méthode utilisée par des plateformes comme *Artificial Analysis* ou *LMSYS* pour les LLM textuels, et elle s'applique parfaitement à la génération d'images.

Biais et reproductibilité :

L'évaluation humaine est coûteuse, lente et sujette à des biais cognitifs massifs. Le *biais de position* (tendance à préférer l'image de gauche), le *biais de saturation* (les images aux couleurs vives attirent l'œil en première seconde), et la fatigue de l'annotateur faussent les résultats.

Pour garantir la reproductibilité, je vous conseille de figer un jeu de test de 500 prompts (couvrant des scènes simples, complexes, des tests

© 2026 Oussama Ghorbel

de compositionnalité et de rendu textuel), d'utiliser des plateformes d'annotation crowdsourcées avec des questions de contrôle (attention checks), et de toujours rapporter les intervalles de confiance (bootstrapping) de vos scores humains. Ne lancez une évaluation humaine que pour les jalons majeurs de votre recherche ; pour le développement quotidien, fiez-vous à votre tableau de bord automatisé.

18.7 — Construire un tableau de bord d'évaluation automatisé

L'ingénierie de l'apprentissage automatique (MLOps) exige que l'évaluation soit automatisée, reproductible et intégrée au cycle de vie du modèle. Un chercheur ne devrait jamais avoir à lancer manuellement un script de 4 heures pour connaître le FID de son dernier checkpoint.

Voici l'architecture d'un pipeline d'évaluation robuste que je recommande pour un modèle autoregressif de type Token-to-Pixel :

1. **Le Trigger :** À la fin de chaque époque d'entraînement (ou toutes les *N* étapes), ou lors d'un commit sur la branche `main`, un job d'évaluation est déclenché sur un cluster GPU.

2. **La Génération :** Le script charge le dernier checkpoint, tire 10 000 prompts aléatoires d'un jeu de validation figé, et génère les images. Les paramètres d'inférence (température, top-k, classifier-free guidance) doivent être strictement versionnés.

3. **Le Calcul des Métriques :**

• *FID & IS :* Utilisation de bibliothèques optimisées comme `clean-fid` ou `torchmetrics.image.fid`. `clean-fid` est particulièrement recommandé car il gère correctement les espaces de couleurs (sRGB vs linéaire) et les redimensionnements, éliminant les écarts artificiels entre les papiers de recherche.

• *CLIP-score :* Chargement d'un modèle CLIP en mode inférence (batch processing sur GPU) pour évaluer les 10 000 paires prompt-image.

• *Diversité :* Calcul d'un sous-échantillon de Precision/Recall sur 5 000 images.

4. **Le Reporting (Dashboard) :** Les métriques sont poussées vers un outil de suivi d'expériences comme Weights & Biases (W&B) ou

© 2026 Oussama Ghorbel

MLflow.

• Créez des panneaux personnalisés : une courbe montrant l'évolution du FID et du CLIP-score en fonction des étapes d'entraînement.

• *Galerie d'artefacts :* Le script sélectionne aléatoirement 16 images, les concatène en une grille avec leurs prompts en surimpression, et les envoie comme image dans W&B. Cela permet de repérer visuellement les artefacts (ex: apparition de grilles de tokens mal décodées) que les métriques scalaires ne capturent pas.

5. **Alerting :** Si le FID dépasse un seuil critique ou si le CLIP-score chute de plus de 5 % par rapport au run précédent, une alerte (Slack, email) est envoyée à l'équipe pour signaler une régression potentielle (ex: instabilité de la loss, effondrement de l'attention).

Automatiser ce processus transforme l'évaluation d'une corvée de fin de projet en un signal continu, vous permettant d'itérer sur vos architectures de LLM et vos tokenizers avec une confiance absolue dans vos résultats.

Exercices de fin de chapitre

Pour consolider les concepts abordés dans ce chapitre, je vous propose trois exercices pratiques. Ils sont conçus pour être réalisés avec Python, PyTorch et les bibliothèques standards de l'écosystème (HuggingFace, `clean-fid`, `torchmetrics`).

Exercice 1 : Calculer le FID sur CIFAR-10

Objectif : Comprendre la manipulation des tenseurs et l'impact de la taille d'échantillon sur le FID.

1. Téléchargez le jeu de données CIFAR-10. Sélectionnez 1 000 images réelles de la classe "avion".

2. Générez 1 000 images "jouet" en ajoutant un bruit gaussien croissant à 1 000 autres images réelles de la classe "avion" (pour simuler un générateur imparfait).

3. Utilisez la bibliothèque `clean-fid` (ou `torchmetrics`) pour calculer le FID entre ces deux ensembles.

© 2026 Oussama Ghorbel

4. **Question :** Recomputez le FID en n'utilisant que 100 images de chaque ensemble. De combien le score a-t-il varié ? Expliquez pourquoi en vous basant sur le biais de l'estimateur de la matrice de covariance.

5. **Indice :** Assurez-vous de convertir correctement les images PIL en tenseurs normalisés attendus par InceptionV3 (`transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`).

Exercice 2 : Implémentation d'un évaluateur CLIP-score

Objectif : Maîtriser l'inférence par batch avec des modèles multimodaux et évaluer l'alignement sémantique.

1. Créez un jeu de test de 50 paires (prompt, image). Vous pouvez utiliser des images de MS-COCO et leurs légendes associées, ou générer vos propres images avec un modèle open-source (ex: Stable Diffusion 1.5 via `diffusers`).

2. Chargez le modèle `openai/clip-vit-base-patch32` via la bibliothèque `transformers` de HuggingFace.

3. Écrivez une fonction `compute_clip_score(prompts, images)` qui :

• Tokenise les textes (attention à la limite de 77 tokens de CLIP).

• Prétraite les images (resize à 224x224, normalisation spécifique à CLIP).

• Calcule les embeddings, normalise les vecteurs, et retourne la similarité cosinus moyenne.

4. **Question :** Modifiez artificiellement 5 images en y incrustant le texte du prompt (via PIL.ImageDraw). Comment le CLIP-score réagit-il ? Que cela vous apprend-il sur la robustesse de la métrique face au *reward hacking* typographique ?

Exercice 3 : Le rapport PDF d'évaluation automatisé

Objectif : Concevoir un livrable d'ingénierie complet pour le suivi des expériences.

1. Écrivez un script Python qui prend en entrée un dossier contenant des images générées et un fichier JSON associant chaque nom de fichier à son prompt.

© 2026 Oussama Ghorbel

2. Le script doit calculer :

• Le FID (en supposant un dossier d'images de référence fourni).

• Le CLIP-score moyen.

• L'Inception Score.

3. Utilisez une bibliothèque comme `reportlab` ou `matplotlib` (sauvegardé en PDF via `PdfPages`) pour générer un rapport structuré d'une ou deux pages contenant :

• Un résumé exécutif avec les 3 métriques scalaires.

• Une grille de 16 images échantillons (4x4) avec leur prompt affiché en dessous et leur CLIP-score individuel.

• Un histogramme de la distribution des CLIP-scores pour identifier les prompts "pathologiques" (ceux que le modèle échoue systématiquement à comprendre).

4. **Livrable :** Un script exécutable en ligne de commande (`python eval_report.py --gen_dir ./outputs --ref_dir ./real --out report.pdf`) qui pourrait être directement intégré dans un pipeline CI/CD.

18.8 — Le déclin d'Inception et l'avènement des fondations visuelles (FDD, SigLIP)

Il est temps d'aborder une vérité qui dérange dans la littérature de la génération d'images : InceptionV3 est un réseau de 2015. Bien qu'il ait révolutionné l'évaluation à l'époque en introduisant le FID et l'IS, son architecture et son entraînement (supervisé sur les 1 000 classes d'ImageNet) le rendent aujourd'hui partiellement aveugle aux subtilités des images synthétiques modernes. InceptionV3 a tendance à ignorer les artefacts de haute fréquence, les incohérences structurelles locales et les défauts spécifiques aux tokenizers visuels (comme les grilles de patchs mal décodées). Pire encore, il récompense souvent la "texture" au détriment de la "géométrie".

Pour pallier ces limites, la recherche s'est massivement tournée vers les modèles de fondation visuels (Visual Foundation Models) pour extraire les caractéristiques de référence. Deux approches dominent désormais les protocoles d'évaluation de pointe :

1. La Fréchet DINOv2 Distance (FDD) :

© 2026 Oussama Ghorbel

DINOv2, entraîné par auto-supervision sur des milliards d'images, possède une compréhension de la structure spatiale et sémantique infiniment supérieure à celle d'Inception. Contrairement à CLIP qui a tendance à "aplatisir" l'image en un vecteur global pour la comparer à du texte, les *patch tokens* de DINOv2 conservent la topologie de l'image.

En remplaçant InceptionV3 par le backbone ViT-L/14 de DINOv2 pour extraire les vecteurs de caractéristiques avant de calculer la distance de Fréchet, on obtient la FDD. Les expériences d'ablation menées lors de l'adaptation de LLM à la génération visuelle montrent que la FDD corrèle beaucoup mieux avec le jugement humain concernant la cohérence anatomique (ex: le nombre correct de doigts ou de pattes) et la structure 3D des scènes.

2. SigLIP et les métriques sans Softmax :

CLIP standard utilise une fonction de perte basée sur le softmax sur un batch d'images et de textes, ce qui force une compétition artificielle au sein du batch et peut saturer les similarités. SigLIP (Sigmoid Loss for Language Image Pre-Training) remplace ce softmax par une loss sigmoïde binaire indépendante. Les embeddings de SigLIP sont bien plus calibrés pour évaluer la similarité absolue et locale. Utiliser SigLIP pour calculer un *SigLIP-FID* ou un *SigLIP-score* offre une évaluation bien plus robuste, particulièrement pour les modèles génératifs où les variations subtiles de composition doivent être pénalisées ou récompensées sans être noyées dans le bruit du batch.

18.9 — Évaluer la compositionnalité spatiale : le talon d'Achille de l'autorégression

L'un des défis majeurs de la modélisation autorégressive d'images (que nous approfondirons dans le Chapitre 19) est le biais inductif lié à l'ordre de balayage. Si votre LLM génère des tokens dans un ordre *raster* (de gauche à droite, de haut en bas), il "dessine" le haut de l'image avant le bas. Face à un prompt comme *"un chien sous un parapluie rouge"*, le modèle doit anticiper la position du chien avant même d'avoir généré les tokens du parapluie. Cette contrainte spatiale stricte conduit souvent à des échecs de compositionnalité : le modèle générera un chien *à côté* d'un parapluie, ou fusionnera les deux concepts.

© 2026 Oussama Ghorbel

Le FID et le CLIP-score sont totalement impuissants face à ce problème. Le CLIP-score donnera un score presque identique pour *"un chien sous un parapluie"* et *"un parapluie sous un chien"*, car son espace latent est fondamentalement "bag-of-words" (il reconnaît la présence des concepts, mais pas leur agencement).

Le protocole d'évaluation compositionnelle :

Pour évaluer rigoureusement la capacité de votre LLM à respecter la syntaxe spatiale du prompt, vous devez construire un pipeline basé sur la détection d'objets *open-vocabulary* (comme Grounding DINO ou GLIP). Voici comment je recommande de structurer ce test :

1. **Le Dataset de Test :** Générez un ensemble de 1 000 prompts hautement compositionnels (inspirés de benchmarks comme *T2I-CompBench* ou *DrawBench*). Ces prompts doivent inclure des relations spatiales explicites (*sur, sous, à gauche de*), des attributs de couleur liés à des objets spécifiques (*un cube rouge et une sphère bleue*), et des tests de négation (*une rue sans voitures*).

2. **L'Extraction de Bounding Boxes :** Pour chaque image générée, utilisez Grounding DINO pour localiser les objets mentionnés dans le prompt.

3. **Le Calcul du Score de Cohérence Spatiale (Spatial Binding Score) :**

• Pour les relations spatiales, calculez les centres géométriques des boîtes englobantes (x, y). Si le prompt demande "A à gauche de B", vérifiez si $x_A < x_B$.

• Pour les attributs (couleur), utilisez un modèle de *Visual Question Answering* (VQA) comme BLIP-2 pour interroger la région délimitée par la bounding box : "Quelle est la couleur de cet objet ?".

4. **La Métrique Finale :** Le score de compositionnalité est le pourcentage de relations spatiales et d'attributs correctement respectés sur l'ensemble du jeu de test.

C'est une métrique lourde en calcul (elle nécessite l'inférence de modèles de détection et de VQA sur chaque image générée), mais elle est absolument indispensable pour diagnostiquer les limites de votre tokenizer et de votre ordre de balayage.

© 2026 Oussama Ghorbel

18.10 — De la métrique à la récompense : RLHF, DPO et la loi de Goodhart

L'évaluation ne sert pas uniquement à mesurer les performances d'un modèle figé ; elle est le moteur de son alignement. Lorsque vous adaptez un LLM textuel (comme LLaMA ou Mistral) pour qu'il devienne un générateur visuel, l'entraînement supervisé standard (next-token prediction sur des paires texte-image) ne suffit pas à garantir l'esthétique ou l'absence d'artefacts. Il faut aligner le modèle via l'Apprentissage par Renforcement avec Rétroaction Humaine (RLHF) ou l'Optimisation Directe des Préférences (DPO).

Pour ce faire, vos métriques d'évaluation doivent être transformées en **Fonctions de Récompense (Reward Models)**.

Construire une fonction de récompense composite :

Une récompense scalaire $R(x, y)$ pour une image générée x et un prompt y est généralement une combinaison linéaire pondérée de nos métriques :

$$R(x, y) = w_1 \cdot \text{CLIP-Score}(x, y) + w_2 \cdot \text{HPS}(x) - w_3 \cdot \text{Penalty}_{\text{artefacts}}(x)$$

où HPS est le *Human Preference Score* (pour l'esthétique) et la pénalité peut être un classificateur binaire entraîné pour détecter les anomalies anatomiques ou les grilles de tokens mal décodées.

Le piège mortel : La Loi de Goodhart

En économie, la loi de Goodhart stipule que *"lorsqu'une mesure devient un objectif, elle cesse d'être une bonne mesure"*. En apprentissage automatique, c'est une catastrophe quotidienne. Si vous utilisez le CLIP-score comme fonction de récompense unique pour entraîner votre LLM via PPO (Proximal Policy Optimization), le modèle va rapidement découvrir des *exploits* (des failles) dans l'espace latent de CLIP.

Il est fréquemment observé, et la littérature abonde d'exemples similaires, des modèles qui apprennent à incrustéer le texte du prompt en filigrane sur l'image, ou à générer systématiquement des signatures d'artistes connus (ex: "by Greg Rutkowski") ou des logos de banques d'images ("ArtStation", "Getty Images"). Pourquoi ? Parce que dans

© 2026 Oussama Ghorbel

l'espace d'embedding de CLIP, ces éléments textuels et ces signatures sont statistiquement très proches des concepts de "haute qualité", "détaillé" et "artistique". Le modèle maximise sa récompense mathématique tout en produisant des images fonctionnellement inutilisables pour un utilisateur final.

Comment s'en prémunir ?

1. **Utiliser un Reward Model entraîné spécifiquement :** Plutôt que d'utiliser le CLIP-score brut, entraînez une tête de régression sur les embeddings CLIP/DINOv2 en utilisant des dizaines de milliers de comparaisons par paires (A vs B) effectuées par des humains. C'est le principe de l'ImageReward.

2. **Pénalités de divergence (KL Penalty) :** Comme pour les LLM textuels, ajoutez une pénalité de divergence de Kullback-Leibler entre la distribution des tokens du modèle aligné et celle du modèle de référence (SFT - Supervised Fine-Tuning). Cela empêche le modèle de s'éloigner trop radicalement de la distribution naturelle des images pour aller "hacker" la fonction de récompense.

3. **Évaluation hors distribution (OOD) :** Testez régulièrement votre modèle sur des prompts qui ne contiennent *aucun* mot-clé esthétique ("une photo floue d'un mur en briques", "un croquis rapide au crayon"). Si votre modèle "refuse" de générer une image de basse qualité esthétique même quand le prompt l'exige explicitement, c'est que votre fonction de récompense a sur-optimisé l'esthétique au détriment du suivi d'instruction (instruction following).

Conclusion du chapitre 18

L'évaluation d'un générateur d'images basé sur un Grand Modèle de Langage n'est pas une simple formalité de fin de pipeline ; c'est la boussole qui dicte chaque décision architecturale. Le FID (et son successeur moderne, la FDD) vous renseigne sur la santé globale de votre distribution et l'efficacité de votre tokenizer. Le CLIP-score et ses dérivés compositionnels vous indiquent si votre LLM a véritablement appris à "comprendre" le texte pour guider les pixels, ou s'il se contente de recracher des motifs statistiques mémorisés. Enfin, le LPIPS et les

© 2026 Oussama Ghorbel

métriques de diversité (Precision/Recall) vous protègent contre l'effondrement de mode et les artefacts de reconstruction.

Construire un tableau de bord automatisé, intégrant ces métriques à chaque étape de votre cycle d'entraînement, est ce qui transforme une expérience de recherche hasardeuse en une ingénierie reproductible. Cependant, gardez toujours à l'esprit que ces métriques sont des cartes, pas le territoire. La carte du FID peut vous dire que vous êtes dans une vallée de haut réalisme, mais seule l'évaluation humaine et l'analyse qualitative des artefacts vous diront si cette vallée est habitable.

Maintenant que nous disposons des outils pour mesurer rigoureusement la qualité, la diversité et l'alignement de nos générations, nous sommes prêts à affronter la question qui divise la communauté de l'apprentissage automatique depuis l'avènement de Stable Diffusion et de DALL-E 2. Dans le chapitre suivant, nous mettrons nos métriques à l'épreuve pour une comparaison approfondie et sans concession : l'approche autoregressive par tokens, que nous défendons dans cet ouvrage, est-elle réellement capable de rivaliser avec les modèles de diffusion en termes de fidélité, de coût computationnel et de flexibilité ? Les chiffres, et notre tableau de bord, vont parler.

© 2026 Oussama Ghorbel

Chapitre 19 – Comparaison approfondie : autoregression par tokens vs diffusion

Au fil de notre parcours dans cet ouvrage, nous avons exploré comment un modèle de langage, initialement conçu pour prédire le mot suivant, peut être détourné, adapté et étendu pour prédire le pixel suivant — ou plus précisément, le token visuel suivant. Mais cette approche, bien que séduisante par son élégance théorique et son unification conceptuelle, se heurte à une réalité industrielle implacable : l'existence d'un paradigme concurrent, massivement adopté et optimisé, celui des modèles de diffusion.

En tant qu'ingénieurs et chercheurs, nous ne pouvons pas nous permettre de choisir une architecture sur la base d'une simple préférence esthétique ou d'une volonté d'unification à tout prix. Le choix entre l'autoregression sur tokens discrets et la diffusion sur des latents continus dictera la topologie de votre cluster GPU, la latence de votre API, la fidélité de vos générations et la flexibilité de vos outils de contrôle. Dans ce chapitre, je vous propose de mettre ces deux géants face à face, d'analyser leurs compromis fondamentaux et de définir un arbre de décision rationnel pour vos futurs projets.

19.1 — Qualité : FID comparable à grande échelle, différences à petite échelle

Pendant longtemps, la communauté de la vision par ordinateur a considéré l'autoregression sur tokens comme une approche de seconde zone pour la génération d'images haute fidélité. Les premiers modèles, comme VQGAN couplé à un Transformer, peinaient à descendre sous un score FID (Fréchet Inception Distance) de 3 ou 4 sur ImageNet 256×256, là où les modèles de diffusion (DDPM, LDM) atteignaient rapidement des scores inférieurs à 2.

Pourquoi cette disparité ? La réponse réside dans la nature même des fonctions de perte et des espaces de représentation. La diffusion opère dans un espace continu (ou latent continu) et optimise une erreur quadratique moyenne (MSE) ou une perte de type score-matching. Cette approche a tendance à « moyenner » les modes de la distribution de

© 2026 Oussama Ghorbel

données. Le résultat est une image globalement cohérente, structurellement juste, mais qui peut souffrir d'un léger flou ou d'un manque de netteté texturale à haute fréquence. L'autoregression, en revanche, opère sur un espace discret (le codebook). La perte d'entropie croisée (cross-entropy) force le modèle à choisir un mode précis parmi les tokens disponibles. Cela élimine le flou de moyennage, produisant des textures extrêmement nettes, mais introduit un risque majeur : la confusion sémantique et les artefacts de quantification. Si le modèle se trompe de token, l'erreur est catégorique, pas progressive.

Cependant, à grande échelle, la donne change radicalement. Des architectures comme Parti (3B paramètres), LlamaGen ou plus récemment VAR (Visual Autoregressive Modeling) ont démontré que l'autoregression sur tokens, lorsqu'elle est dotée d'un budget computationnel massif et de codebooks hiérarchiques ou de très haute qualité, peut égaler, voire surpasser, la diffusion sur les métriques FID et IS (Inception Score). À petite échelle (modèles < 300M paramètres), la diffusion reste supérieure car elle tire mieux parti de l'information spatiale continue. À grande échelle (> 1B paramètres), l'autoregression bénéficie pleinement des lois d'échelle des Transformers, capturant des distributions sémantiques complexes que la diffusion peine parfois à modéliser sans un conditionnement textuel lourd.

19.2 — Vitesse : O(N) steps autoregressifs vs O(T) steps de diffusion

C'est sur le terrain de la vitesse d'inférence que le fossé architectural est le plus profond, et c'est souvent ici que se joue la décision de déploiement en production.

L'autoregression est intrinsèquement séquentielle. Pour générer une image de 256×256 patchifiée en tokens de 16×16, le modèle doit effectuer $N = 256$ passes avant (forward passes) séquentielles. Pour une résolution de 1024×1024, ce nombre explose à $N = 4096$ (ou plus selon le taux de compression du tokenizer). Chaque étape i nécessite de calculer l'attention sur les $i - 1$ tokens précédents. Grâce au mécanisme de KV-cache, le coût computationnel par étape est constant (si l'on fait abstraction de la croissance de l'attention), mais le coût total en FLOPs scales de manière quadratique $O(N^2)$ par rapport à la longueur de la

© 2026 Oussama Ghorbel

séquence. Le temps de génération est donc directement proportionnel au nombre de tokens.

La diffusion, à l'inverse, est itérative mais parallèle spatialement. Le modèle génère l'intégralité de la grille de latents en T étapes de débruitage, où T est typiquement compris entre 20 et 50 avec les échantillonneurs modernes (DPM-Solver, DDIM). À chaque étape t, le réseau (U-Net ou DiT) traite l'intégralité de la carte spatiale en parallèle. Le coût total est donc $O(T \times \text{Coût}_{\text{réseau}})$.

Comparons les ordres de grandeur : 4096 passes séquentielles pour l'AR contre 30 passes parallèles pour la diffusion. Sur le papier, la diffusion écrase l'AR en temps total de génération. Cependant, l'analyse matérielle (roofline model) nuance ce tableau. Les passes de diffusion sont massivement limitées par le calcul (compute-bound) : les multiplications de matrices denses saturent les cœurs Tensor des GPU. Les passes autoregressives, en particulier vers la fin de la séquence où le KV-cache est volumineux, sont limitées par la bande passante mémoire (memory-bound) : le GPU passe son temps à charger les clés et les valeurs depuis la HBM plutôt qu'à calculer.

Pour accélérer l'AR, les chercheurs explorent le décodage spéculatif, la génération parallèle (Jacobi decoding) ou les modèles semi-autoregressifs. Pour la diffusion, la distillation de modèle (LCM, SDXL-Turbo) permet de réduire T à 4 ou 8 étapes, voire 1 (consistency models), creusant encore l'écart en latence pure.

19.3 — Mémoire : KV-cache croissant vs débruitage itératif à taille fixe

La gestion de la mémoire VRAM est le cauchemar quotidien de l'ingénieur en déploiement. Ici encore, les deux paradigmes présentent des profils d'occupation radicalement différents.

Dans un LLM autoregressif, la mémoire est dominée par le KV-cache. Pour chaque couche du Transformer, pour chaque tête d'attention, le modèle doit stocker les vecteurs de clés (Keys) et de valeurs (Values) de tous les tokens générés jusqu'à présent. La formule du KV-cache pour une séquence de longueur N, avec L couches, une dimension cachée D et une précision FP16 (2 octets) est :

© 2026 Oussama Ghorbel

$$\text{Mémoire}_{\text{KV}} = 2 \times L \times D \times N \times 2 \text{ octets}$$

Cette empreinte mémoire croît linéairement avec N. Pour un modèle de 7B paramètres (ex: LLaMA 2) générant une séquence de 4096 tokens visuels, le KV-cache seul peut consommer plusieurs gigaoctets de VRAM par requête, limitant drastiquement la taille du batch (batch size) que le serveur peut accepter simultanément.

La diffusion, en revanche, opère à taille fixe. Le modèle de débruitage prend en entrée un tenseur de latents de taille $[B, C, H, W]$ et produit un tenseur de même taille. La mémoire d'inférence est dominée par les poids du modèle et les activations intermédiaires des couches convolutionnelles ou d'attention spatiale. Puisque la résolution spatiale des latents est généralement faible (ex: 128×128 pour une image de 1024×1024 avec un facteur de compression de 8), l'empreinte mémoire des activations est modeste et, surtout, elle ne croît pas au fil des étapes T.

Cela signifie qu'en inférence, un pipeline de diffusion peut supporter des tailles de batch beaucoup plus importantes qu'un pipeline autoregressif sur la même carte GPU, maximisant ainsi le débit (throughput) global du serveur. L'AR souffre d'un goulot d'étranglement mémoire qui s'aggrave à mesure que la résolution (et donc le nombre de tokens) augmente.

19.4 — Contrôle : inpainting, outpainting, layout — facilité relative

La génération d'images ne se limite plus à la création ex nihilo à partir d'un prompt textuel. Les applications industrielles exigent un contrôle fin : inpainting (remplir un masque), outpainting (étendre les bords), et guidage spatial (layout, ControlNet).

La diffusion excelle nativement dans ces tâches. L'inpainting par diffusion est d'une élégance redoutable : il suffit d'ajouter du bruit aux pixels masqués à l'étape t, de laisser les pixels non masqués intacts, et de laisser le réseau prédire le bruit. Les propriétés des SDE (Stochastic Differential Equations) garantissent que la transition entre la zone masquée et la zone conservée sera harmonieuse. L'outpainting et le guidage par croquis (ControlNet) s'intègrent tout aussi naturellement via

© 2026 Oussama Ghorbel

des mécanismes de conditionnement spatialisé ou d'injection de features dans les couches croisées (cross-attention).

L'autoregression sur tokens, régie par un masque causal strict (le token i ne peut voir que les tokens $< i$), est fondamentalement hostile à l'inpainting spatial arbitraire. Si vous voulez modifier le centre d'une image déjà générée, le masque causal est violé : les tokens du bas de l'image dépendent du centre, mais le centre dépendrait des bords dans un scénario de remplissage. Pour contourner cela, les modèles AR doivent recourir à des astuces coûteuses : génération séquentielle par blocs, utilisation de tokens spéciaux de « masquage » (comme dans les modèles de type BERT ou MaskGIT), ou abandon pur de l'autoregression stricte au profit de l'attention bidirectionnelle pour les tâches d'édition.

Si votre cas d'usage exige des outils d'édition spatiale robustes et interactifs (comme un éditeur type Photoshop), la diffusion offre un avantage architectural massif. L'AR est mieux adapté à la génération conditionnée par des séquences (texte, bounding boxes séquentielles, ou génération layout-to-image où le layout est fourni en préfixe de la séquence).

19.5 — Scalabilité : lois d'échelle (scaling laws) pour chaque paradigme

L'un des arguments les plus puissants en faveur de l'adaptation d'un LLM en générateur d'images réside dans les lois d'échelle (scaling laws). Nous savons, depuis les travaux de Kaplan et l'approche Chinchilla, comment allouer de manière optimale le budget computationnel entre la taille du modèle et le nombre de tokens d'entraînement pour les LLM.

En transposant l'image au statut de token, nous héritons directement de ce cadre théorique. Des études récentes ont montré que les Transformers autoregressifs appliqués aux tokens visuels suivent des lois d'échelle prévisibles en fonction de la perte d'entropie croisée. Si vous doublez le calcul, vous savez exactement quelle amélioration de FID attendre, et comment ajuster la taille du codebook et du modèle en conséquence. Cette prévisibilité est un atout majeur pour les laboratoires disposant de budgets massifs, car elle réduit le risque empirique lors du passage à l'échelle (scaling up).

© 2026 Oussama Ghorbel

La diffusion a longtemps semblé échapper à ces lois claires, la qualité dépendant d'un équilibre complexe entre le bruit, l'architecture U-Net, et le schedule de diffusion. Cependant, l'avènement des Diffusion Transformers (DiT) et des formulations comme EDM (Elucidating the Design Space of Diffusion-Based Generative Models) a changé la donne. Les chercheurs ont démontré que, pour un budget de calcul donné, augmenter la profondeur du Transformer et le nombre d'itérations d'entraînement suit une courbe de puissance (power law) très similaire à celle des LLM.

La différence fondamentale réside dans le « rendement » du calcul. L'AR bénéficie de l'optimisation extrême des bibliothèques de LLM (FlashAttention, Megatron-LM, vLLM), ce qui rend l'entraînement de modèles de 10B+ paramètres sur des milliers de GPU beaucoup plus mature et stable que l'entraînement de DiT géants, où l'ingénierie distribuée est encore en cours de consolidation.

19.6 — Hybrides récents : Transfusion, MAR, Chameleon — le meilleur des deux mondes

Face à ces compromis, la frontière entre AR et diffusion s'estompe. La recherche récente ne cherche plus à opposer les deux paradigmes, mais à les fusionner au sein d'une même architecture, capitalisant sur les forces de chacun.

Transfusion, proposé par Meta, est une avancée conceptuelle majeure. L'idée est d'entraîner un seul et unique Transformer à traiter du texte et des images, mais en utilisant des fonctions de perte distinctes selon la modalité. Pour les tokens de texte, le modèle est entraîné par maximum de vraisemblance (NLL, autoregression classique). Pour les patches d'image, le modèle est entraîné avec une perte de diffusion (prédiction du bruit sur des représentations latentes continues). Cela permet au modèle de générer du texte token par token, tout en « débruitant » l'image en parallèle, contournant ainsi la lenteur de l'AR pour la vision tout en conservant l'unification du Transformer.

MAR (Masked Autoregressive Image Generation) adopte une approche différente. Plutôt que de générer les tokens de gauche à droite, MAR génère l'image en plusieurs étapes de démasquage, préditant de multiples tokens à la fois. Cela réduit le nombre de passes avant (se

© 2026 Oussama Ghorbel

rapprochant du T de la diffusion) tout en conservant la modélisation sur des tokens discrets et la perte d'entropie croisée, évitant ainsi le flou inhérent à la MSE.

Chameleon, également issu des laboratoires de Meta, pousse l'early fusion à son paroxysme en traitant les images comme de simples tokens discrets dès la première couche, mélangés indifféremment avec du texte. Bien qu'il utilise l'autoregression pure, son tokenizer et son architecture optimisée démontrent qu'avec suffisamment de données et de paramètres, l'AR peut rivaliser avec la diffusion en qualité, tout en offrant une capacité de compréhension multimodale (vision-language) native que les pipelines de diffusion (qui séparent le encodeur CLIP du générateur) peinent à égaler en termes de raisonnement spatial profond.

Ces modèles hybrides représentent, à mon sens, l'avenir de la génération. Ils prouvent que le choix n'est plus binaire, mais qu'il s'agit de concevoir des objectifs de perte adaptatifs au sein d'une même colonne vertébrale attentionnelle.

19.7 — Tableau récapitulatif et arbre de décision

Pour synthétiser ces analyses, voici un tableau comparatif des deux paradigmes dans leur forme standard (AR sur tokens discrets vs Diffusion sur latents continus) :

Critère	Autoregression (Tokens Discrets)	Diffusion (Latents Continus)
Qualité (FID)	Excellente à très grande échelle, textures nettes, risque d'artefacts de quantification.	Excellente à toutes les échelles, cohérence globale supérieure, léger flou possible.
Vitesse (Inférence)	Lente. $O(N)$ passes séquentielles. Limité par la bande passante mémoire (KV-cache).	Rapide. $O(T)$ passes parallèles ($T \approx 20 - 50$). Limité par le calcul (FLOPs).
Mémoire (VRAM)	Élevée et croissante (KV-cache). Limite la taille du batch.	Modeste et fixe (activations spatiales). Permet de grands batches.
Contrôle Spatial	Difficile (inpainting natif contre-intuitif avec masque causal).	Natif et trivial (SDEdit, masquage, ControlNet).

© 2026 Oussama Ghorbel

Unification Texte/Image	Native et élégante (même vocabulaire, même perte NLL).	Complexe (nécessite un encodeur de texte séparé et cross-attention).
Écosystème & Outils	Hérite de l'écosystème LLM (vLLM, quantification, LoRA).	Écosystème vision riche (A1111, ComfyUI, ControlNet, IP-Adapter).

Arbre de décision pour l'ingénieur :

1. **Votre objectif principal est-il l'édition spatiale fine (inpainting, outpainting, contrôle strict par croquis) ?**

• *Oui* → Choisissez la **Diffusion**.

• *Non* → Passez à la question 2.

2. **Disposez-vous d'un LLM pré-entraîné massif (ex: LLaMA, Mistral) que vous souhaitez étendre à la vision sans ré-entraîner depuis zéro ?**

• *Oui* → Choisissez l'**Autoregression sur tokens** (fine-tuning du LLM sur des tokens visuels).

• *Non* → Passez à la question 3.

3. **La latence de génération (Time-to-First-Image) est-elle le critère bloquant pour votre application (ex: génération en temps réel sur mobile ou web) ?**

• *Oui* → Choisissez la **Diffusion** (avec distillation type LCM/Turbo).

• *Non* → Passez à la question 4.

4. **Votre application nécessite-t-elle un raisonnement multimodal complexe (compréhension et génération dans le même prompt, agents autonomes) ?**

• *Oui* → Choisissez l'**Autoregression** (ou un hybride type Transfusion).

• *Non* → La **Diffusion** reste le choix par défaut pour la pure génération esthétique.

Exercices pratiques

© 2026 Oussama Ghorbel

Pour ancrer ces concepts et vous préparer aux réalités du déploiement, je vous invite à réaliser les trois exercices suivants. Ils sont conçus pour tester votre intuition mathématique et votre capacité de synthèse architecturale.

Exercice 1 : Tableau comparatif chiffré (Estimations)

Remplissez le tableau ci-dessous avec des estimations réalistes pour un modèle de taille équivalente (environ 1B de paramètres) générant une image. Justifiez vos ordres de grandeur.

Hypothèses : AR avec tokenizer de compression 16x (256 tokens pour 256x256, 4096 tokens pour 1024x1024). Diffusion avec compression 8x (latents 32x32 et 128x128) et $T = 30$ étapes.

Métrique	AR (256×256)	AR (1024×1024)	Diffusion (256×256)	Diffusion (1024×1024)
FLOPs approx. par image	?	?	?	?
Mémoire KV-cache / Activations (FP16)	?	?	?	?
Latence estimée sur 1x A100	?	?	?	?

Exercice 2 : Paragraphe de synthèse

Rédigez un paragraphe de synthèse (environ 150-200 mots) répondant à la question suivante, en adoptant le ton d'un lead engineer justifiant un choix technique auprès de son CTO :

« Dans quel cas choisir l'autoregression sur tokens pour un LLM existant, malgré le désavantage apparent en vitesse et en mémoire par rapport à la diffusion ? »

Consigne : Mettez en avant les notions d'unification des espaces latents, de raisonnement séquentiel et d'infrastructure MLOps.

Exercice 3 : Simulation du coût mémoire du KV-cache

© 2026 Oussama Ghorbel

Le goulot d'étranglement de l'autoregression réside dans le KV-cache. Calculez l'empreinte mémoire exacte du KV-cache pour un Transformer de 32 couches, avec une dimension cachée de 4096 (et 32 têtes d'attention, bien que la formule globale dépende de la dimension totale), en précision FP16 (2 octets par valeur).

Utilisez la formule simplifiée pour le KV-cache total (Keys + Values pour toutes les couches) :

$$\text{Taille (octets)} = 2 \times \text{Nb_couches} \times \text{Dim_cachée} \times N \times 2$$

(Note : le premier facteur 2 représente Keys et Values, le dernier facteur 2 représente les octets en FP16).

Calculez cette taille en Mégaoctets (Mo) pour les longueurs de séquence N suivantes, correspondant à différentes résolutions d'images ou longueurs de contexte :

1. $N = 256$ (Image basse résolution ou petit patch)
2. $N = 1\,024$ (Image moyenne résolution)
3. $N = 4\,096$ (Image haute résolution 1024x1024)

Question bonus : Si votre serveur GPU dispose de 80 Go de VRAM et que les poids du modèle occupent 14 Go, combien de requêtes (batch size) pouvez-vous traiter simultanément pour $N = 4096$ en laissant une marge de sécurité de 10 Go pour les activations et le système ?

Éléments de réponse et pistes de correction

Les exercices proposés à la fin de ce chapitre ne sont pas de simples applications académiques. Ils sont conçus pour vous confronter aux réalités matérielles et stratégiques que vous rencontrerez lors de la conception de vos pipelines de génération. Voici les éléments de correction détaillés, accompagnés de réflexions architecturales essentielles pour l'ingénieur en apprentissage automatique.

Correction de l'Exercice 1 : Tableau comparatif chiffré

Voici une estimation réaliste des ordres de grandeur pour un modèle d'environ 1 milliard de paramètres, fonctionnant sur un GPU de type NVIDIA A100 (80 Go).

© 2026 Oussama Ghorbel

Métrique	AR (256×256)	AR (1024×1024)	Diffusion (256×256)	Diffusion (1024×1024)
FLOPs approx. par image	~0,5 TFLOPs	~8 TFLOPs	~0,06 TFLOPs	~1,5 TFLOPs
Mémoire KV-cache / Activations (FP16)	~128 Mo	~2 Go	~50 Mo	~800 Mo
Latence estimée sur 1x A100	~50 - 80 ms	~1,5 - 2,5 s	~15 - 25 ms	~200 - 350 ms

Analyse et justification :

1. **FLOPs (Floating Point Operations) :** Pour l'autoregression, le coût computationnel d'une passe avant (forward pass) sur un Transformer est approximativement de $2 \times P$ (où P est le nombre de paramètres) par token généré. Pour N tokens, le total est de $2 \times P \times N$. Avec $P = 10^9$ et $N = 256$, nous obtenons environ 500 GFLOPs (0,5 TFLOPs). Pour $N = 4096$, le coût explose à 8 TFLOPs. La diffusion, en revanche, traite l'ensemble de la grille spatiale en une seule passe. Le coût par étape est fixe (environ 2 GFLOPs pour un modèle 1B sur une petite résolution spatiale de latents). Avec $T = 30$ étapes, le total pour la basse résolution est d'environ 60 GFLOPs. Pour la haute résolution, l'attention spatiale quadratique fait grimper ce coût, mais il reste bien inférieur à l'accumulation séquentielle de l'AR.

2. **Mémoire :** Les chiffres de l'AR correspondent strictement au KV-cache (calculé dans l'Exercice 3). La diffusion ne stocke pas d'historique séquentiel ; sa mémoire est dominée par les activations des couches intermédiaires du U-Net ou du DiT. Bien que les activations spatiales croissent avec la résolution, elles sont réécrites à chaque étape de débruitage et ne s'accumulent pas, d'où une empreinte mémoire bien plus faible et stable.

3. **Latence et le « Memory Wall » :** C'est ici que l'analyse purement théorique des FLOPs atteint ses limites. Si l'on divise les FLOPs de l'AR 1024×1024 (8 TFLOPs) par la puissance crête de l'A100 en FP16 (~312 TFLOPS), on obtiendrait un temps théorique de 25 millisecondes. Pourtant, la latence réelle dépasse la seconde.

© 2026 Oussama Ghorbel

Pourquoi ? Parce que la génération autoregressive est massivement **memory-bound** (limitée par la bande passante mémoire). Le GPU passe l'essentiel de son temps à transférer les poids du modèle et le KV-cache depuis la mémoire HBM vers les cœurs de calcul, plutôt qu'à effectuer des multiplications. La diffusion, traitant de gros tenseurs en parallèle, est **compute-bound** et exploite donc les cœurs Tensor (Tensor Cores) avec une efficacité proche du maximum théorique.

Correction de l'Exercice 2 : Paragraphe de synthèse

Voici une proposition de synthèse, rédigée sous la forme d'un pitch technique destiné à une direction technique (CTO) :

> *« Bien que les modèles de diffusion offrent aujourd'hui une latence inférieure et une qualité esthétique immédiate pour la génération d'images isolées, l'autoregression sur tokens discrets s'impose comme le choix stratégique incontournable pour notre architecture multimodale de prochaine génération. En étendant notre LLM 7B existant plutôt qu'en entraînant un pipeline de diffusion ex nihilo, nous unifications radicalement notre espace de représentation. Le modèle acquiert la capacité de raisonner par chaîne de pensée (Chain-of-Thought) textuelle avant d'émettre les tokens visuels, garantissant une cohérence sémantique et une composition spatiale que la diffusion peine à atteindre sans guidage externe lourd. De plus, nous capitalisons directement sur notre infrastructure MLOps mature : nous pouvons réutiliser nos outils de serving (comme vLLM), nos techniques de quantification et nos pipelines d'alignement (RLHF). L'AR permet l'interleaving natif (texte, image, texte) indispensable pour nos agents autonomes, là où la diffusion nécessiterait un pipeline de routage complexe, fragile et coûteux à maintenir. Le surcoût en latence est un compromis acceptable pour nos cas d'usage B2B, où la fiabilité du raisonnement et l'unification du système priment sur le temps de réponse pur. »*

Commentaire de l'auteur : Ce paragraphe met en lumière un principe fondamental de l'ingénierie en IA : le choix d'une architecture ne se fait jamais sur la base d'une seule métrique (ici, la vitesse). L'unification des espaces latents, la réutilisation de l'infrastructure de déploiement et la capacité d'orchestration multimodale sont des

© 2026 Oussama Ghorbel

avantages systémiques qui justifient souvent le coût computationnel supérieur de l'autoregression.

Correction de l'Exercice 3 : Simulation du coût mémoire du KV-cache

Appliquons la formule fournie pour calculer l'empreinte mémoire exacte du KV-cache.

Rappel de la formule : $\text{Taille (octets)} = 2 \times \text{Nb_couches} \times \text{Dim_cachée} \times N \times 2$

Avec $\text{Nb_couches} = 32$ et $\text{Dim_cachée} = 4096$, le facteur constant par token est :

$2 \times 32 \times 4096 \times 2 = 524\,288$ octets par token.

1. **Pour** $N = 256$ **(Image basse résolution) :**

$524\,288 \times 256 = 134\,217\,728$ octets.

Conversion en Mégaoctets (Mo) : $134\,217\,728/(1024 \times 1024) =$ **128** Mo.

2. **Pour** $N = 1\,024$ **(Image moyenne résolution) :**

$524\,288 \times 1\,024 = 536\,870\,912$ octets.

Conversion : $536\,870\,912/(1024 \times 1024) =$ **512** Mo.

3. **Pour** $N = 4\,096$ **(Image haute résolution 1024×1024) :**

$524\,288 \times 4\,096 = 2\,147\,483\,648$ octets.

Conversion : $2\,147\,483\,648/(1024 \times 1024) =$ **2 048** Mo, soit **2 Go**.

Réponse à la question bonus (Calcul du Batch Size) :

- Budget VRAM total : 80 Go
- Poids du modèle : 14 Go
- Marge de sécurité (activations, système, fragmentation) : 10 Go
- VRAM disponible pour le KV-cache : $80 - 14 - 10 = 56$ Go.
- Coût du KV-cache par requête pour $N = 4096$: 2 Go.
- Nombre de requêtes simultanées (Batch Size) : $56/2 =$ **28** requêtes.

© 2026 Oussama Ghorbel

Note avancée sur l'optimisation (GQA et PagedAttention) :

En tant qu'ingénieur, vous devez savoir que ce calcul correspond à une architecture Multi-Head Attention (MHA) standard. Aujourd'hui, la quasi-totalité des LLM modernes (LLaMA 2/3, Mistral) utilisent le **Grouped-Query Attention (GQA)**. Si le modèle utilise 8 têtes pour les clés/valeurs au lieu de 32 (soit un ratio de 4), la taille du KV-cache est divisée par 4. Pour $N = 4096$, le KV-cache ne pèserait alors que 512 Mo, faisant passer le batch size théorique à plus de 100 requêtes simultanées ! De plus, des moteurs d'inférence comme vLLM utilisent la **PagedAttention**, qui alloue le KV-cache par blocs non contigus (à la manière de la mémoire virtuelle d'un OS), éliminant la fragmentation interne et permettant d'approcher ce maximum théorique en production.

Conclusion : La fin du dichotome texte-image

Au terme de cette analyse comparative, un constat s'impose : la guerre de tranchées entre l'autoregression sur tokens discrets et la diffusion sur latents continus touche à sa fin. Les premières années de la génération d'images par IA ont été marquées par une spécialisation stricte. D'un côté, les LLM dominaient le royaume du texte, de la logique et de la séquence. De l'autre, les modèles de diffusion régnaient sur le pixel, la texture et la continuité spatiale.

Mais à mesure que nous poussons les limites de ces architectures, leurs trajectoires convergent inévitablement. Les modèles de diffusion intègrent des Transformers (DiT) et adoptent des formulations qui ressemblent de plus en plus à des chaînes de Markov discrètes. Inversement, les LLM étendent leurs vocabulaires pour ingérer des patches visuels, et les chercheurs développent des objectifs de perte hybrides (comme Transfusion) pour contourner les goulots d'étranglement séquentiels de la vision.

Le choix que vous ferez pour votre prochain projet ne devra donc plus être dicté par un dogme académique, mais par une évaluation froide de vos contraintes de déploiement. Avez-vous besoin d'un éditeur spatial interactif ? La diffusion reste votre meilleure alliée. Cherchez-vous à bâtir un agent multimodal capable de planifier, raisonner et illustrer sa

© 2026 Oussama Ghorbel

pensée dans un flux continu ? L'autoregression, adossée à un tokenizer visuel robuste, est la voie royale.

Cependant, qu'il s'agisse de prédire le prochain mot ou le prochain patch pixel, ces modèles massifs partagent des faiblesses structurelles communes. Ils hallucinent, ils peinent à respecter les lois de la physique élémentaire, et leur coût computationnel soulève des questions écologiques et économiques majeures. Maintenant que nous avons choisi nos armes architecturales et compris comment transformer un LLM en générateur visuel, il est temps d'affronter les limites de cette magie mathématique. Dans le chapitre suivant, nous lèverons le voile sur les artefacts, les échecs de cohérence et les défis de sécurité qui définissent la frontière actuelle de la recherche en génération multimodale.

© 2026 Oussama Ghorbel

Chapitre 20 – Limites actuelles : artefacts, cohérence, coût et sécurité

Tout au long de cet ouvrage, nous avons décortiqué la mécanique fascinante qui permet à un modèle de langage, initialement conçu pour prédire le mot suivant, de s'emparer de la génération d'images. Nous avons construit des tokenizers, exploré des architectures d'attention, et défini des protocoles d'entraînement rigoureux. Pourtant, en tant qu'ingénieurs et chercheurs, notre devoir est de conserver un regard critique sur nos propres créations. L'approche autorégressive pour la génération visuelle, bien qu'élégante et unificatrice, se heurte aujourd'hui à une série de verrous scientifiques et techniques.

Ce chapitre a pour vocation d'identifier honnêtement ces limites. Il ne s'agit pas de discréditer la méthode, mais de cartographier les zones d'ombre qui définissent le périmètre de la recherche actuelle. Comprendre pourquoi un modèle échoue est souvent plus instructif que de célébrer ses réussites. Nous allons explorer les artefacts inhérents à la quantification, les défis de la cohérence spatiale, le mur des coûts d'inférence, ainsi que les enjeux éthiques et sécuritaires qui accompagnent le déploiement de ces technologies.

20.1 — Artefacts de quantification : postérisation, couleurs délavées, textures répétitives

La première limite, et peut-être la plus fondamentale, ne réside pas dans le modèle de langage lui-même, mais dans l'étape préalable de tokenisation. Lorsque nous utilisons un auto-encodeur vectoriel (VQ-VAE ou VQGAN) pour projeter l'espace continu des pixels dans un espace discret de tokens, nous introduisons inévitablement une perte d'information. Cette quantification vectorielle est à l'origine de plusieurs artefacts visuels caractéristiques qui trahissent immédiatement l'origine synthétique de l'image.

Le premier de ces artefacts est la postérisation, ou effet de bandes de couleurs (color banding). Dans une image naturelle, les gradients de couleur (comme un ciel au crépuscule) sont continus. Le tokenizer, contraint par la taille finie de son codebook (par exemple, 8 192 ou 16

© 2026 Oussama Ghorbel

384 vecteurs), ne peut représenter ces variations subtiles qu'en sélectionnant le vecteur le plus proche dans l'espace latent. Lors de la déquantification par le décodeur, cette approximation grossière se traduit par des transitions abruptes entre des plages de couleurs uniformes, détruisant la douceur des dégradés.

Ensuite, nous observons fréquemment un phénomène de couleurs délavées et de flou global, souvent qualifié d'effet "aquarelle". Cet artefact est une conséquence directe de la fonction de perte utilisée pour entraîner l'auto-encodeur. Si l'on optimise le décodeur avec une erreur quadratique moyenne (MSE) ou une perte L1 dans l'espace des pixels, le modèle apprend à minimiser l'erreur en prédisant la moyenne des possibles. Face à une incertitude texturale, le décodeur "moyenne" les pixels, ce qui se traduit visuellement par une désaturation et une perte de contraste. Bien que l'ajout de pertes perceptuelles (comme LPIPS) ou de discriminateurs multi-échelles (comme dans VQGAN) atténue ce problème, il reste difficile à éradiquer complètement sans introduire d'autres instabilités d'entraînement.

Enfin, l'approche autorégressive souffre d'un biais bien connu en modélisation de séquences : la répétition de motifs. Lorsqu'un LLM génère du texte, il peut tomber dans des boucles de répétition. Transposé à l'image, cela se manifeste par des textures répétitives et artificielles. Si le modèle commence à générer les écailles d'un reptile ou les brins d'herbe d'une prairie, l'attention autorégressive tend à copier-coller le motif local qui minimise la perte de prédiction du prochain token. Le résultat est une texture qui semble "tamponnée" (stamped) de manière périodique, manquant cruellement de la variance stochastique et organique que l'on trouve dans les véritables textures naturelles.

20.2 — Cohérence globale : difficulté à maintenir la structure sur de longues séquences

Le cœur de l'approche autorégressive repose sur la factorisation de la distribution conjointe : $p(x) = \prod p(x_i | x_{<i})$. Cette formulation impose un ordre strict de génération, généralement un balayage de type raster (de gauche à droite, de haut en bas). Si cette méthode excelle dans la prédiction locale, elle peine terriblement à maintenir une cohérence globale sur de longues séquences.

© 2026 Oussama Ghorbel

Considérons la génération d'une image de résolution 512 × 512 avec un patch size de 16 × 16. Cela représente une séquence de 1 024 tokens. Lorsque le modèle génère le token numéro 800 (situé en bas à droite de l'image), il doit conditionner sa prédiction sur les 799 tokens précédents. Cependant, les mécanismes d'attention standard ont tendance à souffrir d'un "biais de récence" (recency bias) et d'une dilution de l'attention sur les séquences longues. L'information structurelle globale — par exemple, le fait que l'objet généré est un chien vu de profil, avec une seule queue et quatre pattes — se dilue dans la fenêtre de contexte.

C'est pourquoi il est si fréquent d'observer des anomalies anatomiques ou structurelles dans les images générées par autoregression pure : un animal avec trois pattes arrière, un bâtiment dont les fenêtres changent de style à mi-hauteur, ou une symétrie faciale qui se dégrade progressivement du côté gauche vers le côté droit du visage. Contrairement aux modèles de diffusion qui traitent l'intégralité de la grille spatiale de manière parallèle à chaque étape de débruitage (préservant ainsi une conscience globale de la composition à chaque instant t), le modèle autorégressive "oublie" littéralement le début de l'image à mesure qu'il progresse vers la fin.

Les tentatives pour résoudre ce problème par des courbes de balayage alternatives (comme la courbe de Hilbert ou le balayage en spirale) améliorent la localité spatiale, mais ne résolvent pas le problème fondamental de la modélisation 1D d'un signal intrinsèquement 2D. La cohérence globale nécessite une planification hiérarchique (générer d'abord un croquis grossier, puis les détails), ce que l'autoregression à plat, token par token, ne sait pas faire nativement sans architectures multi-échelles complexes qui alourdissent considérablement le pipeline.

20.3 — Coût inference : latence séquentielle, mémoire KV-cache, throughput

Au-delà des considérations purement visuelles, le déploiement en production de LLMs générateurs d'images se heurte à un mur économique et computationnel. Le coût de l'inférence est le verrou industriel le plus sévère de cette approche.

La génération autorégressive est fondamentalement séquentielle. Pour générer une image de 1 024 tokens, le modèle doit effectuer 1 024

© 2026 Oussama Ghorbel

passes forward complètes. Même sur des GPU modernes équipés de Tensor Cores, cette sérialisation empêche toute parallélisation temporelle. Là où un modèle de diffusion latent (comme Stable Diffusion) peut générer une image en 20 à 30 étapes parallélisables sur l'ensemble de la grille latente, le LLM doit "payer" le coût computationnel de l'attention pour chaque token individuel. La latence de génération (Time-To-Last-Token) s'étire souvent sur plusieurs secondes, voire dizaines de secondes, ce qui est rédhibitoire pour des applications interactives en temps réel.

À ce problème de latence s'ajoute celui de l'empreinte mémoire, spécifiquement lié au cache des clés et des valeurs (KV-cache). Lors de la génération, pour éviter de recalculer les attentions sur les tokens passés, le système stocke les états intermédiaires de chaque couche du transformeur. Pour un modèle de 7 milliards de paramètres en précision FP16, avec une fenêtre de contexte de 1 024 tokens, le KV-cache peut consommer plusieurs gigaoctets de VRAM par requête simultanée. Lorsqu'on souhaite servir des centaines d'utilisateurs en parallèle (high throughput), la mémoire du GPU sature bien avant que les capacités de calcul (FLOPs) ne soient pleinement exploitées. Le "memory-bound" de l'inférence autorégressive limite drastiquement le batch size effectif.

Des techniques comme la génération spéculative (speculative decoding) ou l'utilisation de modèles "draft" plus petits pour proposer des blocs de tokens visuels commencent à être explorées, mais elles ajoutent une complexité d'ingénierie majeure. De même, les approches semi-autorégressives (comme MaskGIT ou MAR) réduisent le nombre d'étapes de génération en prédissant plusieurs tokens par pas, mais elles sacrifient une partie de la flexibilité et de la richesse distributionnelle du modèle purement autorégressif. Le compromis entre qualité visuelle, latence et coût serveur reste une équation non résolue.

20.4 — Biais des données de training et reproduction de stéréotypes visuels

Un modèle n'est que le reflet statistique de ses données d'entraînement. Les LLMs visuels sont généralement pré-entraînés sur des milliards de paires texte-image extraites du web (comme LAION-5B ou DataComp). Ces ensembles de données, bien que massifs, sont le

© 2026 Oussama Ghorbel

miroir déformant de nos sociétés, de nos histoires et de nos préjugés. L'approche autorégressive, par sa nature même de mémorisation et de reproduction de distributions conjointes, amplifie ces biais de manière particulièrement insidieuse.

Lorsqu'un utilisateur requiert la génération d'un "PDG" ou d'un "médecin", le modèle, ayant internalisé les co-occurrences statistiques de son corpus, aura une probabilité écrasante de générer un homme blanc d'âge mûr en costume. À l'inverse, des requêtes associées à des rôles subalternes ou stéréotypés produiront des représentations ethniques et de genre biaisées. Contrairement aux biais textuels qui peuvent être partiellement atténués par des techniques d'alignement comme le RLHF (Reinforcement Learning from Human Feedback) ou des filtres de sécurité sémantiques, les biais visuels sont profondément encodés dans la topologie de l'espace latent et dans les probabilités de transition du codebook.

Désintraire (debiasing) un modèle de génération d'images autorégressif est un défi scientifique ouvert. Modifier le prompt en ajoutant "une femme noire PDG" force le modèle, mais cela ne corrige pas la distribution sous-jacente ; cela ne fait que déplacer le conditionnement. De plus, les filtres sémantiques appliqués sur le texte d'entrée (prompt filtering) sont facilement contournables par des synonymes ou des descriptions indirectes.

La reproduction de stéréotypes visuels pose également la question de l'uniformisation esthétique. Les modèles tendent à converger vers un "style moyen" des images les plus populaires sur internet (photos de stock, rendus 3D de type ArtStation, photographies suréclairées). Cette homogénéisation culturelle et esthétique appauvrit la diversité visuelle générée et marginalise les styles artistiques minoritaires ou les représentations culturelles non occidentales, qui sont sous-représentés dans les données de training. En tant qu'ingénieurs, nous devons reconnaître que la curation des données n'est pas seulement un problème technique de nettoyage de bruit, mais un acte éditorial aux conséquences sociétales profondes.

20.5 — Sécurité : deepfakes, watermarking, détection de contenu généré

© 2026 Oussama Ghorbel

La capacité de générer des images photoréalistes à partir de simples descriptions textuelles a ouvert la boîte de Pandore de la désinformation visuelle. Les deepfakes, la création de fausses preuves photographiques, et l'usurpation d'identité visuelle sont des menaces imminentes. L'approche autorégressive, de par la haute fidélité de ses tokenizers modernes, est tout à fait capable de produire des visages et des scènes indiscernables de la réalité, posant un défi majeur en matière de sécurité et de confiance numérique.

Pour contrer cette menace, la communauté de la recherche s'oriente vers deux axes : le tatouage numérique (watermarking) et la détection forensique. Le watermarking dans un pipeline autorégressif est complexe. Contrairement aux modèles de diffusion où l'on peut injecter un signal dans le bruit initial (comme la méthode Tree-Ring Watermark), le LLM génère des tokens discrets. Une approche prometteuse consiste à biaiser légèrement la distribution de probabilité (les logits) lors de l'échantillonnage pour forcer la sélection de tokens qui encodent un message binaire caché, sans dégrader la qualité perceptuelle de l'image. Cependant, ce tatouage latent doit survivre aux attaques du monde réel : recadrage, compression JPEG, ajout de bruit, ou même capture par l'écran (screen-cam attack).

La détection de contenu généré est une course aux armements asymétrique. Les détecteurs analysent souvent les artefacts de haute fréquence, les anomalies dans le domaine de Fourier, ou les motifs d'utilisation spécifiques du codebook. Mais à chaque fois qu'un détecteur est publié, les architectures de génération évoluent pour lisser ces anomalies. De plus, les faux positifs peuvent avoir des conséquences désastreuses (accuser à tort un photographe humain).

L'industrie tente d'imposer des standards de provenance cryptographique, comme la coalition C2PA (Coalition for Content Provenance and Authenticity), qui attache une signature cryptographique aux métadonnées de l'image dès sa création. Mais cette approche repose sur une infrastructure de confiance et ne résout pas le problème des images générées par des modèles open-source déployés localement, sans aucune supervision ni signature. La sécurité des LLMs visuels ne pourra pas être résolue par la seule technique ; elle nécessitera un cadre légal et une éducation du public à la méfiance visuelle.

© 2026 Oussama Ghorbel

20.6 — Limites du tokenizer : plafond de reconstruction, généralisation hors distribution

Il est crucial de se rappeler une vérité mathématique simple : le modèle autorégressif ne génère pas des pixels, il génère des indices de codebook. L'image finale est le résultat exclusif du décodeur de l'auto-encodeur. Par conséquent, la qualité maximale de l'image générée est strictement bornée par le "plafond de reconstruction" du tokenizer.

Si votre VQGAN a un FID (Fréchet Inception Distance) de reconstruction de 1.5 sur votre jeu de données de test, il est mathématiquement impossible pour le LLM, aussi puissant et bien entraîné soit-il, de générer des images avec un FID inférieur à 1.5. Le LLM ne peut qu'approcher la distribution des tokens de l'ensemble d'entraînement du tokenizer. Ce goulot d'étranglement signifie que des efforts massifs doivent être consacrés à la première étape (l'auto-encodeur), qui est souvent reléguée au second plan dans la littérature focalisée sur les LLMs.

De plus, le tokenizer définit le périmètre de la généralisation hors distribution (OOD). Le codebook est un vocabulaire fermé, appris sur un corpus spécifique (généralement des images naturelles de type ImageNet ou LAION). Si vous demandez au système de générer une image hors de cette distribution — par exemple, une radiographie médicale, une carte topographique, ou une image capturée par un microscope électronique — le tokenizer échouera à encoder correctement ces concepts visuels. Les patches de ces images OOD seront projetés de force sur les vecteurs les plus proches du codebook (par exemple, une texture cellulaire pourrait être quantifiée comme du "bruit" ou du "gravier"). Le LLM modélisera alors cette représentation erronée, et le décodeur produira une aberration visuelle.

Cette limite empêche l'application directe des LLMs visuels à des domaines scientifiques ou industriels de niche sans un réentraînement complet et coûteux de l'auto-encodeur sur des données spécialisées. La rigidité du codebook discret s'oppose à la flexibilité continue que l'on aimerait avoir pour modéliser n'importe quel signal visuel arbitraire. C'est d'ailleurs l'une des raisons pour lesquelles les tokenizers continus (comme ceux utilisés dans certains modèles de diffusion latente ou les

© 2026 Oussama Ghorbel

approches basées sur les MAE) regagnent en popularité pour contourner ce plafond de verre de la quantification.

20.7 — Ce que l'approche ne sait pas encore faire : texte lisible, mains, géométrie précise

Malgré les progrès fulgurants, il existe des "angles morts" cognitifs et spatiaux que l'autoregression par tokens peine radicalement à surmonter. Le plus célèbre d'entre eux est la génération de texte lisible intégré dans l'image (par exemple, une enseigne de magasin ou un titre de journal).

Pour qu'un mot soit lisible, il requiert une précision typographique absolue au niveau du pixel. Une erreur d'un seul pixel ou d'un seul token peut transformer un "A" en "R", ou briser la connectivité d'une lettre cursivce. Dans un modèle autorégressif, la génération est stochastique et séquentielle. Le modèle n'a pas de "mémoire de travail" spatiale rigide pour planifier la bounding box d'un mot et y aligner parfaitement des glyphes. Il traite les tokens de texte de l'image comme de simples motifs visuels locaux. Le résultat est un charabia pseudo-typographique, souvent qualifié de "glossolalie visuelle", qui imite l'apparence globale de l'écriture mais s'effondre dès qu'on tente de le déchiffrer. Les modèles de diffusion récents contournent partiellement ce problème grâce à des encodeurs de texte (comme T5) qui injectent des cartes d'attention spatiale précises via cross-attention, un mécanisme beaucoup plus difficile à intégrer efficacement dans un pipeline purement autorégressif unifié.

Un autre échec notoire concerne l'anatomie complexe, en particulier les mains, et la géométrie précise (comme les rayons d'une roue de vélo ou les barreaux d'une clôture). Ces structures nécessitent une compréhension 3D sous-jacente et des contraintes globales strictes (une main a cinq doigts, une roue est un cercle parfait avec des rayons équidistants). Le balayage 1D des tokens détruit ces relations géométriques 2D/3D. Le modèle prédit le prochain patch en fonction des précédents, sans "comprendre" qu'il est en train de dessiner une structure articulée ou un objet rigide. Il se retrouve souvent à générer sept doigts, des articulations impossibles, ou des lignes parallèles qui convergent de manière non euclidienne.

© 2026 Oussama Ghorbel

Ces limites révèlent que l'approche token-pixel modélise brillamment les *textures* et les *distributions statistiques de couleurs*, mais échoue à modéliser la *physique*, la *géométrie* et la *sémantique structurelle* du monde réel. Jusqu'à ce que des inducteurs de biais (inductive biases) géométriques ou des mécanismes de planification hiérarchique explicite soient intégrés aux architectures de transformeurs visuels, ces erreurs grossières continueront de trahir la machine derrière le pixel.

Exercices de fin de chapitre

Pour ancrer ces concepts théoriques dans la pratique de l'ingénierie, je vous propose les trois exercices suivants. Ils sont conçus pour développer votre intuition critique face aux sorties des modèles génératifs.

Exercice 1 : Catalogage typologique des artefacts

Générez un ensemble de 50 images diversifiées (paysages, portraits, textures, scènes urbaines) en utilisant un modèle autorégressif open-source (comme LlamaGen ou un VQGAN couplé à un GPT-2).

• **Tâche :** Construisez un tableau typologique détaillant les artefacts récurrents. Classez-les en catégories (ex: postérisation, répétition de texture, incohérence anatomique, objets hybrides).

• **Analyse :** Pour chaque catégorie, calculez la fréquence d'apparition et tentez de corréler l'artefact avec le type de prompt ou la région spatiale de l'image (ex: les artefacts de cohérence sont-ils plus fréquents en bas à droite de l'image, là où la séquence est la plus longue ?).

Exercice 2 : Mesure de la dégradation de cohérence structurelle

Cet exercice vise à quantifier l'impact de la longueur de la séquence sur la mémoire à long terme du modèle.

• **Tâche :** Configurez un pipeline de génération pour une même classe d'objets symétriques (ex: "un papillon vu de dessus", "un visage de face") à deux résolutions différentes : une nécessitant 256 tokens (ex: 256×256 avec patch de 16) et une autre nécessitant 1 024

© 2026 Oussama Ghorbel

tokens (ex: 512×512 avec patch de 16).

• **Mesure :** Utilisez un modèle de segmentation sémantique ou de détection de points clés (comme MediaPipe pour les visages) pour évaluer la symétrie et la cohérence structurelle des images générées.

• **Objectif :** Tracez la courbe de dégradation de la symétrie en fonction de l'index du token généré. Démontrez empiriquement comment le biais de récence et la dilution de l'attention affectent la fin de la séquence.

Exercice 3 : Protocole de watermarking par perturbation de codebook

Explorez les limites de la sécurité et de la traçabilité des tokens discrets.

• **Tâche :** Implémentez un script d'inférence qui modifie subtilement la fonction d'échantillonnage (sampling function) du LLM. Au lieu d'échantillonner strictement selon la distribution softmax, forcez le modèle à choisir, pour chaque 10ème token, un indice de codebook dont la parité (pair/impair) correspond à un bit d'un message binaire caché (ex: votre signature "011010").

• **Test de robustesse :** Décodez les images tatouées, puis appliquez-leur des transformations destructives : compression JPEG à différents taux de qualité (Q=90, Q=50, Q=20), recadrage de 20%, et ajout de bruit gaussien.

• **Analyse :** Ré-extrayez le message binaire en repassant les images corrompues dans l'encodeur du tokenizer. Calculez le taux d'erreur binaire (BER) du message extrait en fonction de la sévérité de l'attaque. Concluez sur la viabilité de cette méthode de watermarking dans l'espace des tokens discrets par rapport à l'espace des pixels continus.

20.8 — Vers des architectures hybrides : contourner les verrous de l'autorégression pure

Face à ce constat lucide, une question brûle les lèvres de tout chercheur en apprentissage automatique : faut-il abandonner l'approche autorégressive pour la génération d'images ? La réponse est

© 2026 Oussama Ghorbel

catégoriquement non. Les limites que nous venons de détailler ne sont pas des condamnations à mort, mais des balises qui orientent la recherche contemporaine. Plutôt que de rejeter le paradigme du *next-token prediction*, la communauté scientifique s'attelle à l'hybrider et à le réinventer.

L'une des pistes les plus prometteuses pour résoudre le dilemme de la quantification et le plafond de reconstruction réside dans l'abandon partiel de la discrétisation stricte. Les travaux récents sur les tokenizers continus, ou sur la modélisation autorégressive de distributions de probabilité latentes (plutôt que d'indices de codebook discrets), ouvrent la voie à des modèles qui conservent la planification séquentielle du LLM tout en s'affranchissant des artefacts de postérisation. En prédisant les paramètres d'une distribution gaussienne mixte ou en utilisant des espaces latents non quantifiés, on réintroduit la nuance du signal continu. Des approches comme la quantification scalaire finie (Finite Scalar Quantization, FSQ) tentent également de lisser l'espace d'optimisation des codebooks pour éviter l'effondrement de codes (*codebook collapse*) qui mène aux textures répétitives.

Pour pallier le déficit de cohérence globale et l'incapacité à modéliser la géométrie 3D, l'hybridation avec les modèles de diffusion ou les modèles de flux (*flow matching*) devient la norme implicite. L'idée n'est plus d'opposer autorégression et diffusion, mais de les empiler dans une hiérarchie cognitive. Un LLM peut être utilisé comme un "planificateur sémantique" qui génère une séquence de tokens de bas niveau, des boîtes englobantes (*bounding boxes*) ou des cartes de profondeur, tandis qu'un modèle de diffusion prend le relais pour le rendu pixelique final. Cette architecture en cascade délègue la cohérence globale et la composition spatiale au LLM — qui excelle dans la compréhension des relations à longue portée grâce à son entraînement textuel — et laisse au diffuseur le soin de synthétiser les textures haute fréquence et les détails photoréalistes.

D'autres approches explorent la génération multi-échelle autorégressive, où le modèle prédit d'abord une grille de tokens grossière (par exemple 16 × 16), puis s'autorégresse sur les détails de chaque patch. Cela permet de maintenir une "mémoire de travail" globale avant de plonger dans les détails locaux, contournant ainsi le biais de récence

© 2026 Oussama Ghorbel

inhérent au balayage raster classique.

Enfin, concernant le mur de l'inférence, l'ingénierie matérielle et logicielle progresse à pas de géant. L'émergence d'architectures séquentielles alternatives, comme les modèles d'espace d'état (*State Space Models*, SSM, dont Mamba est l'archétype), promet de réduire la complexité de l'attention de $O(N^2)$ à $O(N)$. Pour la génération d'images, cela signifierait un KV-cache dont l'empreinte mémoire ne croîtrait plus de manière quadratique avec la résolution, permettant enfin d'imaginer la génération native d'images 4K ou de séquences vidéo longues sur du matériel grand public, sans asphyxier la VRAM des serveurs.

Synthèse : le pragmatisme face à la magie

En tant qu'ingénieurs, nous devons nous méfier de la "magie" souvent mise en avant dans les démonstrations technologiques. Les réseaux sociaux et les articles de presse se nourrissent d'images spectaculaires, soigneusement sélectionnées (*cherry-picked*) pour masquer les échecs cuisants que nous avons explorés dans ce chapitre. Notre rôle, lorsque nous concevons un pipeline de génération visuelle, est d'anticiper ces échecs et de concevoir des systèmes résilients.

Il n'existe pas de modèle universel, parfait et peu coûteux. Le triangle d'incompatibilité de la génération d'images — qualité perceptuelle, diversité distributionnelle et vitesse d'inférence — s'applique avec une rigueur impitoyable aux LLMs visuels. Si vous concevez une application pour le divertissement ou l'exploration créative, vous pourrez tolérer les mains à six doigts et les textes illisibles, car l'impact émotionnel et la sérendipité priment. En revanche, si vous développez un outil pour le design industriel, l'architecture, la génération d'interfaces utilisateur ou l'imagerie médicale, les limites de cohérence géométrique et de généralisation hors distribution rendent l'approche autorégressive par tokens inapte au déploiement autonome. Dans ces domaines critiques, le modèle ne peut être qu'un assistant suggestif, nécessitant un contrôle humain strict (*human-in-the-loop*) ou des couches de post-traitement symbolique et géométrique.

Ce chapitre se voulait un miroir sans complaisance. Les artefacts, les biais, les coûts et les failles de sécurité ne sont pas de simples bugs à corriger par une mise à jour de poids ou un ajustement du taux

© 2026 Oussama Ghorbel

d'apprentissage ; ils sont intrinsèques à la façon dont nous représentons le monde visuel sous forme de séquences de probabilités discrètes. Comprendre ces verrous est la condition sine qua non pour innover. C'est en sachant exactement pourquoi et comment le modèle échoue que nous pourrons concevoir les architectures de demain.

Pour aller plus loin : lectures et axes de recherche

Pour approfondir les concepts abordés dans ce chapitre et vous tenir à la frontière de la recherche, je vous recommande d'explorer les axes suivants, qui constituent aujourd'hui le front de science :

1. **Analyse des espaces latents et quantification** : Étudiez les évolutions récentes des auto-encodeurs vectoriels, notamment les travaux sur la quantification scalaire et les méthodes de régularisation du codebook qui tentent de maximiser l'utilisation de l'espace latent sans sacrifier la fidélité de la reconstruction.

2. **Évaluation et atténuation des biais visuels** : Consultez les études sociotechniques menées sur les ensembles de données à grande échelle (comme les audits de LAION ou DataComp) et les frameworks d'alignement visuel. La littérature sur le *Concept Erasure* (effacement de concepts) dans les espaces latents est particulièrement cruciale pour comprendre comment désintraire un modèle sans détruire ses capacités génératives.

3. **Stéganographie neuronale et watermarking** : Lisez les papiers fondateurs sur le tatouage dans l'espace des logits et l'espace latent, ainsi que les analyses de robustesse face aux attaques par re-génération (où un modèle de diffusion est utilisé pour "nettoyer" et effacer le tatouage d'une image autorégressive).

4. **Architectures séquentielles pour la vision** : La littérature sur les *State Space Models* (SSM) et les mécanismes d'*Ring Attention* appliqués à la vision par ordinateur, qui remettent en cause l'hégémonie du Transformeur standard pour les séquences visuelles ultra-longues et la génération vidéo.

La route du token au pixel est encore semée d'embûches, mais c'est précisément cette complexité qui rend ce domaine de l'apprentissage automatique si stimulant. Nous avons posé les limites physiques,

© 2026 Oussama Ghorbel

mathématiques et éthiques de notre approche. Dans le prochain chapitre, nous quitterons la théorie des verrous pour entrer dans la salle des machines du diagnostic : nous apprendrons à démonter un modèle entraîné pour comprendre, par des ablations rigoureuses, ce qui compte vraiment dans son apprentissage.

© 2026 Oussama Ghorbel

Chapitre 21 – Ablations et diagnostic : comprendre ce qui compte vraiment

L'entraînement d'un modèle autorégressif capable de générer des images à partir de texte est une entreprise colossale. Lorsque vous lancez une expérience qui mobilise des dizaines de GPU pendant plusieurs semaines, l'obtention d'un score FID (Fréchet Inception Distance) ou d'un CLIP-score n'est que la partie émergée de l'iceberg. La véritable question de l'ingénieur et du chercheur n'est pas seulement « Quelle est la performance ? », mais « *Pourquoi* cette performance ? ».

Dans la pratique, on voit souvent des équipes attribuer une amélioration de 2 points de FID à une nouvelle architecture d'attention, alors qu'elle provient en réalité d'un changement subtil dans le taux d'abandon (dropout) du conditionnement textuel ou d'une initialisation différente du codebook. Le pipeline de génération d'images par LLM est un système hautement couplé : le tokeniseur, l'ordre de balayage, le mécanisme de conditionnement et la stratégie d'échantillonnage interagissent de manière non linéaire.

Ce chapitre est consacré à l'art et à la science du diagnostic. Nous allons structurer une méthodologie d'ablation rigoureuse pour isoler les variables, quantifier leur impact réel et éviter les illusions statistiques qui pullulent dans la littérature de l'apprentissage automatique. Comme le rappelait le statisticien George Box : « Tous les modèles sont faux, mais certains sont utiles. » L'ablation sert à mesurer exactement en quoi, et pourquoi, notre modèle est utile.

21.1 — Méthodologie d'ablation : un facteur à la fois, plans fractionnaires

L'approche par défaut dans de nombreux laboratoires est la méthode OFAT (*One-Factor-At-a-Time* : un facteur à la fois). On fixe une configuration de base (le *baseline*), on modifie la taille du codebook, on réentraîne, on évalue. Puis on revient à la base, on modifie l'ordre de balayage, on réentraîne, on évalue.

© 2026 Oussama Ghorbel

Cette méthode présente deux défauts majeurs. Premièrement, elle est exponentiellement coûteuse en temps de calcul. Deuxièmement, et c'est le plus grave, elle est aveugle aux **interactions** entre les facteurs. Il est fort possible qu'un codebook de grande taille ($K = 16384$) dégrade les performances avec un balayage raster, mais les améliore considérablement avec une courbe de Hilbert, car la localité spatiale compense la difficulté d'optimisation d'un grand vocabulaire. L'approche OFAT rate complètement cette synergie.

Pour pallier cela, nous devons emprunter aux statistiques industrielles la théorie des **plans d'expériences** (Design of Experiments, DoE). Plutôt que de tester toutes les combinaisons possibles (plan factoriel complet), qui nécessiterait N^k expériences (où N est le nombre de niveaux et k le nombre de facteurs), nous utilisons des **plans fractionnaires**.

Un plan fractionnaire 2^{k-p} permet d'estimer les effets principaux et les interactions d'ordre faible avec une fraction des expériences. Par exemple, pour 7 facteurs à 2 niveaux, un plan complet exige 128 entraînements. Un plan fractionnaire de résolution III ou IV permet d'obtenir les effets principaux avec seulement 8 ou 16 expériences, en acceptant de confondre (aliaser) certaines interactions d'ordre supérieur, généralement négligeables dans les réseaux de neurones profonds.

Cependant, en apprentissage profond, même 16 expériences complètes de 500 000 itérations sont prohibitives. C'est ici qu'intervient la notion de **métriques proxy** (substituts). L'ablation ne doit pas toujours se faire sur le modèle final. On peut concevoir des expériences à petite échelle :

1. **Résolution réduite** : Entraîner sur des images 128×128 au lieu de 256×256.
2. **Budget d'epochs limité** : Évaluer la vitesse de convergence (loss de validation à l'epoch 10) plutôt que la performance asymptotique.
3. **Modèles surrogates** : Utiliser un LLM de 100M de paramètres pour valider des choix architecturaux avant de les porter sur un modèle de 7B.

La règle d'or est la suivante : on utilise les plans fractionnaires et les métriques proxy pour *sélectionner* les configurations prometteuses, et on

© 2026 Oussama Ghorbel

réserve le budget computationnel massif (plan factoriel complet sur le modèle cible) uniquement pour les 2 ou 3 facteurs dont l'interaction semble critique.

21.2 — Ablation du tokenizer : K, d, taux de compression, type de quantification

Le tokeniseur est la fondation de notre pipeline. Si la représentation discrète de l'image est pauvre, le meilleur LLM du monde ne pourra pas générer de bonnes images. L'ablation du tokeniseur se concentre sur quatre axes principaux.

1. La taille du codebook (K)

Le vocabulaire visuel typique varie de 1024 à 16384 tokens.

• Un K faible (ex: 1024) facilite la modélisation autorégressive : le LLM a moins de mal à estimer la distribution de probabilité sur un petit vocabulaire, et les gradients sont moins clairsemés. Cependant, la reconstruction (rFID) plafonne rapidement.

• Un K élevé (ex: 16384 ou 32768) capture des détails fins et des textures complexes, mais souffre du **codebook collapse** (effondrement du dictionnaire) : le modèle n'utilise qu'une petite fraction des tokens disponibles, rendant l'entraînement du LLM instable.

• *Diagnostic* : Tracez la courbe d'utilisation du codebook (perplexité du codebook) pendant l'entraînement du VQGAN. Si elle stagne en dessous de $K/2$, votre ablation doit se porter sur les techniques de réinitialisation des codes morts ou sur le remplacement de la quantification.

2. La dimension latente (d)

Chaque token est un vecteur de dimension d (souvent 256 ou 512) avant d'être mappé sur le codebook. Augmenter d augmente la capacité de représentation mais alourdit le décodeur et le coût de la projection dans l'espace d'embedding du LLM. En pratique, $d = 256$ est un point de rendements décroissants pour des résolutions standards.

3. Le taux de compression (f)

© 2026 Oussama Ghorbel

Le facteur de suréchantillonnage spatial (downsampling) $f \in \{8, 16, 32\}$ détermine la longueur de la séquence $N = (H/f) \times (W/f)$.

• $f = 8$ pour une image 256×256 donne $N = 1024$ tokens. C'est long pour un LLM, mais la reconstruction est excellente.

• $f = 16$ donne $N = 256$ tokens. Le LLM respire, l'attention $\mathcal{O}(N^2)$ est rapide, mais le tokeniseur doit compresser 16 fois plus d'information par token, ce qui détruit les hautes fréquences.

• *Diagnostic* : L'ablation de f est un arbitrage pur entre le coût d'inférence du LLM (lié à N) et le plafond de qualité visuelle (lié au rFID du tokeniseur).

4. Le type de quantification

C'est l'ablation la plus cruciale de ces dernières années. La quantification vectorielle classique (VQ) avec perte d'engagement (*commitment loss*) est fragile.

En pratique, comparer le VQ-VAE avec des approches plus récentes comme **FSQ** (Finite Scalar Quantization) ou **LFQ** (Lookup-Free Quantization) change radicalement la donne. FSQ remplace le codebook par une fonction d'arrondi sur une grille hyper-rectangulaire définie par des niveaux discrets pour chaque dimension.

• *Résultat typique* : FSQ élimine totalement le codebook collapse (l'utilisation est toujours de 100 %) et simplifie l'optimisation, bien que la taille effective du vocabulaire soit le produit cartésien des niveaux (ce qui peut créer un vocabulaire théorique immense mais très structuré, que le LLM apprend bien grâce aux biais sémantiques des canaux).

21.3 — Ablation de l'ordre de balayage et de la taille de patch

Une fois l'image transformée en grille de tokens $H' \times W'$, il faut l'aplatir en une séquence 1D pour le LLM. L'ordre de balayage dicte la façon dont la localité spatiale 2D est préservée dans la séquence 1D.

Les ordres de balayage

© 2026 Oussama Ghorbel

• **Raster** (ligne par ligne) : C'est le standard. Son défaut majeur est la rupture de localité en fin de ligne : le dernier pixel de la ligne i est adjacent au premier pixel de la ligne $i+1$ dans l'espace 2D, mais très éloigné dans la séquence 1D si la résolution est grande.

• **Serpentin** (Boustrophédon) : Alternance gauche-droite et droite-gauche. Il préserve la continuité entre les lignes. C'est une ablation simple qui coûte zéro paramètre supplémentaire et qui améliore souvent la cohérence des textures horizontales.

• **Hilbert** (Courbe de remplissage d'espace) : Préserve au mieux la distance 2D dans l'espace 1D. Cependant, son implémentation dans les noyaux optimisés comme FlashAttention est cauchemardesque, et le masquage causal devient complexe.

• **Spiral** (de l'extérieur vers l'intérieur ou inversement) : Utile pour la génération progressive où le modèle "esquisse" d'abord les contours globaux avant de remplir les détails centraux.

Diagnostic : Pour ablater l'ordre de balayage, ne regardez pas seulement le FID global. Analysez les artefacts directionnels. Si votre modèle génère des images avec des "coutures" horizontales visibles, le balayage raster en est probablement la cause, et le serpentin corrigera le tir.

La taille de patch (Patchification)

Inspirée par les Vision Transformers (ViT), la patchification consiste à regrouper plusieurs pixels ou tokens latents en un seul macro-token avant de l'injecter dans le LLM. Si le tokeniseur a un facteur $f=8$, on peut encore regrouper les tokens par blocs de 2×2.

• *Avantage* : Réduction drastique de la longueur de séquence N (divisée par 4 dans cet exemple).

• *Inconvénient* : Le LLM doit prédire un token "épais" qui contient beaucoup d'information, ce qui peut nécessiter d'augmenter la dimension des embeddings du modèle.

• *Ablation* : Testez des tailles de patch de 1×1, 2×2 et 4×4. Vous observerez généralement que le FID s'améliore avec des patches plus grands jusqu'à un point de bascule où la perte d'information spatiale fine dégrade la génération des petits objets.

© 2026 Oussama Ghorbel

21.4 — Ablation du conditionnement : cross-attention vs préfixe vs AdaLN

Comment le texte guide-t-il la génération des pixels ? C'est le cœur de l'alignement texte-image. Trois grandes familles architecturales s'affrontent ici, et leur ablation est indispensable pour optimiser le ratio qualité/coût computationnel.

1. Le Préfixe (Prefix Tuning / Prompting)

Les tokens de texte (extraits par un encodeur comme T5 ou CLIP) sont simplement concaténés avant les tokens de l'image.

• *Mécanisme* : $[T_1, T_2, \ldots, T_m, I_1, I_2, \ldots, I_n]$.

• *Avantage* : Aucune modification de l'architecture du LLM. On utilise un transformer décodeur standard.

• *Défaut* : Les tokens de texte consomment du cache KV et du temps de calcul d'attention $\mathcal{O}((m+n)^2)$ à chaque étape de génération. De plus, l'attention du LLM tend à "oublier" le début du prompt (le texte) lorsque la séquence d'image s'allonge (phénomène de *lost in the middle*).

2. La Cross-Attention

On insère des couches d'attention croisée entre les couches d'auto-attention du LLM, exactement comme dans l'architecture U-Net des modèles de diffusion (Stable Diffusion).

• *Avantage* : Le texte est réinjecté à chaque profondeur du réseau. L'alignement sémantique (CLIP-score) est généralement supérieur.

• *Défaut* : Cela brise la pureté du LLM. On ne peut plus utiliser le modèle pour du texte pur sans désactiver ces couches. L'ajout de paramètres est significatif (environ 15 à 20 % de paramètres en plus).

3. AdaLN (Adaptive Layer Normalization)

Popularisé par les Diffusion Transformers (DiT), cette approche conditionne les paramètres de scale (γ) et de shift (β) de la normalisation de couche (LayerNorm) à l'aide d'un vecteur de conditionnement textuel projeté par un petit MLP.

© 2026 Oussama Ghorbel

• *Mécanisme* : Au lieu d'ajouter des tokens ou des couches d'attention, on module le flux d'information existant.

• *Avantage* : Extrêmement économe en paramètres et en calcul (pas d'augmentation de la complexité de l'attention).

• *Ablation* : Dans un pipeline AR, AdaLN-Zero (où les couches sont initialisées pour être des fonctions identités au démarrage) converge souvent plus vite que la cross-attention, bien que le CLIP-score final puisse être légèrement inférieur pour des prompts très complexes et longs.

Mon conseil d'ingénieur : Si vous adaptez un LLM existant (comme LLaMA ou Mistral) et que vous voulez préserver ses capacités textuelles, le préfixe est obligatoire. Si vous construisez un modèle dédié uniquement à la vision, AdaLN couplé à un pooling du texte est la solution la plus élégante et la plus rapide en inférence.

21.5 — Ablation de la stratégie d'échantillonnage : température, top-k, guidance scale

Une fois le modèle entraîné, la génération est un processus stochastique. Le LLM produit des logits sur le vocabulaire K, et nous devons échantillonner le token suivant. Les hyperparamètres d'inférence ont un impact souvent plus grand sur le FID que des semaines d'entraînement supplémentaire.

Température (T) et Top-k

La température contrôle l'entropie de la distribution de softmax : $p_i = \exp(z_i/T)/\sum \exp(z_j/T)$.

• $T \to 0$: Échantillonnage glouton (argmax). L'image est nette mais manque de diversité, et les artefacts s'accumulent (le modèle s'enferme dans des modes locaux).

• $T > 1$: Distribution aplatie. Le modèle explore, mais génère du bruit visuel (tokens sémantiquement incohérents).

• *Top-k* : Restreint l'échantillonnage aux k tokens les plus probables. Pour les images, un k élevé (ex: 200 à 500 sur un vocabulaire de 8192) est souvent nécessaire pour permettre la diversité des textures, contrairement au texte où $k = 40$ suffit.

© 2026 Oussama Ghorbel

Classifier-Free Guidance (CFG) pour l'Autoregression

La CFG est connue dans la diffusion, mais elle s'applique aussi à l'autoregression. Pendant l'entraînement, on applique un dropout sur le conditionnement textuel (ex: 10 % des prompts sont remplacés par un token `[NULL]`).

À l'inférence, on calcule les logits pour le prompt réel (z_{cond}) et pour le prompt nul (z_{uncond}), puis on interpole :

$$z_{guided} = z_{uncond} + w \cdot (z_{cond} - z_{uncond})$$

où w est le *guidance scale*.

• *Ablation de w* :

• $w = 1.0$: Pas de guidance (équivalent à l'entraînement standard).

• $w \in [3.0, 5.0]$: Amélioration drastique de l'alignement texte-image (CLIP-score) et de la netteté.

• $w > 7.0$: Saturation des couleurs, artefacts de brûlure (*oversaturation*), et dégradation du FID.

• *Coût* : La CFG double le coût de l'inférence (deux passes avant dans le LLM, bien que le cache KV du prompt nul puisse être pré-calculé). C'est une ablation critique pour le déploiement en production.

21.6 — Interprétation des résultats : significativité statistique, intervalles de confiance

C'est ici que la rigueur scientifique sépare les véritables avancées du bruit aléatoire. Le FID et l'Inception Score (IS) sont des estimateurs biaisés et à haute variance.

Le problème de la variance de seed

Si vous entraînez le même modèle avec la même configuration mais avec trois seeds aléatoires différentes (initialisation des poids, ordre de chargement des données), vous obtiendrez trois FID différents. On observe régulièrement des écarts de ± 1.5 points de FID sur des modèles 1B entraînés sur LAION. Si votre nouvelle architecture améliore le FID de 0.8 points par rapport au *baseline*, **vous n'avez rien prouvé**. Vous

© 2026 Oussama Ghorbel

êtes dans la marge d'erreur.

Bootstrapping et Intervalles de Confiance

Pour évaluer la significativité d'une ablation, il ne faut pas calculer le FID sur un seul ensemble de 50 000 images générées. Il faut utiliser le **bootstrapping** :

1. Générez N images (ex: $N = 10000$).
2. Tirez aléatoirement avec remise M sous-ensembles de taille n (ex: 1000 sous-ensembles de 5000 images).
3. Calculez le FID pour chaque sous-ensemble.
4. Calculez la moyenne et l'écart-type de ces FID pour obtenir un intervalle de confiance à 95 %.

De plus, la communauté s'éloigne progressivement du FID basé sur InceptionV3 (entraîné sur ImageNet, donc biaisé vers la classification d'objets sur fond blanc) au profit du **CLIP-FID**, qui utilise les embeddings de CLIP pour calculer la distance de Fréchet. Le CLIP-FID est beaucoup plus corrélé avec la qualité sémantique et la perception humaine, et sa variance est généralement plus faible.

Règle de reporting

Toute table d'ablation dans un rapport ou un article doit inclure la notation $X \pm Y$, où X est la moyenne et Y l'intervalle de confiance ou l'écart-type sur au moins 3 seeds d'entraînement ou 5 seeds d'inférence. Si l'intervalle de confiance de votre méthode chevauche celui du *baseline*, l'ablation est non concluante.

Exercices pratiques

Pour ancrer ces concepts, je vous propose trois exercices qui simulent le quotidien d'un chercheur en IA générative.

Exercice 1 : Concevoir un plan d'ablation complet

Objectif : Estimer le budget computationnel et structurer un plan d'expérience.

© 2026 Oussama Ghorbel

Vous disposez d'un budget de 1000 heures de GPU A100. Un entraînement complet de votre modèle proxy (100M de paramètres, 50 epochs) prend 10 heures. Vous souhaitez ablater 6 facteurs, chacun ayant 3 niveaux :

1. Taille du codebook $K \in \{1024, 4096, 16384\}$
2. Facteur de compression $f \in \{8, 16, 32\}$
3. Ordre de balayage $\in \{\text{Raster, Serpentin, Hilbert}\}$
4. Type de conditionnement $\in \{\text{Préfixe, Cross-Attn, AdaLN}\}$
5. Dropout de conditionnement $\in \{0.0, 0.1, 0.2\}$
6. Taux d'apprentissage $\in \{1e-4, 3e-4, 1e-3\}$

Questions :

1. Combien d'expériences un plan factoriel complet (3^6) nécessiterait-il ? Quel serait le temps de calcul total ?
2. Proposez une stratégie utilisant un plan fractionnaire ou des métriques proxy pour réduire ce temps à moins de 200 heures de GPU, tout en permettant d'identifier les 2 facteurs les plus influents.

Pistes de solution : Un plan complet exige 729 expériences, soit 7290 heures (incompatible avec le budget). L'utilisation d'un plan de Taguchi (tableau orthogonal L_{18} ou L_{27}) ou d'un plan fractionnaire de résolution III permet de tester les effets principaux en 18 ou 27 expériences. Couplé à l'évaluation sur les 10 premières epochs (métrique proxy de vitesse de convergence, prenant 2h au lieu de 10h), le budget tombe sous les 60 heures, laissant le temps d'affiner les interactions sur les meilleurs candidats.

Exercice 2 : Ablation de la température et tracé de courbe

Objectif : Comprendre la dynamique de l'échantillonnage.

Vous avez un modèle AR pré-entraîné. Vous devez réaliser une ablation sur la température $T \in \{0.5, 0.7, 1.0, 1.3\}$ avec un top-k fixe à 300.

1. Décrivez le protocole exact pour calculer le FID et le CLIP-score pour chaque température.

© 2026 Oussama Ghorbel

2. Dessinez (ou décrivez précisément) l'allure théorique des courbes $FID = f(T)$ et $CLIP\text{-}score = g(T)$.

Pistes de solution :

Le protocole exige de générer un ensemble fixe d'images (ex: 5000) à partir des mêmes seeds de bruit et des mêmes prompts pour chaque T, afin de réduire la variance.

Courbe FID : Elle a généralement une forme de "U" ou de "J". À $T = 0.5$, le FID est mauvais (manque de diversité, mode collapse). Il atteint un optimum autour de $T = 1.0$ (ou légèrement en dessous selon le modèle), puis remonte en flèche à $T = 1.3$ à cause de l'incohérence spatiale et du bruit.

Courbe CLIP-score : Elle est souvent décroissante ou en cloche asymétrique. Les basses températures ($T = 0.5$) génèrent des images très "moyennes" mais qui correspondent bien aux concepts génériques du prompt (haut CLIP-score). Les hautes températures détruisent la sémantique (bas CLIP-score). L'intersection des optima dicte le choix de T en production.

Exercice 3 : Template de rapport d'ablation

Objectif : Standardiser la communication des résultats au sein de votre équipe.

Rédigez un template Markdown réutilisable pour documenter toute future expérience d'ablation. Ce template doit forcer la rigueur scientifique.

Proposition de template :

En adoptant ce template, vous forcez votre équipe (et vous-même) à ne jamais prendre de décision architecturale sur la base d'un seul chiffre isolé, mais toujours dans le contexte d'une variance mesurée et d'une hypothèse claire. C'est ainsi que l'on transforme l'alchimie de l'IA en une véritable discipline d'ingénierie.

© 2026 Oussama Ghorbel

Partie VI — Protocoles Pratiques, Déploiement et Pistes de Recherche

© 2026 Oussama Ghorbel

Chapitre 22 – Protocole complet : adapter un LLM 7B en générateur 256 × 256

Nous avons exploré en détail les fondations théoriques, les mécanismes de tokenisation visuelle et les architectures autorégressives qui permettent à un modèle de langage de « comprendre » et de « générer » des pixels. Il est maintenant temps de passer de la théorie à la pratique. Dans ce chapitre, je vous propose de retrousser nos manches pour bâtir un pipeline complet, bout-en-bout. L'objectif est concret : prendre un grand modèle de langage (LLM) de 7 milliards de paramètres, initialement conçu pour prédire du texte, et le transformer en un générateur d'images conditionné par du texte, produisant des images de 256 × 256 pixels.

Ce protocole est le fruit de nombreuses itérations et des pièges classiques que les équipes de recherche et d'ingénierie rencontrent. Il ne s'agit pas de créer le prochain modèle commercial capable de rivaliser avec les systèmes fermés générant des images en 4K, mais de construire un système fonctionnel, cohérent et évaluable, qui vous donnera les clés pour itérer vers des résolutions et des complexités supérieures.

22.1 — Cahier des charges : résolution, latence cible, matériel disponible

Avant d'écrire la moindre ligne de code, il est impératif de définir le cadre physique et mathématique de notre projet. La génération d'images par autoregression est gourmande en calculs, et l'optimisation du triangle résolution-latence-matériel dictera la faisabilité de l'entraînement.

Résolution et séquence visuelle

Nous ciblons une résolution de 256 × 256 pixels. C'est un choix pragmatique. À cette résolution, les structures globales des objets sont clairement discernables, tout en maintenant une séquence de tokens visuels suffisamment courte pour tenir dans la fenêtre de contexte standard d'un LLM de 7B (généralement 2048 ou 4096 tokens). Avec un taux de compression spatial de 8 (que nous détaillerons à l'étape 1), une image de 256 × 256 sera représentée par une grille latente de 32 × 32,

soit 1024 tokens visuels. Ajoutés à une invite textuelle (prompt) de longueur variable (disons 50 à 100 tokens en moyenne), notre séquence d'entraînement culminera autour de 1150 tokens, ce qui est parfaitement gérable.

Matériel disponible et contraintes mémoire

Un modèle de 7 milliards de paramètres, stocké en précision `bfloat16`, pèse environ 14 Go. Cependant, l'entraînement nécessite bien plus que le simple stockage des poids. Avec l'optimiseur AdamW, les états de l'optimiseur (moyennes mobiles et variances) en `float32` ajoutent 8 octets par paramètre, soit 56 Go supplémentaires. Les gradients et les activations lors de la passe avant (forward pass) peuvent facilement doubler cette empreinte.

Pour ce protocole, je pars du principe que vous disposez d'un nœud de calcul standard dans le cloud ou en laboratoire : 8 GPU NVIDIA A100 (80 Go de VRAM) ou H100. L'utilisation de techniques de parallélisme comme FSDP (Fully Sharded Data Parallel) de PyTorch ou ZeRO-3 de DeepSpeed sera obligatoire pour fragmenter les états de l'optimiseur et les activations à travers les 8 GPU.

Latence cible

En inférence, générer 1024 tokens de manière séquentielle prend du temps. Sur un seul GPU A100, un LLM de 7B optimisé (avec FlashAttention et KV-cache) peut générer environ 50 à 80 tokens par seconde. La génération pure des tokens visuels prendra donc entre 12 et 20 secondes. Le décodage par le VQGAN ajoutera moins de 100 millisecondes. Notre latence cible pour une image complète est donc d'environ 15 à 25 secondes, ce qui est acceptable pour un prototype de recherche ou un service asynchrone.

22.2 — Étape 1 : entraîner ou charger un VQGAN (codebook 8 192, compression ×8)

Le VQGAN (Vector Quantized Generative Adversarial Network) est notre traducteur : il fait le pont entre le monde continu des pixels et le monde discret des tokens. Pour ce protocole, nous avons besoin d'un VQGAN qui compresse l'image spatialement d'un facteur 8 et qui utilise un codebook (dictionnaire de vecteurs) de taille 8 192.

© 2026 Oussama Ghorbel

Architecture du VQGAN

L'encodeur du VQGAN, généralement basé sur une architecture ResNet, prend une image $x \in \mathbb{R}^{3 \times 256 \times 256}$ et produit une carte de caractéristiques $z \in \mathbb{R}^{C \times 32 \times 32}$, où C est la dimension des embeddings latents (souvent 256). Le module de quantification remplace ensuite chaque vecteur de dimension C par le vecteur le plus proche dans le codebook de 8 192 entrées. Le décodeur reconstructeur prend cette grille quantifiée et génère l'image finale.

Faut-il l'entraîner de zéro ?

Je vous le dis sans détour : n'entraînez pas votre VQGAN de zéro si vous pouvez l'éviter. L'entraînement d'un VQGAN de haute fidélité nécessite des millions d'images et des semaines de calcul sur plusieurs GPU pour stabiliser la perte d'engagement (commitment loss) et la perte antagoniste (adversarial loss).

Pour ce protocole, nous allons charger un VQGAN pré-entraîné, par exemple celui fourni par la bibliothèque *Taming Transformers* (comme le modèle `vqgan_imagenet_f16_16384` que nous pouvons adapter, ou un modèle équivalent avec un downsampling de 8 et un codebook de 8192).

Validation du tokenizer

Avant de passer à la suite, vous devez valider que votre VQGAN reconstruit correctement vos données cibles. Chargez 100 images de votre jeu de données d'entraînement, encodez-les, décodez-les et calculez le PSNR (Peak Signal-to-Noise Ratio) et le LPIPS (Learned Perceptual Image Patch Similarity). Un PSNR supérieur à 25 dB et un LPIPS inférieur à 0.15 sur des images de 256 × 256 indiquent que le tokenizer est suffisamment fidèle pour que le LLM puisse apprendre une distribution visuelle propre. Si les images reconstruites sont floues ou présentent des artefacts de couleur majeurs, le LLM héritera de ces défauts, peu importe la qualité de son fine-tuning.

22.3 — Étape 2 : étendre le vocabulaire du LLM et initialiser les embeddings visuels

© 2026 Oussama Ghorbel

C'est ici que la chirurgie cérébrale commence. Un LLM comme LLaMA-2 7B ou Mistral 7B possède un vocabulaire textuel d'environ 32 000 tokens. Nous devons lui apprendre à parler le langage des 8 192 tokens visuels de notre VQGAN.

Extension de la matrice d'embedding

La couche d'embedding du LLM, `model.embed_tokens`, est une matrice de poids $W \in \mathbb{R}^{V \times D}$, où V est la taille du vocabulaire (ex: 32000) et D la dimension cachée du modèle (ex: 4096 pour un 7B). Nous devons redimensionner cette matrice pour accueillir les nouveaux tokens.

Nous ajoutons 8 192 tokens visuels, plus trois tokens spéciaux indispensables pour le balisage : `<im_start>`, `<im_end>`, et `<im_pad>`. La nouvelle taille de vocabulaire devient $V_{new} = V + 8195$.

Initialisation des nouveaux poids

Comment initialiser les lignes de la matrice correspondant aux tokens visuels ? Une initialisation purement aléatoire (bruit gaussien) force le modèle à apprendre ces représentations à partir de zéro, ce qui peut déstabiliser les premières étapes du fine-tuning et provoquer des explosions de gradient.

Je recommande une approche plus douce : initialiser les embeddings visuels en tirant des échantillons de la distribution statistique (moyenne et écart-type) des embeddings textuels existants. Cela place les nouveaux tokens dans le même espace géométrique initial que le texte.

Il est également crucial de redimensionner la couche de sortie (`lm_head`), qui projette les états cachés de dimension D vers le vocabulaire de taille V_{new} pour calculer les logits. Si le modèle utilise le "weight tying" (le partage de poids entre la couche d'embedding et la couche de sortie), assurez-vous que votre framework de deep learning propage correctement le redimensionnement aux deux couches simultanément.

Gel partiel et adaptateurs

Pour un modèle de 7B, un fine-tuning complet (Full Parameter Fine-Tuning) est coûteux. Bien que ce protocole suppose un fine-tuning

© 2026 Oussama Ghorbel

complet pour maximiser la qualité, sachez que l'utilisation de LoRA (Low-Rank Adaptation) sur les matrices d'attention et la couche `lm_head`, combinée à l'entraînement exclusif des nouvelles couches d'embedding, est une alternative très populaire qui réduit drastiquement l'empreinte mémoire.

22.4 — Étape 3 : fine-tuning sur un sous-ensemble de 500 k paires texte-image

Avec notre modèle chirurgicalement modifié, nous passons à l'apprentissage. Nous utilisons un sous-ensemble de 500 000 paires texte-image de haute qualité (par exemple, un filtrage esthétique de LAION ou un sous-ensemble de CC3M).

Formatage des séquences

Le LLM est un modèle de langage causal : il prédit le token t en fonction des tokens $< t$. Nous devons formater nos données pour qu'elles reflètent cette causalité. Une séquence d'entraînement typique ressemblera à ceci :

`[BOS] [Tokens Texte] [EOS] <im_start> [1024 Tokens Visuels] <im_end>`

Le token `[BOS]` (Begin of Sequence) et `[EOS]` (End of Sequence) sont standards. Le LLM apprendra d'abord à modéliser la distribution du texte (ce qui agit comme une régularisation), puis, conditionné par le texte et le token `<im_start>`, il prédira les 1024 tokens visuels dans l'ordre de balayage défini (par exemple, raster scan : de gauche à droite, de haut en bas).

Stratégie de masquage de la perte (Loss Masking)

C'est un point critique où de nombreuses erreurs se produisent. Faut-il calculer la perte d'entropie croisée sur les tokens textuels et visuels, ou uniquement sur les tokens visuels ?

Si vous calculez la perte sur tout, le modèle passera une grande partie de sa capacité à mémoriser des faits textuels, ce qui n'est pas notre objectif principal. Je recommande de masquer les tokens textuels dans le calcul de la perte. La perte ne sera rétropropagée que lorsque le modèle essaie de prédire les tokens visuels (et le token `<im_end>`). Cela force

© 2026 Oussama Ghorbel

le réseau à allouer ses gradients à l'apprentissage de la distribution conditionnelle $P(\text{Image}|\text{Texte})$.

Hyperparamètres et stabilité

• **Taux d'apprentissage (Learning Rate)** : Fixez-le entre `1e-5` et `5e-5`. Un taux trop élevé détruira les capacités linguistiques du modèle (catastrophic forgetting) et fera diverger la perte.

• **Batch Size** : Visez un batch size global de 256 à 512 séquences. Avec 8 GPU et une taille de lot par GPU de 4, vous utiliserez l'accumulation de gradients sur 8 à 16 pas.

• **Précision mixte** : Utilisez impérativement le `bfloat16`. Le `float16` classique pose des problèmes de dépassement (overflow) avec les valeurs de logits élevées lors de la génération de longues séquences visuelles.

• **Gradient Checkpointing** : Activez-le. Il sacrifie un peu de temps de calcul (environ 20 %) pour réduire massivement la mémoire des activations, vous permettant d'augmenter la taille des séquences.

• **Classifier-Free Guidance (CFG)** : Pour permettre l'utilisation de CFG à l'inférence (qui améliore drastiquement l'alignement texte-image), remplacez aléatoirement 10 % des invites textuelles par un token `<unconditional>` ou un embedding nul pendant l'entraînement.

22.5 — Étape 4 : inférence, décodage VQGAN, post-traitement

L'entraînement est terminé. Le modèle a convergé. Il est temps de générer. Le processus d'inférence se décompose en trois phases distinctes.

1. Génération autorégressive des tokens

L'utilisateur fournit une invite textuelle : "Un chat astronaut flottant dans l'espace, style réaliste".

Nous tokenisons ce texte, ajoutons `<im_start>`, et nourrissons le LLM. Le modèle génère le premier token visuel, l'ajoute à son KV-cache, puis génère le second, et ainsi de suite jusqu'à produire 1024

© 2026 Oussama Ghorbel

tokens ou rencontrer `<im_end>`.

Paramètres d'échantillonnage : Contrairement au texte où la température est souvent basse (0.7) pour garantir la cohérence factuelle, les images bénéficient d'une stochasticité légère. Utilisez une température de `1.0` avec un *Top-p* (nucleus sampling) de `0.9` ou un *Top-k* de `100`. Cela évite les images ternes et sur-lissées.

Classifier-Free Guidance : Si vous avez entraîné avec du dropout textuel, générez deux séquences de logits en parallèle (ou par batch) : une conditionnée par le texte, une non conditionnée. Interpolez les logits : $L_{final} = L_{uncond} + w \times (L_{cond} - L_{uncond})$, avec une échelle de guidance w généralement fixée entre 3.0 et 5.0.

2. Décodage VQGAN

Vous avez maintenant un tenseur de 1024 entiers (les indices du codebook). Remodelez ce tenseur en une grille de 32×32. Remplacez chaque indice par le vecteur de dimension C correspondant dans le codebook du VQGAN. Passez cette grille de caractéristiques continues dans le décodeur du VQGAN pour obtenir un tenseur de pixels de $3 \times 256 \times 256$.

3. Post-traitement

Les images issues d'un VQGAN souffrent souvent d'artefacts de haute fréquence (un léger effet de "grille" ou de bruit coloré) dus à la quantification. Appliquez un filtre léger, ou mieux, faites passer l'image dans un modèle de super-résolution léger (comme ESRGAN x2) suivi d'un redimensionnement bicubique vers 256 × 256. Cela lisse les artefacts de quantification tout en préservant les structures sémantiques générées par le LLM.

22.6 — Étape 5 : évaluation (FID, CLIP-score) et itération

Comment savoir si notre générateur de 7B est performant ? L'évaluation de la génération d'images est notoirement difficile, mais nous nous reposerons sur deux métriques standards de l'industrie.

Frechet Inception Distance (FID)

Le FID mesure la qualité et la diversité des images générées en comparant les statistiques (moyenne et covariance) des caractéristiques

© 2026 Oussama Ghorbel

extraites par un réseau InceptionV3 sur un ensemble d'images réelles et sur un ensemble d'images générées.

Protocole : Générez 30 000 images à partir d'invites aléatoires tirées de votre jeu de validation. Calculez le FID par rapport aux 30 000 images réelles correspondantes. Pour un modèle de 7B entraîné sur 500k images, un FID compris entre 15 et 25 est un résultat honorable. En dessous de 10, vous approchez de l'état de l'art pour cette échelle de données.

CLIP-score

Le FID ne dit rien de l'alignement entre le texte et l'image. Pour cela, nous utilisons le CLIP-score. Nous passons chaque paire (texte généré, image générée) dans le modèle CLIP (par exemple ViT-L/14) et calculons la similarité cosinus entre les embeddings textuels et visuels.

Un CLIP-score moyen supérieur à 0.28 indique une bonne compréhension sémantique de base. Au-delà de 0.32, le modèle respecte très bien les concepts complexes (couleurs, positions relatives, objets multiples).

Boucle d'itération et diagnostic

Si le FID est bon mais le CLIP-score mauvais, votre modèle génère de belles images mais ignore le texte : augmentez le poids de la CFG à l'inférence, ou vérifiez que le masquage de la perte textuelle n'a pas isolé le texte du contexte visuel.

Si le CLIP-score est bon mais le FID mauvais (images cohérentes mais pleines d'artefacts, couleurs saturées), le problème vient probablement du VQGAN ou d'un taux d'apprentissage trop élevé qui a créé des instabilités dans les couches profondes du LLM.

22.7 — Pseudo-code intégral et checklist de déploiement

Pour ancrer ces concepts, voici la structure du code d'entraînement et la checklist que je vous invite à imprimer et à garder sur votre bureau.

Checklist de déploiement et de production

1. **Filtrage des données** : Avez-vous supprimé les images NSFW, les filigranes (watermarks) et les images de trop basse résolution de

© 2026 Oussama Ghorbel

votre jeu de 500k ?

2. **Gel du VQGAN** : Le VQGAN est-il bien figé (`requires_grad=False`) pour éviter de corrompre le codebook pendant l'entraînement du LLM ?

3. **Gestion du KV-Cache** : Votre script d'inférence utilise-t-il le KV-cache ? Sans lui, la génération de 1024 tokens prendra des minutes au lieu de secondes.

4. **Serving** : Pour le déploiement, avez-vous exporté le modèle vers un format optimisé (ex: ONNX, TensorRT) ou utilisé un serveur d'inférence comme vLLM adapté pour supporter les tokens personnalisés ?

5. **Sécurité** : Avez-vous implémenté un filtre en entrée (pour bloquer les prompts malveillants) et en sortie (pour détecter les générations inappropriées) ?

Exercices pratiques

La lecture de ce chapitre vous a fourni la carte routière, mais c'est en forgeant qu'on devient forgeron. Je vous invite à réaliser les trois exercices suivants pour confronter la théorie à la réalité du hardware et des données.

Exercice 1 : Le protocole à échelle réduite

L'entraînement d'un modèle de 7B nécessite des ressources importantes. Pour cet exercice, adaptez le protocole décrit ci-dessus en utilisant un LLM de 1 à 2 milliards de paramètres (comme GPT-2 XL, ou une version réduite de LLaMA/Qwen). Remplacez le dataset de 500k images par un sous-ensemble de 10 000 images (par exemple, la classe "paysages" d'ImageNet ou un subset de COCO). L'objectif n'est pas la performance absolue, mais la validation mécanique du pipeline : de l'extension du vocabulaire jusqu'à l'obtention d'une image décodée.

Exercice 2 : Rapport d'expérimentation

À l'issue de l'exercice 1, rédigez un rapport technique concis de 2 pages. Ce rapport doit inclure :

© 2026 Oussama Ghorbel

• La configuration matérielle et logicielle exacte.

• Les courbes de perte (loss) d'entraînement et de validation.

• Une galerie de 8 images générées avec leurs invites textuelles correspondantes.

• Le calcul du CLIP-score moyen sur ces 8 images.

Ce format est celui qu'il convient d'exiger des équipes lors des revues de modèles : des faits, des métriques, et des résultats visuels.

Exercice 3 : Chasse aux points de défaillance

L'entraînement de modèles multimodaux est instable. Identifiez et corrigez (sur le papier ou dans votre code) au moins deux points de défaillance classiques de ce pipeline. Par exemple :

• *NaN Loss* : Que se passe-t-il si le LLM prédit un index de token visuel supérieur à 8191 ou inférieur à 0 ? Comment votre couche `lm_head` et votre fonction de perte gèrent-elles les indices hors limites (out-of-bounds) lors des premières étapes chaotiques de l'entraînement ?

• *Codebook Collapse* : Si vous décidiez d'entraîner le VQGAN conjointement avec le LLM (ce que je déconseille pour débuter), comment détectez-vous que le codebook s'effondre (c'est-à-dire que le modèle n'utilise plus que 50 des 8192 tokens disponibles) et quelle technique de régularisation (ex: EMA, entropy regularization) implémenteriez-vous pour y remédier ?

En surmontant ces obstacles, vous ne serez plus de simples utilisateurs de boîtes noires génératives, mais de véritables ingénieurs capables de sculpter la matière même de l'intelligence artificielle visuelle.

22.8 — Analyse et résolution des points de défaillance (Corrigé de l'exercice 3)

Dans l'exercice précédent, je vous ai mis au défi d'identifier et de corriger des points de rupture critiques dans le pipeline. L'entraînement d'un modèle autoregressif sur des tokens visuels est un terrain miné. Voici mon analyse détaillée des trois pièges les plus fatals que vous rencontrerez inévitablement, et les solutions architecturales pour les

© 2026 Oussama Ghorbel

désamorcer.

Piège 1 : La divergence des logits et la perte NaN (Out-of-Bounds)

Lors des toutes premières itérations du fine-tuning, les poids de la couche `lm_head` (qui projette les états cachés vers le nouveau vocabulaire de taille $V + 8195$) sont instables. Le modèle, n'ayant pas encore appris la topologie du codebook VQGAN, peut produire des logits d'une magnitude extrême pour certains indices visuels.

Le problème : Si vous utilisez une perte d'entropie croisée standard (`CrossEntropyLoss` de PyTorch) sans précaution, des logits aberrants combinés à une précision `bfloat16` peuvent provoquer un dépassement de capacité (overflow) lors du calcul du `softmax`, résultant en une perte `NaN`. De plus, si le modèle génère un index supérieur à 8191 (ou inférieur à 0) lors d'une inférence précoce, le décodeur VQGAN plantera avec une erreur d'indexation (`IndexError`), brisant net votre pipeline de génération.

La solution :

1. **Clamp des logits** : Avant d'appliquer le `softmax` ou de calculer la perte, appliquez un clamp strict sur les logits : `logits = torch.clamp(logits, min=-10.0, max=10.0)`. Cela borne les gradients et stabilise l'optimiseur AdamW.

2. **Masque de validité à l'inférence** : Lors de la génération, forcez la probabilité des tokens hors de l'intervalle `[0, 8191]` à zéro (ou à $-\infty$ avant le softmax) en utilisant un `logits_processor` personnalisé. Le modèle ne doit physiquement pas pouvoir émettre un token qui n'existe pas dans le codebook.

Piège 2 : L'effondrement du Codebook (Codebook Collapse)

Bien que j'aie recommandé de figer le VQGAN, imaginons que vous choisissiez une approche plus ambitieuse : l'entraînement conjoint du LLM et du VQGAN (ou le fine-tuning du décodeur VQGAN pour l'adapter à un domaine visuel très spécifique, comme l'imagerie médicale ou satellitaire).

Le problème : Le phénomène de *codebook collapse* survient lorsque l'encodeur n'utilise qu'une infime fraction des 8 192 vecteurs disponibles

© 2026 Oussama Ghorbel

(par exemple, seulement 50 à 100 tokens "morts" qui capturent toute la variance). Le LLM se retrouve alors à modéliser une distribution visuelle sur un vocabulaire effectif de 50 mots, détruisant la richesse sémantique et la diversité des images générées. Les sorties deviennent monotones et les détails s'effondrent.

La solution :

1. **Régularisation par entropie** : Ajoutez une perte auxiliaire qui maximise l'entropie de la distribution d'utilisation des tokens du codebook sur un batch. Si le modèle tend à toujours choisir le même token, la pénalité augmente, forçant l'encodeur à explorer l'espace latent.

2. **Réinitialisation aléatoire (Random Restart)** : Surveillez la fréquence d'utilisation de chaque token du codebook. Si un token n'a pas été sélectionné par l'encodeur depuis N itérations, réinitialisez son vecteur en tirant au hasard un embedding parmi les caractéristiques z actives du batch courant. Cette technique recycle les tokens morts et maintient le codebook vivant.

3. **Exponential Moving Average (EMA)** : Mettez à jour les vecteurs du codebook non pas par rétropropagation classique, mais par une moyenne mobile exponentielle des vecteurs de l'encodeur qui leur sont assignés. Cela lisse l'entraînement et réduit drastiquement les effondrements.

Piège 3 : La fuite du masque d'attention (Attention Mask Leakage)

C'est une erreur d'implémentation subtile mais dévastatrice, observée à de nombreuses reprises, même dans des implémentations open source populaires.

Le problème : Dans un LLM causal, le masque d'attention est triangulaire inférieur. Cependant, lors de la construction de notre séquence hybride `[Texte] <im_start> [Image]`, si vous utilisez un remplissage (padding) à droite pour aligner les batchs et que vous oubliez de mettre à jour le `attention_mask` pour inclure les tokens `<im_pad>`, le modèle portera attention au padding. Pire encore, si vous découplez mal les masques, les tokens de la fin de l'image pourraient "voir" le début de la séquence d'une autre image dans le même batch

© 2026 Oussama Ghorbel

(cross-contamination). Le résultat se manifeste par des artefacts récurrents, souvent en bas à droite de chaque image générée, ou par une dégradation sévère du FID.

La solution :

Validez visuellement votre masque d'attention. Imprimez (ou plottez avec `matplotlib`) la matrice d'attention pour un batch d'entraînement. Vérifiez que les tokens textuels ne peuvent pas voir les tokens visuels futurs, que les tokens visuels voient le texte et les tokens visuels précédents, et surtout, que la diagonale et le padding sont strictement masqués (valeur $-\infty$ dans le calcul de l'attention). Une simple erreur d'indice dans le tenseur `attention_mask` peut ruiner des semaines de calcul sur un cluster de GPU.

22.9 — Optimisation de la boucle de rétroaction et gestion des données

Un protocole d'ingénierie ne s'arrête pas à la compilation du code et à la configuration des hyperparamètres. La gestion du flux de données est souvent le véritable goulot d'étranglement d'un entraînement sur LLM multimodal, transformant des GPU surpuissants en de simples spectateurs attendant les disques durs.

Le goulot d'étranglement du VQGAN en temps réel

Encoder 500 000 images à la volée, à chaque epoch, avec un VQGAN, est un gaspillage monumental de cycles de calcul. Le VQGAN n'étant pas mis à jour (puisqu'il est figé), la tokenisation d'une image donnée sera toujours strictement identique. Faire tourner un encodeur ResNet sur des A100 pendant l'entraînement du LLM est une hérésie économique.

La bonne pratique : Avant de lancer le fine-tuning du LLM de 7B, lancez un script de pré-calcul (preprocessing) sur un cluster CPU ou sur des GPU d'inférence bon marché. Parcourez l'intégralité de votre dataset, encodez chaque image en ses 1024 tokens VQGAN, et sauvegardez ces séquences d'entiers dans un format binaire rapide à lire et à sérialiser (comme des fichiers `parquet` compressés ou des tenseurs PyTorch en mémoire partagée via `lmdb`).

© 2026 Oussama Ghorbel

Votre `DataLoader` pour le LLM ne fera alors que charger des tenseurs d'entiers et du texte tokenizé. Vous éliminerez ainsi toute latence d'E/S et garantirez que vos 8 GPU A100 seront saturés à 100 % par le calcul des gradients du LLM.

Curriculum Learning pour les séquences visuelles

Le LLM est habitué à des distributions textuelles où la cohérence locale prime. Le forcer à apprendre d'un coup 1024 tokens visuels hautement structurés spatialement peut provoquer une divergence précoce ou un mode collapse (où le modèle génère toujours la même image moyenne). D'excellents résultats sont obtenus en appliquant un *curriculum learning* sur la résolution.

• **Phase 1 (10 % des steps)** : Entraînez le modèle sur des images redimensionnées à 128 × 128 (soit une grille latente de 16 × 16 = 256 tokens visuels). Le modèle apprend rapidement la cohérence globale, les formes primaires et l'alignement texte-image de base.

• **Phase 2 (90 % des steps)** : Passez à la résolution cible de 256 × 256 (1024 tokens). Le modèle, ayant déjà acquis une boussole sémantique, se concentre sur les détails fins, les textures et la cohérence spatiale à longue portée.

Cette approche réduit le temps de convergence global de près de 30 % et stabilise la perte d'entropie croisée en évitant au modèle de se perdre dans l'espace des hautes fréquences dès le premier pas.

Conclusion du Chapitre 22

Nous voici au terme de ce protocole complet. En suivant ces étapes — de la définition rigoureuse du cahier des charges à la chirurgie du vocabulaire, en passant par le masquage stratégique de la perte, le pré-calcul des latents et le débogage des points de défaillance — vous avez assemblé bien plus qu'un simple générateur d'images. Vous avez forgé un pont fonctionnel, robuste et évaluable entre la modélisation probabiliste du langage et la synthèse de signaux visuels continus.

Ce que nous avons construit ensemble dans ce chapitre est un système autoregressif pur. Il prédit le pixel (via le token) suivant avec la même mécanique implacable que celle qui lui fait prédire le mot suivant dans une dissertation philosophique ou un script en Python. Les

© 2026 Oussama Ghorbel

résultats, évalués par le FID et le CLIP-score, prouvent que la compression sémantique opérée par le VQGAN est suffisamment riche pour être "lue" et "écrite" par un transformeur initialement conçu pour le texte. Vous avez démontré qu'avec le bon tokenizer, une image n'est rien d'autre qu'une phrase d'une langue étrangère que le LLM peut apprendre à parler.

Cependant, une fois le modèle entraîné, évalué et validé sur votre jeu de test, un nouveau défi, tout aussi redoutable, se dresse devant l'ingénieur : celui de la mise en production. Un modèle de 7 milliards de paramètres qui met 20 secondes à générer une image de 256 × 256 est un triomphe de recherche, mais un échec commercial s'il doit servir des milliers d'utilisateurs simultanés dans une application web. La mémoire VRAM requise pour charger les poids, la gestion dynamique du KV-cache pour des séquences de plus de 1000 tokens, la latence réseau, et la quantification des poids pour accélérer l'inférence sans sacrifier la fidélité du codebook VQGAN sont des problèmes qui nécessitent une expertise pointue en optimisation système et en déploiement distribué.

C'est précisément l'objet de notre prochain chapitre. Nous allons quitter le laboratoire de recherche pour entrer dans la salle des machines de l'inférence à grande échelle. Préparez-vous à compresser, quantifier et déployer.

© 2026 Oussama Ghorbel

Chapitre 23 – Optimisation et déploiement : inférence efficace, quantification, serving

Dans le chapitre précédent, nous avons franchi une étape majeure : adapter un modèle de langage de 7 milliards de paramètres pour qu'il génère des images de 256 × 256 pixels. Sur un banc d'essai de recherche, avec un seul utilisateur et un notebook Jupyter, c'est une victoire. Mais en tant qu'ingénieur ou chercheur, vous savez que la vraie bataille commence lorsque ce modèle doit affronter la réalité de la production.

Faire tourner un Grand Modèle de Langage (LLM) pour générer du texte est déjà un défi d'infrastructure. Le forcer à prédire séquentiellement des centaines, voire des milliers de tokens visuels pour former une image amplifie ces défis de manière exponentielle. La latence, le débit (throughput) et le coût de l'infrastructure deviennent les trois piliers de notre équation de déploiement. Dans ce chapitre, je vous propose de plonger dans les entrailles de l'optimisation système. Nous allons transformer notre prototype académique en un service robuste, capable de servir des images à la demande, sans faire exploser le budget cloud ni tester la patience des utilisateurs.

23.1 — Quantification des poids du LLM : INT8, INT4, GPTQ, AWQ

Un modèle de 7 milliards de paramètres, stocké en précision FP16 (16 bits en virgule flottante), occupe environ 14 Go de mémoire VRAM. Si vous ajoutez à cela la mémoire nécessaire pour le KV-cache (que nous détaillerons plus loin) et les activations intermédiaires, un GPU standard de 16 Go suffira à peine pour une seule requête, sans aucun batch. La quantification des poids est notre première ligne de défense.

L'objectif de la quantification post-entraînement (Post-Training Quantization, PTQ) est de réduire la précision des poids du modèle (par exemple, de FP16 à INT8 ou INT4) tout en préservant la qualité de la génération. Pour un LLM générant des images, la question cruciale est la suivante : la perte de précision dégrade-t-elle le raisonnement spatial du modèle ?

© 2026 Oussama Ghorbel

L'approche **INT8**, popularisée par l'algorithme LLM.int8() (Dettmers et al., 2022), repose sur un constat empirique : certaines caractéristiques cachées (hidden states) présentent des valeurs aberrantes (outliers) qui, si elles sont tronquées en INT8, détruisent la performance. LLM.int8() résout ce problème en effectuant une multiplication matricielle mixte : les poids et activations standards sont quantifiés en INT8, tandis que les canaux présentant des outliers sont isolés et calculés en FP16. Le gain mémoire est de 50 %, et la dégradation du FID (Fréchet Inception Distance) sur nos images générées est généralement imperceptible.

Cependant, pour déployer notre modèle sur des GPU grand public ou des instances cloud plus modestes, l'**INT4** est souvent indispensable. Deux algorithmes dominent actuellement ce domaine : **GPTQ** et **AWQ**.

GPTQ (Frantar et al., 2022) est une méthode de quantification de second ordre. Elle quantifie les poids colonne par colonne en utilisant la matrice hessienne inverse pour compenser l'erreur de quantification sur les poids suivants. C'est une méthode extrêmement efficace, mais elle traite tous les poids de manière relativement uniforme.

AWQ (Activation-Aware Weight Quantization, Lin et al., 2023) introduit une nuance fondamentale : tous les poids n'ont pas la même importance. AWQ observe que 1 % des poids les plus saillants (ceux qui sont multipliés par les activations les plus fortes) sont responsables de la majeure partie de la qualité du modèle. AWQ protège ces poids en les mettant à l'échelle (scaling) avant la quantification, réduisant ainsi drastiquement l'erreur.

Dans la pratique du déploiement de générateurs d'images autoregressifs, AWQ en 4 bits offre le meilleur compromis. Le modèle de 7B tombe à environ 3,5 Go de VRAM. La cohérence globale de l'image (la structure, les proportions) est dictée par ces poids saillants qui sont préservés, tandis que les textures fines, gérées en aval par le décodeur VQGAN (qui, lui, reste en FP16 car il est très léger), ne souffrent d'aucune dégradation.

23.2 — KV-cache optimisé : paged attention, sliding window, quantification du cache

© 2026 Oussama Ghorbel

L'autorégression a un coût caché mais redoutable : le KV-cache. À chaque étape de génération, le modèle doit calculer l'attention entre le token courant et tous les tokens précédents. Pour éviter de recalculer les clés (Keys) et les valeurs (Values) des couches passées, on les stocke en mémoire.

Pour un LLM textuel standard, une séquence de 500 tokens est courante. Mais pour notre générateur d'images, un prompt textuel de 50 tokens suivi d'une image de 256 × 256 (tokenisée en 256 ou 1024 patches selon le tokenizer) représente une séquence de 1074 tokens. La mémoire requise pour le KV-cache croît linéairement avec la longueur de la séquence et le nombre de couches. Sur un modèle 7B avec 32 couches et une dimension cachée de 4096, le KV-cache pour une seule séquence de 1024 tokens en FP16 peut facilement dépasser 1 ou 2 Go. Si vous essayez de servir 10 utilisateurs simultanément, le KV-cache saturera la VRAM bien avant les poids du modèle.

C'est ici qu'intervient la **PagedAttention** (Kwon et al., 2023), le moteur du framework vLLM. L'analogie avec la mémoire virtuelle des systèmes d'exploitation est frappante. Au lieu d'allouer un bloc de mémoire contigu et maximal pour chaque séquence (ce qui crée une fragmentation interne et externe massive), la PagedAttention divise le KV-cache en blocs fixes (par exemple, 16 tokens). Ces blocs sont alloués à la demande dans une table de pages. Cette élimination de la fragmentation permet d'augmenter le débit du système de 2 à 4 fois, un gain inestimable lorsque l'on génère de longues séquences de tokens visuels.

Qu'en est-il de la **fenêtre glissante** (sliding window), popularisée par des modèles comme Mistral ? L'idée est de ne conserver l'attention que sur les W derniers tokens. Si cela fonctionne merveilleusement pour le texte (où la syntaxe locale prime), c'est un piège mortel pour l'image. Dans une génération autoregressive 2D, le token en bas à droite de l'image doit pouvoir "voir" les tokens en haut à gauche pour maintenir la cohérence de la composition globale (par exemple, la symétrie d'un visage ou la continuité d'un horizon). La fenêtre glissante détruit cette cohérence à longue portée. Pour l'image, la PagedAttention couplée à une attention globale est obligatoire.

© 2026 Oussama Ghorbel

Enfin, la **quantification du KV-cache** (en FP8 ou INT8) est une piste d'optimisation mémoire supplémentaire. Contrairement aux poids du modèle qui sont statiques, les activations du cache sont dynamiques et présentent une variance élevée. Les approches récentes comme KIVI ou KVQuant utilisent des stratégies de quantification asymétriques (par canal pour les Keys, par token pour les Values) pour compresser le cache de 2 à 4 fois avec un impact minimal sur le CLIP-score des images générées.

23.3 — Compilation et fusion de kernels : torch.compile, TensorRT, FlashAttention

Une fois la mémoire maîtrisée, il faut s'attaquer au temps de calcul. Le goulot d'étranglement d'un Transformer n'est pas toujours la puissance de calcul brute (FLOPs), mais souvent la bande passante mémoire (Memory Bandwidth Bound).

FlashAttention (Dao et al., 2022) a révolutionné ce domaine. L'algorithme d'attention standard nécessite de lire et d'écrire la matrice d'attention $N \times N$ (où N est la longueur de la séquence) depuis la mémoire haute bande passante (HBM) du GPU vers la mémoire cache (SRAM). Pour nos séquences de 1024+ tokens, ces allers-retours saturent le bus mémoire. FlashAttention utilise une approche "IO-aware" : il divise les matrices en blocs qui tiennent dans la SRAM, effectue les calculs de softmax et de multiplication matricielle directement dans ce cache ultra-rapide, et ne renvoie en HBM que le résultat final. L'utilisation de FlashAttention 2 ou 3 est non négociable pour notre pipeline : elle réduit la latence de la couche d'attention de 30 à 50 %.

Au-delà de l'attention, le reste du modèle (les couches feed-forward, les normalisations, les activations SiLU) peut être optimisé via la compilation de graphes. **`torch.compile`**, introduit dans PyTorch 2.0, utilise le backend Inductor pour analyser le graphe de calcul du modèle et générer des kernels Triton optimisés. La fusion de kernels (kernel fusion) est le bénéfice principal : au lieu de lancer un kernel GPU pour la normalisation, un autre pour la multiplication, et un troisième pour l'activation (ce qui oblige à lire et écrire en mémoire globale à chaque étape), Inductor fusionne ces opérations en un seul kernel qui s'exécute en un seul passage mémoire. Sur la partie LLM de notre générateur,

© 2026 Oussama Ghorbel

`torch.compile` offre généralement un gain de vitesse de 15 à 25 % sans aucune modification du code source.

Pour le décodeur VQGAN (qui transforme les tokens discrets en pixels continus), la donne est différente. Ce n'est pas un Transformer, mais un réseau convolutif (CNN) profond. Ici, **TensorRT** de NVIDIA reste le roi incontesté. En exportant le décodeur vers un moteur TensorRT, on peut appliquer des fusions de couches convolutives, optimiser les formats de tenseurs (NHWC vs NCHW) et forcer l'exécution en FP16 ou INT8 avec une latence de l'ordre de la milliseconde. Séparer le pipeline en deux moteurs d'inférence (vLLM/TorchCompile pour le LLM, TensorRT pour le VQGAN) est une architecture que je recommande vivement.

23.4 — Batching dynamique et continuous batching pour le serving

Le *batching* (regroupement de requêtes) est essentiel pour saturer les cœurs Tensor d'un GPU. Mais le *static batching* traditionnel est inadapté à notre cas d'usage. Imaginez que vous regroupiez 4 requêtes dans un batch. La requête A demande une courte description textuelle (50 tokens). Les requêtes B, C et D demandent la génération d'une image (1024 tokens). En static batching, la requête A terminera en une fraction de seconde, mais son slot GPU restera occupé (ou paddé) jusqu'à ce que la requête D ait fini de générer son millième token visuel. Le GPU gaspille alors des cycles de calcul précieux.

La solution réside dans le **Continuous Batching** (ou *iteration-level scheduling*), introduit par le système Orca (Yu et al., 2022) et popularisé par vLLM et TensorRT-LLM. Au lieu d'attendre la fin d'une séquence entière pour libérer un slot, le continuous batching opère au niveau de l'étape de génération (le token). Dès que la requête A génère son token de fin (EOS), elle est immédiatement évincée du batch, et une nouvelle requête en attente dans la file peut insérer son premier token dans le batch à l'itération suivante.

Pour un générateur d'images, cette granularité fine est cruciale. La génération d'une image prend du temps (souvent plusieurs secondes). Le continuous batching permet de maintenir un taux d'occupation du GPU proche de 100 % en injectant en continu de nouveaux prompts textuels

© 2026 Oussama Ghorbel

ou de nouveaux débuts de séquences d'images pendant que les autres séquences terminent leur balayage spatial.

De plus, le continuous batching moderne intègre des mécanismes de **préemption**. Si la mémoire du KV-cache vient à manquer à cause d'un afflux soudain de requêtes générant de très grandes images, le système peut "swapper" le KV-cache des requêtes les moins prioritaires vers la mémoire RAM du serveur (CPU), libérant ainsi la VRAM pour les requêtes actives, avant de les recharger plus tard. Cette élasticité est la clé d'un serving résilient.

23.5 — Architecture de serving : FastAPI + GPU worker, file d'attente, cache de résultats

Concevoir l'architecture logicielle requiert de découpler la gestion des requêtes HTTP de l'exécution lourde sur GPU. Une architecture monolithique où FastAPI charge directement le modèle en mémoire est une recette pour le désastre (blocage du GIL en Python, incapacité à scaler, timeouts).

Je préconise une architecture en trois tiers :

1. **Le Routeur API (FastAPI)** : Léger, asynchrone, il reçoit les requêtes, valide les payloads (taille du prompt, paramètres de sécurité), et attribue un identifiant unique.

2. **Le Message Broker (Redis / RabbitMQ / Kafka)** : Il agit comme une file d'attente tampon. Il absorbe les pics de trafic (burst) et permet de prioriser les requêtes (par exemple, les utilisateurs premium passent devant).

3. **Les GPU Workers** : Ce sont des processus isolés (gérés par Ray Serve, Triton Inference Server, ou des workers Celery custom) qui écoutent la file, chargent le modèle en VRAM, exécutent le continuous batching, et renvoient le résultat.

Un point d'architecture spécifique à notre générateur d'images concerne le **cache de résultats**. La génération d'image est coûteuse et déterministe (si la graine aléatoire, ou *seed*, est fixée). Implémenter un cache exact (basé sur le hash du prompt + seed + paramètres) dans une base Redis permet de servir instantanément les requêtes redondantes.

© 2026 Oussama Ghorbel

Plus avancé encore : le **cache sémantique**. En calculant l'embedding du prompt textuel via un petit modèle (comme un CLIP text encoder), on peut chercher dans une base vectorielle (comme Milvus ou Qdrant) si un prompt sémantiquement similaire a déjà été généré récemment. Si la distance cosinus est inférieure à un seuil strict, on peut renvoyer l'image existante, économisant ainsi des cycles GPU massifs pour des prompts quasi-identiques.

Enfin, le décodage final (VQGAN) peut être déporté. Le LLM génère un tenseur de tokens discrets. Ce tenseur, léger, peut être envoyé via gRPC à un cluster de "décodeurs" tournant sur des GPU moins chers (ou même des CPU optimisés avec ONNX Runtime), libérant le GPU principal (H100/A100) pour se concentrer uniquement sur la prédiction autoregressive, qui est la vraie goulet d'étranglement.

23.6 — Monitoring : latence P50/P99, taux d'erreur, drift de qualité

Déployer sans monitorer, c'est piloter à l'aveugle. Les métriques système classiques (CPU, RAM, utilisation GPU) sont insuffisantes pour un LLM. Il faut instrumenter le pipeline avec des métriques spécifiques à l'inférence générative, généralement exposées via Prometheus et visualisées dans Grafana.

Pour la latence, nous devons distinguer deux phases :

- **TTFT (Time To First Token)** : Le temps entre la réception du prompt et l'émission du premier token (texte ou visuel). C'est la métrique de perception de la réactivité pour l'utilisateur.
- **TPOT (Time Per Output Token)** : Le temps de calcul pour chaque token subséquent. Pour une image de 1024 tokens, un TPOT de 5 ms signifie environ 5 secondes de génération totale.

Suivre la distribution P50 (médiane) et P99 (le 99ème percentile) est vital. Un P99 qui s'envole indique souvent des problèmes de swapping de KV-cache ou de garbage collection Python.

Le **taux d'erreur** doit être segmenté. Les erreurs `OOM` (Out Of Memory) sur le GPU indiquent une mauvaise configuration du continuous batching ou une séquence maximale mal bornée. Les erreurs

© 2026 Oussama Ghorbel

de timeout indiquent que la file d'attente est saturée.

Mais le monitoring le plus insidieux concerne le **drift de qualité**. Un modèle quantifié ou compilé peut subir une dégradation silencieuse de ses capacités génératives au fil des mises à jour du code ou des changements de drivers CUDA. Pour contrer cela, mettez en place un pipeline d'évaluation asynchrone : échantillonnez aléatoirement 1 % des images générées en production, et calculez en arrière-plan leur CLIP-score (pour la correspondance texte-image) et un FID proxy par rapport à un ensemble de référence. Si le CLIP-score moyen chute de manière statistiquement significative, une alerte doit être déclenchée avant que les utilisateurs ne s'en plaignent.

23.7 — Estimation des coûts cloud pour 1 000 images/heure

Abordons maintenant la réalité économique. *Avertissement : les estimations de coûts cloud présentées ici sont purement informatives, basées sur des tarifs publics approximatifs à un instant T, et ne constituent en aucun cas un conseil financier ou une garantie de prix. Les tarifs des fournisseurs cloud fluctuent et dépendent de négociations contractuelles.*

Prenons un objectif de production : générer 1 000 images de 256 × 256 par heure, avec des prompts de 50 tokens en entrée et 1024 tokens visuels en sortie.

Supposons que notre modèle 7B, quantifié en AWQ 4-bits et optimisé avec FlashAttention et continuous batching, tourne sur un GPU NVIDIA A10G (24 Go VRAM). Dans ce scénario optimisé, le TPOT est d'environ 4 ms.

Générer 1024 tokens prend donc : $1024 \times 4 \text{ ms} \approx 4,1 \text{ secondes}$.

Ajoutons 0,5 seconde pour le décodage VQGAN et la latence réseau. Nous sommes à ~4,6 secondes par image.

Pour générer 1 000 images en une heure (3 600 secondes), le temps de calcul GPU total requis est de : $1000 \times 4,6 = 4600 \text{ secondes}$.

Cela représente $4600/3600 \approx 1,27$ GPU à temps plein.

Pour absorber cette charge avec une marge de sécurité de 30 % (pour gérer les pics de trafic et éviter la saturation du continuous batching), il

© 2026 Oussama Ghorbel

nous faut **2 GPU A10G**.

Sur un cloud provider majeur (comme AWS avec des instances g5.xlarge ou GCP avec des g2-standard), une instance avec un A10G coûte environ 0,80 $ à 1,00 $ de l'heure en tarif "On-Demand".

Coût horaire : $2 \times 1,00 \$ = 2,00 \$$/heure.

Coût mensuel (730 heures) : **1 460 $ / mois**.

Pour réduire cette facture, l'ingénierie système offre des leviers puissants :

1. **Instances Spot / Preemptible** : Puisque la génération d'image est une tâche qui peut être mise en file d'attente et reprise en cas d'interruption (si l'architecture de queue est bien conçue), l'utilisation d'instances Spot peut réduire la facture de 60 à 70 %.
2. **Serveless GPU / Scale-to-Zero** : Si le trafic de 1 000 images/heure n'est pas uniformément réparti (ex: pics le jour, nul la nuit), des providers comme Modal, RunPod ou Replicate permettent de payer à la seconde d'inférence.
3. **Spéculation de tokens (Speculative Decoding)** : Utiliser un petit modèle "draft" (ex: un modèle 1B) pour proposer des séquences de tokens visuels, que le modèle 7B vérifie en parallèle. Cela peut doubler le débit (TPOT) et diviser le coût par deux.

Exercices Pratiques

Pour ancrer ces concepts d'optimisation et de déploiement, je vous propose trois exercices pratiques qui couvrent la chaîne complète, de la compression du modèle à son exposition via une API, jusqu'au dimensionnement système.

Exercice 1 : Quantification et arbitrage Mémoire / Qualité

Objectif : Quantifier le LLM 7B en INT8 et mesurer l'impact réel sur la génération d'images.

1. Utilisez la bibliothèque `bitsandbytes` et l'implémentation `AutoModelForCausalLM` de Hugging Face avec l'argument

© 2026 Oussama Ghorbel

`load_in_8bit=True`.

2. Créez un script qui mesure la mémoire VRAM allouée (via `torch.cuda.memory_allocated()` et `nvidia-smi`) avant et après le chargement.

3. Générez un dataset de test de 100 images avec le modèle FP16, puis 100 images avec les *mêmes* prompts et *mêmes* seeds avec le modèle INT8.

4. Calculez le FID entre les deux ensembles d'images, ainsi que le CLIP-score moyen par rapport aux prompts.

Livrable attendu : Un tableau comparatif (Mémoire VRAM, Temps d'inférence moyen, FID, CLIP-score) et une conclusion sur la viabilité de l'INT8 pour votre cas d'usage spécifique.

Exercice 2 : Déploiement d'un endpoint FastAPI

Objectif : Écrire un service web minimaliste mais robuste pour exposer le générateur.

1. Initialisez un projet FastAPI. Créez un endpoint `POST /generate` qui accepte un payload JSON : `{"prompt": str, "seed": int, "size": str}`.

2. Le modèle doit être chargé en mémoire au démarrage de l'application (dans le `lifespan` de FastAPI pour éviter de le recharger à chaque requête).

3. L'endpoint doit appeler le pipeline de génération, récupérer le tenseur de pixels, le convertir en image PIL, l'encoder en base64, et retourner une réponse JSON : `{"image_base64": str, "generation_time_ms": float}`.

4. Ajoutez une gestion basique des erreurs (try/except) pour capturer les OOM et retourner un code HTTP 503 avec un message explicite.

Livrable attendu : Le code source Python (`main.py`) et un fichier `Dockerfile` permettant de conteneuriser l'application avec les dépendances CUDA appropriées.

Exercice 3 : Dimensionnement d'infrastructure pour 10 000 requêtes/jour

© 2026 Oussama Ghorbel

Objectif : Concevoir une architecture cloud sur le papier pour un trafic soutenu.

1. Calculez le nombre moyen de requêtes par seconde (RPS) pour 10 000 requêtes réparties uniformément sur 24h, puis modélisez un scénario de pic où 40 % du trafic tombe sur une fenêtre de 4 heures.

2. En vous basant sur l'hypothèse d'un TPOT de 5 ms et d'une séquence moyenne de 800 tokens visuels, calculez le nombre de GPU A100 (40 Go) nécessaires pour absorber le pic de trafic avec un continuous batching configuré pour un batch size dynamique maximal de 32.

3. Estimez la bande passante réseau nécessaire pour servir les images (non compressées vs WebP) et la taille de la base Redis nécessaire pour un cache sémantique conservant les embeddings des 7 derniers jours.

Livrable attendu : Un schéma d'architecture (que vous pouvez décrire textuellement ou dessiner) incluant le Load Balancer, les nœuds API, le Broker de messages, les Workers GPU et le Cache, accompagné d'un tableau de dimensionnement matériel et d'une estimation budgétaire mensuelle théorique.

© 2026 Oussama Ghorbel

Chapitre 24 – Frontières de la recherche : ce qui vient après le token-pixel

Nous arrivons au terme de cet ouvrage. Tout au long de ces pages, j'ai cherché à vous transmettre non seulement les mécanismes mathématiques et architecturaux qui permettent à un grand modèle de langage de « rêver » en pixels, mais aussi l'intuition d'ingénieur nécessaire pour dompter ces architectures. Nous avons construit, brique par brique, le pont entre le symbole discret et le signal continu. Pourtant, en science de l'apprentissage automatique, achever une construction revient souvent à observer les fondations de la prochaine. Le paradigme du *token-pixel* tel que nous l'avons formalisé dans la Partie II et III est déjà en pleine mutation.

Dans ce chapitre final, je vous propose de lever les yeux de nos *codebooks* et de nos matrices d'attention pour observer l'horizon. Quelles sont les limites théoriques de l'autorégression visuelle ? Comment les modèles unifiés redéfinissent-ils la frontière entre les modalités ? Et surtout, quelle est notre responsabilité, en tant que praticiens, face à la puissance de ces systèmes ? La recherche n'attend pas, et c'est ici que votre propre travail commence.

24.1 — Scaling laws pour la génération visuelle autoregressive : au-delà de 70 B paramètres

Les lois d'échelle (*scaling laws*) ont été la boussole de la révolution des LLM. Les travaux fondateurs de Kaplan et al. (2020), puis l'approche *compute-optimal* de Hoffmann et al. (2022) avec Chinchilla, ont démontré que la perte en validation suivait une loi de puissance prévisible en fonction de la taille du modèle, du volume de données et du budget computationnel. Mais qu'en est-il lorsque l'espace de sortie n'est plus le langage naturel, mais une grille de tokens visuels ?

La première frontière de la recherche actuelle consiste à établir des lois d'échelle spécifiques à la génération visuelle autoregressive. Contrairement au texte, où l'entropie par token est relativement élevée et structurée par la grammaire et la sémantique, le token visuel (issu d'un VQGAN ou d'un auto-encodeur) possède une redondance spatiale

© 2026 Oussama Ghorbel

massive. Un modèle de 70 milliards de paramètres, entraîné avec les proportions optimales de Chinchilla pour le texte, pourrait être drastiquement sur-paramétré pour une simple génération d'images en 256 × 256 (soit 256 à 1024 tokens selon le taux de compression), mais sévèrement sous-capacitaire pour de la haute résolution (4096 tokens et plus).

Les chercheurs explorent aujourd'hui comment l'erreur de prédiction de tokens visuels scale avec la résolution et la complexité de la scène. L'hypothèse émergente est que la « taille effective » du vocabulaire visuel et la topologie de l'espace latent nécessitent des exposants de loi de puissance différents de ceux du texte. De plus, le coût computationnel de l'attention quadratique devient prohibitif : générer une image 1024 × 1024 tokenisée en 4096 éléments exige une fenêtre de contexte qui rivalise avec l'analyse de documents longs. Les architectures à attention linéaire (comme les State Space Models, Mamba, ou les variantes de RWKV) appliquées aux tokens visuels constituent un axe de recherche majeur pour contourner ce mur computationnel et permettre le passage à l'échelle au-delà de 70 B paramètres sur des séquences visuelles denses.

24.2 — Modèles unifiés natifs (Chameleon, Emu, Transfusion) : texte, image, audio, vidéo

Pendant longtemps, l'approche dominante a consisté à « greffer » un générateur d'images sur un LLM pré-entraîné, utilisant le texte comme conditionnement via des mécanismes de *cross-attention* (comme dans Stable Diffusion conditionné par T5 ou CLIP). Aujourd'hui, la frontière s'efface au profit de modèles unifiés natifs, où le texte, l'image, l'audio et la vidéo partagent la même colonne vertébrale Transformer et le même espace d'embedding.

Des architectures comme Chameleon (Meta) ont démontré la viabilité de la *early fusion* pure : les tokens de texte et les tokens d'images sont entrelacés dans une séquence unique et traités par les mêmes couches d'attention, sans adaptateur spécifique. Le modèle apprend à allouer sa capacité de représentation de manière dynamique selon la modalité. D'autres approches, comme Emu, poussent cette unification en alignant les espaces latents de différentes modalités dans un espace continu commun avant la quantification, permettant une

© 2026 Oussama Ghorbel

transition fluide entre la compréhension et la génération.

L'innovation la plus fascinante de cette frontière est peut-être Transfusion. Ce paradigme reconnaît que l'entropie et la nature probabiliste du texte (discret, symbolique) diffèrent fondamentalement de celles des pixels (continu, structuré par des fréquences spatiales). Transfusion entraîne un Transformer unique avec une fonction de perte hybride : l'entropie croisée (*cross-entropy*) pour les tokens de texte, et une perte de diffusion (ou d'erreur quadratique) pour les patches d'images. Le modèle apprend ainsi à être autoregressif pour le langage et bidirectionnel/diffusionnel pour la vision, le tout au sein des mêmes poids. Cette hybridation des objectifs d'entraînement au sein d'un seul réseau redéfinit ce que signifie « comprendre » et « générer » pour une intelligence artificielle générale.

24.3 — Génération 3D et vidéo par tokens : extension de la tokenisation

Si tokeniser une image 2D revient à aplatir une grille spatiale en une séquence 1D, la vidéo et la 3D introduisent des dimensions supplémentaires qui font exploser la complexité de la factorisation autoregressive. La recherche actuelle s'attaque à la tokenisation spatio-temporelle et volumétrique.

Pour la vidéo, le défi est de capturer la cohérence temporelle sans sacrifier la résolution spatiale. Les tokenizers vidéo (comme ceux utilisés dans les modèles de génération vidéo autoregressive) appliquent des convolutions 3D ou des patchifications spatio-temporelles pour compresser non seulement les redondances spatiales, mais aussi la persistance temporelle (le fait qu'un pixel de fond ne change pas d'une frame à l'autre). L'ordre de balayage (que nous avons détaillé au Chapitre 11) devient crucial : faut-il générer frame par frame (causalité temporelle stricte) ou générer des blocs spatio-temporels ? Les masques d'attention causale 3D permettent au modèle de prédire le futur d'une scène en conditionnant sur le passé spatial et temporel, ouvrant la voie à des simulateurs de monde (*world models*) capables de raisonner sur la physique et la dynamique.

Pour la 3D, la tokenisation est encore plus complexe. Les représentations 3D (maillages, nuages de points, champs de radiation

© 2026 Oussama Ghorbel

neuronaux ou *Gaussian Splats*) n'ont pas la structure de grille régulière d'une image. Les chercheurs développent des tokenizers qui projettent ces structures 3D dans des grilles latentes 3D voxélisées, ou qui tokenisent des vues multiples (multi-view) avec une contrainte de cohérence épipolaire intégrée dans l'attention. L'objectif est de permettre à un LLM de générer un objet 3D manipulable aussi naturellement qu'il génère une phrase, en traitant les coordonnées spatiales, les normales et les textures comme des symboles d'un vocabulaire géométrique.

24.4 — Tokenizers apprenables de bout en bout avec le LLM (end-to-end)

Le pipeline standard que nous avons exploré dans ce livre repose sur une séparation stricte : on entraîne d'abord un auto-encodeur (VQGAN, VQ-VAE) pour obtenir un *codebook* fixe, puis on gèle ce tokenizer pour entraîner le LLM sur les indices discrets. Cette approche en deux étapes est pragmatique, mais elle crée un goulot d'étranglement théorique : le LLM ne peut pas rétropropager le gradient de sa propre perte de génération vers le *codebook* pour l'adapter à ses besoins spécifiques de modélisation autoregressive.

La frontière de la recherche s'oriente vers l'entraînement de bout en bout (*end-to-end*). L'obstacle majeur est l'opération `argmin` lors de la quantification vectorielle, qui n'est pas différentiable. Les premières solutions utilisaient l'estimateur *straight-through* (copiant le gradient de la sortie vers l'entrée), mais cela mène souvent à l'effondrement du *codebook* (*codebook collapse*), où seuls quelques vecteurs sont utilisés.

Aujourd'hui, des techniques de relaxation continue (comme Gumbel-Softmax) ou des mécanismes de quantification douce (*soft quantization*) permettent un flux de gradient stable du LLM vers le tokenizer. Dans ce paradigme, le LLM « sculpte » l'espace latent de l'image pour qu'il soit optimal pour la prédiction du token suivant, et non plus simplement pour la reconstruction pixel. Cela signifie que le tokenizer apprend à ignorer le bruit haute fréquence inutile pour l'autorégression et à se concentrer sur les structures sémantiques et compositionnelles. Bien que le coût en mémoire et la complexité de stabilisation de l'entraînement soient immenses, cette unification de l'encodage et de la modélisation promet des gains drastiques en fidélité

© 2026 Oussama Ghorbel

et en cohérence logique des images générées.

24.5 — Efficacité : génération en 1-4 steps, distillation, consistency models hybrides

L'autorégression pure a un défaut majeur en production : la latence. Générer une image de 1024 tokens exige 1024 passages avant (*forward passes*) dans le réseau, ce qui se traduit par plusieurs secondes, voire dizaines de secondes, d'inférence sur un GPU standard. Face à la rapidité des modèles de diffusion (qui opèrent en 20 à 50 steps parallélisables), l'autorégression visuelle doit évoluer pour survivre dans des applications interactives.

La recherche se concentre sur la réduction radicale du nombre d'étapes de génération. Une première voie est l'autorégression masquée (comme MaskGIT ou MAR), où le modèle génère l'image en quelques itérations de *refinement* plutôt que token par token. En combinant ces approches avec des techniques de distillation, les chercheurs parviennent à créer des *consistency models* discrets. L'idée est d'entraîner un modèle étudiant à prédire le résultat final d'une trajectoire de génération masquée en un seul, ou tout au plus quatre, passages.

Une autre piste fascinante est l'hybridation : utiliser un petit modèle autoregressif pour générer un « plan » sémantique de l'image (une esquisse en basse résolution de tokens latents) en quelques dizaines de steps, puis utiliser un décodeur de type diffusion ou un modèle de consistance pour « upscaler » et détailler ce plan en haute résolution en un temps record. Cette architecture en cascade, où le LLM agit comme un moteur de raisonnement compositionnel et un réseau léger comme moteur de rendu, est en train de devenir le standard pour le déploiement à grande échelle, réduisant la latence à des centaines de millisecondes tout en conservant la capacité de *prompting* complexe des LLM.

24.6 — Interprétabilité : que « voit » le LLM quand il génère une image ?

À mesure que ces modèles gagnent en puissance, leur nature de « boîte noire » devient inacceptable, tant pour le débogage que pour la sécurité. L'interprétabilité mécanistique, qui a fait des bonds de géant dans le domaine du texte (avec l'identification de *features* sémantiques

© 2026 Oussama Ghorbel

via des auto-encodeurs épars), s'attaque désormais à la génération visuelle.

Que se passe-t-il dans les couches intermédiaires du Transformer lorsqu'il s'apprête à générer le token correspondant à l'œil d'un chien ? Les chercheurs utilisent des techniques de *probing* et d'ablation de couches pour cartographier comment le modèle construit la scène. On découvre que les premières couches d'attention se spécialisent dans la syntaxe spatiale (les contours, les grilles de perspective), tandis que les couches profondes encodent la sémantique globale et les relations d'objets.

Plus fascinant encore, l'analyse des vecteurs de résidus (*residual stream*) révèle l'existence de « concepts visuels latents » qui n'ont pas d'équivalent direct en langage naturel. Le LLM ne se contente pas de traduire des mots en pixels ; il manipule des représentations géométriques et éclairagistes internes. Comprendre ces représentations permet non seulement de corriger les artefacts (comme les fameuses mains à six doigts, souvent dus à une mauvaise agrégation de concepts de contours dans les couches intermédiaires), mais aussi d'éditer l'image *pendant* sa génération en intervenant directement sur les activations du réseau, sans même modifier le *prompt* textuel.

24.7 — Éthique, régulation et open-source : responsabilités du praticien

Nous ne pouvons clore cet ouvrage sans aborder la dimension éthique et réglementaire. La capacité de transformer un LLM en générateur d'images haute fidélité soulève des questions profondes concernant la désinformation, les *deepfakes*, et la violation des droits d'auteur. En tant qu'ingénieurs et chercheurs, nous ne sommes pas de simples observateurs de ces impacts ; nous en sommes les architectes.

La régulation, notamment via l'AI Act en Europe, impose désormais des obligations de transparence et de traçabilité pour les contenus générés par l'IA. C'est ici que la recherche sur le *watermarking* (tatouage numérique) des tokens visuels devient cruciale. Contrairement aux images pixeliques où le tatouage peut être détruit par une simple compression JPEG, le tatouage dans l'espace des tokens (par exemple, en biaisant légèrement la distribution de probabilité du *codebook* lors de

© 2026 Oussama Ghorbel

la génération, ou en réservant des sous-espaces du *codebook* à des signatures cryptographiques) offre une robustesse inédite.

De plus, la décision de rendre open-source un modèle autoregressif visuel de grande taille n'est pas anodine. Si l'open-source démocratise l'accès à la technologie et permet à la communauté de découvrir des vulnérabilités, il facilite également la génération de contenus illicites ou non consensuels à l'échelle industrielle. Le praticien doit aujourd'hui intégrer des garde-fous (*safety filters*, *RLHF* appliqué aux préférences visuelles, refus conditionnel) directement dans le protocole d'entraînement. L'alignement d'un LLM ne se limite plus à l'empêcher de générer du texte toxique ; il doit également refuser de composer des scènes visuelles dangereuses, biaisées ou trompeuses, ce qui nécessite des jeux de données de préférence image-texte d'une complexité nouvelle.

Exercices de fin de chapitre

Pour clore ce livre, je vous invite à passer du statut de lecteur à celui de chercheur. Les exercices suivants sont conçus pour tester votre maîtrise des concepts et votre capacité à projeter le protocole que nous avons développé dans les territoires inexplorés de la recherche.

Exercice 1 : Rédaction d'une note de recherche (Extension du protocole)

Rédigez une note de recherche structurée (environ 2 pages, format article de conférence type *NeurIPS* ou *ICLR*) proposant une extension originale du protocole d'adaptation d'un LLM 7B en générateur visuel décrit dans la Partie V.

- **Contrainte :** Vous devez intégrer une des frontières évoquées dans ce chapitre (par exemple, l'utilisation d'un *consistency model* pour la phase de décodage, ou l'introduction d'un masque d'attention spatio-temporel pour la génération de courtes vidéos).
- **Attendu :** Votre note doit inclure un résumé (*abstract*), une introduction motivant le choix de l'extension, une description formelle de la modification architecturale ou de la fonction de perte, et une section discutant les implications computationnelles (FLOPs,

© 2026 Oussama Ghorbel

mémoire VRAM) de votre approche par rapport au protocole de base.

Exercice 2 : Cartographie des axes de recherche récents

L'autorégression visuelle évolue à une vitesse fulgurante. Sans chercher à inventer ou à halluciner des références bibliographiques précises, votre tâche consiste à structurer une veille scientifique.

• **Consigne :** Identifiez et décrivez les 5 grands axes de publication des 12 derniers mois concernant l'autoregression visuelle et les modèles unifiés.

• **Méthodologie :** Pour chaque axe (par exemple, « Tokenisation hiérarchique », « Distillation de modèles masqués », « Alignement sémantique des codebooks »), expliquez le problème fondamental que cet axe tente de résoudre, les limites des approches précédentes, et pourquoi la communauté s'y intéresse aujourd'hui. Cet exercice a pour but de vous forger une méthodologie de lecture critique des *preprints* (arXiv).

Exercice 3 : Protocole expérimental sur la taille du Codebook

Au Chapitre 7, nous avons discuté de l'impact de la taille du *codebook* sur la fidélité de la reconstruction et la modélisation. Considérez l'hypothèse suivante formulée par un collègue chercheur : *« Passer d'un codebook de 8 192 entrées à un codebook de 16 384 entrées améliore le score FID (Fréchet Inception Distance) de 10 % à résolution égale, sans dégrader la perplexité du LLM »*.

• **Tâche :** Concevez un protocole expérimental rigoureux pour valider ou réfuter cette hypothèse.

• **Détails requis :**

1. **Contrôle des variables :** Comment vous assurez-vous que l'amélioration du FID provient bien de la richesse du *codebook* et non d'une augmentation implicite de la capacité du modèle ou d'un biais d'entraînement ?

2. **Métriques secondaires :** Outre le FID et la perplexité, quelles métriques devez-vous suivre pour détecter un éventuel *codebook collapse* ou une explosion du coût de calcul lors de la recherche du plus proche voisin (*nearest neighbor search*) ?

© 2026 Oussama Ghorbel

3. **Ablation :** Décrivez la grille d'ablation (tailles intermédiaires, taux d'apprentissage du *codebook*, initialisation) nécessaire pour tracer la courbe de Pareto entre la taille du vocabulaire visuel et la performance générative.

En réalisant ces exercices, vous ne vous contenterez plus de consommer la littérature scientifique ; vous contribuerez à l'écrire. Le passage du token au pixel n'est qu'une étape. La véritable magie réside dans la capacité de l'ingénieur à orchestrer ces mathématiques pour repousser les limites de la machine. Je vous souhaite d'excellentes expérimentations, et peut-être, à travers vos futures publications, aurons-nous l'occasion de nous croiser sur les frontières de cette discipline qui n'a pas fini de nous surprendre.

Conclusion générale : L'ingénieur face à la toile infinie

Nous voici parvenus à la dernière page de cet ouvrage. Lorsque j'ai entrepris d'écrire « Du Token au Pixel », mon ambition n'était pas seulement de compiler un état de l'art technique, mais de vous offrir une boussole pour naviguer dans l'une des transitions les plus vertigineuses de l'histoire de l'informatique : le moment où les machines, après avoir appris à lire et à écrire le langage humain, ont commencé à voir, à imaginer et à peindre le monde.

Au fil des six parties de ce livre, nous avons parcouru un chemin immense. Nous sommes partis de l'atome fondamental de l'apprentissage profond moderne — le token — pour explorer comment il pouvait être mappé, aligné et transformé en une matrice de pixels cohérente. Nous avons disséqué l'anatomie des grands modèles de langage, compris comment le pixel, malgré sa nature continue et hautement redondante, pouvait être dompté par la quantification vectorielle et les auto-encodeurs. Nous avons exploré les méandres de la modélisation autorégressive, ces ordres de balayage qui obligent un réseau de neurones à « lire » une image comme on lit un roman, de gauche à droite, ou selon les fractales d'une courbe de Hilbert.

Pourtant, au-delà des mathématiques, des fonctions de perte et des architectures en Transformer, ce livre est avant tout une réflexion sur la représentation. Qu'est-ce qu'une image pour une machine ? Ce n'est ni une émotion, ni un souvenir, ni une œuvre d'art au sens humain du

© 2026 Oussama Ghorbel

terme. C'est une distribution de probabilités conditionnelles. C'est la prédiction statistique du prochain patch de couleur, sachant les mille patches qui l'ont précédé et les tokens textuels qui l'ont invoqué. Et c'est précisément dans cette réduction apparente que réside la magie de l'ingénierie : en réduisant la complexité infinie du monde visuel à un vocabulaire discret de quelques dizaines de milliers d'entrées, nous avons donné à la machine la capacité de composer l'inédit.

L'ingénieur en apprentissage automatique d'aujourd'hui n'est plus seulement un optimiseur de gradients ou un nettoyeur de jeux de données. Il est devenu un architecte de la perception synthétique. Lorsque vous concevez un *codebook*, vous ne faites pas que compresser de l'information ; vous définissez les briques élémentaires de l'imagination artificielle. Lorsque vous choisissez un ordre de balayage ou une stratégie de *cross-attention*, vous dictez la manière dont le modèle conceptualise l'espace, le temps et la sémantique. Cette responsabilité est immense. Elle exige une rigueur scientifique absolue, celle que nous avons tenté d'ancrer dans les protocoles pratiques de la Partie VI, mais elle requiert également une humilité constante face à l'émergence de comportements imprévus.

Les frontières que nous avons tracées dans ce chapitre final — des *scaling laws* visuelles aux modèles unifiés natifs, en passant par la génération 3D et l'interprétabilité mécanistique — ne sont pas de simples curiosités académiques. Elles sont les fondations des systèmes qui, dans les mois et les années à venir, redéfiniront nos interfaces, nos industries créatives, nos simulateurs scientifiques et notre rapport à la vérité visuelle. Le passage du token au pixel n'était qu'un prologue. La véritable révolution réside dans la fusion de ces modalités en un tissu cognitif unique, où le texte, l'image, le son et la physique s'entrelacent dans un même espace latent.

Face à cette accélération, il est facile de céder au vertige ou au cynisme. Les modèles deviennent obsolètes en quelques mois, les *benchmarks* sont saturés, et la course aux paramètres semble parfois sans fin. Mais je vous invite à résister à cette fatigue. La recherche n'est pas une course de vitesse, c'est une exploration en profondeur. Les véritables percées ne viendront pas uniquement de l'ajout de couches d'attention ou de l'augmentation du budget computationnel, mais de ceux qui sauront

© 2026 Oussama Ghorbel

remettre en question les postulats mêmes de la tokenisation. Peut-être que le *codebook* de demain ne sera plus un dictionnaire figé, mais une mémoire dynamique et continue. Peut-être que l'autorégression cédera sa place à des champs de diffusion probabilistes guidés par des grammaires visuelles latentes.

C'est à vous, qui tenez ce livre entre vos mains ou qui en faites défiler les pixels sur votre écran, qu'il appartient d'écrire la suite. Les exercices de ce dernier chapitre n'étaient pas de simples vérifications de connaissances, mais des invitations à la subversion scientifique. Testez les limites des *codebooks*, poussez les modèles dans leurs retranchements, cherchez les artefacts qui trahissent la mécanique sous-jacente. C'est en comprenant pourquoi un modèle échoue que l'on découvre comment il « pense ».

Je n'ai jamais conçu cet ouvrage comme une vérité gravée dans le marbre, mais comme un échafaudage. Un échafaudage robuste, je l'espère, qui vous aura permis de vous hisser à la hauteur des défis contemporains, mais qui est destiné à être démonté une fois que vous aurez construit vos propres édifices. Le domaine de l'intelligence artificielle générative est impitoyable avec ceux qui se reposent sur leurs acquis, mais il est infiniment généreux envers ceux qui conservent leur curiosité et leur rigueur.

En refermant ces pages, vous n'êtes plus seulement des lecteurs. Vous êtes les praticiens qui déploieront ces architectures dans le monde réel, qui en atténueront les biais, qui en optimiseront l'empreinte carbone et qui, peut-être, inventeront le paradigme qui rendra le mot « token » lui-même obsolète.

Le pixel est la destination finale de notre voyage, mais il n'est que le point de départ du vôtre. Que vos gradients soient stables, vos *codebooks* riches, et vos générations audacieuses.

— Oussama Ghorbel

© 2026 Oussama Ghorbel

Conclusion

Conclusion : L'Aube d'une Nouvelle Grammaire Visuelle

Synthèse du voyage : du token au pixel et retour

Au terme de cet ouvrage, il est temps de prendre de la hauteur pour contempler le chemin parcouru. Lorsque nous avons entamé cette exploration, notre point de départ était l'architecture familière du Grand Modèle de Langage (LLM) : une machine à prédire le prochain token, forgée dans le creuset du texte humain, optimisée pour la séquentialité et la syntaxe linguistique. Le défi que nous nous sommes fixé était de taille, voire vertigineux : comment contraindre cette architecture, intrinsèquement discrète et unidimensionnelle dans son déploiement, à engendrer la richesse continue, dense et spatialement structurée d'une image ? La réponse, comme nous l'avons vu tout au long de ces pages, ne résidait pas dans une révolution architecturale brutale, mais dans un changement de paradigme concernant la représentation même de l'information visuelle.

Nous avons d'abord dû désapprendre à voir l'image comme une simple grille de valeurs RGB. Le pixel, en tant que signal brut, est porteur d'une redondance massive et d'une entropie spatiale qui le rendent impropre à une modélisation autorégressive directe et efficace. C'est ici que la tokenisation visuelle, longuement abordée dans la deuxième partie de ce livre, a joué son rôle de pierre de Rosette. En compressant l'image via des auto-encodeurs variationnels (VAE) ou en la quantifiant par des modèles comme VQGAN, nous avons transformé le continuum visuel en un alphabet discret. Ce saut conceptuel — traiter un patch d'image ou un vecteur latent comme l'équivalent fonctionnel d'un mot dans une phrase — a été le catalyseur de toute la génération d'images par LLM. Nous avons exploré les codebooks, leurs tailles, leurs dimensions, leurs initialisations, et compris que la qualité du tokenizer conditionne irrémédiablement le plafond de verre du générateur. Un mauvais tokenizer produira des artefacts et des incohérences, quelle que soit la puissance ou la profondeur du modèle de langage qui le suit.

© 2026 Oussama Ghorbel

Ensuite, nous avons plongé dans les méandres de la modélisation autorégressive. Factoriser une image en une séquence de probabilités conditionnelles, $p(x) = \prod p(x_i|x_{<i})$, n'est pas une opération triviale lorsque l'espace sous-jacent est bidimensionnel. Le choix de l'ordre de balayage — qu'il soit raster, serpentin, en courbe de Hilbert, spiralé ou appris — n'est pas qu'une curiosité mathématique ou algorithmique ; il dicte la manière dont le modèle tisse les relations à longue portée, gère la cohérence globale de la scène générée et préserve la structure sémantique des objets. Nous avons également vu que l'autorégression stricte, bien qu'élégante et fondamentalement unificatrice avec le texte, souffre de sa propre linéarité et de sa lenteur inhérente. Cela nous a conduits à explorer les générations parallèles, masquées (MaskGIT) et multi-échelles, repoussant ainsi les limites de la vitesse d'inférence sans sacrifier la fidélité visuelle ni la diversité des échantillons.

Les leçons de l'ingénierie et de la recherche

Au-delà de la théorie et des équations, ce livre s'est voulu un manuel de bord pour l'ingénieur et le chercheur confrontés à la réalité du silicium et des données. La modélisation n'est rien sans un protocole d'entraînement rigoureux. Nous avons disséqué les mécanismes d'alignement entre le texte et l'image, comprenant que l'unification des vocabulaires dans un espace de tokens commun nécessite une attention extrême aux projections, aux normalisations et aux mécanismes de cross-attention. Conditionner la génération visuelle par une invite textuelle (prompt) exige une orchestration fine des embeddings, où la sémantique du langage doit venir sculpter l'espace latent de l'image sans en étouffer la variance créative.

L'un des enseignements majeurs de notre périple réside dans la gestion impitoyable du budget computationnel. Transformer un LLM de 7 milliards de paramètres en générateur d'images haute résolution n'est pas un simple exercice de remplacement de couche de sortie. C'est un exercice d'équilibriste où chaque FLOP compte, où la mémoire VRAM est une denrée rare. Nous avons détaillé les protocoles d'entraînement, l'importance cruciale des curriculums d'apprentissage — commençant par des résolutions basses pour monter progressivement en complexité et en finesse spatiale — et les techniques de stabilisation face à l'effondrement des modes, à la divergence des gradients ou à l'oubli

© 2026 Oussama Ghorbel

catastrophique des capacités linguistiques initiales. L'étude de cas sur l'adaptation de modèles ouverts comme LLaMA ou Mistral a démontré que le transfert de connaissances du langage vers la vision est non seulement possible, mais extraordinairement fructueux. Le LLM apporte avec lui une compréhension du monde, une capacité de raisonnement, de composition et une structuration logique qui, une fois projetées dans l'espace visuel, permettent de générer des scènes respectant des contraintes spatiales complexes, là où les modèles purement convolutifs ou de diffusion non conditionnés peinent parfois à maintenir la cohérence globale.

Cependant, l'ingénierie nous a aussi appris l'humilité face à la complexité des systèmes profonds. Les ablations et les diagnostics, couverts dans la cinquième partie, nous ont rappelé que l'intuition est souvent trompeuse en apprentissage automatique. Augmenter la taille du codebook n'améliore pas toujours le score FID ; ajouter des couches de cross-attention ne garantit pas un meilleur alignement CLIP. C'est en mesurant, en comparant et en isolant les variables que nous avons pu séparer les véritables avancées architecturales des simples artefacts d'hyperparamètres. La rigueur de l'évaluation, à travers des métriques aussi variées que le FID, l'Inception Score, le LPIPS, le CLIP-score ou les évaluations par paires humaines, reste le seul juge de paix de nos créations algorithmiques. Sans cette boussole métrique, l'ingénieur navigue à l'aveugle dans un espace latent de haute dimension.

Perspectives : au-delà de l'autorégression et de la diffusion

Alors que nous refermons les chapitres techniques, il est impératif de tourner notre regard vers l'horizon et d'anticiper les mutations à venir. La frontière entre l'autorégression par tokens et les modèles de diffusion, longtemps perçue comme une ligne de fracture paradigmatique irréconciliable, s'estompe aujourd'hui. Les modèles de flux (Flow Matching), les transformeurs de diffusion et les approches hybrides, qui combinent la planification globale de la diffusion avec la discrétisation sémantique des LLM, dessinent le contour des architectures de demain. Nous ne sommes plus dans l'ère de la compétition stérile entre paradigmes, mais dans celle de leur fusion synergique. La génération masquée et les modèles semi-autorégressifs comme MAR ont prouvé que l'on peut s'affranchir de la tyrannie de l'ordre séquentiel strict,

© 2026 Oussama Ghorbel

ouvrant la voie à des inférences rapides, hautement parallélisables et interactives.

Mais le véritable saut quantique résidera dans la nature même de ce que nous générons et la façon dont nous le représentons. Jusqu'à présent, nous avons principalement parlé d'images statiques en deux dimensions. Pourtant, le token ne demande qu'à s'émanciper du pixel plat pour investir le temps, le volume et la physique. La tokenisation de la vidéo, où la cohérence temporelle et la dynamique des fluides ou des corps s'ajoutent à la cohérence spatiale, est le prochain grand champ de bataille. Modéliser une séquence vidéo comme une séquence de tokens spatio-temporels permet au LLM de devenir un simulateur de dynamiques. Nous nous approchons à grands pas de l'ère des « world models » (modèles du monde), où le générateur d'images n'est plus un simple illustrateur de texte, mais un moteur de simulation capable d'anticiper les conséquences visuelles et physiques d'actions dans un environnement virtuel, un jeu vidéo ou un système robotique.

De même, la tokenisation sémantique, structurelle et abstraite promet de bouleverser nos approches. Plutôt que de compresser l'image en un ensemble de patches visuels locaux, pourquoi ne pas la tokeniser en un graphe de relations, en une scène 3D implicite (comme les NeRF, les Gaussian Splatting ou les champs de distance signés), ou en un ensemble d'instructions de rendu symboliques ? Le LLM, en tant que moteur de raisonnement, est naturellement enclin à manipuler des symboles, des hiérarchies et des structures logiques. Lui fournir une représentation 3D ou orientée objet de l'image pourrait démultiplier sa capacité à composer des scènes complexes, à maintenir l'identité stricte des personnages à travers différents angles de caméra, et à respecter les lois de la perspective, de l'occlusion et de l'éclairage avec une rigueur géométrique que l'approche purement pixelique et statistique ne peut qu'approximer par apprentissage de corrélations.

Enfin, l'optimisation et le déploiement, que nous avons abordés en fin d'ouvrage, dicteront la démocratisation réelle de ces technologies. La quantification post-entraînement, la distillation de connaissances, les techniques de serving asynchrone, le caching de KV et l'inférence spéculative ne sont pas de simples détails d'implémentation pour ingénieurs système ; ce sont les clés qui permettront à ces modèles

© 2026 Oussama Ghorbel

colossaux de tourner sur des appareils périphériques (edge devices), des smartphones ou des navigateurs web, rendant la génération d'images haute fidélité accessible, rapide, privée et respectueuse des ressources réseau.

Les défis éthiques, computationnels et sociétaux

Toutefois, cette marche en avant technologique, aussi fascinante soit-elle, ne saurait nous exonérer d'une réflexion profonde, lucide et continue sur ses implications. Le budget computationnel requis pour entraîner, affiner et faire inférer ces modèles multimodaux est colossal. L'empreinte carbone, la consommation en eau pour le refroidissement des data centers et l'énergie électrique nécessaire à la tokenisation de milliards d'images et à l'optimisation de centaines de milliards de paramètres posent un défi environnemental majeur. En tant que chercheurs et ingénieurs, nous avons la responsabilité morale de concevoir des algorithmes non seulement performants sur les benchmarks, mais aussi efficients. La course aveugle à la taille des modèles (scaling laws) doit être contrebalancée par une recherche acharnée sur l'efficacité des données, l'architecture éparse (sparsity), les réseaux à mélange d'experts (MoE) et l'inférence neuromorphique. La durabilité de l'IA générative est la condition sine qua non de son avenir.

Sur le plan sociétal et éthique, la capacité de transformer un LLM en générateur d'images d'un photoréalisme confondant soulève des questions cruciales qui dépassent le cadre du laboratoire. La prolifération des deepfakes, la désinformation visuelle à l'ère des élections et des conflits géopolitiques, et l'érosion générale de la confiance dans le document photographique exigent des réponses techniques et législatives robustes. Le tatouage numérique (watermarking) invisible mais robuste aux compressions et recadrages, la traçabilité cryptographique des générations (comme les standards C2PA) et le développement de détecteurs d'IA open-source ne sont plus des options de recherche marginales, mais des impératifs absolus de sécurité civile. Nous avons abordé ces limites dans la cinquième partie, mais il convient de marteler que la sécurité de l'IA visuelle doit être intégrée dès la phase de conception (safety by design), et non ajoutée comme un pansement algorithmique a posteriori.

© 2026 Oussama Ghorbel

De plus, l'impact de ces technologies sur les industries créatives, le tissu économique des artistes et la notion même d'auteur ne peut être ignoré ou balayé d'un revers de main technocratique. Le LLM génératif d'images est un outil d'une puissance inouïe, capable d'augmenter les capacités d'un artiste, d'un designer, d'un architecte ou d'un cinéaste, en agissant comme un collaborateur infatigable. Mais il menace également de précariser certains métiers d'exécution, d'inonder les marchés de contenus synthétiques médiocres et de diluer la valeur perçue de la création humaine originale. La question des droits d'auteur, de la transparence sur l'utilisation des données d'entraînement, du droit d'opposition (opt-out) et de la juste compensation des créateurs dont les œuvres ont nourri les codebooks et les espaces latents de nos modèles reste un champ de mines juridique et moral. Il nous appartient, en tant que communauté technique, de participer activement à ce débat, de promouvoir des jeux de données éthiques, et de concevoir des outils qui placent l'humain au centre du processus créatif, en tant que directeur artistique et garant du sens, plutôt qu'en tant que simple opérateur de prompt.

Mot final au lecteur : bâtisseurs de pixels

Nous arrivons ainsi au terme de ce voyage, du token au pixel, et du pixel à l'imagination. Si vous avez parcouru ces chapitres, que vous avez implémenté ces tokenizers, débogué ces fonctions de perte, ajusté ces ordres de balayage et observé, parfois avec émerveillement, parfois avec frustration, les premières images émerger du bruit de votre espace latent, vous n'êtes plus seulement un utilisateur ou un consommateur de l'intelligence artificielle. Vous en êtes devenu un architecte.

Ce livre a été conçu comme une boussole et une boîte à outils dans un territoire en mutation rapide, presque sismique. Les architectures spécifiques, les noms de modèles, les bibliothèques logicielles et les benchmarks que nous avons cités évolueront inévitablement. Certains deviendront obsolètes avant même que l'encre numérique de ces pages ne soit sèche, remplacés par des approches que nous n'imaginons qu'à peine aujourd'hui. Mais les fondations que nous avons bâties ensemble — la compréhension profonde de l'entropie visuelle, la maîtrise de la factorisation autorégressive, l'alignement sémantique entre les modalités, la gestion du budget computationnel et la rigueur implacable

© 2026 Oussama Ghorbel

de l'ablation — ces fondations, elles, sont pérennes. Elles constituent le socle mathématique et conceptuel sur lequel vous inventerez les paradigmes de demain.

N'ayez pas peur de l'expérimentation et de l'erreur. L'apprentissage automatique, en particulier dans le domaine visuel et génératif, est autant un art de l'observation qu'une science de l'optimisation. Les plus grandes découvertes naissent souvent d'une anomalie dans une courbe d'entraînement, d'un artefact inattendu et étrange dans une image générée, d'un effondrement de mode qui révèle une faille dans notre compréhension de l'espace latent, ou d'une intuition qui défie la littérature existante. Soyez curieux, soyez rigoureux dans vos protocoles, et n'oubliez jamais de regarder attentivement les images que vos modèles produisent. Derrière chaque matrice de tenseurs, chaque multiplication de poids et chaque distribution de probabilités se cache une esthétique, une tentative de la machine de capturer, de compresser et de réinventer la beauté, la laideur et la complexité du monde réel.

Transformer un Grand Modèle de Langage en générateur d'images, c'est finalement offrir à la machine le sens de la vue et de l'imagination visuelle. C'est lui apprendre à rêver en couleurs, en textures, en ombres et en lumières, en utilisant le langage humain comme le fil d'Ariane de ses hallucinations contrôlées. C'est une entreprise magnifique, exigeante, parfois ingrate, et fondamentalement sans fin.

Je vous remercie d'avoir partagé cette exploration technique et conceptuelle. Que vos codebooks soient denses et expressifs, que vos gradients soient stables et bien conditionnés, que vos inférences soient rapides, et que vos générations soient toujours à la hauteur de votre imagination. Le futur de la synthèse visuelle et de l'interaction multimodale s'écrit désormais en tokens, et c'est à vous, ingénieurs, chercheurs et créateurs, qu'il appartient de rédiger les prochaines lignes de cette histoire fascinante.

© 2026 Oussama Ghorbel

Bibliographie commentée

Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning* (2016).

Idée maîtresse : Fondements mathématiques des réseaux de neurones : représentation, régularisation, optimisation et modèles génératifs (VAE, GAN, flux).

Apport : Cadre théorique pour formaliser la distribution jointe p(x,y) entre tokens textuels et représentations visuelles latentes ; bases des VAE réutilisées dans la tokenisation.

Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3e éd.)* (2022).

Idée maîtresse : Guide pratique couvrant CNN, auto-encodeurs, GAN, transformers et déploiement, avec du code exécutable à chaque étape.

Apport : Méthodologie d'expérimentation reproductible, gestion des pipelines d'entraînement et astuces d'ingénierie reprises dans nos protocoles pratiques.

François Chollet, *Deep Learning with Python (2e éd.)* (2021).

Idée maîtresse : Introduction intuitive aux architectures profondes, insistant sur la représentation des données et la créativité computationnelle.

Apport : Approche pédagogique de la représentation discrète vs continue ; inspiration pour les schémas de raisonnement token->pixel.

David Foster, *Generative Deep Learning (2e éd.)* (2022).

Idée maîtresse : Panorama des modèles génératifs : VAE, GAN, modèles de diffusion, transformers génératifs, avec implémentations PyTorch.

Apport : Comparaison structurée diffusion vs autoregression ; chapitres sur VQ-VAE et DALL·E directement exploités pour la partie tokenisation.

Lewis Tunstall, Leandro von Werra, Thomas Wolf, *Natural Language Processing with Transformers* (2022).

© 2026 Oussama Ghorbel

Idée maîtresse : Guide pratique des transformers via la bibliothèque Hugging Face : tokenisation, fine-tuning, génération et déploiement.

Apport : Mécaniques internes du tokenizer et de la génération autoregressive ; base pour adapter un LLM pré-entraîné à une nouvelle modalité.

Denis Rothman, *Transformers for Natural Language Processing and Computer Vision (2e éd.)* (2023).

Idée maîtresse : Extension des transformers à la vision : ViT, BEiT, modèles multimodaux, et intégration texte-image.

Apport : Architecture ViT et patchification ; concepts de position embedding 2D réutilisés pour le balayage raster vs serpentin.

Kevin P. Murphy, *Probabilistic Machine Learning: Advanced Topics* (2023).

Idée maîtresse : Traitement probabiliste avancé : modèles à variables latentes, VQ-VAE, modèles autoregressifs, diffusion et normalizing flows.

Apport : Formalisation mathématique de la quantification vectorielle et de la vraisemblance des modèles discrets ; calculs de budget en bits/token.

Simon J. D. Prince, *Understanding Deep Learning* (2023).

Idée maîtresse : Synthèse moderne couvrant transformers, diffusion, VQ-GAN, multimodalité et grands modèles, avec exercices.

Apport : Chapitres sur la tokenisation visuelle et les codebooks ; schémas pédagogiques adaptés à nos figures de raisonnement.

Mohamed Elgendy, *Deep Learning for Vision Systems* (2020).

Idée maîtresse : Architectures CNN et vision par ordinateur : classification, détection, segmentation, GAN et transfert de style.

Apport : Compréhension des représentations convolutionnelles pré-tokenisation ; base pour le chapitre sur les inducteurs visuels (CNN backbone).

Jakub Langr, Vladimir Bok, *GANs in Action* (2019).

© 2026 Oussama Ghorbel

Idée maîtresse : Implémentation progressive de GAN : DCGAN, WGAN, StyleGAN, avec PyTorch.

Apport : Contrepoint génératif : comprendre ce que la diffusion et l'autoregression remplacent ou complètent par rapport aux GAN.

Christopher M. Bishop, *Pattern Recognition and Machine Learning* (2006).

Idée maîtresse : Fondements statistiques : modèles graphiques, inférence variationnelle, mélange de distributions, kernels.

Apport : Cadre d'inférence variationnelle nécessaire à la compréhension des VAE et de la borne ELBO utilisée dans les tokenizers.

David J. C. MacKay, *Information Theory, Inference, and Learning Algorithms* (2003).

Idée maîtresse : Théorie de l'information, codage, entropie, modèles de Markov cachés et réseaux de neurones.

Apport : Calculs d'entropie et de redondance pour dimensionner le codebook ; lien bits/token/pixel utilisé dans les budgets computationnels.

Daniel Jurafsky, James H. Martin, *Speech and Language Processing (3e éd., draft en ligne)* (2024 (draft)).

Idée maîtresse : Référence NLP : tokenisation BPE/SentencePiece, transformers, génération de texte, modèles multimodaux.

Apport : Mécanique BPE et gestion du vocabulaire ; analogie directe avec la construction d'un vocabulaire visuel discret.

Trevor Hastie, Robert Tibshirani, Jerome Friedman, *The Elements of Statistical Learning (2e éd.)* (2009).

Idée maîtresse : Apprentissage statistique : régression, classification, SVM, forêts, boosting, sélection de modèles.

Apport : Critères de sélection de modèle et validation croisée appliqués au choix de la taille de codebook et de la résolution de patch.

Alec Radford et al. (OpenAI) — document technique associé, *Learning Transferable Visual Models From Natural Language Supervision (CLIP)* (2021).

© 2026 Oussama Ghorbel

Idée maîtresse : Alignement contraste texte-image via un espace d'embedding partagé ; base de DALL·E et des scoreurs CLIP.

Apport : Principe d'embedding partagé et de CLIP-score pour l'évaluation ; architecture de cross-attention texte->image. (Note : article, non livre ; incertitude sur le format exact de publication.)

Aditya Ramesh et al. (OpenAI) — document technique associé, *Zero-Shot Text-to-Image Generation (DALL·E)* (2021).

Idée maîtresse : Génération autoregressive d'images via tokens VQ-VAE et un transformer GPT-like sur un vocabulaire unifié texte+image.

Apport : Preuve de concept centrale du livre : un LLM peut générer des images si on lui fournit un tokenizer visuel adéquat. (Article ; incertitude sur détails d'implémentation non publiés.)

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, Björn Ommer, *High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion)* (2022).

Idée maîtresse : Diffusion dans un espace latent compressé par auto-encodeur, conditionnée par texte via cross-attention.

Apport : Contrepoint diffusion : comprendre les compromis qualité/coût entre diffusion latente et autoregression pure. (Article ; chiffres FID à vérifier.)

Andreas Blattmann, Robin Rombach et al., *Stable Video Diffusion / travaux associés sur la tokenisation continue* (2023).

Idée maîtresse : Extension des modèles latents à la vidéo et exploration de tokenizers continus vs discrets.

Apport : Discussion sur les tokenizers continus (KL-regularized) comme alternative aux codebooks discrets. (Incertitude : détails complets non tous publiés.)

© 2026 Oussama Ghorbel